



UNIVERSITÀ DEGLI STUDI DI BERGAMO

Dipartimento di Ingegneria Gestionale,
dell'Informazione e della Produzione

Corso di Laurea Triennale in Ingegneria Informatica

Classe n. L-8 - Ingegneria Informatica

Modellazione Dinamica e Simulazione Real-Time per Velivolo F-16

*Analisi MIMO, Controllo longitudinale
attraverso LQR, MPC e Simulazione Real-Time
per computer di bordo FCS*

Relatore:

Prof. Davide Brugali

Correlatori:

Prof. Antonio Russo

Tesi di Laurea di:

Leonardo Leggeri

Matricola: 1091937

Anno Accademico 2025/2026

Indice

Elenco delle figure	1
Elenco delle tabelle	5
Abstract	7
Introduzione	9
I Modellazione e Simulazione Dinamica dell' F-16	13
1 Modellazione cinematica e dinamica del velivolo	15
1.1 Notazione e sistemi di riferimento	15
1.2 Vettori di stato e cinematica del vento	18
1.3 Modellazione di forze e momenti aerodinamici	19
1.3.1 Forze nel Body Frame (bf)	19
1.3.2 Forze nel Wind Frame (w)	19
1.3.3 Momenti Aerodinamici	20
2 Derivazione Analitica delle Equazioni Del Moto	21
2.1 Equazioni di cinematica	21
2.2 Equazioni di dinamica	21
2.3 Momento angolare	22
3 Modello delle Forze e Momenti Aerodinamici	23
3.1 Pressione Dinamica e Geometria di Riferimento	23
3.2 Modelli di forze aerodinamiche	25
3.3 Adimensionalizzazione dei momenti	25
3.4 Analisi delle derivate di stabilità tramite serie di Taylor	26
3.4.1 Dinamica longitudinale	26
3.4.2 Modello Aerodinamico e Derivate di Stabilità	26
3.4.3 Dinamica Latero-Direzionale e Accoppiamento Aerodinamico	27

4	Equazioni non lineari del moto e integrazione numerica	31
4.1	Equazioni di posizione e assetto	31
4.2	Dinamica non lineare delle forze e dei momenti	32
4.3	Meccanismo di Controllo Longitudinale	36
4.4	Rappresentazione dell'assetto mediante quaternioni	37
4.5	Analisi critica dei metodi di Eulero	40
4.6	Metodo di Runge-Kutta del quarto ordine (RK4)	41
5	Architettura e Struttura del Simulatore	45
5.1	Panoramica Globale dell'Architettura	46
5.2	Dinamica del Corpo Rigido	46
5.3	Sottosistema Aerodinamico	47
5.4	Modello Atmosferico	49
5.5	Parametri di Volo e Assi di Stabilità	49
5.6	Modellazione della Gravità	51
5.7	Condizionamento delle Uscite	52
6	Ambiente di Simulazione Attuazione Hardware-in-the-Loop	53
6.1	Integrazione con periferiche Thrustmaster HOTAS	54
6.2	Interfacciamento: FlightGear e Aerospace Blockset	55
6.2.1	Immagini FlightGear	55
6.2.2	Strumenti di Bordo e Telecamere	57
II	Automazione del controllo longitudinale	59
7	Analisi di Stabilità e Linearizzazione	61
7.1	Definizione dei punti di equilibrio	61
7.1.1	L'Oggetto opspect (Operating Point Specification)	61
7.1.2	Generazione di op e opreport	62
7.2	Linearizzazione dello spazio di stato	64
7.3	Analisi spettrale e autovalori	65
7.4	Disaccoppiamento delle dinamiche	66
7.5	Rappresentazione MIMO e funzioni di trasferimento	69
7.5.1	Derivazione Matematica	70

8	Proprietà Strutturali e Analisi MIMO	71
8.1	Raggiungibilità, osservabilità e realizzazione minima	71
8.1.1	Matrici di Raggiungibilità e Osservabilità	71
8.1.2	Cancellazione Polo-Zero e Comando minreal	74
8.1.3	Matrice di Sistema e Zeri Invarianti	74
8.1.4	La Proprietà Bloccante degli Zeri MIMO	75
8.1.5	Verifica in MATLAB: Estrazione degli Zeri Invarianti	76
	Significato Fisico della Proprietà Bloccante Esponenziale ($\lambda \neq 0$)	76
8.2	Analisi dei poli, zeri e proprietà bloccanti	77
8.2.1	Controprova Computazionale: Analisi di un Sottosistema Sot-	
	toattuato (2×2)	78
	La comparsa della Proprietà Bloccante Esponenziale	78
	La comparsa della Proprietà Bloccante Esponenziale ($\lambda \neq 0$)	79
	Simulazione Multipla della Proprietà Bloccante	79
8.3	Interazioni dinamiche e vincoli strutturali	81
9	Sintesi del Controllo e Limiti del Disaccoppiamento	83
9.1	Progetto del pre-compensatore dinamico	83
9.1.1	Calcolo della Prima Colonna(Velocità)	84
9.1.2	Calcolo della Seconda Colonna	84
9.1.3	Calcolo della Terza Colonna(Becheggio)	85
9.2	Limiti algoritmici: complessità polinomiale	85
9.3	Evoluzione verso il controllo nello spazio di stato	86
10	Controllo LQR (Linear Quadratic Regulator)	87
10.1	Sintesi del regolatore	87
10.1.1	Orizzonte Infinito e Indice di Costo	88
10.1.2	L'Equazione Algebrica di Riccati	88
10.2	Ottimizzazione delle performance tramite la Regola di Bryson	90
10.2.1	Il problema delle scale e l'inadeguatezza della matrice identità	91
10.2.2	Applicazione della Regola di Bryson	92
10.2.3	Conclusioni sulla sintesi LQR	93
10.2.4	Prestazioni Aggressive Con Matrice Q dominante	94
10.2.5	Compromesso Ottimo Con Matrici Q ed R bilanciate	96

10.2.6	Risparmio Energetico Con Matrice R dominante	97
11	Controllo Predittivo (MPC)	99
11.1	Principi del Receding Horizon	99
11.1.1	Formulazione del Problema di Ottimizzazione	99
11.1.2	Varianti del Vincolo Terminale	100
	Vincolo di Uguaglianza $x(N) = 0$	100
	Vincolo di Disuguaglianza $x(N) \in \mathbb{X}_f$	101
11.2	Stabilità e teoremi di Lyapunov	101
11.2.1	Teorema di Lyapunov	101
11.2.2	L'Ipotesi Base di Stabilità nell'MPC	102
11.2.3	La Prova Fondamentale di Stabilità	103
11.2.4	Formulazione Alternativa: Vincolo Terminale di Uguaglianza	104
11.3	Analisi degli insiemi invarianti e stabilità robusta	104
11.3.1	Ingredienti Fondamentali per la Stabilità	104
11.4	Implementazione per Sistemi Lineari: Legame con LQR	105
11.4.1	Il Control Invariant Set e il Maximal Invariant Set $\mathcal{O}(\infty)$	106
	Algoritmo Iterativo per il Calcolo di $\mathcal{O}(\infty)$	106
11.4.2	Definizione del Punto di Equilibrio e Variabili di Trim	107
11.4.3	Impostazione dei Vincoli di ingresso e stato	108
	Vincoli di Ampiezza sugli Attuatori	108
	Vincoli di Rate (Velocità di Attuazione)	109
	Vincoli sugli Stati	109
11.4.4	Definizioni di Insiemi Controllabili e Stabilizzabili	109
	One-step Set $\mathcal{Q}(\Omega)$	109
	i-steps Controllable Set $\mathcal{C}_i(\mathcal{X}, \Omega)$	110
	i-steps Stabilizable Set $\mathcal{S}_i(\mathcal{X}, \Omega)$	110
11.4.5	Proprietà di Monotonicità e Convergenza	110
11.5	Implementazione Pratica: Controllo Longitudinale F-16	111
11.5.1	Analisi dei Risultati e Validazione del Controllore MPC	112
11.5.2	Variazione del Funzionale di Costo	114

III Implementazione Real-Time Multithreading in C++ 117

12 Monitor Real Time	119
12.1 Schedulazione nei Sistemi Real-Time	119
12.1.1 Utilizzo di Rate Monotonic	120
12.2 Obiettivo della Tesi	120
13 Modelli e Strumenti	123
13.1 Il Modello di Task Periodico	123
13.1.1 Definizioni Fondamentali	123
13.1.2 Il Ciclo di Vita di un Task Periodico	124
13.1.3 Metriche Temporal Fondamentali	124
13.2 Schedulazione a Priorità Fissa: Rate Monotonic	125
13.2.1 Algoritmi di Scheduling Real-Time	125
13.2.2 Rate Monotonic: Assegnamento delle Priorità	126
13.3 POSIX e DDS per Attività Periodiche e Distribuite	126
13.3.1 Il Middleware DDS e la Qualità del Servizio	128
13.4 Il Monitor DDS: <i>eProsima Fast DDS Monitor</i>	129
13.4.1 Prerequisito: Abilitazione del Modulo Statistico	129
13.4.2 Configurazione del Dominio nel Codice	130
13.4.3 Domain View: La Mappa della Rete DDS	131
13.4.4 Chart View: Visualizzazione Statistica della Comunicazione	133
Modalità di tracciamento statistico:	133
Dati in Tempo Reale	135
Sistema di Allerta	135
13.4.5 Utilizzo nel Progetto: Validazione Prima del Tracciamento	136
13.5 Strumenti di Tracciamento del Kernel: ftrace , trace-cmd e Kernel-Shark	137
13.5.1 Il Sottosistema ftrace del Kernel Linux	137
I Tracepoint del Sottosistema sched	137
13.5.2 trace-cmd : Interfaccia a Riga di Comando per ftrace	138
Utilizzo in run_trace.sh : Tracciamento di un Singolo Processo	138
Utilizzo in marker.sh : Tracciamento Passivo Multi-Processo	139
Il File Prodotto: trace.dat	140

13.5.3	Il Comando <code>trace-cmd report</code> : Decodifica del Log Binario	141
13.5.4	KernelShark: Visualizzazione Grafica del Tracciato	143
13.5.5	La Dashboard: <code>dashboard.py</code>	144
13.5.6	Prerequisiti di Installazione	145
	Tab 1: Scripts Manager	146
	Tab 2: DDS Studio MultiProcesso	146
	Tab 3: Trace Manager	147
13.5.7	Flow-Chart di Riferimento per ottenere i Documenti di Progetto	149
13.6	Librerie Python Utilizzate	150
13.6.1	Librerie per l'Analisi e Visualizzazione dei Dati	150
13.6.2	Librerie per la GUI e l'Esecuzione di Sistema	150
14	Il Lavoro Svolto	151
14.1	Fase 1 — Architettura del Progetto e Build System	151
14.1.1	Struttura delle Directory	151
	La Directory dei Sorgenti (<code>src/</code>)	152
	La Directory degli Artefatti (<code>bin/</code>)	153
14.1.2	Build System: Architettura Decentralizzata dei Makefile	153
	Perché Non un Makefile Unico?	153
	Perché il Linking Statico	154
	Costruzione Locale e Linking Statico	154
	Gestione Dinamica delle Dipendenze degli Header	155
14.1.3	Libreria <code>time</code> : Gestione del Tempo Monotónico POSIX	156
	Perché <code>CLOCK_MONOTONIC</code> e non <code>CLOCK_REALTIME</code>	156
14.1.4	Libreria <code>trace</code> : Iniezione dei Marker nel Kernel	157
	Perché scrivere i marker nel Kernel e non in un File Utente	157
	Due Versioni: con Stato e Inline	158
14.1.5	Libreria <code>activity</code> : Struttura del Task Periodico	158
	Il Problema del Drift Cumulativo e la sua Soluzione	158
	La Funzione di Carico Computazionale: <code>activity_load</code>	159
14.1.6	Libreria <code>dds_lib</code> : Wrapper al Middleware FastDDS	160
	Modulo <code>Broadcastner</code>	160
	Modulo <code>Listener</code>	161
14.1.7	Libreria <code>generated_msg</code> : Classi di Messaggistica DDS	161

Il Processo di Generazione	161
14.1.8 Libreria math : Funzioni Algebriche Ausiliarie	163
14.2 Fase 3 — Le Applicazioni del Progetto	163
14.2.1 Applicazione app10g : Due Task su Core Distinti	163
Dipendenze e Gestione degli Errori:	164
Inizializzazione del Tracing e Dichiarazione delle Variabili	165
Inizializzazione degli Attributi e Policy RT	165
Affinità di Core	166
Configurazione dei Thread	167
Cleanup e Attesa	168
Caratteristiche Rilevanti	169
PeriodicTask eseguita dall' applicazione:	169
activity_load :	172
14.2.2 Applicazione Broadcastner : Publisher DDS Periodico	173
Include e Variabili Globali di Modulo	173
La Funzione di Pubblicazione con Marker	174
Inizializzazione DDS e Creazione dei Thread	175
14.2.3 Applicazione Listener : Subscriber DDS Event-Driven	179
Include e Namespace	179
La Classe GeometryListener : Architettura a Callback	180
Il main : Avvio della Sottoscrizione	182
14.3 Fase 4 — Infrastruttura di Tracciamento e Automazione	183
14.3.1 Tracciamento Attivo: run_trace.sh	184
Risoluzione dei Percorsi e Alias dei Nomi	184
Discovery Dinamica di Binario e Sorgente	185
Creazione della Directory di Output Univoca	187
Step 1 — Registrazione con trace-cmd record	187
Step 2 — Decodifica con trace-cmd report	188
Step 3 — Compilazione ed Esecuzione del Parser C++	189
Step 4 — Monitor Visivo Python e Chiusura	193
14.3.2 Tracciamento Passivo Multi-Processo: marker.sh	194
Risoluzione dei Percorsi e Lettura degli Argomenti	194
Aggancio Passivo: Estrazione dei TID da /proc	195

Discovery del Sorgente, Directory di Output e Registrazione	197
Decodifica, Analisi, Monitor e KernelShark	200
14.3.3 Le Versioni Didattiche: <code>marker1.sh</code> – <code>marker4.sh</code>	203
<code>marker1.sh</code> : Registrazione	203
<code>marker2.sh</code> : Decodifica	206
<code>marker3.sh</code> : Analisi con Parser C++	207
<code>marker4.sh</code> : Visualizzazione	209
14.4 Fase 5 — Analisi del Tracciato e Strumenti di Monitoraggio	211
14.4.1 L'Analizzatore C++: <code>thread_analysis.cpp</code>	211
Include, Strutture Dati e Helper Functions	211
Fase 1: Estrazione dei Parametri Teorici dal Sorgente C/C++	214
Fase 2: Discovery Dinamica dei Thread dal Trace	216
Fase 3: Estrazione degli Sched_Switch e Logica VPID	219
Fase 4: Costruzione Intervalli, Metriche RT e Lettura file CSV	224
15 Caso di Studio: Il Computer di Bordo dell'F-16	233
15.1 Motivazione Aeronautica e Architettura del Sistema	234
15.2 Librerie Avioniche Utilizzate	235
15.2.1 Modulo F-16: Librerie di Simulazione Aerodinamica	235
Rilevamento Runtime delle Deadline Miss e Flag Skip	235
Elenco Completo dei Moduli F-16	236
15.3 Il Sistema di Controllo del Volo (FCS)	238
15.3.1 Controllo del Beccheggio (PITCH_A, PITCH_B, PITCH_C)	238
15.3.2 Controllo del Rollio (ROLL_A, ROLL_B, ROLL_C)	239
15.3.3 Controllo della Quota (ALT_A, ALT_B)	239
15.4 Il Sistema di Navigazione	239
15.4.1 Navigazione GPS (GPS_1, GPS_2)	240
15.4.2 Navigazione Inerziale (INS_1, INS_2, INS_3)	240
15.5 Il Sistema Radar	241
15.5.1 Radar Aria-Aria (RADAR_A)	241
15.5.2 Radar Aria-Terra (RADAR_G)	241
15.6 Il Sistema FADEC	242
15.7 Il Sistema d'Arma e i Display Avionici	242
15.7.1 Sistema d'ingaggio	242

15.7.2 Display HUD e MFD	243
15.8 Distribuzione dei Core e Analisi dell'Architettura	243
15.9 Sincronizzazione Finale e Terminazione	243
15.10 Risultati dell'Analisi di Tracciamento	245
16 Conclusioni	253
16.1 Risultati Ottenuti	253
16.2 Limiti del Sistema	254
16.3 Sviluppi Futuri	254
Ringraziamenti	257
A Codici MATLAB: Controllo Predittivo MPC	261
B Codici MATLAB: Controllo Ottimo LQR	287
Bibliografia	311

Elenco delle figure

1.1	Definizione dei sistemi di coordinate: la terna solidale al velivolo (Body Frame - FRD) e la terna inerziale terrestre (Earth Frame - NED).	16
3.1	Rappresentazione delle forze aerodinamiche incidenti sul profilo alare e scomposizione del vettore velocità.	24
3.2	Rappresentazione geometrica dell'apertura alare (b) e dei parametri di riferimento del velivolo.	24
4.1	Rappresentazione visiva del fenomeno del Gimbal Lock: allineamento fisico degli assi di rotazione quando l'angolo di beccheggio raggiunge i 90° , con conseguente perdita di un grado di libertà.	38
4.2	Rappresentazione topologica della sfera in S^3	39
5.1	Architettura globale del simulatore F-16 in ambiente Simulink. Sono visibili i sei sottosistemi principali che governano la dinamica del volo.	46
5.2	Dettaglio del blocco 6-DOF (Quaternion).	47
5.3	Dettaglio del sottosistema Aerodinamico: interfaccia tra lo stato del velivolo e la funzione di calcolo dei coefficienti.	48
5.4	Implementazione del modello atmosferico ISA per il calcolo dei parametri ambientali in funzione della quota.	49
5.5	Sottosistema per il calcolo delle variabili cinematiche α , β e della pressione dinamica q	50
5.6	Trasformazione vettoriale della forza peso dal sistema NED agli assi corpo tramite Direction Cosine Matrix.	52
5.7	Condizionamento dei segnali di uscita e calcolo dei parametri di volo derivati (Mach, Gamma).	52
6.1	Panoramica dell'architettura di simulazione: integrazione tra Simulink e motore di rendering FlightGear.	53

6.2	Sottosistema di acquisizione e condizionamento dei segnali provenienti dal sistema Thrustmaster.	54
6.3	Visuale a terra dell'aereo prima del decollo.	56
6.4	Visuale FlightGear in volo di crociera.	56
6.5	Visuale dell'abitacolo su FlightGear in volo di crociera.	57
6.6	Interfacciamento con FlightGear tramite Aerospace Blockset e visualizzazione dell'assetto tramite telecamere esterne e strumentazione di bordo.	58
7.1	Inizio della ricerca dell'algoritmo PSO	62
7.2	Algoritmo PSO a metà del suo funzionamento	63
7.3	Algoritmo PSO alla fine del suo funzionamento	63
7.4	Funzionale di costo dell'algoritmo	64
7.5	Report generato da MATLAB al termine dell'ottimizzazione, mostrando gli stati e gli ingressi di controllo del punto operativo (Trim) validato.	64
7.6	Mappa Poli-Zeri (pzmap) della realizzazione minima longitudinale. Evidenza del polo instabile a parte reale positiva.	66
8.1	Verifica simulata della proprietà bloccante sul sottosistema 2×2 . Solo la perfetta sovrapposizione vettoriale tra ingresso e stato iniziale (Caso 1) garantisce l'annullamento delle uscite. Qualsiasi deviazione dallo stato geometrico ideale (Caso 2 e 3) compromette il bilanciamento delle forze interne.	81
10.1		92
10.2	Risposta del sistema con $Q \gg R$. (a) Il rientro è immediato e aggressivo, (b) ma genera picchi di comando che possono causare la saturazione degli attuatori.	94
10.3	Analisi geometrica con $Q \gg R$. La ripidezza della funzione di Lyapunov (b) forza le traiettorie di fase (a) a una convergenza dalle condizioni iniziali verso l'equilibrio.	95
10.4	Risposta del sistema con $Q \approx R$. El controllore offre un ottimo compromesso tra rapidità di reiezione del disturbo (a) e contenimento dello sforzo di controllo (b).	96

10.5	Analisi geometrica con $Q \approx R$. La conformazione bilanciata della superficie di Lyapunov (b) garantisce traiettorie di fase (a) fluide e ben smorzate verso l'origine.	97
10.6	Risposta del sistema con $R \gg Q$. L'azione di controllo è moderata ed energeticamente efficiente (b), ma il rientro al punto operativo risulta molto più lento (a).	98
10.7	Analisi geometrica con $R \gg Q$. La superficie di Lyapunov risulta quasi piatta (b), permettendo alle traiettorie di fase (a) ampie escursioni e una convergenza molto lenta.	98
11.1	Evoluzione spaziale degli insiemi a i passi. Il grafico confronta visivamente le proprietà matematiche discusse mostrando un target invariante (insiemi stabilizzabili $\mathcal{S}_i$) si ottiene una sequenza in rigorosa espansione monotona, mentre l'utilizzo di un target generico non invariante (insiemi controllabili $\mathcal{C}_i$) non garantisce la proprietà che garantisce la completa controllabilità del sistema in N passi	111
11.2	Rappresentazione del poliedro di invarianza $\mathcal{X}_f$ (fetta $\theta = 0$) e traiettoria di predizione dello stato verso l'origine.	112
11.3	Evoluzione temporale delle variabili di stato: beccheggio, pitch rate e variazioni di velocità lineare.	113
11.4	Sforzo di controllo degli attuatori: si noti l'attivazione dei vincoli di saturazione sull'elevatore e sui flap.	114
11.5	Analisi geometrica del controllore MPC a ciclo chiuso. Il grafico tridimensionale mostra l'evoluzione dello stato e il corrispondente andamento decrescente del funzionale di costo ottimo, che si comporta come una funzione di Lyapunov scendendo lungo la superficie del costo terminale.	115
13.1	La figura che mostra il funzionamento del Domain View	132
13.2	figura che mostra funzionamento del monitor DDS e del Chart View	133
13.3	logo dell'applicazione	143
13.4	Interfaccia grafica di KernelShark	144
13.5	Tab presente nella dashboard per il tracing di singoli processi	146
13.6	tab per il tracing di più processi nella dashboard	147

13.7		148
13.8	Flowchart di riferimento e flusso dati	149
14.1	Estratto di <code>trace_output.txt</code> : ogni riga documenta un evento di scheduling con timestamp, nome processo, PID e stato di transizione.	189
14.2	Output a terminale di <code>thread_analysis</code> : metriche RT per ogni task.	191
14.3	Esempio di risultato ottenuto nella creazione del file <code>timeline.csv</code> .	191
14.4	Diagramma di Gantt generato da <code>monitor.py</code> per <code>app10g</code> .	193
14.5	Output di <code>marker.sh</code> : aggancio congiunto con estrazione di tutti i TID prima di avviare il trace.	198
14.6	Monitor Python con risultato della simulazione Broadcastner-Listener. Si può notare nel monitor la barra arancio riguardante il tempo impiegato per la comunicazione DDS.	202
14.7	Interfaccia KernelShark con marker applicativi visibili sulla timeline.	202
15.1	Diagramma di Gantt del simulatore F-16 prodotto da <code>monitor.py</code> : le 20 corsie temporali mostrano l'alternanza di stati RUN (verde), SLEEP (grigio) e PREEMPT (rosso) per ogni sottosistema avionico.	245

Elenco delle tabelle

1.1	Legenda e Significato Fisico delle Variabili del Moto Longitudinale . .	17
14.1	Descrizione delle metriche statistiche presenti nel file <code>timeline.csv</code> .	192
15.1	Mappa completa dei 20 thread del simulatore F-16	234

Abstract

This thesis presents a comprehensive study on the flight dynamics, advanced multivariable control, and real-time software implementation of an F-16 fighter jet on-board computer. Due to its inherent aerodynamic instability—an engineering design choice aimed at maximizing maneuverability for aerial combat—the aircraft cannot be piloted manually and strictly requires a robust, automatic *Fly-by-Wire Flight Control System*(FCS). The work is logically structured into three main phases:

- **Modeling and Simulation:** the derivation of the non-linear 6-DOF equations of flight dynamics, the integration of NASA aerodynamic models, and the creation of a high-fidelity simulator in MATLAB/Simulink interfaced with FlightGear for 3D visual validation.
- **Multivariable and Optimal Control:** the extraction of the linearized MIMO state-space model at cruise Trim, the study of the structural limits of classic decoupling, and the synthesis of advanced controllers LQR and MPC capable of handling the physical saturation limits of the actuators to ensure closed-loop asymptotic stability.
- **Real-Time Implementation (C++):** the multithreaded C++ architecture developed on multi-core machines, leveraging POSIX primitives, the eProsima DDS middleware for inter-process communication, and Rate Monotonic (RM) scheduling. The system is validated through a Linux kernel tracing infrastructure (ftrace, trace-cmd) interfaced with a Python graphical dashboard.

Introduzione

L'obiettivo di questa tesi è lo studio sistemistico, il controllo multivariabile e l'implementazione software in tempo reale del computer di bordo di un caccia F-16. Essendo progettato con un'instabilità aerodinamica intrinseca per massimizzarne la manovrabilità in combattimento, questo velivolo non può essere pilotato manualmente e richiede un *Flight Control System* (FCS) di tipo *Fly-by-Wire* estremamente affidabile. Il lavoro si articola logicamente in tre macro-aree principali:

- **Modellazione e Simulazione:** la derivazione delle equazioni non lineari (6-DOF) della dinamica di volo, l'integrazione dei modelli aerodinamici NASA e la creazione di un simulatore ad alta fedeltà in MATLAB/Simulink interfacciato con FlightGear per la validazione visiva 3D.
- **Controllo Multivariabile e Ottimo:** l'estrazione del modello lineare MIMO nel punto di *Trim* di crociera, lo studio dei limiti del disaccoppiamento classico, e la sintesi di controllori avanzati LQR e MPC capaci di gestire i vincoli fisici di saturazione degli attuatori per garantire la stabilità asintotica in anello chiuso.
- **Implementazione Real-Time (C++):** l'architettura multithreading in C++ sviluppata su macchine multi-core, avvalendosi di primitive POSIX, del middleware DDS di eProsima per la comunicazione inter-processo e della schedulazione *Rate Monotonic* (RM). Il sistema è validato tramite un'infrastruttura di tracing del kernel Linux (ftrace, trace-cmd) interfacciata con una dashboard grafica in Python.

Struttura della tesi

Parte I — Modellazione e Simulazione Dinamica dell’F-16

Capitolo 1 — Modellazione cinematica e dinamica del velivolo: Definisce i sistemi di riferimento, la nomenclatura ICAO e i vettori di stato cinematici e aerodinamici del velivolo.

Capitolo 2 — Derivazione analitica delle equazioni del moto: Deriva rigorosamente le equazioni differenziali 6-DOF (dinamica e cinematica) e discute l’utilizzo dei quaternioni per evitare il Gimbal Lock.

Capitolo 3 — Modello delle Forze e Momenti Aerodinamici: Costruisce il modello aerodinamico basato sui coefficienti tabulati NASA e discute la pressione dinamica e le derivate di stabilità.

Capitolo 4 — Equazioni non lineari del moto e integrazione numerica: Presenta il sistema di equazioni differenziali esplicito finale e analizza i metodi di integrazione numerica (Runge-Kutta, Eulero).

Capitolo 5 — Architettura e Struttura del Simulatore: Illustra l’architettura a blocchi del simulatore MATLAB/Simulink e i sottosistemi principali (aerodinamica, gravità, corpo rigido).

Capitolo 6 — Ambiente di Simulazione & Attuazione Hardware-in-the-Loop: Descrive l’integrazione del simulatore con l’hardware di controllo (Thrust-master HOTAS) e con il motore di rendering 3D FlightGear per l’analisi visiva.

Parte II — Automazione del controllo longitudinale

Capitolo 7 — Analisi di Stabilità e Linearizzazione: Ricerca numerica del punto operativo (Trim) in crociera, estrazione del modello lineare e analisi di stabilità del sistema (poli e autovalori).

Capitolo 8 — Proprietà Strutturali e Analisi MIMO: Analisi di raggiungibilità e osservabilità, realizzazione minima e studio della proprietà bloccante degli zeri di trasmissione nel sistema MIMO.

Capitolo 9 — Sintesi del Controllo e Limiti del Disaccoppiamento: Studio delle tecniche di disaccoppiamento in avanti e dei limiti matematico-strutturali che impediscono l'uso di semplici PID decentralizzati.

Capitolo 10 — Controllo LQR (Linear Quadratic Regulator): Sintesi del Regolatore Lineare Quadratico (LQR), taratura dei pesi con la regola di Bryson e analisi geometrica del funzionale di Lyapunov.

Capitolo 11 — Controllo Predittivo (MPC): Formulazione e implementazione del Model Predictive Control per gestire esplicitamente i vincoli fisici degli attuatori, con analisi degli insiemi invarianti.

Parte III — Implementazione Real-Time Multithreading in C++

Capitolo 12 — Monitor Real-Time: Introduzione al caso di studio e all'architettura software di monitoraggio in tempo reale.

Capitolo 13 — Modelli e Strumenti: Fondamenti teorici: task periodici, primitive POSIX, middleware DDS, e strumenti di tracing del kernel Linux (ftrace, trace-cmd) con interfacce Python.

Capitolo 14 — Il Lavoro Svolto: Dettaglio implementativo delle librerie core in C++, orchestrazione tramite script Bash, e sviluppo della dashboard di monitoraggio grafico.

Capitolo 15 — Caso di Studio: il Computer di Bordo dell'F-16: Integrazione finale del sistema: emulazione dei sottosistemi avionici tramite thread POSIX e validazione temporale delle tracce kernel per garantire il real-time.

Capitolo 16 — Conclusioni: Sintesi dei traguardi raggiunti, limiti del sistema e proposte per sviluppi futuri.

Parte I

Modellazione e Simulazione Dinamica dell' F-16

Capitolo 1

Modellazione cinematica e dinamica del velivolo

In questo capitolo vengono definiti tutti i vettori di stato, le variabili aerodinamiche, i sistemi di riferimento e le convenzioni direzionali. L'intera nomenclatura adottata in questa trattazione segue rigorosamente lo standard accademico definito da Stevens e Lewis nel testo di riferimento "*Aircraft Control and Simulation*",[3]

1.1 Notazione e sistemi di riferimento

Per semplificare la scrittura delle matrici di trasformazione spaziale (come quelle impiegate nelle equazioni di navigazione), si adotta la seguente notazione compatta per le funzioni trigonometriche degli angoli:

- $c(\cdot) \equiv \cos(\cdot)$
- $s(\cdot) \equiv \sin(\cdot)$

Ad esempio, scriveremo $c\theta$ per indicare $\cos \theta$ e $s\phi$ per indicare $\sin \phi$.

Sistemi di Riferimento e Convenzioni Direzionali

Per descrivere accuratamente il moto nello spazio tridimensionale, si definiscono i seguenti sistemi di riferimento:

- **Earth Frame (F_E) - Convenzione NED:** Sistema inerziale solidale con la superficie terrestre. Gli assi seguono la convenzione **NED** (*North, East, Down*).

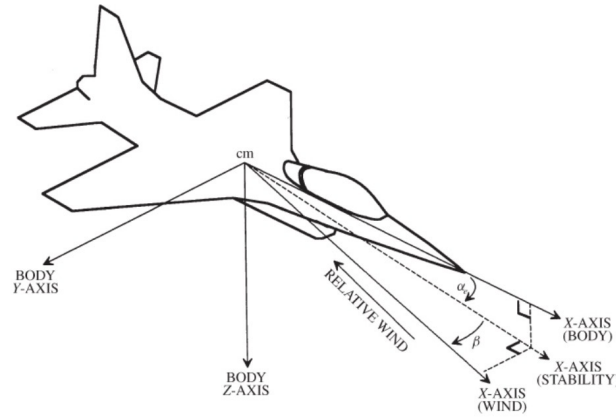


FIGURA 1.1: Definizione dei sistemi di coordinate: la terna solida al velivolo (Body Frame - FRD) e la terna inerziale terrestre (Earth Frame - NED).

- **Target Plane (tp):** Piano di destinazione inerziale utilizzato per la navigazione. I vettori atmosferici vengono spesso definiti in questo sistema.
- **Body Frame (bf o frd) - Convenzione FRD:** Sistema di riferimento non inerziale con origine nel baricentro (*Center of Mass*, CM) del velivolo. Gli assi seguono la convenzione **FRD** (*Forward, Right, Down*):
 - X_{frd} : Asse longitudinale, punta verso il muso (*Forward*).
 - Y_{frd} : Asse trasversale, punta verso l'ala destra (*Right*).
 - Z_{frd} : Asse verticale, punta verso la pancia (*Down*).
- **Wind Frame (w):** Sistema di riferimento allineato con il vettore della velocità relativa dell'aria.
- **Stability Frame (s):** Sistema di riferimento intermedio, allineato con la proiezione della velocità relativa sul piano di simmetria del velivolo.

Simbolo	Descrizione	Unità
V_T	Velocità vera del velivolo rispetto alla massa d'aria (<i>True Airspeed</i>)	[m/s]
α	Angolo d'attacco (<i>Angle of Attack</i>)	[rad]
θ	Angolo di assetto longitudinale rispetto all'orizzonte (<i>Pitch Angle</i>)	[rad]
q	Velocità angolare di beccheggio in assi corpo (<i>Pitch Rate</i>)	[rad/s]
h	Quota altimetrica del velivolo (<i>Altitude</i>)	[m]
a_z	Accelerazione specifica misurata lungo asse Z, usata come feedback primario	[m/s ²]
δ_{th}	Comando di manetta (<i>Throttle</i>)	[0 - 1]
δ_e	Deflessione del piano di coda orizzontale (<i>Elevator</i>)	[rad]
T	Spinta del turbofan (<i>Thrust</i>)	[N]
m	Massa del velivolo	[kg]
g	Accelerazione di gravità	[m/s ²]
$\bar{q}$	Pressione dinamica	[Pa]
S	Superficie alare	[m ²]
$\bar{c}$	Corda media aerodinamica	[m]
I_{yy}	Momento d'inerzia di beccheggio	[kg·m ²]
C_D, C_L, C_m	Coefficienti di Resistenza, Portanza e Momento	[-]
C_{m_q}	Smorzamento aerodinamico derivato dalla rotazione q	[-]
$\Delta C_{m_{cg}}$	Correzione del momento di beccheggio causata dal baricentro (<i>CG</i>)	[-]

TABELLA 1.1: Legenda e Significato Fisico delle Variabili del Moto Longitudinale

1.2 Vettori di stato e cinematica del vento

Vettore di Stato Globale ($\mathbf{X}$)

La dinamica a 6 gradi di libertà è descritta dal **vettore di stato globale** $\mathbf{X} \in \mathbb{R}^{12}$.

$$\mathbf{X} = \begin{bmatrix} \Phi \\ \omega_{b/e}^{frd} \\ \mathbf{V}_{cm/e}^{frd} \\ \mathbf{P}^E \end{bmatrix} = \begin{bmatrix} \phi \\ \theta \\ \psi \\ p \\ q \\ r \\ u \\ v \\ w \\ p_N \\ p_E \\ p_D \end{bmatrix} \quad (1.1)$$

Dalla componente verso il basso (p_D), si definisce la quota altimetrica h e il rateo di salita:

$$\dot{h} = -\dot{p}_D \quad (1.2)$$

Vettori Atmosferici e Cinematica del Vento

Si definisce il vettore velocità del vento misurato nel Target Plane:

$$\mathbf{V}_{w/e}^{tp} = \begin{bmatrix} W_N \\ W_E \\ W_D \end{bmatrix} \quad (1.3)$$

Per ricavare la velocità relativa dell'aria nel sistema FRD ($\mathbf{V}_r^{frd}$), si proietta

il vento tramite la matrice dei coseni direttori (*Direction Cosine Matrix*, DCM) $\mathbf{C}_{frd/tp}$, scritta con la notazione compatta:

$$\mathbf{C}_{frd/tp} = \begin{bmatrix} c\theta c\psi & c\theta s\psi & -s\theta \\ s\phi s\theta c\psi - c\phi s\psi & s\phi s\theta s\psi + c\phi c\psi & s\phi c\theta \\ c\phi s\theta c\psi + s\phi s\psi & c\phi s\theta s\psi - s\phi c\psi & c\phi c\theta \end{bmatrix} \quad (1.4)$$

La velocità relativa del velivolo è quindi:

$$\mathbf{V}_r^{frd} = \mathbf{V}_{cm/e}^{frd} - \mathbf{C}_{frd/tp} \mathbf{V}_{w/e}^{tp} \quad (1.5)$$

1.3 Modellazione di forze e momenti aerodinamici

1.3.1 Forze nel Body Frame (*bf*)

Le forze esterne nel sistema Body si suddividono in effetti aerodinamici (*A*) ed effetti propulsivi o di spinta (*T*, *Thrust*):

$$\mathbf{F}_A^{bf} = \begin{bmatrix} X_A \\ Y_A \\ Z_A \end{bmatrix}, \quad \mathbf{F}_T^{bf} = \begin{bmatrix} X_T \\ Y_T \\ Z_T \end{bmatrix} \quad (1.6)$$

dove X_A, Y_A, Z_A rappresentano l'effetto aerodinamico puro, e X_T, Y_T, Z_T la forza propulsiva.

1.3.2 Forze nel Wind Frame (*w*)

Nel sistema del vento (*w*), le forze aerodinamiche assumono la loro denominazione classica: Resistenza (*D*, *Drag*), Forza Laterale o Vento al Trasverso (*C*, *Cross Wind force*) e Portanza (*L*, *Lift*). Rispettando l'orientamento degli assi, il vettore è definito come:

$$\mathbf{F}_A^w = \begin{bmatrix} -D \\ -C \\ -L \end{bmatrix} \quad (1.7)$$

La relazione matematica che proietta gli effetti aerodinamici dal Body Frame al Wind Frame utilizza la matrice di trasformazione $\mathbf{C}_{w/bf}$ (funzione di incidenza α e derapata β):

$$\mathbf{F}_A^w = \begin{bmatrix} -D \\ -C \\ -L \end{bmatrix} = \mathbf{C}_{w/bf} \begin{bmatrix} X_A \\ Y_A \\ Z_A \end{bmatrix} \quad (1.8)$$

1.3.3 Momenti Aerodinamici

I momenti aerodinamici attorno al baricentro sono raggruppati in un vettore colonna contenente i ratei di rollio (l), beccheggio (m) e imbardata (n):

$$\mathbf{M} = \begin{bmatrix} l \\ m \\ n \end{bmatrix} \quad (1.9)$$

È di cruciale importanza notare che i valori delle componenti cambiano a seconda del sistema di riferimento in cui il vettore viene espresso:

- $\mathbf{M}_A^{bf} = [l^{bf}, m^{bf}, n^{bf}]^T$: Momenti nel Body Frame.
- $\mathbf{M}_A^s = [l^s, m^s, n^s]^T$: Momenti negli Assi di Stabilità.
- $\mathbf{M}_A^w = [l^w, m^w, n^w]^T$: Momenti negli Assi Vento.

All'interno del codice di simulazione, i momenti dimensionali vengono estratti scalando i **coefficienti di momento adimensionali** (C_l, C_m, C_n) ricavati dal database aerodinamico.

Capitolo 2

Derivazione Analitica delle Equazioni Del Moto

In questa sezione si eseguono i calcoli tensoriali e matriciali per derivare le equazioni differenziali del moto, omettendo per semplicità la presenza del vento durante la derivazione cinematica di base (assumendo quindi $\mathbf{V}_{cm/e}^{frd} \approx \mathbf{V}_r^{frd}$),[4]

2.1 Equazioni di cinematica

Per ottenere la velocità del velivolo rispetto al suolo, è necessario proiettare il vettore velocità dal Body Frame (frd) al Target Plane (tp). La matrice di trasformazione $\mathbf{C}_{tp/frd}$ è l'esatta trasposta della matrice dei coseni direttori definita nel Capitolo 1 ($\mathbf{C}_{tp/frd} = \mathbf{C}_{frd/tp}^T$).

Moltiplicando tale matrice per le velocità $\mathbf{V}_{cm/e}^{frd}$, si ottiene:

$$\begin{bmatrix} \dot{p}_N \\ \dot{p}_E \\ \dot{p}_D \end{bmatrix} = \begin{bmatrix} c\theta c\psi & s\phi s\theta c\psi - c\phi s\psi & c\phi s\theta c\psi + s\phi s\psi \\ c\theta s\psi & s\phi s\theta s\psi + c\phi c\psi & c\phi s\theta s\psi - s\phi c\psi \\ -s\theta & s\phi c\theta & c\phi c\theta \end{bmatrix} \begin{bmatrix} u \\ v \\ w \end{bmatrix} \quad (2.1)$$

2.2 Equazioni di dinamica

Applicando la Seconda Legge di Newton nel sistema di riferimento inerziale e derivandola nel sistema rotante (Body Frame) tramite il Teorema di Coriolis, l'equazione vettoriale risulta:

$$\mathbf{F}_{tot}^{bf} = m \left(\dot{\mathbf{V}}_{cm/e}^{frd} + \boldsymbol{\omega}_{b/e}^{frd} \times \mathbf{V}_{cm/e}^{frd} \right) \quad (2.2)$$

Il prodotto vettoriale genera i termini legati alle accelerazioni apparenti (Coriolis e centrifughe):

$$\boldsymbol{\omega}_{b/e}^{frd} \times \mathbf{V}_{cm/e}^{frd} = \begin{bmatrix} p \\ q \\ r \end{bmatrix} \times \begin{bmatrix} u \\ v \\ w \end{bmatrix} = \begin{bmatrix} qw - rv \\ ru - pw \\ pv - qu \end{bmatrix} \quad (2.3)$$

Il vettore delle forze totali $\mathbf{F}_{tot}^{bf}$ è la somma dei contributi aerodinamici ($\mathbf{F}_A^{bf}$), propulsivi ($\mathbf{F}_T^{bf}$) e gravitazionali ($\mathbf{F}_{grav}^{bf}$). La gravità, proiettata nel Body Frame attraverso la matrice $\mathbf{C}_{frd/tp}$, fornisce le componenti:

$$\mathbf{F}_{grav}^{bf} = \mathbf{C}_{frd/tp} \begin{bmatrix} 0 \\ 0 \\ mg \end{bmatrix} = \begin{bmatrix} -mgs\theta \\ mgc\theta s\phi \\ mgc\theta c\phi \end{bmatrix} \quad (2.4)$$

2.3 Momento angolare

L'equazione di conservazione del momento angolare $\mathbf{H}$ per un sistema rotante è:

$$\dot{\mathbf{M}}^{bf} = \dot{\mathbf{H}}^{frd} + (\boldsymbol{\omega}_{b/e}^{frd} \times \mathbf{H}^{frd}) \quad (2.5)$$

Il momento angolare si ottiene moltiplicando il tensore d'inerzia $\mathbf{I}$ (assumendo la simmetria sul piano XZ) per la velocità angolare:

$$\mathbf{H}^{frd} = \mathbf{I} \boldsymbol{\omega}_{b/e}^{frd} = \begin{bmatrix} I_{xx} & 0 & -I_{xz} \\ 0 & I_{yy} & 0 \\ -I_{xz} & 0 & I_{zz} \end{bmatrix} \begin{bmatrix} p \\ q \\ r \end{bmatrix} = \begin{bmatrix} I_{xx}p - I_{xz}r \\ I_{yy}q \\ -I_{xz}p + I_{zz}r \end{bmatrix} \quad (2.6)$$

Derivando $\mathbf{H}^{frd}$ rispetto al tempo si ottiene $\dot{\mathbf{H}}^{frd} = [I_{xx}\dot{p} - I_{xz}\dot{r}, I_{yy}\dot{q}, -I_{xz}\dot{p} + I_{zz}\dot{r}]^T$. Sviluppando il prodotto vettoriale per ottenere i termini giroscopici:

$$\boldsymbol{\omega}_{b/e}^{frd} \times \mathbf{H}^{frd} = \begin{bmatrix} p \\ q \\ r \end{bmatrix} \times \begin{bmatrix} H_x \\ H_y \\ H_z \end{bmatrix} = \begin{bmatrix} qH_z - rH_y \\ rH_x - pH_z \\ pH_y - qH_x \end{bmatrix} \quad (2.7)$$

Capitolo 3

Modello delle Forze e Momenti Aerodinamici

Le forze $\mathbf{F}$ e i momenti $\mathbf{M}$ presenti nelle equazioni dinamiche del Capitolo 3 sono la somma dei contributi aerodinamici (pedice A) e propulsivi (pedice T , *Thrust*). Essendo la gravità già esplicitata nelle equazioni traslazionali, si ha:

$$\mathbf{F} = \mathbf{F}_A + \mathbf{F}_T \quad (3.1)$$

$$\mathbf{M} = \mathbf{M}_A + \mathbf{M}_T \quad (3.2)$$

Questo capitolo definisce le componenti aerodinamiche secondo la convenzione standard di Stevens e Lewis

3.1 Pressione Dinamica e Geometria di Riferimento

Le forze aerodinamiche scalano con l'energia cinetica del flusso d'aria. Si definisce la pressione dinamica $\bar{q}$ attraverso la seguente relazione, la cui scomposizione vettoriale è illustrata concettualmente nella [Figura 3.1](#):

$$\bar{q} = \frac{1}{2} \rho V_T^2 \quad (3.3)$$

dove ρ è la densità dell'aria e V_T la *True Airspeed*. I parametri geometrici di riferimento del velivolo necessari per il calcolo dei coefficienti adimensionali sono:

- S : Superficie alare di riferimento (*Wing reference area*).

- $\bar{c}$: Corda aerodinamica media (*Mean aerodynamic chord*), usata per adimensionalizzare il momento di beccheggio.
- b : Apertura alare (*Wingspan*), usata per adimensionalizzare i momenti di rollio e imbardata (come visibile nella [Figura 3.2](#)).

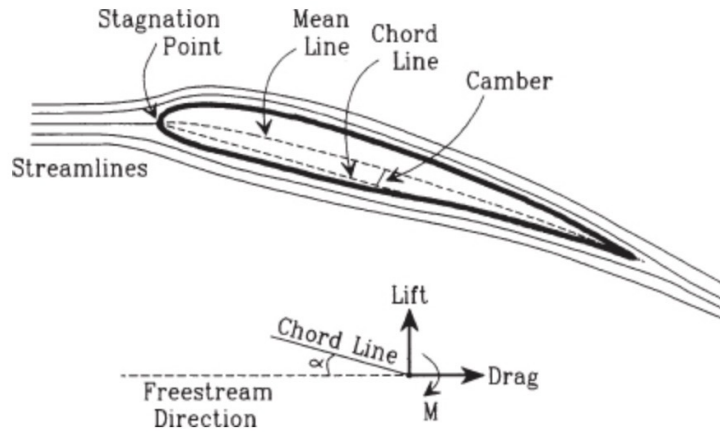


FIGURA 3.1: Rappresentazione delle forze aerodinamiche incidenti sul profilo alare e scomposizione del vettore velocità.

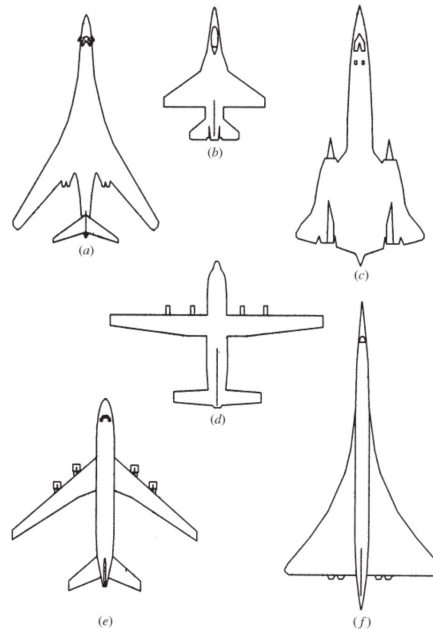


FIGURA 3.2: Rappresentazione geometrica dell'apertura alare (b) e dei parametri di riferimento del velivolo.

3.2 Modelli di forze aerodinamiche

Le forze aerodinamiche nel Body Frame sono espresse tramite i coefficienti adimensionali C_X, C_Y, C_Z :

$$F_{xA} = \bar{q} S C_X \quad (3.4)$$

$$F_{yA} = \bar{q} S C_Y \quad (3.5)$$

$$F_{zA} = \bar{q} S C_Z \quad (3.6)$$

Tradizionalmente, in galleria del vento, i dati vengono estratti nel sistema di riferimento del vento (Wind Axes), dove le forze sono Portanza (*Lift*, L), Resistenza (*Drag*, D) e Forza Laterale (*Side Force*, Y). Trascurando l'angolo di derapata ($\beta \approx 0$) per la dinamica longitudinale, la rotazione dal sistema Wind al sistema Body in funzione dell'angolo d'attacco α è:

$$C_X = -C_D \cos \alpha + C_L \sin \alpha \quad (3.7)$$

$$C_Z = -C_D \sin \alpha - C_L \cos \alpha \quad (3.8)$$

Nota bene: Si presti attenzione al segno di C_Z . Essendo l'asse Z_B puntato verso il basso (convenzione FRD), una portanza positiva (diretta verso l'alto) genera un F_{zA} negativo.

3.3 Adimensionalizzazione dei momenti

I momenti aerodinamici attorno al baricentro dipendono dai coefficienti di rollio (C_l), beccheggio (C_m) e imbardata (C_n). Per adimensionalizzare i ratei di rotazione, si utilizzano i fattori $\frac{b}{2V_T}$ per la dinamica latero-direzionale e $\frac{\bar{c}}{2V_T}$ per la longitudinale:

$$L_A = \bar{q} S b C_l \quad (3.9)$$

$$M_A = \bar{q} S \bar{c} C_m \quad (3.10)$$

$$N_A = \bar{q} S b C_n \quad (3.11)$$

3.4 Analisi delle derivate di stabilità tramite serie di Taylor

Per simulazioni o sintesi di controllori lineari, i coefficienti aerodinamici vengono espansi in serie di Taylor rispetto alle variabili di stato e di controllo

3.4.1 Dinamica longitudinale

I coefficienti longitudinali dipendono principalmente da α , dal rateo di beccheggio q e dal comando dell'equilibratore δ_e :

3.4.2 Modello Aerodinamico e Derivate di Stabilità

Nelle equazioni del moto precedentemente ricavate, le forze e i momenti aerodinamici sono racchiusi all'interno dei coefficienti globali. Per esplicitare la dipendenza di tali carichi dallo stato del velivolo e dall'ingresso di controllo, è necessario espandere i coefficienti tramite un approccio alle piccole perturbazioni (Serie di Taylor arrestata al primo ordine). Come documentato nel modello NASA dell'F-16 (Stevens e Lewis), i dati aerodinamici del velivolo sono tipicamente strutturati fornendo i coefficienti delle forze direttamente proiettati in *Assi Corpo* (C_X, C_Z), piuttosto che in assi vento (C_D, C_L). L'espansione lineare dei coefficienti longitudinali risulta pertanto:

$$C_X = C_{X_0}(\alpha) + C_{X_q} \left(\frac{q\bar{c}}{2V_T} \right) + C_{X_{\delta_e}} \delta_e \quad (3.12)$$

$$C_Z = C_{Z_0}(\alpha) + C_{Z_q} \left(\frac{q\bar{c}}{2V_T} \right) + C_{Z_{\delta_e}} \delta_e \quad (3.13)$$

$$C_m = C_{m_0} + C_{m_\alpha} \alpha + C_{m_q} \left(\frac{q\bar{c}}{2V_T} \right) + C_{m_{\delta_e}} \delta_e \quad (3.14)$$

Questa formulazione scompone matematicamente l'aerodinamica in tre contributi fisici distinti, fondamentali per la successiva costruzione delle matrici di stato $\mathbf{A}_{long}$ e di ingresso $\mathbf{B}_{long}$:

- **Aerodinamica Statica Base** ($C_{X_0}, C_{Z_0}, C_{m_0} + C_{m_\alpha} \alpha$): Rappresentano i carichi generati dal velivolo in volo stazionario. Il termine C_{m_α} è la derivata

di stabilità statica longitudinale: per un velivolo convenzionale stabile deve essere negativo (un aumento di incidenza genera un momento picchiante di richiamo), ma nel caso dell'F-16, intrinsecamente instabile nel volo subsonico, tale termine risulta positivo, rendendo obbligatoria la presenza di un sistema SAS (Stability Augmentation System).

- **Derivate di Smorzamento Dinamico** ($C_{X_q}, C_{Z_q}, C_{m_q}$): Moltiplicate per il rateo di beccheggio adimensionalizzato ($q\bar{c}/2V_T$), modellano gli effetti istantanei dovuti alla rotazione del velivolo. Il termine dominante è C_{m_q} (*Pitch Damping*), che quantifica l'opposizione aerodinamica del piano di coda al moto di beccheggio, agendo come uno smorzatore viscoso naturale.
- **Derivate di Controllo** ($C_{X_{\delta_e}}, C_{Z_{\delta_e}}, C_{m_{\delta_e}}$): Isolano l'effetto puro dell'elevatore. Il termine $C_{m_{\delta_e}}$ costituisce l'efficacia di comando primaria per governare l'accelerazione angolare $\dot{q}$, mentre $C_{Z_{\delta_e}}$ modella la variazione istantanea di forza normale sull'asse Z (responsabile del salto nell'accelerazione a_z e della dinamica a non-minimo fase discussa in precedenza).

Nota di trasformazione: Qualora le equazioni di stato impieghino le forze in assi vento (come nel caso delle derivate $\dot{V}_T$ e $\dot{\alpha}$), i coefficienti del modello tabellare in assi corpo vengono ruotati tramite le usuali trasformazioni trigonometriche:

$$C_D = -C_X \cos \alpha + C_Z \sin \alpha$$

$$C_L = -C_Z \cos \alpha - C_X \sin \alpha$$

Il termine C_{m_α} rappresenta la rigidezza in beccheggio e definisce la stabilità statica longitudinale del velivolo, mentre C_{m_q} (damping di beccheggio) fornisce la stabilità dinamica. Queste trasformazioni verranno utilizzate all'interno del capitolo 6 per ottenere le equazioni puntate del controllo longitudinale in assi del corpo, si spiega meglio in questo paragrafo [4.2](#).

3.4.3 Dinamica Latero-Direzionale e Accoppiamento Aerodinamico

A differenza del moto longitudinale, che gode di un disaccoppiamento naturale dalle variabili laterali in condizioni di volo simmetrico, la dinamica latero-direzionale è

intrinsecamente accoppiata. Un moto di rollio induce effetti sull'asse di imbardata e viceversa. Per modellare matematicamente le forze laterali (Y) e i momenti di rollio (L) e imbardata (N), i coefficienti adimensionali globali C_Y, C_l, C_n vengono espansi in Serie di Taylor rispetto all'angolo di derapata (o slittamento) β , ai ratei angolari p, r e agli ingressi di controllo asimmetrici (alettoni δ_a e timone δ_r).

È fondamentale notare che, per i coefficienti latero-direzionali dipendenti dai ratei angolari, il parametro geometrico di adimensionalizzazione non è più la corda media aerodinamica $\bar{c}$ (utilizzata nel piano longitudinale), bensì l'apertura alare b . Le equazioni linearizzate assumono la seguente forma:

$$C_Y = C_{Y_\beta}\beta + C_{Y_p}\left(\frac{pb}{2V_T}\right) + C_{Y_r}\left(\frac{rb}{2V_T}\right) + C_{Y_{\delta_a}}\delta_a + C_{Y_{\delta_r}}\delta_r \quad (3.15)$$

$$C_l = C_{l_\beta}\beta + C_{l_p}\left(\frac{pb}{2V_T}\right) + C_{l_r}\left(\frac{rb}{2V_T}\right) + C_{l_{\delta_a}}\delta_a + C_{l_{\delta_r}}\delta_r \quad (3.16)$$

$$C_n = C_{n_\beta}\beta + C_{n_p}\left(\frac{pb}{2V_T}\right) + C_{n_r}\left(\frac{rb}{2V_T}\right) + C_{n_{\delta_a}}\delta_a + C_{n_{\delta_r}}\delta_r \quad (3.17)$$

Questo sistema evidenzia la natura intrinsecamente *Multi-Input Multi-Output* (MIMO) dell'aerodinamica latero-direzionale. I contributi possono essere fisicamente scomposti in tre categorie principali:

- **Derivate Statiche di Derapata (β):** Rappresentano la reazione del velivolo a un vento laterale.
 - C_{n_β} (*Stabilità Direzionale o Effetto Banderuola*): Garantita dalla deriva verticale, deve essere rigidamente positivo per un volo stabile. Un $\beta > 0$ (vento da destra) deve generare un momento imbardante positivo ($n > 0$) per riportare il muso nel vento.
 - C_{l_β} (*Effetto Diedro*): Modella l'accoppiamento rollio-derapata. In aerei stabili è tipicamente negativo: uno scivolamento a destra ($\beta > 0$) genera una maggiore portanza sulla semiala destra, creando un momento di rollio a sinistra ($l < 0$) che tende a raddrizzare il velivolo. L'equilibrio tra C_{l_β} e C_{n_β} determina le caratteristiche dei modi dinamici di *Dutch Roll* e spirale divergente.

- **Derivate di Smorzamento e Accoppiamento (p, r) :** Modellano l'aerodinamica instazionaria durante le rotazioni.
 - *Smorzamenti diretti* (C_{l_p}, C_{n_r}) : Sono lo smorzamento di rollio (dovuto all'imponente resistenza delle semiali a ruotare) e lo smorzamento di imbardata. Sono termini fortemente stabilizzanti (negativi).
 - *Accoppiamenti incrociati* (C_{l_r}, C_{n_p}) : Modellano l'interazione tra gli assi. Ad esempio, C_{l_r} (rollio indotto dall'imbardata) descrive come un'imbardata verso destra aumenti la velocità relativa della semiala sinistra, generando più portanza asimmetrica e causando un momento di rollio indotto.
- **Derivate di Controllo (δ_a, δ_r) :**
 - $C_{l_{\delta_a}}$ e $C_{n_{\delta_r}}$ rappresentano rispettivamente l'autorità primaria degli alettoni sul rollio e del timone sull'imbardata.
 - $C_{n_{\delta_a}}$ modella il fenomeno dell'*Imbardata Inversa* (Adverse Yaw): una deflessione degli alettoni per rollare genera un differenziale di resistenza indotta (l'alettone abbassato crea più resistenza) che trascina il muso dell'aereo nella direzione opposta alla virata voluta. Questo termine parassita impone la progettazione di un interconnettore alettoni-timone (ARI - Aileron Rudder Interconnect) nelle leggi di controllo fly-by-wire.

Capitolo 4

Equazioni non lineari del moto e integrazione numerica

In questo capitolo vengono riportate le 12 equazioni differenziali ordinarie (ODE) che costituiscono il modello completo del velivolo, scritte nella forma esplicita pronta per l'integrazione numerica che avverrà all'interno del capitolo [4.5](#).

4.1 Equazioni di posizione e assetto

Sviluppando il prodotto matriciale di trasformazione $\mathbf{C}_{tp/frd}$, le derivate della posizione nel Target Plane risultano:

$$\dot{p}_N = u(c\theta c\psi) + v(s\phi s\theta c\psi - c\phi s\psi) + w(c\phi s\theta c\psi + s\phi s\psi) \quad (4.1)$$

$$\dot{p}_E = u(c\theta s\psi) + v(s\phi s\theta s\psi + c\phi c\psi) + w(c\phi s\theta s\psi - s\phi c\psi) \quad (4.2)$$

$$\dot{p}_D = u(-s\theta) + v(s\phi c\theta) + w(c\phi c\theta) \quad (4.3)$$

Significato Fisico: La terza equazione $\dot{p}_D$ governa il rateo di discesa. Di conseguenza, il rateo di salita effettivo del velivolo è $\dot{h} = u(s\theta) - v(s\phi c\theta) - w(c\phi c\theta)$.

Equazioni Cinematiche dell'Assetto

Per aggiornare l'orientamento spaziale del velivolo Φ , le velocità angolari di rollio, beccheggio e imbardata misurate in assi FRD vengono proiettate sulle derivate degli angoli di Eulero:

$$\dot{\phi} = p + q(s\phi \tan \theta) + r(c\phi \tan \theta) \quad (4.4)$$

$$\dot{\theta} = q(c\phi) - r(s\phi) \quad (4.5)$$

$$\dot{\psi} = q\left(\frac{s\phi}{c\theta}\right) + r\left(\frac{c\phi}{c\theta}\right) \quad (4.6)$$

Gimbal Lock: L'evidente divisione per $c\theta$ nell'equazione di $\dot{\psi}$ dimostra la singolarità matematica a $\theta = \pm 90^\circ$ (volo verticale), risolvibile nell'implementazione software unicamente tramite la cinematica a quaternioni.

4.2 Dinamica non lineare delle forze e dei momenti

Sostituendo $\mathbf{F}_{tot}^{bf} = \mathbf{F}_A^{bf} + \mathbf{F}_T^{bf} + \mathbf{F}_{grav}^{bf}$ e isolando le derivate del vettore velocità $\mathbf{V}_{cm/e}^{frd}$, si ottengono le accelerazioni lineari:

$$\dot{U} = \frac{X_A + X_T}{m} - g s \theta - (q w - r v) \quad (4.7)$$

$$\dot{V} = \frac{Y_A + Y_T}{m} + g c \theta s \phi - (r u - p w) \quad (4.8)$$

$$\dot{W} = \frac{Z_A + Z_T}{m} + g c \theta c \phi - (p v - q u) \quad (4.9)$$

Significato Fisico: Le equazioni dimostrano come le forze esterne combinate (aerodinamica X_A, Y_A, Z_A e propulsione X_T, Y_T, Z_T) e il vettore gravità continuamente ripartito dagli angoli di assetto, si sommino all'effetto intrinseco di accoppiamento rotazionale (es. $q w - r v$).

Equazioni dei Momenti

Dal bilancio vettoriale dei momenti $\mathbf{M}^{bf} = [l^{bf}, m^{bf}, n^{bf}]^T$, si estraggono le accelerazioni angolari:

$$l^{bf} = I_{xx}\dot{p} - I_{xz}\dot{r} + (I_{zz} - I_{yy})qr - I_{xz}pq \quad (4.10)$$

$$m^{bf} = I_{yy}\dot{q} + (I_{xx} - I_{zz})pr + I_{xz}(p^2 - r^2) \quad (4.11)$$

$$n^{bf} = I_{zz}\dot{r} - I_{xz}\dot{p} + (I_{yy} - I_{xx})pq + I_{xz}qr \quad (4.12)$$

Significato Fisico: Il tensore asimmetrico I_{xz} accoppia matematicamente il rollio p con l'imbardata r , mentre i termini non lineari come qr e pr modellano le potenti coppie giroscopiche intrinseche della massa rotante del velivolo.

Equazioni Non-Lineari del Moto Longitudinale

Applicando la proprietà di disaccoppiamento mostrate all'interno del libro **Aircraft Control and Simulation** nello stato di volo livellato, le ODE originarie collassano in un sistema di 5 equazioni differenziali che rappresentano la dinamica del moto longitudinale dell'aereo mostrando come gli ingressi interagiscono con il sistema. In questa formulazione si utilizzano i coefficienti aerodinamici espressi nativamente in assi corpo (C_X e C_Z), opportunamente proiettati in assi vento tramite l'angolo d'attacco α . Inoltre, si evidenzia esplicitamente la dipendenza dei coefficienti dall'ingresso di controllo dell'elevatore (δ_e) e si include il termine di traslazione del momento $\Delta C_{m_{cg}}$:

$$\dot{V}_T = \frac{1}{m} \left[T \cos \alpha + \bar{q} S (C_X(\alpha, \delta_e) \cos \alpha + C_Z(\alpha, \delta_e) \sin \alpha) \right] - g \sin(\theta - \alpha) \quad (4.13)$$

$$\dot{\alpha} = q - \frac{1}{m V_T} \left[T \sin \alpha + \bar{q} S (C_X(\alpha, \delta_e) \sin \alpha - C_Z(\alpha, \delta_e) \cos \alpha) \right] + \frac{g}{V_T} \cos(\theta - \alpha) \quad (4.14)$$

$$\dot{\theta} = q \quad (4.15)$$

$$\dot{q} = \frac{\bar{q} S \bar{c}}{I_{yy}} \left[C_m(\alpha, \delta_e) + \frac{\bar{c}}{2 V_T} C_{m_q} q + \Delta C_{m_{cg}} \right] \quad (4.16)$$

$$\dot{h} = V_T \sin(\theta - \alpha) \quad (4.17)$$

Trasformazione Integrata delle Forze Aerodinamiche:

Si presti particolare attenzione alla formulazione utilizzata nelle equazioni 4.13 e 4.14. Dal punto di vista della meccanica del volo della galleria del vento, i dati estratti dalle gallerie del vento (database NASA dell'F-16 utilizzato nei codici matlab) misurano le reazioni vincolari torsionometriche bilanciate in modo solidale alla fusoliera, restituendo perciò coefficienti aerodinamici in *Assi Corpo* (C_X, C_Z). Tuttavia, la sintesi delle leggi di controllo necessita di governare variabili cinematiche espresse in *Assi Vento* (V_T, α).

Per conciliare questi due sistemi di riferimento senza avere errori di interpolazione derivanti da conversioni a priori del database, le equazioni differenziali fungono da "trasformatore di coordinate" in tempo reale. I blocchi trigonometrici racchiusi nelle parentesi tonde operano una proiezione vettoriale :

- Il termine $(C_X \cos \alpha + C_Z \sin \alpha)$ isola la componente delle forze aerodinamiche tangenziale al vettore velocità, e corrisponde rigorosamente all'opposto del

coefficiente di Resistenza ($-C_D$).

- Il termine $(C_X \sin \alpha - C_Z \cos \alpha)$ isola la componente perpendicolare al vettore velocità (converso positivo verso il basso), corrispondendo rigorosamente all'opposto del coefficiente di Portanza ($-C_L$).

Questa modalità di procedere garantisce che durante l'integrazione e la linearizzazione numerica che viene eseguita negli script matlab, il calcolatore legga i dati corretti e li proietta istantaneamente nella direzione del moto, fornendo le derivate di stato esatte da poter utilizzare. Questo set di ODE regola la dinamica longitudinale e su di esso verrà calcolato il punto di Trim in volo di crociera e tramite linearizzazione si estrarranno le matrici del sistema in spazio di stato ($\mathbf{A}_{long}$, $\mathbf{B}_{long}$, $\mathbf{C}_{long}$, $\mathbf{D}_{long}$). È fondamentale notare l'utilizzo di $\mathbf{B}_{ctrl}$ in luogo della matrice di ingresso totale $\mathbf{B}_{long}$: per la sintesi dei controllori, infatti, è strettamente necessario estrarre e separare le sole colonne relative agli ingressi fisicamente comandabili dal computer di volo (attuatori: manetta, elevatore e flap), isolandole dagli ingressi di disturbo non controllabili (come la dinamica delle raffiche di vento, che andranno a formare la matrice separata $\mathbf{B}_{wind}$). La spiegazione completa e l'implementazione analitica di questa scomposizione matriciale è rimandata al **Capitolo 10** (si veda la Sezione 10). *Nota metodologica:* L'implementazione software di questa procedura, che prevede prima la linearizzazione numerica del modello completo a 6-DOF e il successivo filtraggio algoritmico per isolare le sole componenti longitudinali utili alla sintesi dei controllori, verrà trattata in dettaglio nel **Capitolo 7** (si veda la Sezione 7.4).

Equazione di Uscita: Accelerazione Normale (a_z)

Le variabili di stato V_T, α, θ, q e h rappresentano gli stati del sistema governati da equazioni differenziali, l'accelerazione misurata dal sensore inerziale lungo l'asse Z del corpo (a_z) è una variabile di uscita algebrica e di seguito si mostra il modo in cui si ottiene.

Come ampiamente discusso in *Aircraft Control and Simulation* (Stevens e Lewis)[3], un accelerometro di bordo non misura l'accelerazione cinematica totale, bensì la

forza specifica. Essa rappresenta la risultante delle sole forze esterne non gravitazionali (aerodinamiche e propulsive) agenti sulla massa sismica del sensore, divisa per la massa del velivolo. Matematicamente, lungo l'asse Z del corpo:

$$a_z = \frac{F_{z,aero} + F_{z,prop}}{m} \quad (4.18)$$

Assumendo che il vettore spinta sia allineato con l'asse longitudinale del velivolo, il segnale a_z deriva interamente dalle forze aerodinamiche. Utilizzando direttamente i coefficienti aerodinamici espressi in assi corpo (C_X e C_Z), che costituiscono l'output nativo dei modelli di volo a sei gradi di libertà, la forza aerodinamica normale si esprime banalmente come:

$$F_{z,aero} = \bar{q} S C_Z(\alpha, \delta_e)$$

A differenza della formulazione in assi vento (che richiederebbe la complessa scomposizione trigonometrica $F_{z,aero} = -L \cos \alpha - D \sin \alpha$), l'uso degli assi corpo evita ulteriori trasformazioni. Considerando come ingresso di controllo aerodinamico primario esclusivamente l'elevatore (δ_e), l'equazione di uscita definitiva risulta:

$$a_z = \frac{\bar{q} S}{m} C_Z(\alpha, \delta_e) \quad (4.19)$$

Intervento dell'Elevatore e Matrice D_{long} : Importanza di a_z nella Teoria dei Controlli: L'equazione 4.19 ha un'importanza per la sintesi dei sistemi di controllo di volo, per due ragioni distinte: una di natura matematica e una di natura operativa.

1. **Legame Diretto Ingresso-Uscita (Matrice D_{long}):** Dal punto di vista della modellistica in spazio di stato, poiché il coefficiente di portanza C_L dipende istantaneamente dalla deflessione dell'elevatore δ_e , una variazione del comando produce un salto immediato nel segnale misurato a_z , prima ancora che l'assetto del velivolo inizi a variare. Questa relazione crea una **matrice di trasmissione diretta D_{long} non nulla**. Fisicamente, questo feed-through modella la dinamica, dove un comando per cabrare genera una temporanea perdita di portanza sulla coda, spingendo inizialmente il baricentro del velivolo verso il basso prima di innescare la salita.

2. **Gestione del Fattore di Carico:** Nel moto longitudinale reale, l'accelerazione a_z è strettamente legata al fattore di carico verticale n_z percepito dal pilota e dalla struttura. L'utilizzo di a_z come variabile di feedback primaria nell'anello di controllo serve a:

- **Protezione Strutturale e Fisiologica:** Il *Flight Control System* (FCS) dell'F-16 utilizza la lettura continua di a_z per implementare algoritmi di protezione dell'involucro di volo (*G-Limiting*). Il controllore "taglia" automaticamente l'autorità dell'elevatore se la manovra rischia di superare i limiti strutturali della cellula (tipicamente $+9g/-3g$), prevenendo cedimenti catastrofici o la perdita di coscienza indotta dall'accelerazione nel pilota (G-LOC).
- **Logica di Comando del Pilota:** Nei moderni caccia *fly-by-wire*, la cloche (side-stick) non comanda fisicamente la deflessione dell'elevatore. Nel volo a medie e alte velocità, la trazione del pilota sulla barra si traduce in una richiesta esplicita di accelerazione (sistema *G-command*). Il controllore regola l'elevatore affinché l' a_z misurata inseguia l' a_z comandata dal pilota. Controllare l'accelerazione sull'asse Z significa quindi governare direttamente la curvatura della traiettoria e i "G" percepiti nell'abitacolo, garantendo risposte tattiche prevedibili e letali.

4.3 Meccanismo di Controllo Longitudinale

Le equazioni 4.13-4.17 e l'equazione 4.19 dimostrano matematicamente come l'ingresso primario dell'elevatore (δ_e) interagisca con l'intera dinamica del velivolo. Nel Capitolo 6 sono mostrati proprio i coefficienti dinamici utilizzati sia per moto longitudinale e latero-direzionale dell'aereo, mostrando come l'ingresso dell'elevatore condiziona i coefficienti aerodinamici al Capitolo (3.4.1):

1. **Azione Primaria su $\dot{q}$ e a_z :** La deflessione δ_e modifica istantaneamente il coefficiente di momento C_m (innescando l'accelerazione angolare $\dot{q}$) e il coefficiente di portanza C_L (misurato istantaneamente dall'accelerometro come variazione di a_z).

2. **Integrazione su q e $\dot{\theta}$ (Variazione di Assetto):** L'accelerazione $\dot{q}$ si integra nella velocità di beccheggio q , la quale forza la rotazione fisica del velivolo variando l'angolo di assetto $\dot{\theta}$ (Equazione 4.15).
3. **Azione Diretta su $\dot{\alpha}$ (Variazione di Traiettoria):** La variazione di portanza crea l'effetto di *non-minimo fase*: un comando per alzare il muso riduce momentaneamente la portanza totale sulla coda prima che l'angolo α complessivo aumenti.
4. **Azione Secondaria su $\dot{V}_T$ (Resistenza Indotta):** Infine, la deflessione meccanica della superficie aggiunge resistenza aerodinamica, alterando C_D e causando una lieve decelerazione iniziale del velivolo.

4.4 Rappresentazione dell'assetto mediante quaternioni

Come evidenziato dalle equazioni cinematiche dell'assetto, l'uso degli angoli di Eulero (Φ) per l'integrazione numerica introduce la singolarità del *Gimbal Lock* a $\theta = \pm 90^\circ$. Inoltre, il motore di rendering (simulatore 3D su flightGear 6.2.2) all'interno del simulatore affronta un problema correlato ma distinto nell'applicare la rotazione (ϕ, θ, ψ) al modello 3D del velivolo. Se si utilizzassero direttamente le matrici di rotazione di Eulero, si riscontrerebbe il fenomeno del Gimbal Lock un problema che potrebbe causare gravi **disturbi visivi** durante la simulazione: il modello 3D "scatterebbe" improvvisamente tra orientamenti diversi, poiché variazioni infinitesime di ϕ e ψ produrrebbero variazioni numeriche nella matrice di trasformazione risultante. Per risolvere matematicamente e visivamente questo problema, la formulazione moderna della meccanica del volo [5], sostituisce l'integrazione degli angoli di Eulero con l'utilizzo dei **Quaternioni** costituiscono una rappresentazione matematica che evita le singolarità degli angoli di Eulero, riducendo il costo computazionale e garantendo una resa grafica priva di artefatti visivi all'interno del simulatore.

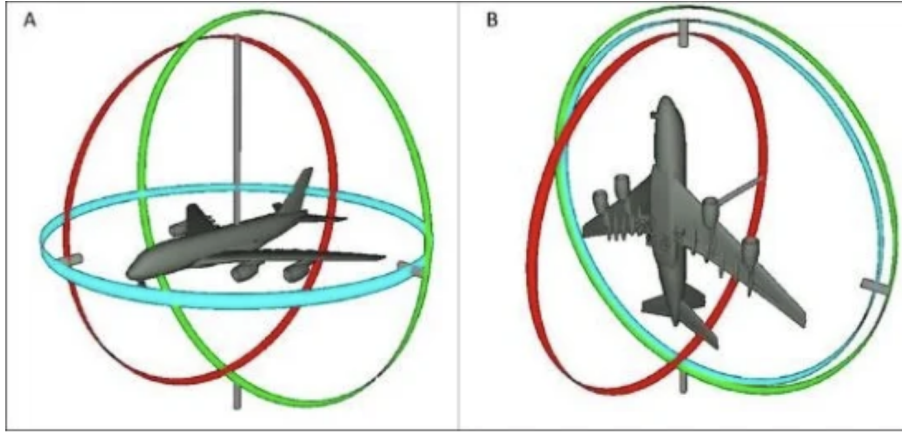


FIGURA 4.1: Rappresentazione visiva del fenomeno del Gimbal Lock: allineamento fisico degli assi di rotazione quando l'angolo di beccheggio raggiunge i 90° , con conseguente perdita di un grado di libertà.

La trattazione teorica di questa sezione si basa sui concetti esposti nel corso online del MIT.[6]

Definizione matematica del quaternione: Un quaternione è un numero iper-complesso a quattro componenti, ideato per rappresentare una rotazione nello spazio tridimensionale. Viene definito come:

$$\mathbf{q} = q_0 + q_1\mathbf{i} + q_2\mathbf{j} + q_3\mathbf{k} \quad (4.20)$$

dove q_0 (spesso indicato con w) rappresenta la parte scalare e (q_1, q_2, q_3) (corrispondenti a x, y, z) la parte vettoriale. Le unità immaginarie obbediscono alle identità fondamentali: $\mathbf{i}^2 = \mathbf{j}^2 = \mathbf{k}^2 = \mathbf{ijk} = -1$.

In notazione vettoriale, scriveremo $\mathbf{q} = [q_0, q_1, q_2, q_3]^T$. Per il Teorema di Rotazione di Eulero, qualsiasi orientamento può essere raggiunto tramite una singola rotazione di un angolo θ attorno a un asse arbitrario descritto dal versore unitario $\hat{u} = [u_x, u_y, u_z]^T$. Il quaternione esprime questa rotazione come:

$$\mathbf{q} = \begin{bmatrix} \cos(\theta/2) \\ u_x \sin(\theta/2) \\ u_y \sin(\theta/2) \\ u_z \sin(\theta/2) \end{bmatrix} \quad (4.21)$$

Per rappresentare una rotazione fisica pura, il quaternione deve essere unitario, imponendo il vincolo di norma geometrica:

$$|\mathbf{q}| = \sqrt{q_0^2 + q_1^2 + q_2^2 + q_3^2} = 1 \quad (4.22)$$

Topologia Continua e Assenza di Singularità

La motivazione matematica per cui i quaternioni non soffrono del *Gimbal Lock* risiede nella topologia dello spazio che generano. La rappresentazione a 4 parametri vincolata dalla condizione $|\mathbf{q}| = 1$ definisce una varietà matematica (la 3-sfera S^3) che è **liscia e priva di singularità**. In questo spazio non esistono punti degeneri. La mappa differenziale dai quaternioni unitari al gruppo delle rotazioni tridimensionali $SO(3)$ è un rivestimento doppio regolare ovunque. Ciò significa che ogni orientamento fisico corrisponde esattamente a due quaternioni antipodali ($\mathbf{q}$ e $-\mathbf{q}$, che applicano la medesima rotazione), ma il passaggio da un assetto all'altro avviene sempre percorrendo un arco continuo sulla superficie di S^3 , garantendo l'assoluta fluidità visiva nel rendering e la stabilità nell'integrazione differenziale.

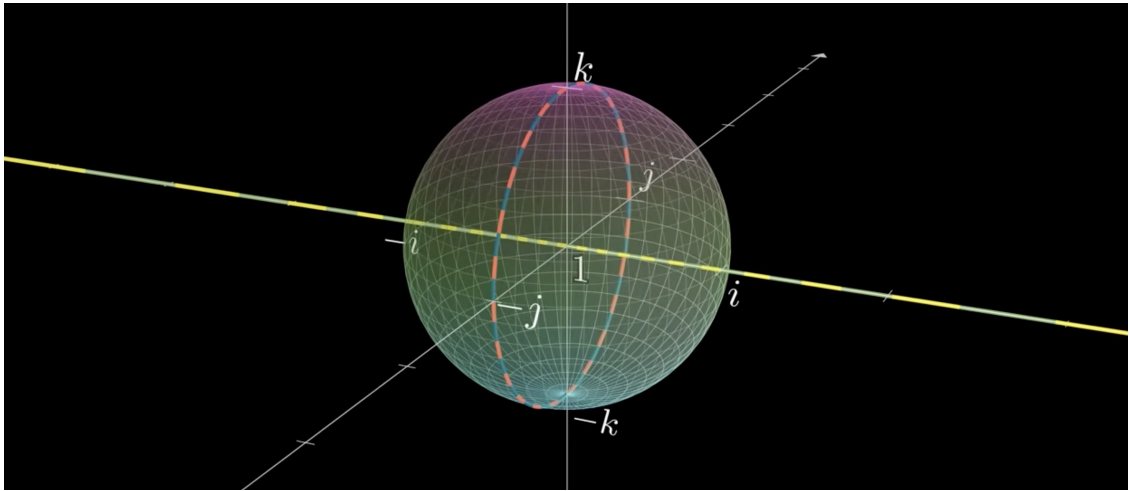


FIGURA 4.2: Rappresentazione topologica della sfera in S^3

Composizione delle Rotazioni

Un ulteriore vantaggio computazionale dei quaternioni rispetto alle matrici dei coseni direttori risiede nella composizione delle rotazioni, che avviene tramite il **Prodotto di Hamilton** ($\otimes$). Tale operatore, analogamente alle rotazioni fisiche, gode della

proprietà associativa ma non di quella commutativa. Dati due quaternioni $\mathbf{p}$ e $\mathbf{q}$, la loro composizione è data da:

$$\mathbf{p} \otimes \mathbf{q} = \begin{bmatrix} p_0q_0 - p_1q_1 - p_2q_2 - p_3q_3 \\ p_0q_1 + p_1q_0 + p_2q_3 - p_3q_2 \\ p_0q_2 - p_1q_3 + p_2q_0 + p_3q_1 \\ p_0q_3 + p_1q_2 - p_2q_1 + p_3q_0 \end{bmatrix} \quad (4.23)$$

All'interno dell'architettura di simulazione, le velocità angolari misurate nel Body Frame ($\omega_{b/e}^{frd}$) vengono utilizzate per propagare nel tempo il quaternionе tramite una semplice equazione differenziale lineare, ricavando gli angoli di Eulero (ϕ, θ, ψ) a posteriori, esclusivamente per l'interfaccia utente o per le leggi di controllo lineari, isolando così il nucleo matematico del simulatore dal rischio di discontinuità numerica.

4.5 Analisi critica dei metodi di Eulero

I metodi numerici più semplici appartengono alla famiglia di Eulero. Tuttavia, nell'ambito delle simulazioni aerospaziali considerate in questa tesi, tali metodi sono stati esclusi a causa delle loro limitazioni in termini di stabilità e conservazione dell'energia del sistema dinamico, le quali generano un'incertezza tale da rendere inattendibili i risultati ottenuti. **Eulero in Avanti e Divergenza**

Il metodo di Eulero Esplicito calcola lo stato futuro approssimando la curva della soluzione con la sua retta tangente calcolata all'istante iniziale t_n :

$$X_{n+1} = X_n + h \cdot f(X_n, U_n, t_n) \quad (4.24)$$

Dal punto di vista geometrico, questo approccio ignora la curvatura della traiettoria di stato all'interno dell'intervallo h . Fisicamente, l'errore di troncamento locale, proporzionale a $\mathcal{O}(h^2)$, si traduce in un drammatico effetto di non conservazione dell'energia. Nei sistemi con dinamiche oscillatorie scarsamente smorzate (come il modo Phugoid o il Dutch Roll di un velivolo), Eulero Esplicito agisce come un iniettore artificiale di energia. Ad ogni passo, l'errore di approssimazione tangenziale spinge lo stato leggermente all'esterno dell'orbita reale nello spazio delle fasi. Nel

tempo, questa energia spuria si accumula, portando a una divergenza numerica della soluzione, anche se il velivolo reale è fisicamente stabile.

Eulero all'Indietro (Implicito) e Smorzamento Numerico

Per ovviare all'instabilità, si potrebbe ricorrere al metodo di Eulero Implicito, che valuta la derivata all'istante futuro:

$$X_{n+1} = X_n + h \cdot f(X_{n+1}, U_{n+1}, t_{n+1}) \quad (4.25)$$

Sebbene questo metodo sia incondizionatamente stabile, esso è computazionalmente costoso poiché richiede la risoluzione di un sistema di equazioni non lineari (ad esempio tramite Newton-Raphson) ad ogni step. Peggio ancora, dal punto di vista fisico, Eulero all'Indietro soffre del problema opposto rispetto alla sua variante esplicita: introduce uno smorzamento numerico artificiale. Esso dissipa virtualmente l'energia del sistema, falsando la percezione dei modi dinamici del velivolo, che appariranno più smorzati e stabili di quanto non siano nella realtà aerodinamica.

4.6 Metodo di Runge-Kutta del quarto ordine (RK4)

Per garantire la fedeltà fisica della simulazione rispettando la dinamica reale delle energie in gioco, il testo di Stevens e Lewis[3] si affida al metodo di Runge-Kutta del quarto ordine (RK4). Questo metodo esplicito raggiunge un bilanciamento eccellente tra stabilità numerica, accuratezza ed efficienza computazionale. L'errore di troncamento locale è ridotto a $\mathcal{O}(h^5)$, mentre l'errore globale scala come $\mathcal{O}(h^4)$, permettendo di utilizzare passi di integrazione più ampi senza incorrere in divergenze o distorsioni energetiche.

Formulazione Matematica

Il metodo RK4 calcola lo stato futuro valutando il campo vettoriale delle derivate f

in quattro punti distinti all'interno dell'intervallo temporale $[t_n, t_{n+1}]$, per poi mediareli con pesi specifici derivati dallo sviluppo in serie di Taylor. I quattro incrementi (slopes) vettoriali sono calcolati in sequenza come segue:

$$k_1 = f(X_n, U_n, t_n) \quad (4.26)$$

$$k_2 = f\left(X_n + \frac{h}{2}k_1, U_{n+1/2}, t_n + \frac{h}{2}\right) \quad (4.27)$$

$$k_3 = f\left(X_n + \frac{h}{2}k_2, U_{n+1/2}, t_n + \frac{h}{2}\right) \quad (4.28)$$

$$k_4 = f(X_n + hk_3, U_{n+1}, t_n + h) \quad (4.29)$$

Il significato fisico dei quattro predittori è il seguente:

- k_1 è la pendenza calcolata all'inizio dell'intervallo (equivalente a Eulero in avanti).
- k_2 è una stima della pendenza al punto medio dell'intervallo, utilizzando k_1 per proiettare lo stato.
- k_3 è una seconda stima (più raffinata) della pendenza al punto medio, utilizzando k_2 .
- k_4 è la stima della pendenza alla fine dell'intervallo, utilizzando k_3 .

Infine, lo stato al tempo t_{n+1} viene aggiornato calcolando una media ponderata delle quattro pendenze. I pesi assegnati (1, 2, 2, 1) sono mutuati dalla regola di quadratura di Simpson, che assegna maggior rilevanza alle stime calcolate nel punto medio per massimizzare la fedeltà della traiettoria:

$$X_{n+1} = X_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \quad (4.30)$$

Applicazione al Modello 6-DOF

Nell'architettura del simulatore, ad ogni step temporale, il blocco matematico che implementa le equazioni descritte viene richiamato quattro volte consecutive dal solutore RK4. Ad ogni chiamata, il vettore delle forze aerodinamiche e propulsive viene ricalcolato sulla base degli stati intermedi generati dai coefficienti k_i . Questo approccio garantisce che la complessa rete di accoppiamenti non lineari (come le

coppie giroscopiche e le dipendenze da α e β) venga tracciata con una precisione tale da prevenire l'instabilità energetica tipica dei modelli aeronautici complessi ad alti ratei di rotazione.[3]

Capitolo 5

Architettura e Struttura del Simulatore

L'instabilità intrinseca dell'F-16 e la forte non linearità delle sue equazioni del moto vengono rappresentate all'interno di questo simulatore che serve come strumento di validazione e per la linearizzazione e la validazione delle leggi di controllo che verranno create all'interno di questi capitoli Cap.10 e Cap.11. Il simulatore è stato sviluppato in ambiente MATLAB/Simulink dal professore Raktim Bhattacharya [1], adottando un approccio accademico e con una organizzazione che permette di isolare i singoli fenomeni fisici (aerodinamica, cinematica, gravità, atmosfera).

5.1 Panoramica Globale dell'Architettura

La struttura principale del simulatore, riportata in Figura 5.1, rappresenta il *plant* del velivolo in anello aperto. Esso è composto da sei macro-blocchi interconnessi che si scambiano segnali cinematici e dinamici ad ogni istante di campionamento, risolvendo di fatto il sistema di equazioni differenziali ordinarie (ODE) che governa il moto nello spazio.

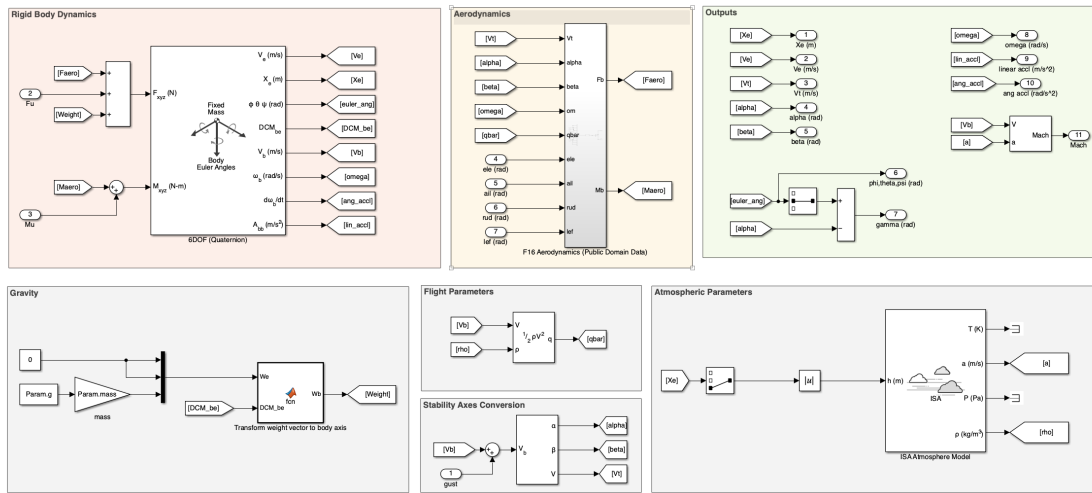


FIGURA 5.1: Architettura globale del simulatore F-16 in ambiente Simulink. Sono visibili i sei sottosistemi principali che governano la dinamica del volo.

Nelle sezioni seguenti verranno analizzati nel dettaglio i singoli sottosistemi, definendone gli ingressi, le uscite e le responsabilità computazionali all'interno del ciclo di simulazione.

5.2 Dinamica del Corpo Rigido

Il blocco **Rigid Body Dynamics** presente in `matlab` costituisce il nucleo cinematico del simulatore. Il suo scopo è integrare le equazioni di Newton-Eulero (che verranno discusse nel Capitolo 4) per propagare lo stato del velivolo nel tempo, pertanto, le equazioni differenziali non sono state implementate analiticamente riga per riga, bensì integrate direttamente tramite i solutori numerici di questo blocco Simulink.

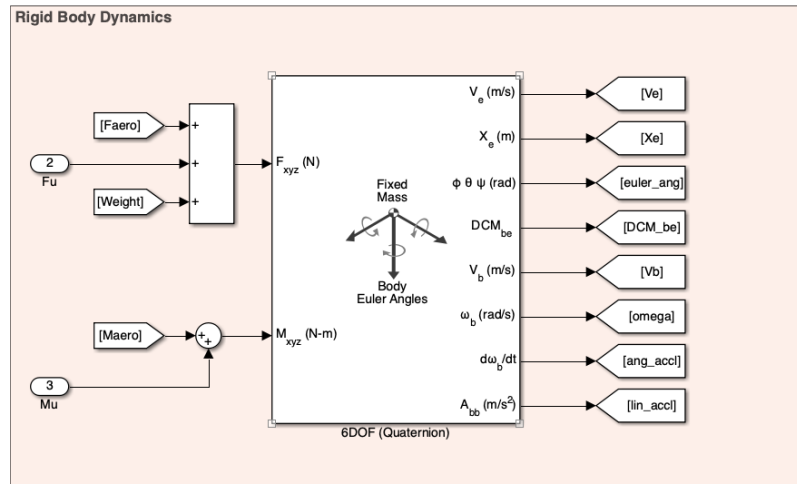


FIGURA 5.2: Dettaglio del blocco 6-DOF (Quaternion).

- **Ingressi:** Il blocco riceve la sommatoria delle forze esterne applicate sul baricentro, composta dalle forze aerodinamiche (**Faero**) e dalla forza peso vettoriale (**Weight**). Parallelamente, riceve i momenti aerodinamici (**Maero**) generati dai ratei di rotazione e dalle superfici di controllo, questi valori vengono ottenuti utilizzando lo script MATLAB **F16AeroFM.m** (dettagliato nella Sezione 5.3), che computa le forze e i momenti risultanti.
- **Elaborazione:** Utilizza il blocco nativo di Simulink *6DOF (Quaternion)*. L'integrazione numerica avviene sfruttando i parametri di Eulero (quaternioni) al fine di evitare le singolarità matematiche (*Gimbal Lock*) tipiche del volo verticale che verranno trattati nel Capitolo 4.4.
- **Uscite:** Restituisce lo stato cinematico completo. Fornisce le velocità nel sistema corpo (**Vb**), le velocità e le posizioni nel sistema terrestre (**Ve**, **Xe**), gli angoli di Eulero classici per la telemetria (**euler_ang**), i ratei angolari (**omega**) e le accelerazioni lineari e angolari e inoltre Esporta la *Direction Cosine Matrix* (**DCM_be**), fondamentale per le proiezioni tra i sistemi di riferimento.

5.3 Sottosistema Aerodinamico

Il blocco **Aerodynamics** funge da ponte tra la cinematica istantanea del velivolo e i dati sperimentali di galleria del vento che sono stati utilizzati per il calcolo dei

coefficienti aerodinamici (Sezione 3.4.1, pubblicati nei database NASA).

- **Ingressi:** L'aerodinamica dipende dalle variabili in ingresso ricevendo così la *True Airspeed* (V_t), gli angoli di incidenza e derapata (α , β), i ratei di rotazione (ω), la pressione dinamica (q_{bar}) e, soprattutto, i comandi del pilota: deflessione di equilibratore (ϵ), alettoni (δ), timone (δ_r) e flap del bordo d'attacco (δ_{le}).
- **Elaborazione:** Al suo interno viene richiamata la funzione MATLAB `F16AeroFM.m`. Questa funzione interpola le tabelle dei coefficienti adimensionali, somma i contributi dei *Leading Edge Flaps* e i termini di smorzamento, scalando infine i coefficienti mediante la geometria dell'ala per ottenere forze dimensionali in Newton.
- **Uscite:** Genera i vettori di forza aerodinamica (F_{aero}) e momento aerodinamico (M_{aero}) espressi nel Body Frame, che verranno inviati al solutore 6-DOF.

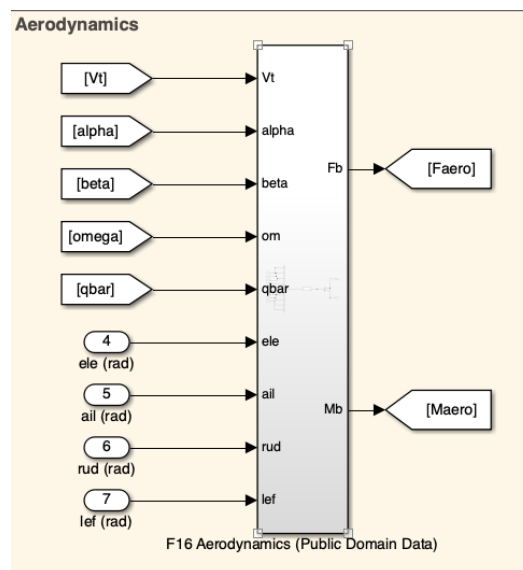


FIGURA 5.3: Dettaglio del sottosistema Aerodinamico: interfaccia tra lo stato del velivolo e la funzione di calcolo dei coefficienti.

5.4 Modello Atmosferico

Le prestazioni dell' F-16 variano drasticamente in base alla quota di volo a causa delle variazioni di densità e temperatura dell'aria. Questo blocco modella l'ambiente in cui il velivolo si muove.

- **Ingressi:** Riceve la posizione inerziale del velivolo ($\mathbf{x}_e$), estraendo specificamente la terza componente cambiata di segno, ovvero la quota altimetrica geometrica h , l'equazione usata per descrivere questa variabile si trova al Capitolo [4.1](#).
- **Elaborazione:** Implementa il modello *ISA* (*International Standard Atmosphere*). Calcola i gradienti termici e barometrici in base all'altitudine a cui si trova l'aereo.
- **Uscite:** Fornisce al sistema la temperatura assoluta T , la velocità del suono locale a , la pressione statica P e la densità dell'aria ρ . Quest'ultima è un parametro critico per la determinazione della portanza.

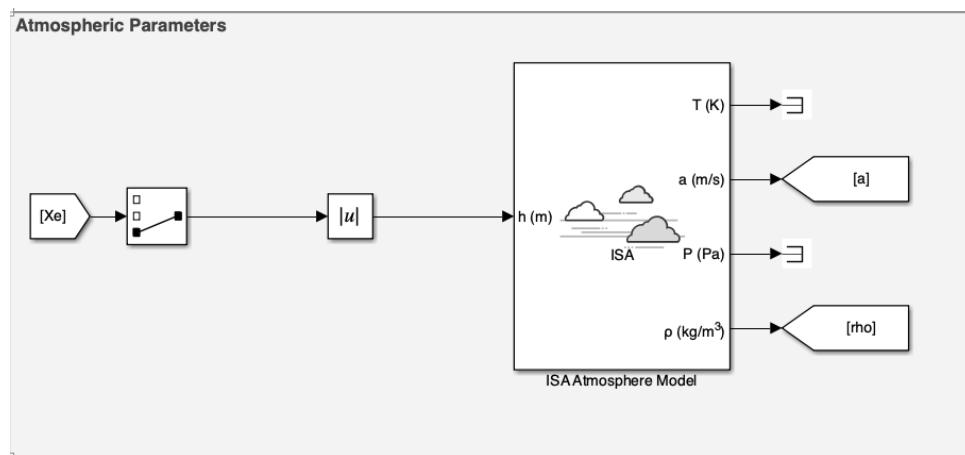


FIGURA 5.4: Implementazione del modello atmosferico ISA per il calcolo dei parametri ambientali in funzione della quota.

5.5 Parametri di Volo e Assi di Stabilità

Questa sezione del modello calcola le variabili aerodinamiche secondarie necessarie alla simulazione.

- **Conversione agli Assi di Stabilità:** Riceve il vettore velocità letto nel Body Frame (V_b) e lo scompone. In tale sezione è predisposto un nodo di somma (**gust**) che permette l'iniezione di modelli stocastici di raffiche di vento per valutare la robustezza del velivolo. Infine, l'uscita fornisce l'angolo d'attacco α , l'angolo di derapata β e la velocità totale V_t .
- **Pressione Dinamica:** Sfruttando la velocità V_t appena calcolata e la densità ρ proveniente dal modello atmosferico e calcola la pressione dinamica $\bar{q} = \frac{1}{2}\rho V_t^2$. Questo segnale viene poi instradato verso il blocco aerodinamico.

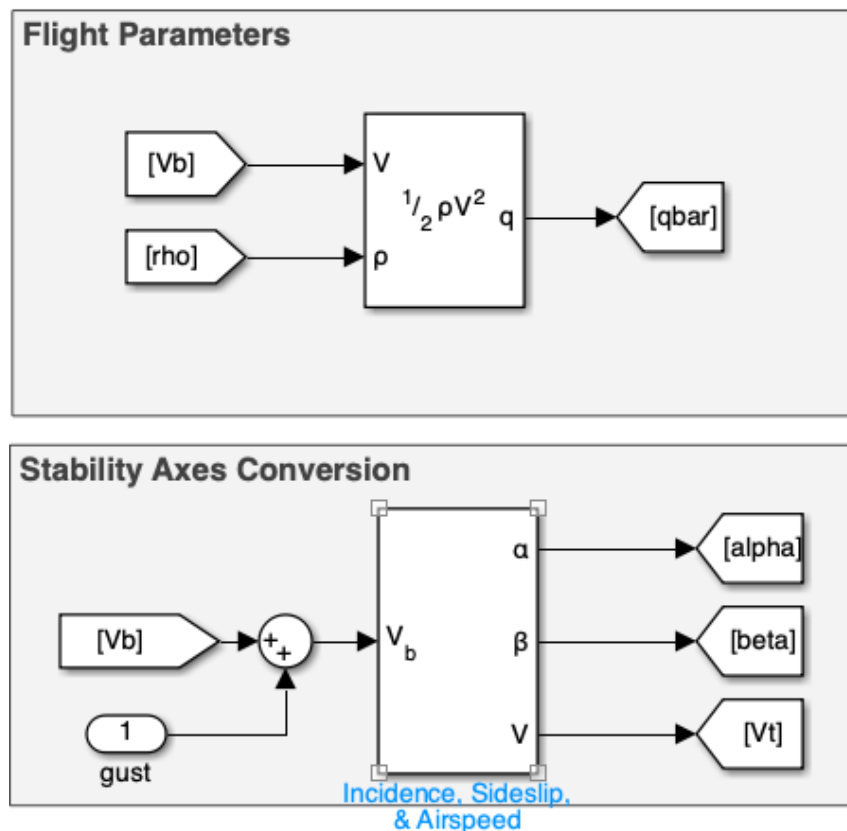


FIGURA 5.5: Sottosistema per il calcolo delle variabili cinematiche α , β e della pressione dinamica q .

5.6 Modellazione della Gravità

Il calcolo della forza peso richiede particolare attenzione poiché l'integrazione dinamica avviene in un sistema rotante e solidale all'aereo (Body Frame).

- **Logica:** Nel sistema di riferimento terrestre (NED), il vettore peso ha una sola componente attiva diretta verso il basso, pari a $[0, 0, m \cdot g]^T$ come presente inoltre all'interno del modello `matlab`.
- **Elaborazione:** Il blocco riceve la massa costante del velivolo moltiplicata per l'accelerazione di gravità g . Utilizzando la matrice di rotazione `DCM_be` presa dal blocco alla sezione 5.2 nella sezione delle `Uscite`, calcolata dal blocco 6-DOF e poi proiettata istantaneamente il vettore gravità negli assi inerziali agli assi corpo.
- **Uscita:** Il vettore `Weight` risultante, che alle elevate incidenze o durante i rollii si distribuisce sugli assi X_{body} e Y_{body} influenzando pesantemente la dinamica longitudinale e laterale.

All'interno del sottosistema è presente un blocco *MATLAB Function* che esegue materialmente il prodotto matriciale. Il codice implementato riceve il vettore peso inerziale (`We`) e la *Direction Cosine Matrix* (5.2) (`DCM_be`), restituendo il vettore proiettato negli assi corpo (`Wb`):

```

1  function Wb = fcn(We, DCM_be)
2  Wb = DCM_be * We; % Weight in body coordinate system.
3  % DCM_be is correct, no need for transpose. They must have
   fixed it in this version.

```

LISTING 5.1: Proiezione vettoriale della forza peso tramite blocco MATLAB Function.

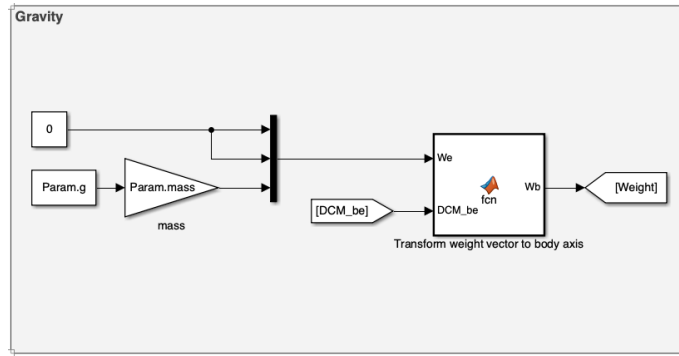


FIGURA 5.6: Trasformazione vettoriale della forza peso dal sistema NED agli assi corpo tramite Direction Cosine Matrix.

5.7 Condizionamento delle Uscite

Tale blocco ha lo scopo di condizionare i segnali di uscita destinati al motore di rendering visivo FlightGear (trattato nel Capitolo 6. Oltre a riordinare i flussi dei bus il blocco esegue due calcoli derivati essenziali:

1. **Numero di Mach:** Calcolato come rapporto tra la velocità relativa (V_b) e la velocità del suono locale (a) fornita dal blocco atmosferico 5.4.
2. **Angolo di Rampa (Flight Path Angle, γ):** Estrae l'angolo di beccheggio θ dal vettore di Eulero e vi sottrae l'angolo d'attacco α . Il parametro $\gamma = \theta - \alpha$.

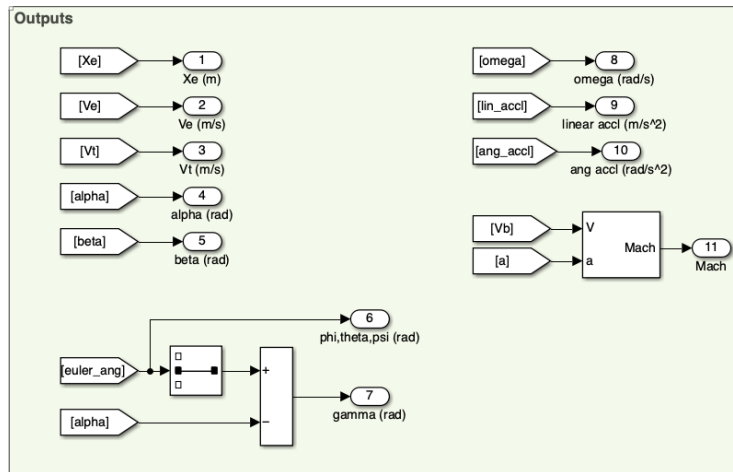


FIGURA 5.7: Condizionamento dei segnali di uscita e calcolo dei parametri di volo derivati (Mach, Gamma).

Una volta consolidato il modello dinamico all'interno dell'ambiente MATLAB/Simulink (descritto nel Capitolo 5), l'infrastruttura software è stata espansa per supportare l'esecuzione in tempo reale. L'obiettivo di questa fase è trasformare il modello Simulink in un vero e proprio simulatore di volo. Questo simulatore, sviluppato nel presente lavoro di tesi estendendo il modello del Prof. Raktim Bhattacharya[1], hardware e software, visibile nella sua interezza in Figura 6.1, costituisce lo strumento definitivo per validare le dinamiche di volo, e in un secondo momento, per testare la robustezza delle strategie di controllo automatico (LQR, MPC) che verranno affrontate nei capitoli successivi.

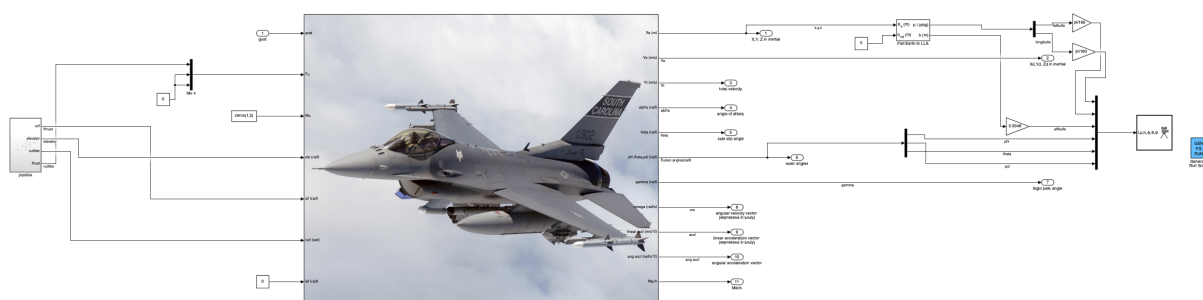


FIGURA 6.1: Panoramica dell'architettura di simulazione: integrazione tra Simulink e motore di rendering FlightGear.

6.1 Integrazione con periferiche Thrustmaster HOTAS

Per interagire fisicamente e in tempo reale con il modello non lineare, il computer che esegue Simulink è stato interfacciato con una periferica di pilotaggio professionale **Thrustmaster**. L'acquisizione dei comandi avviene tramite appositi blocchi Simulink strutturati in un **Sub-System** appositamente implementato per l'acquisizione delle periferiche di gioco (es. *Pilot Joystick* o *Analog Input*). I segnali elettrici analogici provenienti dagli assi vengono acquisiti con un tempo di campionamento rigoroso e sottoposti a condizionamento (normalizzazione, filtraggio del rumore e applicazione di zone morte). Nello specifico, la mappatura degli assi permette al pilota di comandare i quattro ingressi primari del velivolo:

- **Manetta (Throttle):** Regola la spinta del propulsore (**Thrust**).
- **Barra di comando (Stick):** Il movimento longitudinale agisce sull'equilibratore (**elev**) per il controllo del beccheggio, mentre il movimento laterale comanda gli alettoni (**ail**) per il rollio.
- **Pedaliera (Pedals):** Comanda il timone di direzione (**rud**) per l'imbardata.

Questi segnali condizionati vengono inviati direttamente agli ingressi del sottosistema aerodinamico e propulsivo, influenzando istantaneamente le derivate di stato del blocco 6-DOF.

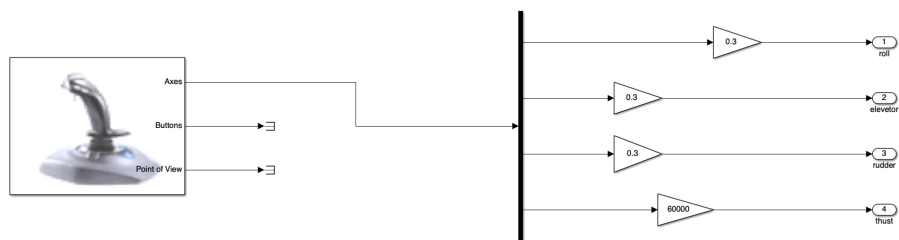


FIGURA 6.2: Sottosistema di acquisizione e condizionamento dei segnali provenienti dal sistema Thrustmaster.

6.2 Interfacciamento: FlightGear e Aerospace Blockset

Per fornire un riscontro visivo che sia robusto è importante e di vitale importanza registrare le variabili di assetto e delle velocità durante la manovra, per farlo il calcolatore matematico (Simulink) è stato interfacciato con il simulatore di volo open-source **FlightGear**. La comunicazione avviene sfruttando i blocchi di rete forniti dall'**Aerospace Blockset** di MATLAB. Ad ogni istante di campionamento, il vettore di stato globale generato dal blocco 6-DOF viene spaccettato ed inviato a FlightGear tramite protocollo UDP. I dati trasmessi includono:

- **Cinematica Inerziale:** Latitudine, Longitudine e Altitudine.
- **Assetto:** Angoli di Eulero (Rollio ϕ , Beccheggio θ , Imbardata ψ).

6.2.1 Immagini FlightGear

In questa sezione vengono riportate le schermate ottenute utilizzando il simulatore FlightGear. Per la simulazione è stata utilizzata la versione 2020.3, integrata con il pacchetto dedicato all'F-16 per permetterne il pilotaggio e il test dei sistemi.

Immagine prima del decollo: Prima di avviare la simulazione è necessario inizializzare il velivolo sulla pista. Successivamente, tramite l'apposita interfaccia, si attiva la procedura automatizzata che esegue i controlli pre-volo necessari al decollo.



FIGURA 6.3: Visuale a terra dell'aereo prima del decollo.

Volo di crociera La Figura 6.4 illustra l'aereo in volo di crociera, dimostrando che, anche in assenza di input esterni, il sistema riesce a mantenere il proprio assetto di volo livellato con le tecniche di controllo avanzate sviluppate ai capitolo Cap.10 e Cap.11.



FIGURA 6.4: Visuale FlightGear in volo di crociera.

Visuale interna dell'abitacolo Come visibile nella Figura 6.5, l'HUD dell'aereo in volo di crociera mostra i valori telemetrici. Tali dati vengono gestiti dal monitor real-time e dal computer di bordo, argomenti trattati diffusamente nel Capitolo 15.



FIGURA 6.5: Visuale dell'abitacolo su FlightGear in volo di crociera.

6.2.2 Strumenti di Bordo e Telecamere

L'impiego di FlightGear non si limita al mero rendering del terreno esterno. Trasmettendo i parametri fisici corretti, FlightGear è in grado di animare in tempo reale il *cockpit* virtuale dell'F-16. Questo permette al pilota di leggere direttamente dall'*Head-Up Display* (HUD) e dagli strumenti analogici valori critici come il numero di Mach, l'altitudine barometrica e l'angolo di rampa (γ). Inoltre, il sistema permette l'utilizzo delle **telecamere virtuali (cineprese)** integrate in FlightGear. Il pilota o l'ingegnere di sistema possono variare liberamente il punto di vista durante la simulazione (es. visuale dall'abitacolo, *chase camera* posteriore, visuale *fly-by*) per analizzare visivamente fenomeni dinamici complessi, quali il *Dutch Roll* [8] o le oscillazioni indotte dai comandi (PIO - *Pilot Induced Oscillations*), che risulterebbero difficili da interpretare unicamente attraverso i grafici temporali.

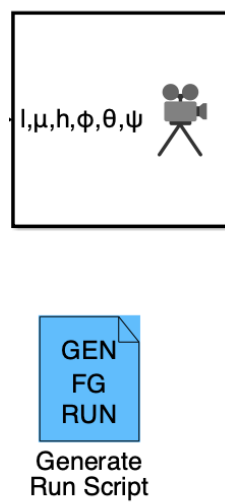


FIGURA 6.6: Interfacciamento con FlightGear tramite Aerospace Blockset e visualizzazione dell'assetto tramite telecamere esterne e strumentazione di bordo.

Parte II

Automazione del controllo longitudinale

Capitolo 7

Analisi di Stabilità e Linearizzazione

l'analisi di stabilità e la sintesi dei sistemi di controllo di volo (FCS) richiedono un modello lineare tempo-invariante (LTI). In questo capitolo si descrive la procedura matematica, validata tramite gli script MATLAB, per estrarre il modello nello *Spazio degli Stati* e valutarne la stabilità intrinseca intorno a un punto di *Trim*. Sarà proprio su questo punto operativo che verranno sintetizzati i controllori avanzati LQR (Cap.10) e MPC (Cap.11), valutandone il Control-Invariant-Set e il dominio di attrazione.

7.1 Definizione dei punti di equilibrio

Il volo in condizione di *Trim* è definito come uno stato in cui il velivolo mantiene una traiettoria costante senza accelerazioni nette ($\dot{\mathbf{V}} = \mathbf{0}$, $\dot{\boldsymbol{\omega}} = \mathbf{0}$). Matematicamente, dato il sistema $\dot{\mathbf{X}} = \mathbf{f}(\mathbf{X}, \mathbf{U})$, si ricerca un punto operativo (Operating Point, *op*) identificato dalla coppia $(\mathbf{X}^*, \mathbf{U}^*)$ tale per cui il campo vettoriale delle derivate si annulli.

7.1.1 L'Oggetto *opspec* (Operating Point Specification)

Prima di avviare l'algoritmo di ricerca, il software deve conoscere i vincoli del problema. L'oggetto **opspec** funge da specifica per la ricerca del punto operativo, istruendo il solutore su quali variabili sono fisse e quali sono libere di variare:

- **Stati Fissati (`Known = true`):** Si impone una condizione di volo specifica, ad esempio una velocità costante $V_t = 91.44$ m/s e quota fissa.
- **Derivate Nulle (`SteadyState = true`):** Si impone che le accelerazioni siano zero. Fanno eccezione le coordinate di posizione Nord ed Est (`SteadyState =`

false), poiché l'aereo si sposta nello spazio, ma questo non altera l'equilibrio delle forze aerodinamiche.

- **Limiti (Min / Max):** Si impongono vincoli fisici, come la saturazione della spinta ($T \in [1000, 19000]$ lbf) o il fincorsa meccanico dell'equilibratore ($\pm 25^\circ$). Tali limiti fisici costituiranno i vincoli per la definizione dell'Invariant Set nella sintesi del controllo MPC, come verrà approfondito nel Capitolo 11.

7.1.2 Generazione di op e oreport

Completato il setup, la funzione MATLAB `findop` viene richiamata per calcolare il punto di equilibrio. Data la forte non linearità del modello aerodinamico dell'F-16 e i stringenti vincoli fisici imposti tramite l'oggetto `opspec`, i tradizionali algoritmi di ottimizzazione locale (come il **Gradient-Descent** e **Active-Set**), tendono spesso a bloccarsi in minimi locali o a fallire la convergenza, tali algoritmi, testati in ambiente MATLAB, hanno restituito punti operativi sub-ottimi che non soddisfano le rigorose specifiche imposte nell'oggetto `opspec`. Per superare questa criticità e ottenere un punto operativo fisicamente accettabile che rispettasse rigorosamente le specifiche, si è reso necessario l'utilizzo di un algoritmo di ottimizzazione globale ed euristico chiamato (**Particle Swarm Optimization**)[9].

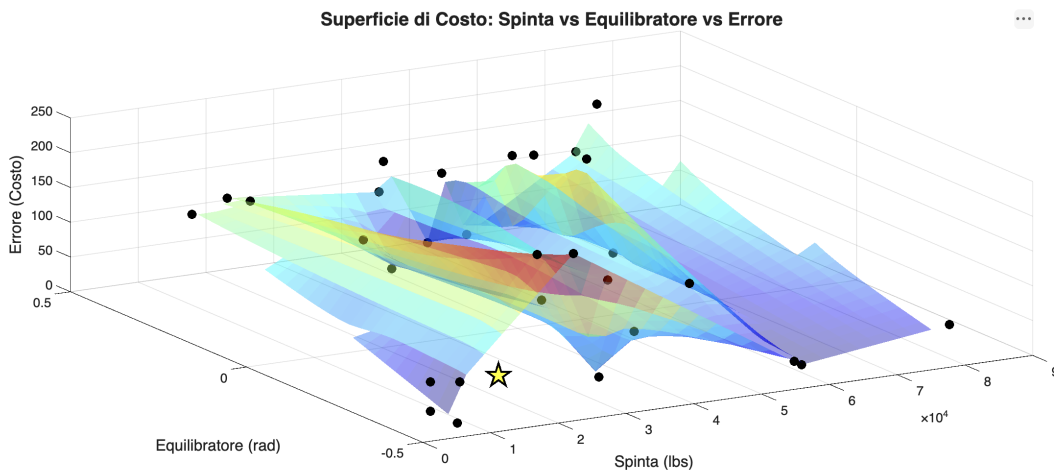


FIGURA 7.1: Inizio della ricerca dell'algoritmo PSO

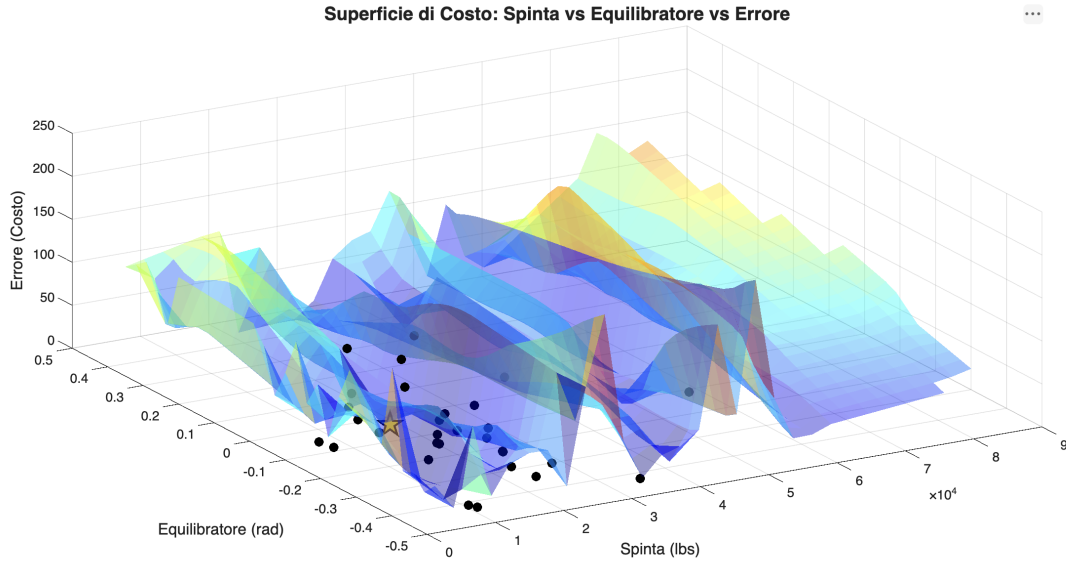


FIGURA 7.2: Algoritmo PSO a metà del suo funzionamento

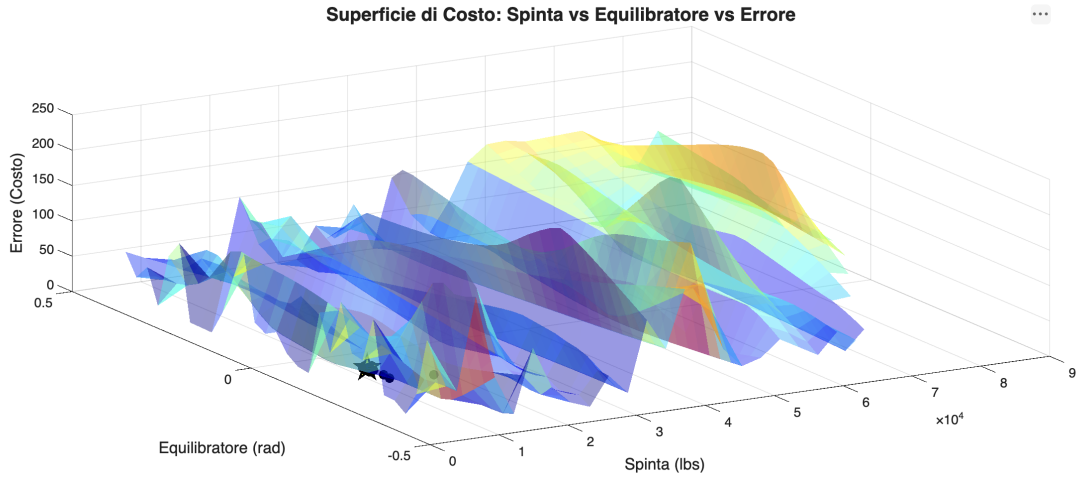


FIGURA 7.3: Algoritmo PSO alla fine del suo funzionamento

Questo algoritmo varia iterativamente gli angoli di assetto (come α e θ) e i comandi (spinta ed equilibratore), esplorando lo spazio delle soluzioni in modo intelligente fino a minimizzare gli errori sulle derivate di stato. Il processo restituisce l'oggetto **op**, che contiene il punto di equilibrio esatto ($\mathbf{X}^*, \mathbf{U}^*$), e l'oggetto **opreport**, che certifica l'avvenuta convergenza matematica mostrando residui tendenti a zero (garanzia di un vero equilibrio fisico in volo stazionario). I valori degli stati e degli

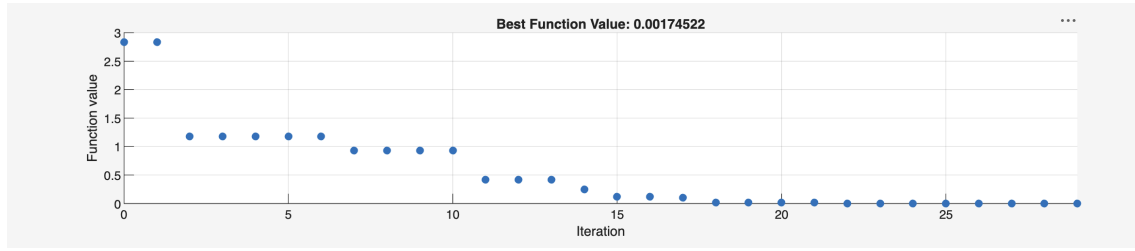


FIGURA 7.4: Funzionale di costo dell'algoritmo

ingressi di controllo calcolati a convergenza dall'algoritmo sono visualizzabili nella Figura 7.5.

Properties	x	dx	Known	SteadyState	Min	Max	dxMin	dxMax	Nx
1	[0;0.2054;0]	[0;0;0]	3×1 logical	3×1 logical	[-Inf;-Inf;-Inf]	[Inf;Inf;Inf]	[-Inf;-Inf;-Inf]	[Inf;Inf;Inf]	3
2	0	-0.0083	1	1	-Inf	Inf	-Inf	Inf	1
3	0	-0.0827	1	1	-Inf	Inf	-Inf	Inf	1
4	0	-0.0034	1	1	-Inf	Inf	-Inf	Inf	1
5	89.5176	-0.0076	0	1	-Inf	Inf	-Inf	Inf	1
6	0	-0.0407	1	1	-Inf	Inf	-Inf	Inf	1
7	18.6516	0.0154	0	1	-Inf	Inf	-Inf	Inf	1
8	0	91.4400	1	0	-Inf	Inf	-Inf	Inf	1
9	0	0	1	0	-Inf	Inf	-Inf	Inf	1
10	-3048	0	1	1	-Inf	Inf	-Inf	Inf	1

FIGURA 7.5: Report generato da MATLAB al termine dell'ottimizzazione, mostrante gli stati e gli ingressi di controllo del punto operativo (Trim) validato.

7.2 Linearizzazione dello spazio di stato

Ottenuto il punto operativo op , si applica la teoria delle piccole perturbazioni. Definendo $\mathbf{x} = \mathbf{X} - \mathbf{X}^*$ e $\mathbf{u} = \mathbf{U} - \mathbf{U}^*$, si espande la funzione non lineare in serie di Taylor troncata al primo ordine, ottenendo il sistema lineare globale:

$$\dot{\mathbf{x}}(t) = A_{full}\mathbf{x}(t) + B_{full}\mathbf{u}(t) \quad (7.1)$$

$$\mathbf{y}(t) = C_{full}\mathbf{x}(t) + D_{full}\mathbf{u}(t) \quad (7.2)$$

Tramite il comando `linearize`, l'algoritmo differenzia numericamente il modello restituendo le matrici Jacobiane complete 12×12 . La matrice $A_{full} \in \mathbb{R}^{12 \times 12}$ mappa

l'effetto che una perturbazione su ciascuno dei 12 stati ha sulle derivate (accelerazioni) di tutti gli altri 12 stati. Questa matrice contiene la dinamica *completamente accoppiata* dell'F-16 in tutte le direzioni dello spazio.

7.3 Analisi spettrale e autovalori

Il comportamento in evoluzione libera del sistema perturbato è governato dall'equazione differenziale omogenea $\dot{\mathbf{x}}_{long} = A_{long}\mathbf{x}_{long}$. La stabilità di tale sistema dipende esclusivamente dalla matrice dinamica A_{long} e viene analizzata calcolandone gli autovalori λ , ovvero le radici del polinomio caratteristico:

$$\det(\lambda I - A_{long}) = 0 \quad (7.3)$$

Per il Teorema di Stabilità di Ljapunov applicato ai sistemi lineari:

- Se tutti gli autovalori $\lambda_i = \sigma_i + j\omega_i$ possiedono parte reale strettamente negativa ($\sigma_i < 0$), il punto di equilibrio è **asintoticamente stabile**. Una perturbazione (es. una folata di vento) verrà smorzata nel tempo, riportando l'aereo in assetto.
- Se almeno un autovalore possiede parte reale positiva ($\sigma_i > 0$), il sistema è **instabile** e la perturbazione divergerà esponenzialmente.

Eseguendo il calcolo degli autovalori sulla realizzazione minima del sistema longitudinale (tramite il comando `pole(sys_long_min)`), i risultati estratti da MATLAB rivelano la vera natura aerodinamica dell'F-16. A differenza degli aerei civili convenzionali — che presentano tipicamente due coppie di autovalori complessi coniugati stabili (identificativi dei classici modi di Fugoide e Corto Periodo utili per studiare la dinamica veloce e lenta del controllo longitudinale), l'F-16 è progettato con *stabilità statica rilassata* per massimizzare l'agilità in combattimento. L'analisi numerica restituisce infatti quattro autovalori: **tre con parte reale strettamente negativa e uno con parte reale positiva**. Questo singolo polo posizionato nel semipiano destro ($s > 0$) genera un modo aperiodico divergente. Ciò dimostra matematicamente che la dinamica longitudinale dell'F-16 ad anello aperto è **intrinsecamente instabile**. Qualsiasi perturbazione esterna, per quanto infinitesima, provocherebbe

un rapido e incontrollabile ribaltamento del muso. Di conseguenza, risulta fisicamente impossibile pilotare il velivolo in sicurezza senza un'assistenza elettronica, rendendo categoricamente obbligatoria la sintesi di un sistema di controllo attivo FCS (Flight Control System) in retroazione per stabilizzare il plant. Il computer di bordo e la sua realizzazione real-time si trova a questo capitolo 12. La distribuzione spaziale degli autovalori è confermata e visualizzabile graficamente tramite la mappa poli-zeri del sistema, generata dal comando `pzmap` e riportata in Figura 7.6. Nel grafico è immediatamente identificabile la radice instabile ("X") posta a destra dell'asse immaginario.

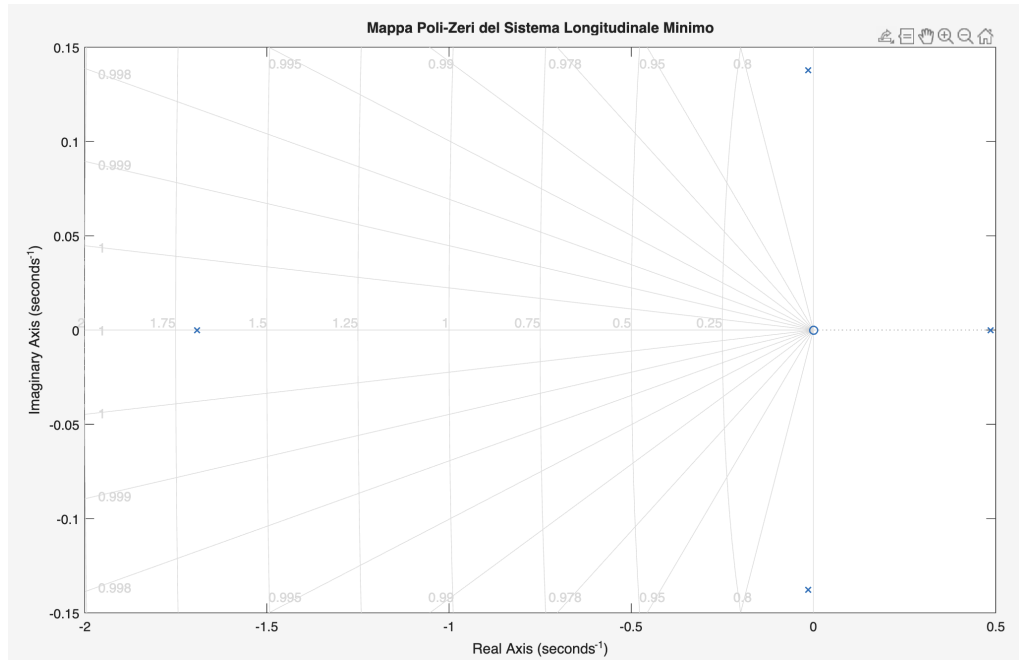


FIGURA 7.6: Mappa Poli-Zeri (pzmap) della realizzazione minima longitudinale. Evidenza del polo instabile a parte reale positiva.

7.4 Disaccoppiamento delle dinamiche

Si procede quindi all'estrazione (partizionamento) del **Sottosistema Longitudinale**. Osservando la convenzione del vettore di stato dell'F-16 di Stevens e Lewis, [3]: Per isolare la dinamica longitudinale dal modello aerodinamico completo a 6 gradi di libertà, le matrici di stato vengono estratte selezionando opportunamente gli

indici associati alle variabili di interesse. L'operazione di partizionamento permette di ottenere il sottosistema ridotto come segue:

$$\begin{aligned}
 A_{long} &= A_{full}(ix, ix) \\
 B_{long} &= B_{full}(ix, iu) \\
 C_{long} &= C_{full}(iy, ix) \\
 D_{long} &= D_{full}(iy, iu)
 \end{aligned} \tag{7.4}$$

Dove i vettori degli indici ix , iu e iy definiscono rispettivamente gli stati, gli ingressi e le uscite del modello longitudinale estratti dal simulatore:

Vettore degli Stati ($ix = [2, 5, 7, 9]$):

- θ (Pitch angle) → Indice **2**
- q (Pitch rate) → Indice **5**
- U (Body X-axis velocity) → Indice **7**
- W (Body Z-axis velocity) → Indice **9**

◦ *Nota: U e W sono le componenti della velocità sugli assi corpo. Da queste si ricavano le variabili aerodinamiche per il controllo di volo, ovvero la velocità totale $V_t = \sqrt{U^2 + W^2}$ e l'angolo d'attacco $\alpha = \arctan(W/U)$.*

Vettore degli Ingressi ($iu = [1, 2, 5, 6, 8]$):

- Thrust (Manetta) → Indice **1**
- F16/ele (Equilibratore / Elevator) → Indice **2**
- F16/lef (Flap al Bordo d'Attacco / LEF) → Indice **5**
- F16 gust1 (Disturbo: Raffica di vento asse X) → Indice **6**
- F16 Gust3 (Disturbo: Raffica di vento asse Z) → Indice **8**

Vettore delle Uscite ($iy = [7, 8, 13, 15, 17, 19]$):

- V_t (True Airspeed) → Indice **7**

- α (Angle of Attack) $\rightarrow$ Indice **8**
- γ (Flight Path Angle) $\rightarrow$ Indice **13**
- q (Pitch rate in uscita) $\rightarrow$ Indice **15**
- a_x (Accelerazione asse X / xbdd) $\rightarrow$ Indice **17**
- a_z (Accelerazione asse Z / zbdd) $\rightarrow$ Indice **19**

Di conseguenza, il vettore di estrazione degli stati è rigorosamente $\mathbf{ix} = [2, 5, 7, 9]$. Allo stesso modo, tramite il vettore $\mathbf{i_u}$, si isolano i comandi che agiscono in modo simmetrico (Spinta, Equilibratore, Flap) e le raffiche di vento, escludendo alettoni e timone; mentre con $\mathbf{i_y}$ si selezionano le sole variabili di output utili al controllo longitudinale.

L'estrazione in MATLAB avviene applicando questi vettori come veri e propri "filtri matematici" sull'impianto completo:

- **Matrice Dinamica (A):** Il comando $\mathbf{A_long} = \mathbf{A_full}(\mathbf{ix}, \mathbf{ix})$ estrae la sottomatrice $\mathbf{A}_{long} \in \mathbb{R}^{4 \times 4}$. Questa descrive esclusivamente come beccheggio, rateo, e le velocità sull'asse corpo interagiscano tra loro, isolando il nucleo puro della dinamica longitudinale transitoria.
- **Matrice degli Ingressi ($\mathbf{B}_{long}$ e sua scomposizione):** Il partizionamento $\mathbf{B_long} = \mathbf{B_full}(\mathbf{ix}, \mathbf{i_u})$ genera inizialmente la sottomatrice $\mathbf{B}_{long} \in \mathbb{R}^{4 \times 5}$. Essa mappa l'efficacia aerodinamica e propulsiva dei comandi longitudinali (e l'impatto dei disturbi del vento) direttamente sulle derivate degli stati, scartando qualsiasi accoppiamento "sporco" con la dinamica latero-direzionale. Tuttavia, ai fini della sintesi delle leggi di controllo, lo script procede con una fondamentale scomposizione algoritmica di tale matrice:

$$\mathbf{B}_{long} = \left[\mathbf{B}_{ctrl} \mid \mathbf{B}_{wind} \right] \quad (7.5)$$

Questa operazione divide la matrice originaria in due sottomatrici distinte: $\mathbf{B}_{ctrl}$ isola esclusivamente le colonne relative agli attuatori fisicamente comandabili dal *Flight Control Computer* (manetta δ_{th} , elevatore δ_e e flap di bordo

d'attacco δ_{lef}), mentre $\mathbf{B}_{wind}$ raggruppa le colonne relative agli ingressi di disturbo atmosferico non controllabili. Come anticipato nel 4.2, e approfondito nel 10, questa separazione è il presupposto matematico obbligatorio affinché l'algoritmo di controllo agisca unicamente sui comandi fisici del velivolo per reiezione dei disturbi, senza tentare matematicamente di "comandare il vento".

- **Matrice di Uscita (C):** L'estrazione $\mathbf{C}_{long} = \mathbf{C}_{full}(\mathbf{i}_y, \mathbf{i}_x)$ produce la matrice $\mathbf{C}_{long} \in \mathbb{R}^{6 \times 4}$. Questa matrice definisce il legame algebrico e cinematico che trasforma gli stati interni del corpo rigido nelle grandezze fisiche misurate dai sensori (come la velocità totale V_t , l'incidenza α e il rateo di beccheggio in uscita).
- **Matrice di Trasmissione Diretta (D):** Infine, $\mathbf{D}_{long} = \mathbf{D}_{full}(\mathbf{i}_y, \mathbf{i}_u)$ fornisce la matrice di feedthrough $\mathbf{D}_{long} \in \mathbb{R}^{6 \times 5}$. La corretta estrazione di questa matrice è vitale per la fedeltà del modello, poiché modella gli effetti istantanei: cattura, ad esempio, come una rapida deflessione dell'equilibratore generi un picco immediato di accelerazione verticale (a_z) prima ancora che il velivolo abbia fisicamente il tempo di ruotare.

7.5 Rappresentazione MIMO e funzioni di trasferimento

Il modello linearizzato nello Spazio degli Stati fornisce una descrizione completa della dinamica interna del velivolo. Tuttavia, per l'analisi nel dominio della frequenza e la progettazione dei classici controllori (come il PID per il mantenimento della quota o del beccheggio), è necessario mappare la relazione diretta tra gli ingressi di controllo (cause) e le uscite misurate (effetti).

Essendo l'F-16 un sistema **MIMO** (*Multiple-Input Multiple-Output*), non è descrivibile da una singola funzione di trasferimento, ma richiede una **Matrice delle Funzioni di Trasferimento**, indicata con $G(s)$.

7.5.1 Derivazione Matematica

Si consideri il sistema lineare tempo-invariante (LTI) precedentemente estratto:

$$\dot{\mathbf{x}}(t) = A\mathbf{x}(t) + B\mathbf{u}(t) \quad (7.6)$$

$$\mathbf{y}(t) = C\mathbf{x}(t) + D\mathbf{u}(t) \quad (7.7)$$

Applicando la Trasformata di Laplace ad ambo i membri e assumendo condizioni iniziali nulle ($\mathbf{x}(0) = \mathbf{0}$), l'equazione di stato diviene:

$$s\mathbf{X}(s) = A\mathbf{X}(s) + B\mathbf{U}(s) \implies \mathbf{X}(s) = (sI - A)^{-1}B\mathbf{U}(s) \quad (7.8)$$

Sostituendo $\mathbf{X}(s)$ nell'equazione di uscita, si ottiene la relazione ingresso-uscita nel dominio di Laplace:

$$\mathbf{Y}(s) = [C(sI - A)^{-1}B + D] \mathbf{U}(s) \quad (7.9)$$

La matrice $G(s) = C(sI - A)^{-1}B + D$ è la Matrice delle Funzioni di Trasferimento. Se il vettore degli ingressi $\mathbf{U}$ ha dimensione m e il vettore delle uscite $\mathbf{Y}$ ha dimensione p , $G(s)$ sarà una matrice di dimensione $p \times m$.

Ogni singolo elemento $G_{i,j}(s)$ della matrice è una funzione di trasferimento SISO (*Single-Input Single-Output*) costituita dal rapporto tra due polinomi complessi, e descrive come il j -esimo ingresso influenzi l' i -esima uscita:

$$G_{i,j}(s) = \frac{Y_i(s)}{U_j(s)} \quad (7.10)$$

Capitolo 8

Proprietà Strutturali e Analisi MI-MO

Una volta ottenuto il modello lineare nello Spazio degli Stati, l'analisi si sposta dal dominio del tempo al dominio complesso della frequenza di Laplace. In questo capitolo si affronta la parte di scrematura del modello (garantendo raggiungibilità e osservabilità caratteristiche fondamentali per un sistema dinamico), il calcolo dei poli e zeri effettivi con analisi della proprietà bloccante degli zeri di trasmissione.

8.1 Raggiungibilità, osservabilità e realizzazione minima

La conversione dalla rappresentazione nello Spazio degli Stati alla Matrice delle Funzioni di Trasferimento $G(s)$ è governata dalla relazione:

$$G(s) = C(sI - A)^{-1}B + D \quad (8.1)$$

L'inversione della matrice $(sI - A)$ produce al denominatore un polinomio caratteristico di ordine pari al numero degli stati originari n . Tuttavia, affinché il sistema sia completamente gestibile, è necessario che tutti i suoi stati interni siano eccitabili e misurabili.

8.1.1 Matrici di Raggiungibilità e Osservabilità

Uno stato si definisce **raggiungibile** (o *controllabile*) se esiste un segnale di ingresso (es. deflessione dell'equilibratore o variazione di manetta) capace di trasferire il sistema da uno stato iniziale nullo a tale stato in un intervallo di tempo finito. La

condizione di completa raggiungibilità si verifica analizzando il rango della matrice di Raggiungibilità $\mathcal{R}$:

$$\mathcal{R} = \begin{bmatrix} B & AB & A^2B & \dots & A^{n-1}B \end{bmatrix} \quad (8.2)$$

Il sistema è completamente raggiungibile se e solo se $\text{rango}(\mathcal{R}) = n$. Dualmente, uno stato è **osservabile** se la sua evoluzione dinamica può essere dedotta unicamente dalla conoscenza degli ingressi applicati e delle uscite misurate dai sensori di bordo. La condizione si verifica calcolando il rango della matrice di Osservabilità $\mathcal{O}$:

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{n-1} \end{bmatrix} \quad (8.3)$$

Il sistema è completamente osservabile se $\text{rango}(\mathcal{O}) = n$. Eseguendo l'analisi sul modello longitudinale dell'F-16 (con $n = 4$ stati), i calcoli computazionali confermano che il rango della matrice uguaglia il numero delle variabili di stato. Di seguito si riporta l'output esatto ricavato dal terminale di calcolo, che mostra le dimensioni complete (4×20) della matrice $\mathcal{R}$ e l'esito della verifica:

```

1  --- Analisi di Raggiungibilita'/Controllabilita' ---
2  Matrice di Raggiungibilita' R:
3  Columns 1 through 14
4      0      0      0      0      0      0      -2.9743      -0.5483
5      -0.0045      0.0220      -0.0000      1.9455      0.3990      0.0054
6      0      -2.9743      -0.5483      -0.0045      0.0220      -0.0000      1.9455      0.3990
7      0.0054      -0.0267      -0.0000      -6.9503      -1.3364      -0.0135
8      0.0001      0.5049      1.3592      -0.0050      0.0245      -0.0000      53.0932      9.8358
9      0.0822      -0.4063      0.0000      -12.4700      -3.0523      -0.0648
10      0      -7.0977      0.6462      0.1002      -0.4949      -0.0000      -243.1015      -45.9982
11      -0.4226      2.0873      -0.0000      292.0728      57.8258      0.6751
12
13  Columns 15 through 20
14      -0.0267      -0.0000      -6.9503      -1.3364      -0.0135      0.0666
15      0.0666      -0.0000      11.4818      2.2467      0.0248      -0.1225
16      0.3200      0.0000      113.1894      21.5677      0.2068      -1.0213
17      -3.3347      0.0000      -735.9921      -142.3933      -1.4850      7.3357
18
19  [OK] Il sistema e' COMPLETAMENTE raggiungibile (Rango R = 4).
20  --- Analisi di Osservabilita' ---
21  Matrice di Osservabilita' O:
22      0      0      0.9801      0.1984
23      0      0      -0.0022      0.0107
24      1.0000      0      0.0022      -0.0107
25      0      1.0000      0      0
26      -9.6110      -17.9109      -0.0012      0.0253
27      -1.9456      82.9949      -0.1053      -0.5364
28      -9.8060      -1.0875      -0.0221      -0.0817
29      0.0000      0.9285      -0.0011      -0.0058
30      -0.0000      0.0715      0.0011      0.0058
31      0      -0.7079      -0.0026      0.0224
32      -0.0377      5.1879      0.0441      -0.4140
33      2.0553      -103.3322      -0.1600      2.1405
34      0.3709      -15.4178      0.0115      0.0189
35      0.0221      -1.1188      -0.0018      0.0238
36      -0.0221      0.4110      -0.0008      -0.0015
37      -0.0184      2.4032      -0.0005      -0.0279
38      0.3818      -38.8595      0.0300      0.3392
39      -2.6267      255.7162      0.0446      -3.4623
40      -0.1470      12.6508      0.0382      -0.3545
41      -0.0290      2.8253      0.0004      -0.0378
42      0.0106      -0.4221      -0.0009      0.0100
43      0.0591      -4.0247      -0.0033      0.0687
44      -0.9481      55.5025      0.0657      -1.0499
45      6.3080      -471.7933      -0.3030      7.5750
46  [OK] Il sistema e' COMPLETAMENTE osservabile (Rango O = 4).

```

Risultato del calcolo delle Matrici di Raggiungibilita' e Osservabilita'

Un medesimo controllo svolto in dualità sulla matrice $\mathcal{O}$ ha confermato la completa osservabilità del sistema longitudinale.

8.1.2 Cancellazione Polo-Zero e Comando `minreal`

L'analisi del rango delle matrici $\mathcal{R}$ e $\mathcal{O}$ è di cruciale importanza pratica. Se un modo dinamico del velivolo risultasse non simultaneamente controllabile e osservabile, esso genererebbe una radice identica sia al numeratore (uno zero) sia al denominatore (un polo) della funzione di trasferimento. Sebbene analiticamente queste coppie si semplifichino, numericamente esse permangono, innalzando in modo fittizio l'ordine del sistema e creando malcondizionamenti. Per risolvere questo problema, si estrae la **Realizzazione Minima** (*Minimal Realization*). Tramite il comando MATLAB `minreal`, l'algoritmo analizza i ranghi, cancellando le dinamiche nascoste e le coincidenze polo-zero. Il risultato è un sistema LTI compatto e strettamente *minimal*, che descrive solo la dinamica effettivamente utile.

- **Poli del Sistema MIMO:** Coincidono con gli autovalori della matrice A_{long} ottenuta dal sistema a realizzazione minima. Essi determinano la stabilità asintotica globale e le frequenze naturali di oscillazione.
- **Zeri Invarianti e di Trasmissione:** Nei sistemi multivariabili, gli zeri non sono semplicemente le radici del numeratore delle singole funzioni di trasferimento, ma godono di specifiche proprietà matriciali e direzionali.

8.1.3 Matrice di Sistema e Zeri Invarianti

Si consideri il sistema MIMO in tempo continuo nello spazio degli stati. Trasformando secondo Laplace, si ottiene la relazione matriciale:

$$\begin{bmatrix} \lambda I - A & -B \\ C & D \end{bmatrix} \begin{bmatrix} \mathbf{X}(s) \\ \mathbf{U}(s) \end{bmatrix} = \begin{bmatrix} \mathbf{0} \\ \mathbf{Y}(s) \end{bmatrix} \quad (8.4)$$

Da questa relazione si definisce la **Matrice di Sistema** $P(\lambda)$:

$$P(\lambda) = \begin{bmatrix} \lambda I - A & -B \\ C & D \end{bmatrix} \quad (8.5)$$

Il **rango normale** della matrice $P(\lambda)$ è definito come il suo rango per ogni valore di λ , eccetto per un numero finito di singolarità. Gli **Zeri Invarianti** di un sistema MIMO sono definiti come i valori complessi λ tali per cui il rango della matrice $P(\lambda)$ è strettamente minore del suo rango normale (ovvero i valori che annullano i minori di rango massimo della matrice). È fondamentale distinguere tra Zeri Invarianti e Zeri di Trasmissione. Se il sistema (A, B, C, D) è una realizzazione minima (completamente raggiungibile e osservabile), allora gli zeri invarianti coincidono perfettamente con gli zeri di trasmissione della matrice $G(s)$. Nel caso in cui il sistema non sia minimale, gli zeri di trasmissione rappresentano solo un sottoinsieme degli zeri invarianti. Si sta lavorando in questo caso con un sistema minimale quindi zeri di trasmissione zeri invarianti coincidono.

8.1.4 La Proprietà Bloccante degli Zeri MIMO

A differenza dei sistemi SISO, gli zeri invarianti MIMO sono associati a specifiche direzioni geometriche nello spazio degli stati e degli ingressi. Questa caratteristica si traduce nel seguente **Teorema**: Se λ è uno zero invariante, allora esiste uno stato iniziale $\mathbf{x}_0 \neq \mathbf{0}$ e un vettore d'ingresso $\mathbf{u}_0 \neq \mathbf{0}$, tali che dato un ingresso $u(t) = \mathbf{u}_0 e^{\lambda t}$, l'uscita del sistema risulta rigorosamente nulla: $\mathbf{y}(t) = \mathbf{0}$, $\forall t \geq 0$. Questa caratteristica è nota come **Proprietà Bloccante degli Zeri**: gli zeri bloccano la trasmissione degli ingressi corrispondenti alle loro pulsazioni spaziali. La dimostrazione si basa sulla perdita di rango della matrice $P(\lambda)$. Essendo λ uno zero invariante, esiste un vettore nel suo spazio nullo (*Nullspace*) tale per cui:

$$\begin{bmatrix} \lambda I - A & -B \\ C & D \end{bmatrix} \begin{bmatrix} \mathbf{x}_0 \\ \mathbf{u}_0 \end{bmatrix} = \begin{bmatrix} \mathbf{0} \\ \mathbf{0} \end{bmatrix} \quad (8.6)$$

[cite: 1083] Sviluppando il sistema lineare omogeneo, si ottengono le direzioni associate:

$$(\lambda I - A)\mathbf{x}_0 - B\mathbf{u}_0 = \mathbf{0} \quad (8.7)$$

$$C\mathbf{x}_0 + D\mathbf{u}_0 = \mathbf{0} \quad (8.8)$$

Fisicamente, se il velivolo viene eccitato con l'esatta proporzione di comandi dettata da $\mathbf{u}_0$ partendo dall'assetto $\mathbf{x}_0$, l'energia entra nel sistema alterando gli stati interni,

ma non si manifesta sui sensori in uscita.

8.1.5 Verifica in MATLAB: Estrazione degli Zeri Invarianti

Per dimostrare analiticamente la proprietà bloccante su uno specifico sottosistema sottoattuato (2×2) del modello longitudinale dell'F-16, è stato implementato uno script per estrarre gli zeri invarianti, calcolare il *Nullspace* della matrice di sistema $P(\lambda)$ ed eseguire una simulazione dinamica nel dominio del tempo.

L'estrazione computazionale ha restituito il seguente risultato vettoriale per gli zeri λ :

```

1   Zeri Invarianti calcolati per il sistema 2x2 (lambda):
2   -3.7283
3   5.7575

```

L'analisi computazionale rivela che il sottosistema possiede **due zeri invarianti reali e ben distinti**. Di particolare criticità ingegneristica è la presenza di uno zero posizionato nel semipiano destro ($\lambda = 5.7575$), noto come zero a fase non minima (instabile), oltre allo zero nel semipiano sinistro (stabile, $\lambda = -3.7283$).

Significato Fisico della Proprietà Bloccante Esponenziale ($\lambda \neq 0$)

La presenza di zeri reali non nulli all'interno di questa configurazione ridotta innesca la proprietà bloccante in forma dinamica. Poiché in questo caso $\lambda \neq 0$, l'eccitazione mantiene la sua forma originale esponenziale:

$$\mathbf{u}(t) = \mathbf{u}_0 e^{\lambda t} \quad (8.9)$$

Questo implica l'esistenza di una combinazione spaziale esatta di spinta propulsiva ed equilibratore governata da un andamento esponenziale ($\mathbf{u}_0 e^{\lambda t}$) che, se applicata a partire dallo specifico assetto iniziale $\mathbf{x}_0$, non produce alcuna variazione misurabile sulle uscite prese in esame (V_t e q). Scegliendo, ad esempio, lo zero instabile ($\lambda = 5.7575$), il comando cresce e diverge esponenzialmente nel tempo. Fisicamente, le variazioni delle forze aerodinamiche indotte dal piano di coda si annullano dinamicamente con il momento indotto dalla spinta al variare dell'assetto interno, raggiungendo un "equilibrio ombra" che blocca la trasmissione del comando e maschera la divergenza interna ai sensori di bordo.

La simulazione del sistema, inizializzato nello stato $\mathbf{x}_0$ e sollecitato da tale comando accoppiato divergente, produce un risultato ingegneristicamente notevole: a fronte di ingressi attivi ed esponenziali, i grafici delle uscite misurate mostrano un tracciato rigorosamente piatto e coincidente con lo zero ($\mathbf{y} \equiv \mathbf{0}$). Questo comportamento valida algebricamente e numericamente il Teorema della proprietà bloccante [10]. Infine, l'esecuzione del codice conferma che l'operazione di prodotto matrice-vettore per il calcolo del *Nullspace* restituisce un residuo con ordine di grandezza prossimo allo zero ($\approx 10^{-15}$). Ciò certifica la correttezza formale delle direzioni $\mathbf{x}_0$ e $\mathbf{u}_0$ individuate, confermando in modo definitivo l'esistenza di direzioni bloccanti per le configurazioni sottoattuate del velivolo.

8.2 Analisi dei poli, zeri e proprietà bloccanti

La sintesi di un sistema di controllo per la dinamica longitudinale dell'F-16 richiede un'attenta analisi preliminare delle proprietà strutturali dell'impianto. In particolare, prima di definire l'architettura di controllo, è fondamentale studiare la presenza di **Zeri di Trasmissione** e l'eventuale **Proprietà Bloccante** del sistema. Il modello in spazio di stato linearizzato originale presenta una matrice degli ingressi B che include sia gli attuatori fisici sia le componenti di disturbo atmosferico. Per valutare le reali capacità di comando del velivolo, si è proceduto a un'operazione di filtraggio degli ingressi: la matrice originaria è stata partizionata separando le due componenti vettoriali di disturbo dovute al vento (B_{Wind}) dalle tre componenti relative agli ingressi di controllo effettivi (B_{ctrl} , [la cui trattazione è approfondita nel Capitolo 10](#)). Eliminando l'effetto incontrollabile del vento e considerando la matrice B_{ctrl} , l'analisi si concentra sul sistema quadrato 3×3 formato dai tre attuatori longitudinali (Spinta, Equilibratore, Flap di bordo d'attacco) e dalle tre uscite prioritarie (V_t, α, q). L'estrazione degli zeri invarianti su questo sistema filtrato 3×3 porta a un risultato strutturale di grande rilievo: **il sistema non possiede zeri di trasmissione**. La matrice di sistema mantiene rango pieno per qualsiasi frequenza complessa s . Per la teoria dei sistemi, la presenza di uno zero invariante λ implica l'esistenza di una direzione bloccante, ovvero una specifica combinazione di ingressi $u(t) = u_0 e^{\lambda t}$ che, partendo da un'opportuna condizione iniziale, viene totalmente assorbita dalla dinamica interna, producendo un'uscita identicamente nulla. L'assenza

totale di zeri nel modello 3×3 in esame indica che il sistema è privo di direzioni bloccanti. Questo garantisce il raggiungimento di un'**autorità di controllo totale**: avendo saturato i gradi di libertà dell'aereo (tre attuatori per tre uscite principali), non esistono "movimenti nascosti" o frequenze cieche. Qualsiasi comando inviato agli attuatori si manifesterà inevitabilmente sulla dinamica longitudinale misurata. Sebbene l'assenza di zeri bloccanti o a fase non minima confermi l'eccellente governabilità strutturale dell'aereo, permane la problematica del forte accoppiamento dinamico (cross-talk) tra i canali. Ad esempio, una variazione della spinta altera non solo la velocità V_t , ma induce marcate variazioni parassite sul rateo di beccheggio q e sull'angolo di attacco α .

8.2.1 Controprova Computazionale: Analisi di un Sottosistema Sottoattuato (2×2)

Come ampiamente dimostrato, l'architettura completa 3×3 è priva di zeri di trasmissione. Per avvalorare ulteriormente questo risultato e validare l'importanza del terzo attuatore (LE Flap), è stata eseguita una controprova computazionale isolando uno specifico sottosistema *sottoattuato* (matrice 2×2), in cui si tenta di controllare V_t e q utilizzando unicamente Spinta ed Equilibratore. Privando il sistema di un grado di libertà di controllo, l'estrazione computazionale in MATLAB rileva immediatamente la comparsa di zeri invarianti:

```

1   Zeri Invarianti calcolati per il sistema 2x2 (lambda):
2   -3.7283
3   5.7575

```

A differenza del sistema completo, questo sottosistema 2×2 possiede **due zeri invarianti reali e ben distinti**. Risulta importante notare la presenza dello zero a fase non minima posizionato nel semipiano destro: $\lambda = 5.7575$.

La comparsa della Proprietà Bloccante Esponenziale

La presenza di zeri di trasmissione in questa configurazione ridotta innesca la proprietà bloccante. Poiché in questo caso $\lambda \neq 0$, il segnale d'ingresso bloccante non è una costante, ma assume la forma di un'eccitazione esponenziale:

$$\mathbf{u}(t) = \mathbf{u}_0 e^{\lambda t} \quad (8.10)$$

Questo risultato ha implicazioni fisiche profonde e pericolose. Scegliendo, ad esempio, lo zero instabile ($\lambda = 5.7575$), la teoria impone che esista una specifica combinazione vettoriale di Spinta ed Equilibratore ($\mathbf{u}_0$) che, applicata allo specifico assetto iniziale $\mathbf{x}_0$, genera un comando che **cresce e diverge esponenzialmente nel tempo**. Tuttavia, a causa del perfetto bilanciamento interno delle forze aerodinamiche indotto da questa specifica frequenza, la divergenza degli stati interni (come l'angolo di attacco α) viene totalmente mascherata sulle uscite. La simulazione del sottosistema sollecitato da tale ingresso produce infatti un tracciato delle uscite rigorosamente piatto ($\mathbf{y} \equiv \mathbf{0}$), validando algebricamente e numericamente il Teorema della proprietà bloccante [10]. In altre parole, il velivolo sta subendo una manovra interna divergente che lo porterebbe rapidamente allo stallo o al cedimento strutturale, ma i sensori di Velocità e Velocità di Beccheggio non rilevano alcuna variazione, rendendo l'instabilità invisibile al controllore.

La comparsa della Proprietà Bloccante Esponenziale ($\lambda \neq 0$)

La presenza di zeri all'interno di questa configurazione ridotta innesca la proprietà bloccante. Poiché gli zeri estratti sono valori reali e non nulli (ad esempio $\lambda = -3.7283$ o $\lambda = 5.7575$), l'eccitazione non degenera in una costante, ma mantiene la sua forma esponenziale:

$$\mathbf{u}(t) = \mathbf{u}_0 e^{\lambda t} \quad (8.11)$$

Questo implica che, senza il Flap, esiste una combinazione esatta e variabile di spinta ed equilibratore creata da un movimento esponenziale che applicata a partire da uno specifico assetto iniziale $\mathbf{x}_0$, non produce alcuna variazione misurabile su V_t e q . Le forze aerodinamiche indotte si annullano a vicenda dinamicamente, raggiungendo una condizione di equilibrio ombra che maschera totalmente la manovra ai sensori di bordo.

Simulazione Multipla della Proprietà Bloccante

Il sottosistema è stato sollecitato esplorando le tre casistiche fondamentali che definiscono geometricamente la proprietà bloccante degli zeri in un sistema multivariabile. I risultati della simulazione sono riportati in Figura 8.1:

- **Caso 1 (Condizione Ideale):** Fornendo al sistema l'esatto vettore d'ingresso $\mathbf{u}_0 e^{\lambda t}$ e partendo rigorosamente dallo stato iniziale calcolato $\mathbf{x}_0$, si assiste alla perfetta cancellazione tra risposta libera e forzata. Le uscite misurate risultano perfettamente nulle per tutto l'orizzonte temporale.
- **Caso 2 (Sensibilità allo Stato Iniziale):** Applicando il medesimo ingresso, ma forzando il sistema a partire dall'origine ($\mathbf{x}(0) = \mathbf{0}$), l'equilibrio viene meno. L'insorgere di transitori sulle uscite dimostra empiricamente che lo zero non sopprime passivamente le frequenze, ma richiede un preciso allineamento dello stato interno.
- **Caso 3 (Sensibilità alla Direzione d'Ingresso):** Partendo dall'assetto ideale $\mathbf{x}_0$, si è provveduto ad alterare leggermente la direzione del vettore di controllo. Come si evince dal grafico, la perdita di allineamento geometrico impedisce il mascheramento delle uscite, portando il sistema a divergere seppure di poco la proprietà bloccante degli zeri non è rispettata.

Questi risultati confermano che gli zeri di trasmissione in ambito MIMO possiedono un comportamento vettoriale, mostrando il legame che intercorre tra lo spettro delle frequenze alla geometria dello spazio di stato e degli ingressi. L'insorgere di queste direzioni bloccanti non appena si rinuncia al terzo attuatore dimostra, per contrasto, la necessità formale e pratica della configurazione 3×3 adottata per il presente progetto di controllo, che risulta essere l'unica in grado di scongiurare la presenza di "movimenti nascosti".

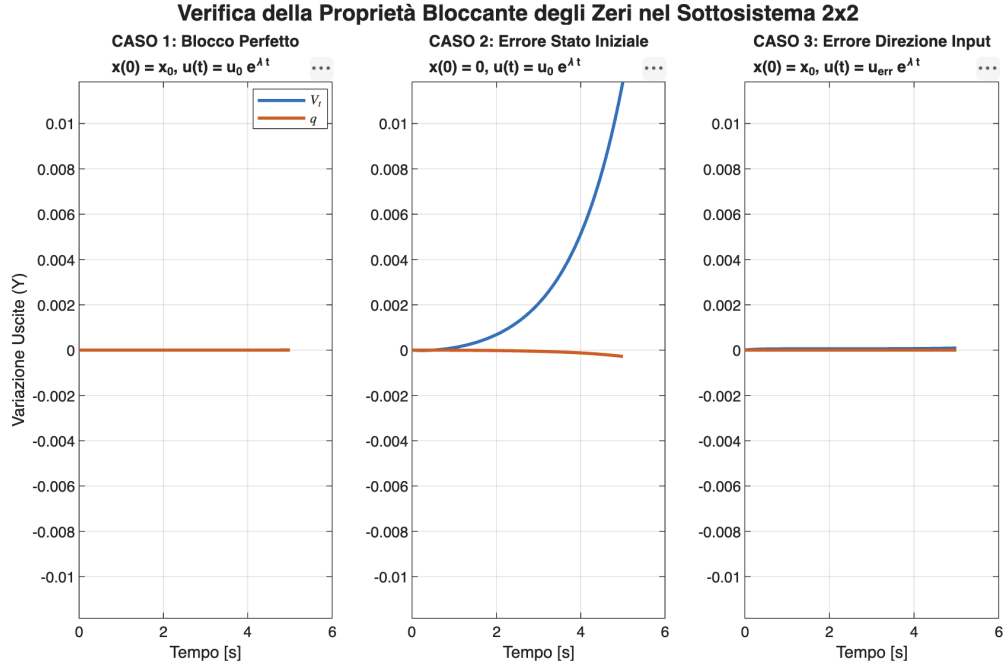


FIGURA 8.1: Verifica simulata della proprietà bloccante sul sottosistema 2×2 . Solo la perfetta sovrapposizione vettoriale tra ingresso e stato iniziale (Caso 1) garantisce l'annullamento delle uscite. Qualsiasi deviazione dallo stato geometrico ideale (Caso 2 e 3) compromette il bilanciamento delle forze interne.

8.3 Interazioni dinamiche e vincoli strutturali

L'esecuzione dell'algoritmo di linearizzazione e realizzazione minima sull'ambiente MATLAB ha prodotto la matrice $G_{min}(s)$ per il velivolo F-16 in condizione di trim.

Un aspetto notevole della realizzazione minima è l'ordine del denominatore comune. Il polinomio caratteristico del sistema ridotto risulta essere di quarto grado:

$$\Delta(s) = s^4 + 1.233s^3 - 1.511s^2 + 0.004179s - 0.02174 \quad (8.12)$$

Questo denota che il sistema ha preservato tutte e quattro le variabili di stato essenziali della dinamica longitudinale (θ, q, U, W) . Inoltre, la presenza di variazioni di segno nei coefficienti del polinomio caratteristico conferma analiticamente, secondo il **criterio di stabilità di Routh-Hurwitz** [11], l'intrinseca instabilità del velivolo

discussa nel capitolo precedente. Data la vastità della matrice completa (costituita da 5 ingressi e 6 uscite, per un totale di 30 funzioni di trasferimento SISO) e considerando la natura sottoattuata del sistema, nei prossimi capitoli si procederà alla selezione di uno specifico sottoinsieme di ingressi e di uscite, definiti rispettivamente dalle matrici B_{ctrl} e C_{ctrl} . Tale scelta risulta fondamentale per garantire l'effettivo controllo della dinamica longitudinale del velivolo, escludendo la componente dei disturbi atmosferici modellata nella matrice B_{wind} (la cui trattazione estesa è rimandata al [Cap.11](#)).

Capitolo 9

Sintesi del Controllo e Limiti del Disaccoppiamento

La dinamica longitudinale dell'F-16 presenta un elevato grado di accoppiamento. Contrariamente a quanto si potrebbe ipotizzare per velivoli da trasporto commerciali, l'impianto risulta fortemente accoppiato anche nella condizione di volo di crociera. Poiché l'ipotesi di un disaccoppiamento senza pre-compensazione non è possibile per governare l'impianto in sicurezza si tenterà pertanto l'approccio della pre-compensazione sulla matrice delle funzioni di trasferimento, al fine di valutare la fattibilità di semplici regolatori decentralizzati (PID indipendenti) da applicare direttamente agli attuatori fisici dell'aereo. L'approccio classico per la sintesi multivariabile prevede l'inserimento di una rete di **pre-compensazione dinamica in avanti** (*Dynamic Feedforward Decoupler*), con l'obiettivo di forzare la diagonalizzazione del sistema e restituire ai controllori dei canali virtuali isolati. Tuttavia, come verrà dimostrato nel prosieguo della trattazione, tale strategia si rivelerà inefficace a causa degli stringenti limiti fisici e numerici, rendendo necessaria l'adozione di tecniche di controllo più avanzate (analizzate nei Capitoli 10 e 11).

9.1 Progetto del pre-compensatore dinamico

L'obiettivo della rete di disaccoppiamento in avanti è determinare una matrice di pre-compensazione $W(s) \in \mathbb{C}^{3 \times 3}$ tale per cui l'impianto apparente $\tilde{G}(s) = G(s)W(s)$ risulti essere una matrice puramente diagonale. Sviluppando il prodotto matriciale tra l'impianto originale $G(s)$ e la matrice incognita $W(s)$, si ottiene:

$$\tilde{G}(s) = \begin{bmatrix} G_{11} & G_{12} & G_{13} \\ G_{21} & G_{22} & G_{23} \\ G_{31} & G_{32} & G_{33} \end{bmatrix} \begin{bmatrix} W_{11} & W_{12} & W_{13} \\ W_{21} & W_{22} & W_{23} \\ W_{31} & W_{32} & W_{33} \end{bmatrix} \quad (9.1)$$

Affinché $\tilde{G}(s)$ sia diagonale, tutti i termini extra-diagonali devono essere identicamente nulli. Questo impone un sistema di 6 equazioni (due zeri per ogni colonna) a fronte di 9 incognite (gli elementi della matrice W). Il sistema presenta quindi 3 gradi di libertà. Dal punto di vista ingegneristico, la scelta più logica è imporre che i canali di comando "diretti" passino inalterati attraverso il disaccoppiatore. Matematicamente, questo si traduce nell'imporre che gli elementi sulla diagonale principale di $W(s)$ siano unitari:

$$W_{11}(s) = W_{22}(s) = W_{33}(s) = 1 \quad (9.2)$$

Fisicamente, ciò significa che un comando virtuale unitario (es. cabrata) genera una richiesta unitaria sull'attuatore primario corrispondente (es. equilibratore), demandando agli elementi fuori diagonale il compito di generare le deflessioni correttive sugli altri attuatori per annullare il cross-talk.

Sfruttando questa condizione, è possibile scindere il problema in tre sistemi lineari indipendenti, uno per ciascuna colonna di $W(s)$.

9.1.1 Calcolo della Prima Colonna(Velocità)

Annullando i termini extra-diagonali della prima colonna di $\tilde{G}(s)$ ($\tilde{G}_{21} = 0$ e $\tilde{G}_{31} = 0$) e ponendo $W_{11} = 1$, si ottiene il sistema:

$$\begin{cases} G_{21}(1) + G_{22}W_{21} + G_{23}W_{31} = 0 \\ G_{31}(1) + G_{32}W_{21} + G_{33}W_{31} = 0 \end{cases} \quad (9.3)$$

Riscrivendo in forma matriciale per isolare le incognite W_{21} e W_{31} :

$$\begin{bmatrix} G_{22} & G_{23} \\ G_{32} & G_{33} \end{bmatrix} \begin{bmatrix} W_{21} \\ W_{31} \end{bmatrix} = \begin{bmatrix} -G_{21} \\ -G_{31} \end{bmatrix} \quad (9.4)$$

9.1.2 Calcolo della Seconda Colonna

Seguendo la medesima logica, si annullano i termini $\tilde{G}_{12}$ e $\tilde{G}_{32}$, ponendo $W_{22} = 1$:

$$\begin{bmatrix} G_{11} & G_{13} \\ G_{31} & G_{33} \end{bmatrix} \begin{bmatrix} W_{12} \\ W_{32} \end{bmatrix} = \begin{bmatrix} -G_{12} \\ -G_{32} \end{bmatrix} \quad (9.5)$$

9.1.3 Calcolo della Terza Colonna(Becheggio)

Infine, per annullare i termini $\tilde{G}_{13}$ e $\tilde{G}_{23}$, ponendo $W_{33} = 1$:

$$\begin{bmatrix} G_{11} & G_{12} \\ G_{21} & G_{22} \end{bmatrix} \begin{bmatrix} W_{13} \\ W_{23} \end{bmatrix} = \begin{bmatrix} -G_{13} \\ -G_{23} \end{bmatrix} \quad (9.6)$$

Risolvendo analiticamente (o numericamente in ambiente MATLAB) questi tre sistemi quadrati 2×2 , si ricavano le 6 funzioni di trasferimento incognite che, assieme agli 1 sulla diagonale, costituiscono la matrice di pre-compensazione esatta $W(s)$.

9.2 Limiti algoritmici: complessità polinomiale

Dal punto di vista puramente teorico, il prodotto $\tilde{G} = G \cdot W_{dinamico}$ restituisce una matrice con zeri perfetti al di fuori della diagonale principale. Tuttavia, la validazione numerica di questo impianto virtuale ha rivelato la totale inapplicabilità pratica di questa soluzione. Estrapolando i canali isolati (ad esempio il canale della velocità V_t rispetto al comando virtuale di spinta, o il rateo di beccheggio q rispetto al comando virtuale dell'equilibratore), si osserva una **esplosione dell'ordine del sistema**. A causa delle operazioni di inversione e moltiplicazione delle funzioni di trasferimento, l'algoritmo calcola il minimo comune multiplo dei denominatori e come riscontrato dai dati estratti in ambiente MATLAB, le funzioni di trasferimento disaccoppiate risultanti generano polinomi a numeratore e denominatore che **superano il 50° grado** (fino a s^{52}).

Inoltre, i coefficienti dei polinomi hanno un intervallo di magnitudo non realistico variando da ordini di grandezza pari a 10^{10} fino a residui di 10^{-30} . Questo fenomeno genera tre criticità letali per il controllo del volo:

1. **Cancellazioni Imperfette:** Forzare l'annullamento dei termini incrociati comporta la creazione di decine di coppie polo-zero coincidenti. Dal punto di vista numerico (floating-point), la cancellazione non è mai perfetta, lasciando nel sistema "modi nascosti" e micro-dinamiche ad altissima frequenza.
2. **Poli Instabili Non di Fase Minima:** L'inversione della matrice aerodinamica introduce sistematicamente radici nel semipiano destro del piano complesso, rendendo l'impianto a ciclo aperto virtuale intrinsecamente instabile.

3. **Fallimento della Sintesi PID:** Il tentativo di sintetizzare regolatori Proporzionali-Integrali-Derivativi (PID) su canali di ordine 52 si è rivelato matematicamente impossibile. Gli algoritmi di *autotuning* (come il PID Tuner di Simulink) collassano di fronte al malcondizionamento numerico, non riuscendo a identificare coefficienti in grado di stabilizzare l'anello chiuso e restituendo verdeti di instabilità strutturale, indipendentemente dai parametri scelti.

9.3 Evoluzione verso il controllo nello spazio di stato

L'impossibilità di estrarre canali SISO governabili decreta il fallimento dell'architettura a disaccoppiamento in avanti e controllo decentralizzato. Costringere l'aereo a cancellare la sua stessa fisica aerodinamica richiederebbe peraltro un'energia e una deflessione degli attuatori che superano ampiamente le saturazioni fisiche di bordo. Per garantire la stabilità e le prestazioni richieste dal *Flight Control System*, è necessario un drastico cambio di paradigma. Anziché tentare di mascherare l'accoppiamento operando manipolazioni su matrici di funzioni di trasferimento, la tesi sposterà il focus su architetture native per sistemi MIMO, operanti interamente nel dominio dello **Spazio di Stato** (tramite matrici A, B, C, D).

Come verrà discusso nei capitoli successivi, il controllo dell'impianto verrà raggiunto con successo tramite due tecniche avanzate:

- **Regolatore Lineare Quadratico (LQR) - (Cfr. Capitolo 10):** Una tecnica di controllo ottimo a retroazione di stato completo che non forza zeri irrealizzabili, ma gestisce organicamente l'accoppiamento posizionando in modo globale i poli a ciclo chiuso, ottimizzando contemporaneamente l'errore di *tracking* e lo sforzo energetico degli attuatori.
- **Model Predictive Control (MPC) - (Cfr. Capitolo 11):** Un'architettura predittiva introdotta per gestire attivamente le interazioni multivariabili nel rispetto rigoroso dei vincoli di inviluppo e delle saturazioni meccaniche, prevenendo il degrado prestazionale alle alte incidenze.

Capitolo 10

Controllo LQR (Linear Quadratic Regulator)

10.1 Sintesi del regolatore

A differenza del classico approccio Proporzionale-Integrale-Derivativo (PID) — il quale opera tipicamente in retroazione sull'uscita (*Output Feedback*), misurando l'errore tra il riferimento e la variabile effettivamente controllata — la logica LQR si basa sulla retroazione completa di stato (*Full State Feedback*). Questo significa che il controllore non si limita a osservare una singola uscita $y(t)$, ma sfrutta l'informazione simultanea di tutte le variabili dinamiche del sistema $\delta\mathbf{x}(t)$ per calcolare l'azione di controllo ottima. Tale approccio multivariabile risulta particolarmente indicato per le dinamiche di volo, in quanto permette di gestire matematicamente e in modo naturale gli accoppiamenti tra i vari stati (come la forte interazione tra velocità, angolo d'attacco e beccheggio). Un passaggio cruciale nella progettazione pratica del sistema consiste nel separare gli ingressi fisicamente comandabili dai disturbi esterni. La dinamica longitudinale completa del velivolo è infatti descritta dall'equazione:

$$\delta\dot{\mathbf{x}} = A_{long}\delta\mathbf{x} + B_{ctrl}\delta\mathbf{u} + B_{wind}\delta\mathbf{d} \quad (10.1)$$

dove B_{ctrl} rappresenta la matrice di ingresso associata esclusivamente ai veri attuatori fisici a disposizione del sistema (es. manetta ed equilibratore), mentre B_{wind} mappa l'effetto dei disturbi atmosferici, come le raffiche di vento $\delta\mathbf{d}$. Poiché l'algoritmo di controllo non ha autorità sull'atmosfera circostante, la sintesi dell'ottimizzatore LQR deve essere effettuata considerando **esclusivamente** la sottomatrice B_{ctrl} . L'inclusione del termine del vento nel calcolo ottico porterebbe infatti alla genesi di un controllore illogico, che tenterebbe di usare il disturbo come se fosse una superficie di governo.

La legge di controllo retroazionata assume pertanto la forma:

$$\delta \mathbf{u}(t) = -K \delta \mathbf{x}(t) \quad (10.2)$$

dove K rappresenta la matrice dei guadagni ottimi riferita unicamente agli attuatori fisici.

10.1.1 Orizzonte Infinito e Indice di Costo

L'obiettivo del controllo LQR è determinare la matrice K che minimizzi un preciso indice di prestazione (o funzione di costo) quadratico. Poiché lo scopo in questa fase (il mantenimento di un punto di trim in volo stazionario) è garantire la stabilità asintotica del velivolo a tempo indeterminato, il problema di ottimizzazione viene formulato su un **orizzonte temporale infinito** ($t_f \rightarrow \infty$). La funzione di costo associata è:

$$J = \int_0^\infty (\delta \mathbf{x}^T Q \delta \mathbf{x} + \delta \mathbf{u}^T R \delta \mathbf{u}) dt \quad (10.3)$$

Il cuore della progettazione risiede nella scelta delle matrici di ponderazione simmetriche Q (semidefinita positiva) ed R (definita positiva), che rappresentano il compromesso diretto tra le prestazioni e l'energia spesa:

- **Matrice Q (Ponderazione degli Stati):** Regola l'*aggressività* del controllo. Aumentare i pesi sulla diagonale obbliga il sistema a penalizzare pesantemente le deviazioni di quello specifico stato dal punto di equilibrio.
- **Matrice R (Ponderazione degli Ingressi):** Regola il *risparmio energetico*. Aumentare i valori penalizza l'uso degli attuatori, prevenendo saturazioni e riducendo l'usura meccanica delle superfici di governo.

10.1.2 L'Equazione Algebrica di Riccati

Per la creazione di un LQR per il controllo di un sistema si sceglie la formulazione di un orizzonte temporale infinito che applicato al sistema linearizzato tempo-invariante (LTI) permette di ottenere una matrice dei guadagni ottimi K risulta **stazionaria** (invariante nel tempo).

La teoria del controllo ottimo dimostra che la soluzione per minimizzare l'indice J si ottiene individuando l'unica matrice P , simmetrica e definita positiva, che soddisfa l'Equazione Algebrica di Riccati stazionaria (ARE - *Algebraic Riccati Equation*):

$$A_{long}^T P + P A_{long} - P B_{ctrl} R^{-1} B_{ctrl}^T P + Q = 0 \quad (10.4)$$

Una volta risolta numericamente l'equazione per ricavare P , la matrice dei guadagni stabilizzante K si calcola analiticamente tramite la relazione:

$$K = R^{-1} B_{ctrl}^T P \quad (10.5)$$

Sostituendo la legge di controllo ottima $\delta \mathbf{u} = -K \delta \mathbf{x}$ nel sistema lineare originale (e ipotizzando per l'analisi di stabilità nominale un'assenza di raffiche $\delta \mathbf{d} = 0$), l'evoluzione dell'aereo è governata dalla nuova dinamica a ciclo chiuso:

$$\delta \dot{\mathbf{x}} = (A_{long} - B_{ctrl} K) \delta \mathbf{x} = A_{cl} \delta \mathbf{x} \quad (10.6)$$

Le proprietà di stabilità globale di questa nuova dinamica sono matematicamente assicurate dal Teorema di Ljapunov, e la dimostrazione deriva in modo elegante dal Principio di Ottimalità di Bellman. Nello specifico, la funzione di costo ottimo, valutata a partire da uno stato generico, assume la forma quadratica definita proprio dalla matrice P :

$$V(\delta \mathbf{x}) = \delta \mathbf{x}^T P \delta \mathbf{x} \quad (10.7)$$

Essendo la matrice P simmetrica e definita positiva (per costruzione dell'Equazione Algebrica di Riccati), la funzione $V(\delta \mathbf{x})$ gode della proprietà fondamentale di essere **definita positiva**: è sempre maggiore di zero per ogni perturbazione $\delta \mathbf{x} \neq 0$ e si annulla solo nell'origine. Questo la rende la perfetta candidata come funzione di Ljapunov, paragonabile all'energia "virtuale" associata all'errore del sistema. Applicando l'equazione di Bellman (nella sua formulazione differenziale continua di Hamilton-Jacobi-Bellman), derivando $V(\delta \mathbf{x})$ rispetto al tempo lungo le traiettorie del sistema, si ottiene:

$$\dot{V}(\delta \mathbf{x}) = \delta \dot{\mathbf{x}}^T P \delta \mathbf{x} + \delta \mathbf{x}^T P \delta \dot{\mathbf{x}} = -\delta \mathbf{x}^T (Q + K^T R K) \delta \mathbf{x} \quad (10.8)$$

Poiché le matrici di peso Q e R sono rispettivamente semidefinita positiva e definita positiva, l'espressione risultante dimostra che la derivata $\dot{V}(\delta\mathbf{x})$ è strettamente negativa. Il decadimento monotono e continuo di questa funzione certifica matematicamente l'**asintotica stabilità** dell'anello chiuso: indipendentemente dalla perturbazione iniziale, le dinamiche di Corto Periodo e Phugoide verranno smorzate e il velivolo convergerà inesorabilmente verso il punto di equilibrio desiderato ($\delta\mathbf{x} = 0$).

10.2 Ottimizzazione delle performance tramite la Regola di Bryson

Per valutare la robustezza del controllore ottimo e la sua capacità di riportare il sistema in equilibrio da diverse configurazioni di volo, tutte le simulazioni e i grafici mostrati di seguito (dinamiche temporali, ritratti di fase e traiettorie sulle superfici di Lyapunov) partono da un set multiplo di condizioni iniziali. Nello specifico, è stata imposta una matrice $\mathbf{X}_{0,\text{mult}}$ composta da otto diversi vettori di stato perturbati. Ogni colonna della matrice rappresenta un vettore di stato iniziale $\mathbf{x}_0 = [\theta, q, u, w]^T$:

$$\mathbf{X}_{0,\text{mult}} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1.2 & -1.2 & 0.5 & -0.5 & 0.8 & -0.8 & 0.3 & -0.3 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 20 & -20 & 15 & -15 & -10 & 10 & 22 & -22 \end{bmatrix} \quad (10.9)$$

Tale set simula una serie di transitori longitudinali estremi, caratterizzati da scostamenti severi e simmetrici sulla velocità di beccheggio (q in rad/s) e sulla velocità verticale (w in ft/s), mantenendo inizialmente nulli gli scostamenti di angolo di beccheggio (θ) e velocità orizzontale (u). A differenza delle tecniche predittive avanzate (che gestiscono esplicitamente i vincoli di disuguaglianza), il regolatore LQR standard non ha consapevolezza dei limiti fisici di inviluppo o delle saturazioni meccaniche degli attuatori. Il rispetto dei margini di sicurezza deve essere garantito *implicitamente* attraverso un'opportuna taratura dell'indice di costo. Di seguito vengono analizzati tre scenari di sintesi per evidenziare l'impatto dei pesi Q ed R sulle prestazioni a ciclo chiuso, partendo sempre dal set di condizioni iniziali $\mathbf{X}_{0,\text{mult}}$, i grafici mostrati si basano sulle condizioni della prima colonna della matrice

di condizioni $\mathbf{X}_{0,\text{mult}}$.

10.2.1 Il problema delle scale e l'inadeguatezza della matrice identità

In una prima fase esplorativa, è comune impostare le matrici di costo come proporzionali alla matrice identità, ad esempio $Q = \rho I$. Tuttavia, questa scelta si rivela inadeguata per sistemi fisici complessi come il velivolo F-16, in cui le variabili di stato presentano ordini di grandezza, unità di misura e limiti fisici profondamente eterogenei.

Considerando il vettore di stato longitudinale dell'F-16, costituito da $x = [\theta, q, U, W]^T$, si osserva una marcata disparità numerica tra le variabili:

- Il rateo di beccheggio q è espresso in radianti al secondo (rad/s). Una variazione di 1 rad/s corrisponde a circa $57^\circ/\text{s}$, che per un velivolo rappresenta una manovra estremamente violenta, prossima ai limiti strutturali.
- La velocità verticale W è espressa in piedi al secondo (ft/s). Una variazione di 1 ft/s è trascurabile, e durante i transienti aerodinamici è comune osservare scostamenti di 20 o 30 ft/s.

Avendo la matrice Q identità per la penalizzazione degli stati il controllore assegna lo stesso "costo" unitario a un errore di 1 rad/s su q e a un errore di 1 ft/s su W . Di conseguenza, di fronte a un normale transiente aerodinamico in cui $W = 20$ ft/s, il controllore percepisce un errore enorme ($20^2 \times 1 = 400$) rispetto a un gravissimo errore angolare di $q = 0.5$ rad/s ($0.5^2 \times 1 = 0.25$). Il risultato è una legge di controllo totalmente sbilanciata e non realistica, che satura gli attuatori per annullare innocue variazioni di velocità, prestando scarsa attenzione alle dinamiche reali dell'aereo. Questo sbilanciamento è evidente anche analizzando la funzione di Lyapunov a ciclo chiuso $V(x) = x^T P x$. In assenza di normalizzazione, la superficie di energia associata alla matrice P risulta deformata a forma di "valle" o cilindro parabolico lungo l'asse delle velocità, poiché i termini quadratici della velocità dominano numericamente su quelli angolari.

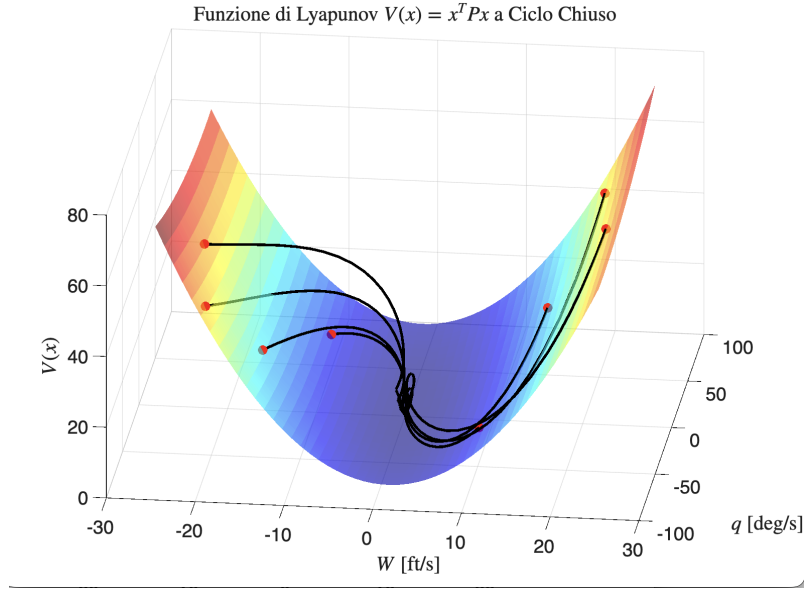


FIGURA 10.1

10.2.2 Applicazione della Regola di Bryson

Per avere un controllo realistico per la fisica dell' aereo il controllore adotta la **Regola di Bryson**[2]. Questo approccio prevede di costruire le matrici Q ed R come matrici diagonali, i cui elementi non nulli sono inversamente proporzionali al quadrato della massima variazione tollerabile per la rispettiva variabile:

$$Q_{ii} = \frac{1}{(x_{i,\max})^2}, \quad R_{jj} = \frac{1}{(u_{j,\max})^2} \quad (10.10)$$

quindi quando una variabile di stato o di controllo raggiunge il proprio limite fisico massimo il suo contributo all'indice di costo J è normalizzato esattamente a 1. Sulla base delle caratteristiche fisiche dell'F-16, vengono utilizzati i seguenti limiti per le variabili di stato del controllo longitudinale:

- Angolo di beccheggio (θ): $\theta_{\max} = 15^\circ \approx 0.2618 \text{ rad}$
- Rateo di beccheggio (q): $q_{\max} = 30^\circ/\text{s} \approx 0.5236 \text{ rad/s}$
- Velocità asse X (U): $U_{\max} = 30 \text{ ft/s}$
- Velocità asse Z (W): $W_{\max} = 20 \text{ ft/s}$

Dunque, la matrice Q che si ottiene risulta:

$$Q = \text{diag} \left(\frac{1}{(0.2618)^2}, \frac{1}{(0.5236)^2}, \frac{1}{(30)^2}, \frac{1}{(20)^2} \right) \quad (10.11)$$

In modo analogo il vettore di controllo $u = [\delta_{\text{th}}, \delta_e, \delta_{\text{lef}}]^T$, i limiti fisici degli attuatori impongono:

- Variazione di Spinta (δ_{th}): $\text{Thrust}_{\text{max}} = 5000 \text{ lbs}$
- Equilibratore (δ_e): $\text{Elev}_{\text{max}} = 25^\circ \approx 0.4363 \text{ rad}$
- Flap bordo d'attacco (δ_{lef}): $\text{LEF}_{\text{max}} = 25^\circ \approx 0.4363 \text{ rad}$

La matrice di peso sulle azioni di controllo R diviene quindi:

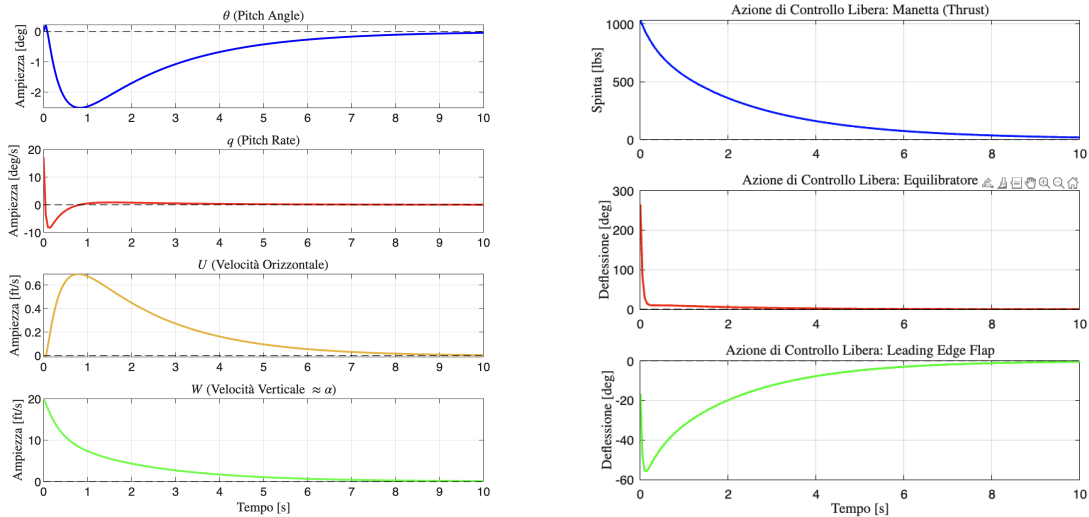
$$R = \text{diag} \left(\frac{1}{(5000)^2}, \frac{1}{(0.4363)^2}, \frac{1}{(0.4363)^2} \right) \quad (10.12)$$

10.2.3 Conclusioni sulla sintesi LQR

L'adozione della Regola di Bryson ha permesso di ricondurre il problema matematico dell'ottimizzazione quadratica a un significato puramente fisico e più realistico. Il controllore LQR così sintetizzato distribuisce l'energia di controllo in maniera ottimale, reagendo in modo bilanciato sia alle perturbazioni aerodinamiche lente che ai moti transitori veloci di beccheggio. La conferma visiva nel ritratto di fase della funzione di Lyapunov $V(x) = x^T P x$. Con la normalizzazione, la mappa di energia nello spazio degli stati assume la conformazione teorica di un paraboloide, confermando che l'algoritmo sta valutando e penalizzando l'energia del sistema tenendo conto della fisica dell'aereo e delle diverse unità di misura delle variabili di stato.

10.2.4 Prestazioni Aggressive Con Matrice Q dominante

Sbilanciando la taratura a favore della matrice Q (*State Penalty*), la priorità del controllore è azzerare l'errore degli stati nel minor tempo matematicamente possibile. La richiesta di un rientro immediato genera comandi estremamente repentini. In questo scenario emerge la criticità dell'LQR: richiedendo un'azione di controllo troppo aggressiva per rimettere l'aereo in asse, i comandi teorici generati dal guadagno K rischiano di superare ampiamente le reali capacità fisiche delle superfici aerodinamiche, mandando gli attuatori in saturazione e degradando la stabilità reale del velivolo. Il comportamento dinamico è illustrato nella Figura 10.2.

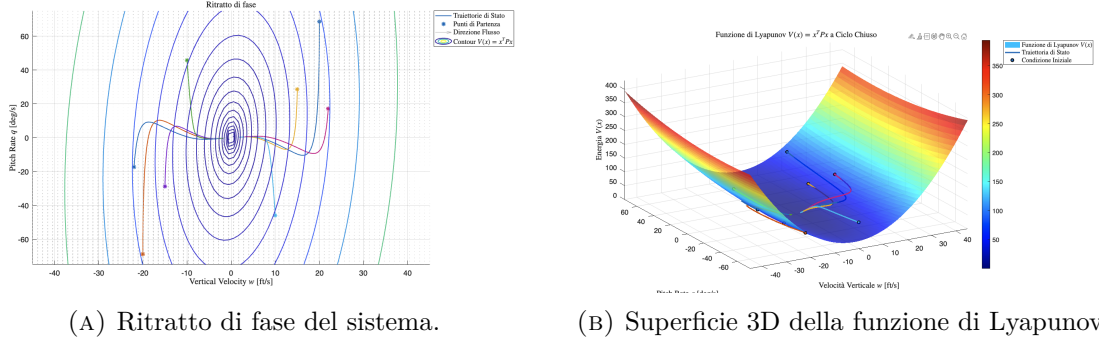


(A) Dinamica degli stati a ciclo chiuso.

(B) Sforzo di controllo richiesto agli attuatori.

FIGURA 10.2: Risposta del sistema con $Q \gg R$. (a) Il rientro è immediato e aggressivo, (b) ma genera picchi di comando che possono causare la saturazione degli attuatori.

Valutando l'analisi dinamica si nota che il comportamento del sistema può essere valutato attraverso il ritratto di fase e la forma quadratica della funzione ottima di Lyapunov utilizzata, definita come $V(x) = x^T P x$ (dove P è la soluzione dell'Equazione Algebrica di Riccati), quindi i grafici che vengono plottati cambieranno in base a come vengono penalizzate le matrici Q ed R . In questo scenario aggressivo, la superficie 3D di Lyapunov si presenta particolarmente ripida e scavata, riflettendo l'elevato costo associato alla deviazione degli stati dall'equilibrio. Il ritratto di



(A) Ritratto di fase del sistema.

(B) Superficie 3D della funzione di Lyapunov.

FIGURA 10.3: Analisi geometrica con $Q \gg R$. La ripidezza della funzione di Lyapunov (b) forza le traiettorie di fase (a) a una convergenza dalle condizioni iniziali verso l'equilibrio.

fase evidenzia traiettorie che precipitano verso l'origine con pendenze elevate: la convergenza è estremamente rapida, ma la geometria delle curve conferma l'assenza di smorzamento graduale, sintomo della forte richiesta energetica che il sistema richiede. Valutando l'analisi dinamica si nota che el comportamento del sistema può essere valutato attraverso el ritratto di fase e la forma quadratica della funzione ottima di Lyapunov utilizzata, definita come $V(x) = x^T P x$ (dove P è la soluzione dell'Equazione Algebrica di Riccati), quindi i grafici che vengono plottati cambieranno in base a come vengono penalizzate le matrici Q ed R . In questo scenario aggressivo, la superficie 3D di Lyapunov si presenta particolarmente ripida e scavata, riflettendo l'elevato costo associato alla deviazione degli stati dall'equilibrio. El ritratto di fase evidenzia traiettorie che precipitano verso l'origine con pendenze elevate: la convergenza è estremamente rapida, ma la geometria delle curve conferma l'assenza di smorzamento graduale, sintomo della forte richiesta energetica che el sistema richiede.

10.2.5 Compromesso Ottimo Con Matrici Q ed R bilanciate

Per ottenere un comportamento realistico e applicabile in volo, è necessario individuare un bilanciamento tra le due matrici ($Q \approx R$, al netto delle dovute normalizzazioni di scala). In questo scenario di compromesso, l'LQR fornisce la sua prestazione ottimale: garantisce un rientro sufficientemente rapido dell'assetto e lo smorzamento delle oscillazioni (tipiche del modo Phugoide e Corto Periodo), mantenendo contemporaneamente le richieste di deflessione dell'equilibratore entro limiti operativi sicuri. Questo set di guadagni, analizzato nella Figura 10.4, rappresenta la taratura nominale scelta per el controllore a ciclo chiuso.

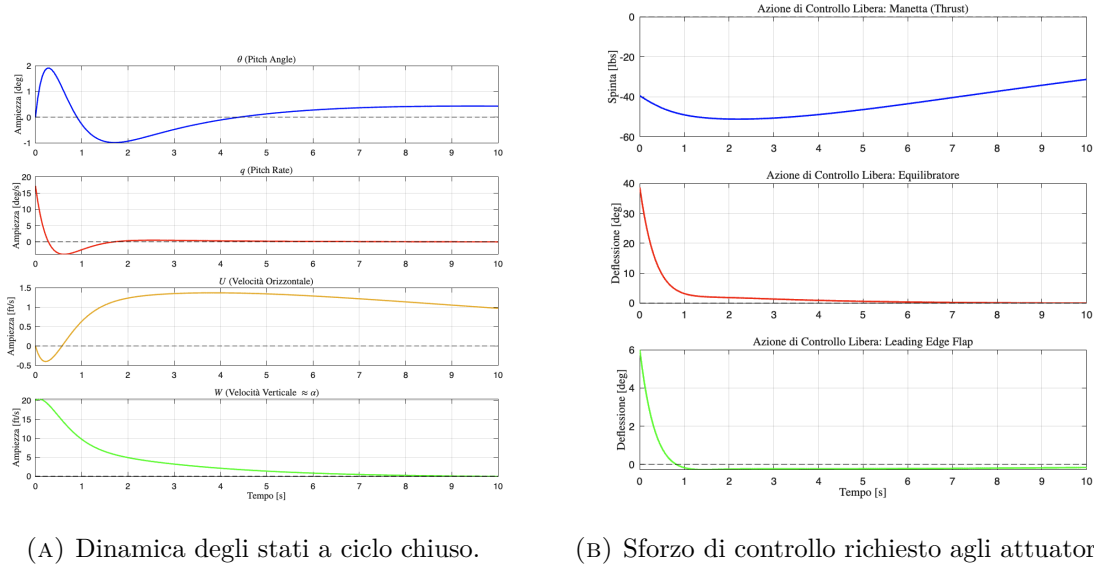
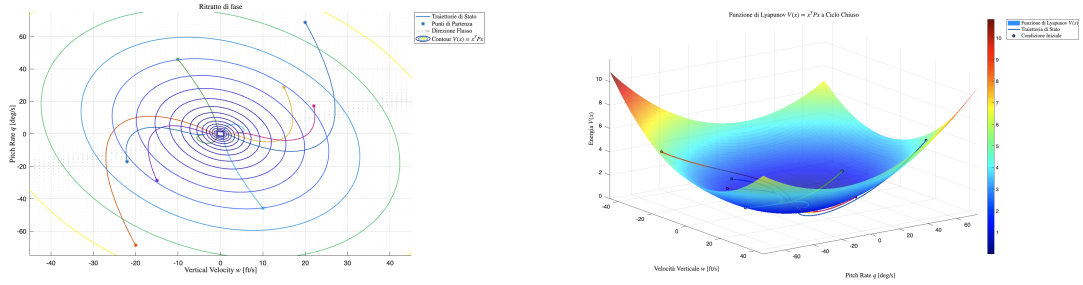


FIGURA 10.4: Risposta del sistema con $Q \approx R$. El controllore offre un ottimo compromesso tra rapidità di reiezione del disturbo (a) e contenimento dello sforzo di controllo (b).

L'efficacia di questo compromesso si riflette in modo evidente nella topologia dello spazio degli stati. Osservando la funzione di Lyapunov per el caso bilanciato, si nota una conformazione della superficie 3D più armoniosa e ben proporzionata. La discesa delle traiettorie di stato lungo la "conca" definita da $V(x)$ avviene in modo fluido e diretto verso el punto di equilibrio. El ritratto di fase conferma l'ottimo smorzamento introdotto dal controllore: le traiettorie spiraleggiano o convergono stabilmente verso

l'origine senza le deviazioni brusche del caso precedente, garantendo stabilità senza stressare l'aerodinamica del velivolo.



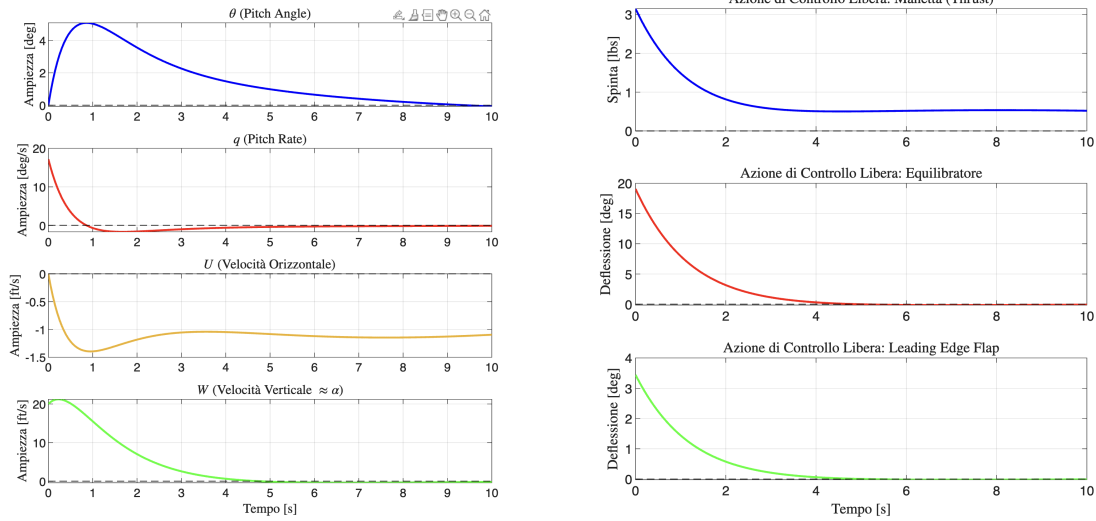
(A) Ritratto di fase del sistema.

(B) Superficie 3D della funzione di Lyapunov.

FIGURA 10.5: Analisi geometrica con $Q \approx R$. La conformazione bilanciata della superficie di Lyapunov (b) garantisce traiettorie di fase (a) fluide e ben smorzate verso l'origine.

10.2.6 Risparmio Energetico Con Matrice R dominante

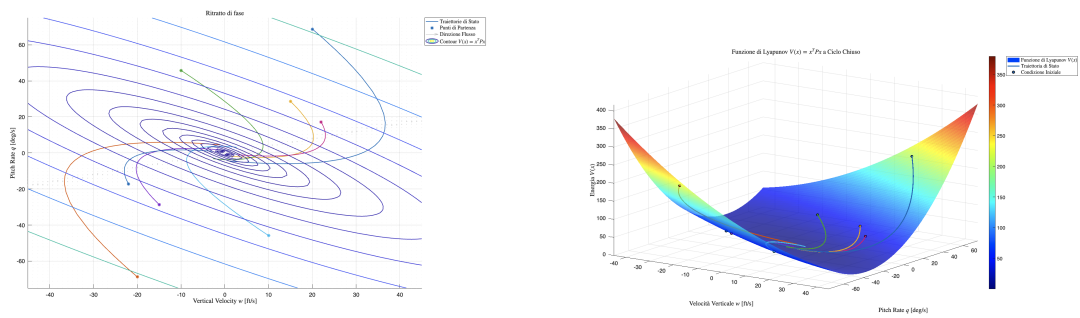
All'estremo opposto, impostando valori elevati sulla diagonale della matrice R (*Control Penalty*), l'algoritmo di ottimizzazione penalizza fortemente l'utilizzo degli attuatori rispetto all'errore di stato. La priorità del solutore è il risparmio di energia di manovra: l'azione di controllo dell'equilibratore risulta dolce e priva di picchi. Di conseguenza, il controllore accetta un transitorio molto più lungo per riportare l'aereo nel punto di equilibrio, lasciando decadere le perturbazioni gradualmente ed evitando qualsiasi rischio di saturazione, a discapito della prontezza di risposta. I risultati sono riportati nella Figura 10.6. Coerentemente con la penalizzazione dello sforzo di controllo, la topologia del sistema subisce una variazione significativa. La predominanza della matrice R si traduce in una funzione ottima di Lyapunov molto "piatta" e con pendenze dolci. Poiché il costo associato all'errore di stato è marginale, il solutore permette alle traiettorie nel ritratto di fase di compiere ampie escursioni prima di chiudersi sull'origine. La discesa lungo la superficie 3D avviene con estrema lentezza: il sistema scivola dolcemente verso il punto operativo, privilegiando la totale minimizzazione dello sforzo di deflessione rispetto alla prontezza aerodinamica.



(A) Dinamica degli stati a ciclo chiuso.

(B) Sforzo di controllo richiesto agli attuatori.

FIGURA 10.6: Risposta del sistema con $R \gg Q$. L'azione di controllo è moderata ed energeticamente efficiente (b), ma il rientro al punto operativo risulta molto più lento (a).



(A) Ritratto di fase del sistema.

(B) Superficie 3D della funzione di Lyapunov.

FIGURA 10.7: Analisi geometrica con $R \gg Q$. La superficie di Lyapunov risulta quasi piatta (b), permettendo alle traiettorie di fase (a) ampie escursioni e una convergenza molto lenta.

Capitolo 11

Controllo Predittivo (MPC)

11.1 Principi del Receding Horizon

Il Model Predictive Control (MPC) è controllo avanzato in cui si ottiene l'azione di controllo ottimale risolvendo ad ogni istante di campionamento k , un problema di ottimizzazione in anello aperto basato sulla previsione del comportamento futuro del sistema. A differenza delle tecniche di controllo in retro azione dello stato (come LQR Cap.10), in cui la legge di controllo è calcolata esplicitamente offline ovvero a priori, l'MPC invece lavora on-line con un algoritmo che valuta un'intera sequenza ottima di ingressi futuri. Applica così al sistema solo il primo elemento di tale sequenza e all'istante successivo $(k + 1)$, viene acquisita una nuova misura dello stato e il problema di ottimizzazione viene risolto nuovamente, traslando l'orizzonte temporale in avanti di un passo. Questo meccanismo, noto come *Receding Horizon*, è fondamentale per garantire la chiusura dell'anello di controllo (feedback) e compensare eventuali disturbi che possono essere presenti nel sistema che si osserva.

11.1.1 Formulazione del Problema di Ottimizzazione

Il problema di ottimizzazione dell'MPC nasce dal problema di dovere guidare lo stato del sistema verso l'origine minimizzando un funzionale di costo. Per un sistema a tempo discreto del tipo $x(k + 1) = f(x(k), u(k))$, la cifra di merito è definita su un orizzonte di predizione di lunghezza N (passi):

$$J(x(k), u(\cdot)) = \sum_{j=0}^{N-1} l(x(j), u(j)) + V_f(x(N)) \quad (11.1)$$

dove $l(x, u)$ rappresenta il costo di stadio (definito positivo e nullo solo nell'origine) e $V_f(x(N))$ rappresenta il costo terminale.

Il problema di ottimizzazione che il controllore deve risolvere ad ogni istante k è:

$$\begin{aligned}
 \min_u \quad & J(x(k), u(\cdot)) \\
 \text{s.t.} \quad & x(0) = x(k) \\
 & x(j+1) = f(x(j), u(j)) \\
 & x(j) \in \mathcal{X}, \quad u(j) \in \mathcal{U} \\
 & x(N) \in \mathbb{X}_f
 \end{aligned} \tag{11.2}$$

I vincoli impongono il rispetto della dinamica del sistema, i limiti fisici sugli stati $\mathcal{X}$ e sugli attuatori $\mathcal{U}$ (che si assumono essere insiemi compatti, ovvero chiusi e limitati, e forzano lo stato finale a terminare in un opportuno insieme invariante $\mathbb{X}_f$).

11.1.2 Varianti del Vincolo Terminale

La stabilità, come dimostrato, richiede di forzare lo stato N -esimo in una regione in cui le proprietà della CLF sono valide, ovvero quando viene applicata l'ultima azione di controllo bisogna assicurare di trovarsi in una posizione in cui essa riesca a riportare il sistema al set-point. Esistono due formulazioni principali per creare il problema di ottimizzazione dell' mpc:

Vincolo di Uguaglianza $x(N) = 0$

In questa formulazione alternativa usata principalmente utilizzata per sistemi non lineari e non linearizzabili intorno a un punto di equilibrio dove non si riesce a calcolare una CLF di controllo, il problema di ottimizzazione viene vincolato affinché lo stato raggiunga esattamente l'origine al termine dell'orizzonte predittivo ($x(N) = 0$). In questo modo, l'insieme terminale degenera in un punto $\mathbb{X}_f = \{0\}$ e non è necessario calcolare esplicitamente alcuna CLF. Tuttavia, richiedere di annullare l'errore in esattamente N passi restringe drasticamente il dominio di attrazione del controllore, riducendo l'insieme delle condizioni iniziali per cui l'ottimizzazione ammette soluzione quindi diventano di numero minore le soluzioni al problema di ottimizzazione che l' mpc può trovare.

Vincolo di Disuguaglianza $x(N) \in \mathbb{X}_f$

Questo vincolo è quello normalmente usato nei sistemi lineari e linearizzati, l'implementazione mpc che segue infatti impone che lo stato finale entri all'interno di una regione di sicurezza (il Maximal Invariant Set $\mathcal{O}_\infty$). Ciò permette al controllore di avere un bacino di attrazione (o Dominio di Attrazione $\mathbb{X}_N$, 11.5.1 molto più ampio, dando così la possibilità all' mpc di trovare ad ogni passo k una soluzione al problema di ottimizzazione riportando lo stato nel sistema nella regione di convergenza del Control-Invariant-Set dove può essere applicata la legge di controllo locale dell'LQR $u = -Kx$.

11.2 Stabilità e teoremi di Lyapunov

L'architettura del Model Predictive Control, per sua natura, agisce in anello aperto calcolando un'intera sequenza di azioni ottime, ma chiude il ciclo di retroazione (*Feedback di stato*) applicando al sistema esclusivamente il primo elemento della sequenza, seguendo il principio del *Receding Horizon*. Come per ogni strategia di controllo, è fondamentale progettare il regolatore in modo da garantire formalmente che il sistema in anello chiuso sia **asintoticamente stabile**. Per dimostrare questa proprietà, ci si avvale del Teorema di Lyapunov.

11.2.1 Teorema di Lyapunov

Si consideri un sistema non lineare a tempo discreto del tipo $x(k+1) = f(x(k), u(k))$, assunto per semplicità in forma autonoma $x(k+1) = f(x(k))$, con $f(x)$ continua e $\bar{x} = 0$ un punto di equilibrio. Il teorema afferma che, se esiste una funzione continua $V(x)$, definita positiva in $\bar{x}$, tale per cui la sua variazione rispetto alla traiettoria del sistema $\Delta V(x)$ sia semidefinita o definita negativa, ovvero:

$$\Delta V(x) = V(f(x)) - V(x) \leq 0 \quad (11.3)$$

allora l'origine $\bar{x} = 0$ è un punto di equilibrio stabile del sistema. Inoltre, se $\Delta V(x) < 0$, l'equilibrio risulta **asintoticamente stabile**.

11.2.2 L'Ipotesi Base di Stabilità nell'MPC

Senza perdita di generalità, si consideri il problema di regolazione dello stato all'origine ($\bar{x} = 0$). Il problema di ottimizzazione dell'MPC prevede la minimizzazione della seguente cifra di merito:

$$J(x(k), u(\cdot)) = \sum_{j=0}^{N-1} \ell(x(j), u(j)) + V_f(x(N)) \quad (11.4)$$

soggetta ai vincoli sul modello $x(j+1) = f(x(j), u(j))$, e ai vincoli chiusi e limitati (*compatti*) sugli stati $x(j) \in \mathcal{X}$ e sugli ingressi $u(j) \in \mathcal{U}$. L'idea centrale per la prova di stabilità è dimostrare che il **costo ottimo** calcolato al tempo k , definito come $J^0(x(k))$, rispetti le due proprietà fondamentali per essere una funzione di Lyapunov.

1. Definitezza Positiva: Questa prima proprietà è garantita *by design* scegliendo un costo di stadio $\ell(x, u)$ e un costo terminale $V_f(x)$ che siano funzioni definite positive (tali che $\ell(0, 0) = 0$ e $V_f(0) = 0$). Nel caso specifico si adotteranno forme quadratiche: $\ell(x, u) = \|x\|_Q^2 + \|u\|_R^2$.

2. Variazione Definita Negativa: Per dimostrare che il costo decresce strettamente nel tempo ($\Delta J(x) = J^0(x^+) - J^0(x(k)) \leq 0$), supponiamo che al tempo k l'ottimizzatore abbia trovato la sequenza ottima:

$$\mathbf{u}^0(x) = \{u^0(0; x), u^0(1; x), \dots, u^0(N-1; x)\} \quad (11.5)$$

che genera la corrispondente sequenza ottima di stati predetti $x^0(j; x)$. Il costo ottimo valutato è quindi $J^0(x(k))$. All'istante successivo $k+1$ (dove lo stato attuale diviene x^+), il risultato dell'ottimizzazione non è ancora noto. Per ovviare a questo problema, si costruisce una sequenza **fattibile, ma non ottima** traslando la sequenza precedente di un passo e accodando un'azione di controllo finale definita da una legge locale $v = \kappa_f(x) \in \mathcal{U}$:

$$\tilde{\mathbf{u}} = \{u^0(1; x), u^0(2; x), \dots, u^0(N-1; x), \kappa_f(x^0(N; x))\} \quad (11.6)$$

Affinché questa sequenza sia effettivamente fattibile per il sistema vincolato, si introduce l'**Ipotesi base di stabilità**: deve esistere una legge di controllo locale

$\kappa_f(x) \in \mathcal{U}$ tale che:

1. $\forall x \in \mathbb{X}_f$, allora $f(x, \kappa_f(x)) \in \mathbb{X}_f$, ovvero $\mathbb{X}_f$ deve essere un *Control Invariant Set*.
2. $V_f(f(x, \kappa_f(x))) - V_f(x) \leq -\ell(x, \kappa_f(x))$, ovvero il costo terminale deve agire da Control Lyapunov Function (CLF) locale.

11.2.3 La Prova Fondamentale di Stabilità

Impiegando la sequenza sub-ottima $\tilde{\mathbf{u}}$, è possibile calcolarne il costo associato $J(x^+, \tilde{\mathbf{u}})$. Confrontando la sequenza ottima $\mathbf{u}^0(x)$ all'istante k con la sequenza sub-ottima $\tilde{\mathbf{u}}$ all'istante $k+1$, si evince che esse differiscono esclusivamente per il primo e l'ultimo elemento. Sottraendo i due costi, i termini della sommatoria intermedia si elidono perfettamente, restituendo:

$$J(x^+, \tilde{\mathbf{u}}) - J^0(x(k)) = \ell(x^0(N; x), \kappa_f) + V_f(f(x^0(N; x), \kappa_f)) - \ell(x^0(0; x), u^0(0; x)) - V_f(x^0(N; x)) \quad (11.7)$$

Raggruppando i termini, si ottiene:

$$= -\ell(x^0(0; x), u^0(0; x)) + \underbrace{\ell(x^0(N; x), \kappa_f) + V_f(f(x^0(N; x), \kappa_f)) - V_f(x^0(N; x))}_{\leq 0 \text{ (per l'ipotesi base di stabilità)}} \quad (11.8)$$

Poiché $\tilde{\mathbf{u}}$ è una soluzione fattibile ma non ottima, per definizione sappiamo che il costo ottimo al passo $k+1$ sarà minore o uguale a quello sub-ottimo: $J^0(x^+) \leq J(x^+, \tilde{\mathbf{u}})$. Di conseguenza, la variazione del costo ottimo è limitata da:

$$\Delta J(x) = J^0(x^+) - J^0(x(k)) \leq J(x^+, \tilde{\mathbf{u}}) - J^0(x(k)) \leq -\ell(x^0(0; x), u^0(0; x)) \quad (11.9)$$

Siccome $\ell(\cdot)$ è definita positiva, $-\ell(\cdot)$ è strettamente negativa. Questo conclude la prova: il costo decresce in modo monotono ad ogni passo di campionamento ($\Delta J(x) \leq 0$), agendo formalmente come funzione di Lyapunov e assicurando la totale convergenza asintotica del sistema in anello chiuso (QED). L'analisi per i controllori lineari coinciderà poi con la legge LQR analizzata nel Capitolo 11.4.

11.2.4 Formulazione Alternativa: Vincolo Terminale di Uguaglianza

In taluni casi può risultare difficile calcolare analiticamente una CLF locale o un *control invariant set* adeguato. Una formulazione alternativa per garantire la stabilità dell'MPC consiste nell'imporre un rigido vincolo terminale di uguaglianza:

$$x(N) = 0 \quad (11.10)$$

In questo scenario non è più necessario determinare una funzione $V_f(x)$, poiché imporre che lo stato terminale sia nullo garantisce intrinsecamente che $V_f(x(N)) = 0$, implicando anche che il *terminal invariant set* sia confinato al solo punto $\mathbb{X}_f = \{0\}$. Lo sviluppo della prova di stabilità rimane concettualmente identico. Si accoda l'azione nulla $\tilde{\mathbf{u}} = \{u^0(1; x), \dots, u^0(N-1; x), 0\}$, che genera la transizione in equilibrio $f(0, 0) = 0$. Il calcolo della differenza tra i costi si riduce quindi a:

$$J(x^+, \tilde{\mathbf{u}}) - J^0(x(k)) = -\ell(x^0(0; x), u^0(0; x)) \quad (11.11)$$

Dimostrando nuovamente la decrescita monotona e rigorosa del costo ottimo.

11.3 Analisi degli insiemi invarianti e stabilità robusta

Per implementare correttamente un MPC lineare è necessario garantire la stabilità asintotica dell'anello chiuso attraverso una scelta rigorosa degli elementi che compongono il problema di ottimizzazione infatti tale stabilità è garantita rispettando specifici requisiti.

11.3.1 Ingredienti Fondamentali per la Stabilità

Per un sistema lineare o linearizzato intorno a un punto di equilibrio a tempo continuo viene discretizzato a tempo discreto $x(k+1) = Ax(k) + Bu(k)$ e poi soggetto a vincoli deve essere progettato del regolatore deve includere i seguenti cinque ingredienti:

1. **Continuità:** Le funzioni $f(x, u) = Ax + Bu$ e il costo di un passo $l(x, u)$ devono essere continue. Nel caso di sistemi lineari, questa proprietà è intrinsecamente soddisfatta.
2. **Compattezza dei Vincoli:** Gli insiemi $\mathcal{X}$ e $\mathcal{U}$ devono essere compatti (chiusi e limitati) e devono contenere l'origine al loro interno, i vincoli sono espressi in questo modo: $x_{min} \leq x \leq x_{max}$ e $u_{min} \leq u \leq u_{max}$.
3. **Definitezza Positiva:** Il costo di stadio $l(x, u) = \|x\|_Q^2 + \|u\|_R^2$ e il costo terminale $V_f(x) = \|x\|_P^2$ devono essere definiti positivi ($Q \geq 0, R > 0, P \geq 0$).
4. **Esistenza della CLF Locale:** Deve esistere una legge di controllo locale $\kappa_f(x)$ tale per cui il costo terminale soddisfi la disuguaglianza di Lyapunov:

$$V_f(f(x, \kappa_f(x))) - V_f(x) \leq -l(x, \kappa_f(x)) \quad (11.12)$$

Nel caso lineare, si sceglie la legge LQR $\kappa_f(x) = -Kx$ e $V_f(x) = x^T Px$, dove P risolve l'equazione di Riccati.

5. **Invarianza del Vincolo Terminale:** Lo stato finale deve essere forzato in un insieme terminale $\mathbb{X}_f$ che sia un *control invariant set*.

Funzione MATLAB: La funzione MATLAB sviluppata per calcolare ed impostare questi requisiti si chiama `mpc_ingredients`, la cui implementazione completa è consultabile in Appendice ??.

11.4 Implementazione per Sistemi Lineari: Legame con LQR

Nel caso di sistemi lineari $x(k+1) = Ax(k) + Bu(k)$ con funzione di costo quadratica $l(x, u) = \|x\|_Q^2 + \|u\|_R^2$, gli ingredienti per garantire la stabilità derivano direttamente dalla teoria del Controllo Lineare Quadratico (LQR).

La legge di controllo locale stabilizzante viene scelta come $\kappa_f(x) = -Kx$, dove K è la matrice di guadagno ottima dell'LQR. Di conseguenza, il costo terminale che funge da CLF è dato da:

$$V_f(x) = \|x\|_P^2 = x^T Px \quad (11.13)$$

La matrice P è la soluzione dell'equazione algebrica di Riccati (DARE). Questa scelta è perfetta poiché misura esattamente il costo "da N a infinito". Inserendo P e K nell'ipotesi base di stabilità si verifica che:

$$(A - BK)^T P (A - BK) - P = -(Q + K^T R K) \quad (11.14)$$

che rappresenta esattamente l'uguaglianza ricavata dall'equazione di Riccati stazionaria, dimostrando la stabilità.

Per completare la formulazione, l'insieme terminale $\mathbb{X}_f$ viene calcolato iterativamente costruendo il *Invariant Set Massimo* $\mathcal{O}_\infty$, ovvero il più grande insieme di stati per i quali l'evoluzione governata dal controllore lineare $(A - BK)$ rispetta tutti i vincoli di stato e di ingressi fisici dell'aereo X_{max} e U_{max} .

11.4.1 Il Control Invariant Set e il Maximal Invariant Set $\mathcal{O}(\infty)$

Un **Control Invariant Set** $\mathbb{X}_f$ è una regione dello spazio degli stati tale per cui, se lo stato vi appartiene allora esiste un comando ammissibile capace di mantenere durante l'evoluzione del sistema all'interno dell'insieme.

$$\mathbb{X}_f = \{x \in \mathcal{X} : \text{se } x \in \mathbb{X}_f \implies (Ax + B\kappa_f(x)) \in \mathbb{X}_f, \kappa_f(x) \in \mathcal{U}\} \quad (11.15)$$

Per l'MPC, questo insieme funge da *ancora* per la fattibilità ricorsiva (*recursive feasibility*): garantisce che se il problema è risolvibile al tempo k , lo stato terminale $x(N)$ atterrerà in una zona da cui è sempre possibile calcolare una traiettoria futura sicura.

Algoritmo Iterativo per il Calcolo di $\mathcal{O}(\infty)$

Per determinare il **Invariant Set** $\mathcal{O}(\infty)$, ovvero il più grande insieme invariante possibile data una legge di controllo LQR $\kappa_f(x) = -Kx$, si utilizza un algoritmo

iterativo. Si raccolgono i vincoli sullo stato e sugli ingressi in forma matriciale $Gx \leq g$:

$$G = \begin{bmatrix} I \\ -I \\ -K \\ K \end{bmatrix}, \quad g = \begin{bmatrix} X_{max} \\ -X_{min} \\ U_{max} \\ -U_{min} \end{bmatrix} \quad (11.16)$$

Una volta fatto l'algoritmo procede come segue:

1. **Passo 0:** Si definisce $\mathcal{O}(0) = \{x : Gx \leq g\}$.
2. **Passo j :** Si aggiungono i vincoli sullo stato proiettato al passo j tramite la dinamica in loop chiuso $A_{cl} = (A - BK)$:

$$\mathcal{O}(j) = \{x \in \mathcal{O}(j-1) : G(A - BK)^j x \leq g\} \quad (11.17)$$

3. **Convergenza:** L'algoritmo termina quando $\mathcal{O}(j+1) = \mathcal{O}(j)$, indicando che sono state incluse tutte le limitazioni dinamiche. Il set risultante è l'invariante massimo $\mathcal{O}(\infty)$.

11.4.2 Definizione del Punto di Equilibrio e Variabili di Trim

Prima di procedere con la sintesi del controllore Model Predictive Control (MPC), è fondamentale definire il punto di equilibrio (o *Trim Point*) attorno al quale il modello matematico non-lineare dell'F-16 è stato linearizzato. Il velivolo in volo livellato non accelerato deve bilanciare perfettamente le forze aerodinamiche (portanza e resistenza) con la spinta del motore e il peso. Le variabili di trim principali calcolate dall'algoritmo di ottimizzazione sono:

- V_{t0} (**Velocità Totale di Trim**): La velocità vera a cui il velivolo mantiene il volo livellato.
- θ_{trim} (**o best_theta**): L'angolo di beccheggio di equilibrio. In volo livellato, poiché l'angolo di rampa γ è nullo, l'angolo di beccheggio coincide esattamente con l'angolo di attacco α_{trim} . Questo valore rappresenta l'inclinazione "a muso in sù" necessaria per generare la portanza necessaria a sostenere il peso del velivolo alla velocità V_{t0} .

- $\mathbf{u}_{trim}$ (o **best_u**): Il vettore degli ingressi di equilibrio, composto da:
 - Spinta del motore (T_{trim}): necessaria per contrastare la resistenza aerodinamica.
 - Deflessione dell'elevatore ($\delta_{e,trim}$): necessaria per azzerare il momento di beccheggio ($C_m = 0$).
 - Deflessione dei flaperon ($\delta_{f,trim}$): contribuisce al bilanciamento longitudinale e al profilo di portanza.

Poiché l'MPC utilizza un modello lineare dinamico tempo-invariante (LTI), il vettore di stato $\mathbf{x}$ e il vettore di ingresso $\mathbf{u}$ del controllore non rappresentano valori assoluti, bensì **perturbazioni** rispetto al punto di trim:

$$\mathbf{x} = \mathbf{x}_{assoluto} - \mathbf{x}_{trim}, \quad \mathbf{u} = \mathbf{u}_{assoluto} - \mathbf{u}_{trim} \quad (11.18)$$

11.4.3 Impostazione dei Vincoli di ingresso e stato

Il vantaggio principale del controllo predittivo è la sua capacità di gestire esplicitamente i vincoli del sistema durante l'ottimizzazione. Nell'applicazione all'F-16, i vincoli sono stati definiti tenendo conto sia dell'involuppo di volo sicuro sia dei limiti fisici degli attuatori.

Vincoli di Ampiezza sugli Attuatori

Gli attuatori meccanici (elevatore e flaperon) e il motore hanno escursioni minime e massime. Per applicare questi vincoli all'ottimizzatore, i limiti assoluti vengono traslati rispetto alla condizione di trim:

$$\mathbf{u}_{min} - \mathbf{u}_{trim} \leq \mathbf{u}(k) \leq \mathbf{u}_{max} - \mathbf{u}_{trim} \quad (11.19)$$

Ad esempio, se l'elevatore può muoversi fisicamente tra -25° e $+25^\circ$, e il valore di trim è $+2^\circ$, l'MPC avrà a disposizione un'escursione asimmetrica $\delta_e \in [-27^\circ, +23^\circ]$. Questo assicura che il controllore non calcoli mai azioni fisicamente irrealizzabili.

Vincoli di Rate (Velocità di Attuazione)

I servomotori reali non possono variare la deflessione istantaneamente. Lo stesso vale per la dinamica della turbina. È stato quindi introdotto un vincolo stringente sulla variazione dell'ingresso tra un istante di campionamento e il successivo:

$$-\Delta \mathbf{u}_{max} \leq \mathbf{u}(k) - \mathbf{u}(k-1) \leq \Delta \mathbf{u}_{max} \quad (11.20)$$

Questo vincolo è essenziale non solo per il realismo ingegneristico, ma previene anche il chattering (oscillazioni ad alta frequenza) nel segnale di controllo, prolungando la vita utile degli attuatori.

Vincoli sugli Stati

I vincoli sugli stati impediscono al velivolo di entrare in regimi aerodinamici pericolosi. Nello specifico, si è limitata l'escursione dell'angolo di beccheggio θ e della velocità angolare q per evitare il superamento dell'angolo di stallo e per mantenere le ipotesi di linearità del modello attorno al punto di equilibrio. Anche in questo caso, i bounds vengono centrati sullo stato di trim:

$$\mathbf{x}_{min} \leq \mathbf{x}(k) \leq \mathbf{x}_{max} \quad (11.21)$$

11.4.4 Definizioni di Insiemi Controllabili e Stabilizzabili

Per definire gli insiemi controllabili e stabilizzanti serve una caratteristica molto importante ovvero il **Dominio di Attrazione** $\mathbb{X}_N$, ovvero, (l'insieme di tutti gli stati iniziali da cui lo stato iniziale di un sistema può essere controllato in N passi e riportato all'origine o a un set-point diverso dall'origine), è necessario quindi definire la gerarchia degli insiemi:

One-step Set $\mathcal{Q}(\Omega)$

Questo insieme rappresenta l'insieme degli stati $x \in \mathcal{X}$ per i quali esiste almeno un comando ammissibile $u \in \mathcal{U}$ capace di portare il sistema nel set target Ω in un solo passo:

$$\mathcal{Q}(\Omega) = \{x \in \mathcal{X} : \exists u \in \mathcal{U} \text{ s.t. } (Ax + Bu) \in \Omega\} \quad (11.22)$$

i-steps Controllable Set $\mathcal{C}_i(\mathcal{X}, \Omega)$

Questo insieme mostra gli stati che possono essere portati in Ω in esattamente i passi, rispettando i vincoli di stato lungo tutta la traiettoria:

$$\mathcal{C}_{i+1}(\mathcal{X}, \Omega) = \mathcal{Q}(\mathcal{C}_i(\mathcal{X}, \Omega)) \cap \mathcal{X}, \quad \text{con } \mathcal{C}_0 = \Omega \quad (11.23)$$

i-steps Stabilizable Set $\mathcal{S}_i(\mathcal{X}, \Omega)$

Un insieme si dice *stabilizzabile* in i passi se il target Ω è un **invariant set**.

11.4.5 Proprietà di Monotonicità e Convergenza

Esiste una differenza cruciale tra $\mathcal{C}_i$ e $\mathcal{S}_i$ che determina il successo dell'MPC:

- **Monotonicità di $\mathcal{S}_i$:** Poiché Ω è invariante, vale sempre la proprietà di inclusione:

$$\mathcal{S}_i(\mathcal{X}, \Omega) \subseteq \mathcal{S}_{i+1}(\mathcal{X}, \Omega) \quad (11.24)$$

La proprietà mostra che all'aumentare dell'orizzonte di predizione N il dominio di attrazione $\mathbb{X}_N$ non può restringersi. Se uno stato raggiunge il set di sicurezza in i passi, l'invarianza garantisce che possa rimanerci anche al passo $i + 1$. Questa inclusione non è garantita per i set controllabili generici che non hanno un target invariante.

- **Convergenza:** Entrambe le sequenze $\mathcal{S}_i$ e $\mathcal{C}_i$, per $i \rightarrow \infty$, convergono allo stesso insieme limite: il **Maximal Control Invariant Set** $\mathcal{O}_\infty(\mathcal{X})$, che rappresenta la regione massima di governabilità del sistema.

L'implementazione dell'MPC utilizza queste proprietà per espandere il dominio di attrazione $\mathbb{X}_N$ rispetto al semplice set invariante locale $\mathbb{X}_f$. Più lungo è l'orizzonte N , maggiore è la capacità del controllore di recuperare l'aereo da perturbazioni severe, portandolo gradualmente all'interno della regione di sicurezza $\mathcal{O}_\infty(\mathcal{X})$.

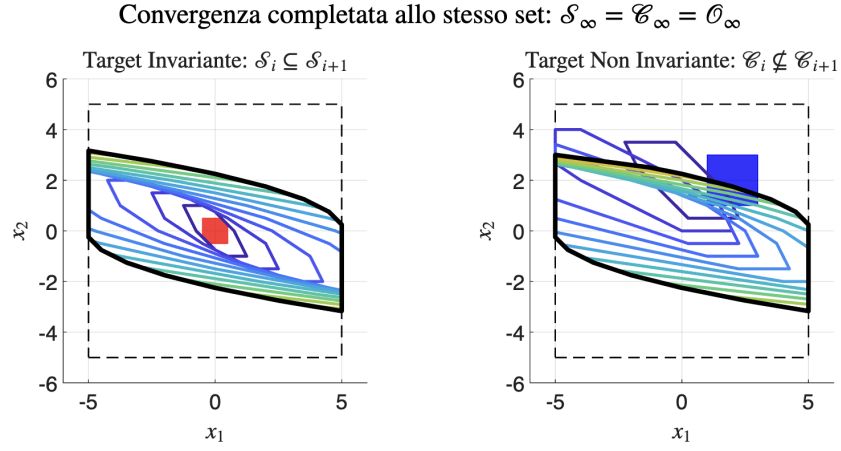


FIGURA 11.1: Evoluzione spaziale degli insiemi a i passi. Il grafico confronta visivamente le proprietà matematiche discusse mostrando un target invariante (insiemi stabilizzabili $\mathcal{S}_i$) si ottiene una sequenza in rigorosa espansione monotona, mentre l'utilizzo di un target generico non invariante (insiemi controllabili $\mathcal{C}_i$) non garantisce la proprietà che garantisce la completa controllabilità del sistema in N passi

11.5 Implementazione Pratica: Controllo Longitudinale F-16

Alla luce della teoria esposta precedentemente, si è implementato in MATLAB un controllore MPC per la dinamica longitudinale del velivolo F-16. Il vettore di stato è stato definito secondo l'ordine $[\theta, q, u, w]$, al fine di modellare l'assetto di beccheggio, il pitch rate e le componenti di velocità nel sistema di riferimento *body* del velivolo. A causa delle variabili di stato con grandezze fisiche diverse (ad esempio, le velocità u e w misurate in piedi al secondo contro angoli misurati in radianti), le matrici di costo Q ed R sono state penalizzate secondo la teoria di Bryson[2], normalizzando gli errori massimi tollerabili per garantire la corretta penalizzazione delle matrici Q ed R dell'ottimizzatore `quadprog`.

11.5.1 Analisi dei Risultati e Validazione del Controllore MPC

L'implementazione del controllo predittivo basato su modello (MPC) ha permesso di gestire la dinamica longitudinale del velivolo F-16 garantendo il rispetto rigoroso dei vincoli fisici sugli attuatori e sugli stati, superando i limiti intrinseci del controllo LQR.

La simulazione è stata condotta a partire da una condizione iniziale molto perturbata, definita nel vettore di stato iniziale $\mathbf{x}_0$ come:

$$\mathbf{x}_0 = [\theta, q, u, w]^T = [25^\circ, 3^\circ/s, 2 \text{ ft/s}, 5 \text{ ft/s}]^T \quad (11.25)$$

Nonostante il setpoint iniziale l'algoritmo di ottimizzazione `quadprog` riesce ad identificare una sequenza di controllo ammissibile all'interno dell'orizzonte predittivo $N = 30$ (corrispondente a 1.5 secondi con $T_s = 0.05 \text{ s}$).

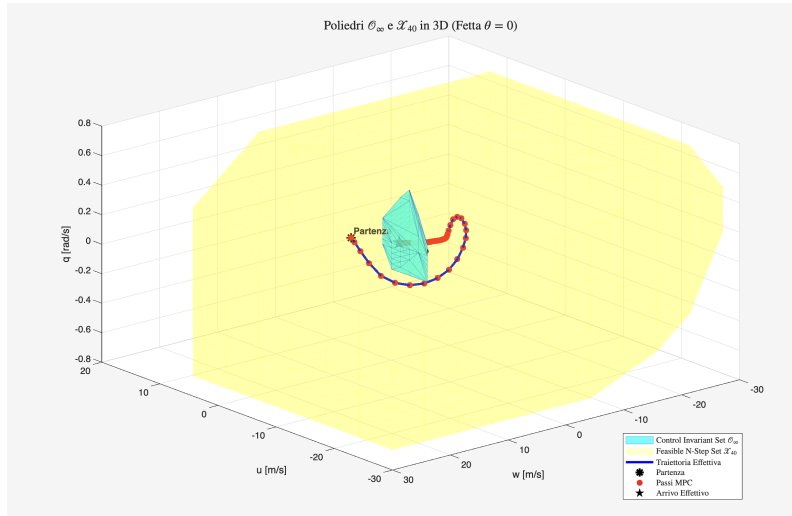


FIGURA 11.2: Rappresentazione del poliedro di invarianza $\mathcal{X}_f$ (fetta $\theta = 0$) e traiettoria di predizione dello stato verso l'origine.

Come evidenziato nella Figura 11.2, la solidità matematica del controllore è garantita dal calcolo dell'insieme invariante massimale O_∞ . Lo script MATLAB ha verificato la convergenza del poliedro, definendo la regione dello spazio di stato entro la quale il sistema può essere stabilizzato senza violare i vincoli. La traiettoria di predizione (visualizzata in rosso) mostra come l'MPC pianifichi come riportare

lo stato del sistema verso l'origine forzando lo stato finale a cadere all'interno del *terminal set*, inoltre si mostra la nuvola gialla di possibili stati del sistema che mostra il dominio di attrazione dell' MPC impostato in questa simulazione con $N=40$, essa mostra l'argomento trattato nella sezione 11.4.4.

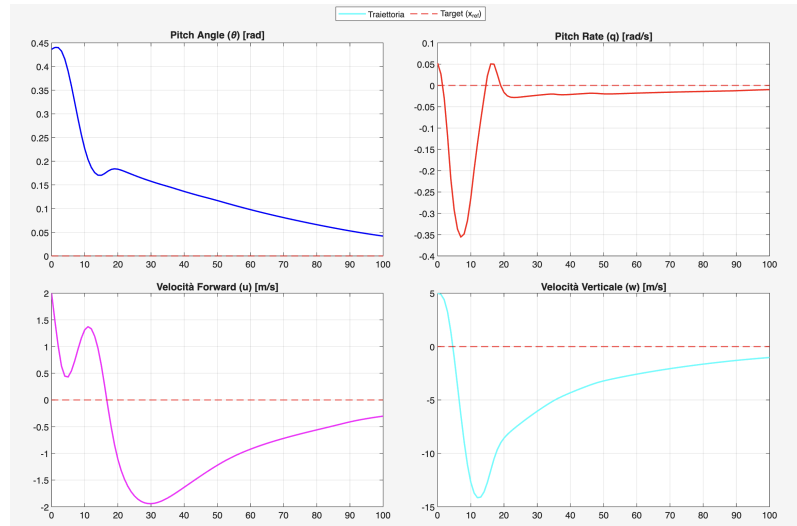


FIGURA 11.3: Evoluzione temporale delle variabili di stato: beccheggio, pitch rate e variazioni di velocità lineare.

L'analisi degli stati al passare del tempo (Figura 11.3) mostra un transitorio fluido e privo di sovra-elongazioni critiche per l'angolo di beccheggio e le velocità lineari. Il tempo di assestamento complessivo è coerente con la penalizzazione applicata tramite la matrice Q (calcolata secondo la regola di Bryson)[2], bilanciando correttamente la rapidità di risposta con la stabilità del velivolo. L'aspetto più significativo emerge dalla Figura 11.4, relativa allo sforzo di controllo. A differenza di un regolatore lineare standard, l'MPC sfrutta l'intera autorità di comando disponibile "appoggiandosi" ai vincoli di saturazione ($\pm 25^\circ$ per l'elevatore e $\pm 12^\circ$ per i flap) senza mai superarli. La gestione dei vincoli di rateo (*Rate Limiting*), integrata nella matrice delle disuguaglianze del problema QP, previene variazioni impulsive di spinta, proteggendo l'integrità strutturale dei servomeccanismi dell'aereo. Inoltre, avendo tarato le matrici Q ed R attraverso la regola di Bryson, la matrice P ottenuta dall'equazione di Riccati, su cui si basa la CLF all'interno del control-invariant set, genera un'azione di controllo realistica. Ciò permette al velivolo di compiere

manovre rispettando rigorosamente i limiti fisici, pur sacrificando una convergenza più rapida verso l'equilibrio.

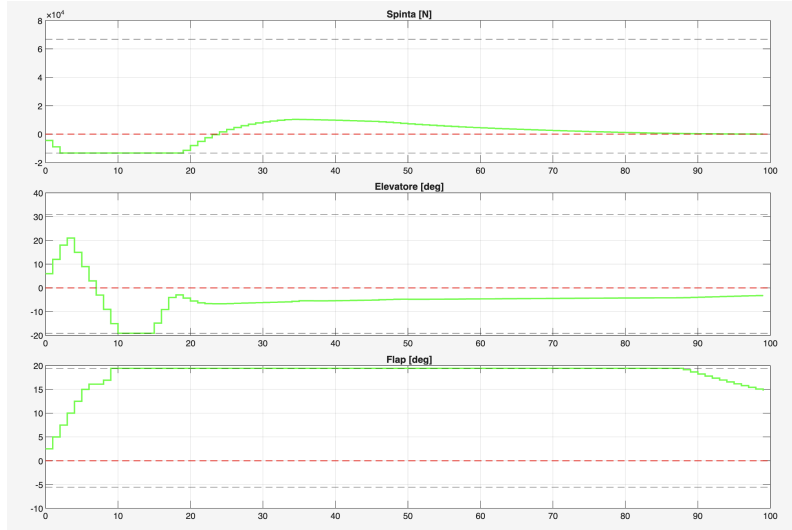


FIGURA 11.4: Sforzo di controllo degli attuatori: si noti l'attivazione dei vincoli di saturazione sull'elevatore e sui flap.

11.5.2 Variazione del Funzionale di Costo

Per completare la validazione del controllore predittivo ed effettuare un confronto diretto con le prestazioni del regolatore LQR analizzate in precedenza, è stata valutata la topologia dello spazio degli stati sottoponendo il sistema al medesimo set di condizioni iniziali multiple $\mathbf{X}_{0,\text{mult}}$. Osservando il comportamento del sistema (Figura 11.5), si nota in modo evidente come il funzionale di costo ottimo calcolato ad ogni passo dal controllore MPC assuma il ruolo di una **funzione di Lyapunov** per il sistema. Dal grafico tridimensionale emerge chiaramente che, a partire da ogni condizione iniziale, il costo scende monotonamente lungo la superficie definita dalla funzione di costo terminale $V(x) = x^T P x$ (dove P è la matrice di costo finale associata al controllore LQR non vincolato).

Questo andamento strettamente decrescente nel tempo è la prova geometrica che garantisce la **stabilità asintotica in anello chiuso**. Infatti, come prescritto dal Teorema di Lyapunov, la variazione negativa del funzionale ad ogni passo ($\Delta J^0(x) < 0$) assicura che le traiettorie siano "attratte" in modo inesorabile verso l'origine, senza la possibilità di divergere o di creare cicli instabili. La stabilità è

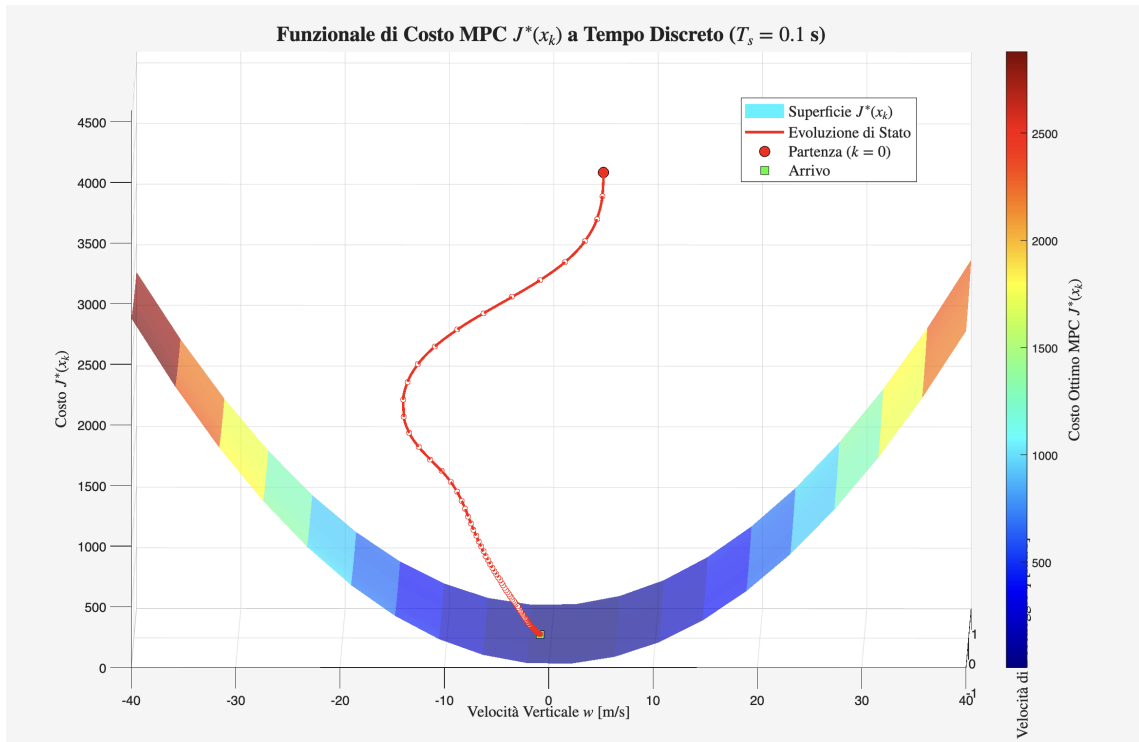


FIGURA 11.5: Analisi geometrica del controllore MPC a ciclo chiuso. Il grafico tridimensionale mostra l'evoluzione dello stato e il corrispondente andamento decrescente del funzionale di costo ottimo, che si comporta come una funzione di Lyapunov scendendo lungo la superficie del costo terminale.

permessa dalla corretta definizione del tempo di campionamento e del dominio di attrazione dell'MPC, unita alla capacità del *solver* numerico di trovare una soluzione ottima fattibile tra un istante discreto e il successivo. Inoltre, emerge in modo palese la natura ibrida dell'ottimizzazione MPC: man mano che le traiettorie convergono verso l'origine e lo stato rientra all'interno del *Control Invariant Set*, i vincoli smettono di essere attivi (ovvero il sistema non satura più gli attuatori). In quest'area la soluzione dell'MPC converge a quella del regolatore lineare, affidandosi alla legge di controllo ottima locale $u = -Kx$ (identica a quella discussa per il controllore LQR) per guidare definitivamente l'aereo al punto di equilibrio desiderato. In conclusione, i risultati dimostrano che l'architettura MPC è in grado di trasformare un impianto multivariabile fortemente accoppiato in un sistema governabile e robusto.

Il controllore garantisce di operare in sicurezza anche in presenza di forti perturbazioni iniziali, assicurando contemporaneamente il rigoroso rispetto dei vincoli fisici e l'assoluta stabilità asintotica del volo.

Parte III

Implementazione Real-Time Multithreading in C++

Capitolo 12

Monitor Real Time

12.1 Schedulazione nei Sistemi Real-Time

I sistemi *Real-Time* (RT) sono caratterizzati dalla necessità di produrre, entro un preciso vincolo temporale, l'esecuzione di una task periodica all'interno di un vincolo detto **deadline**. Nei sistemi *Hard Real-Time*, come i computer di bordo avionici i quali saranno trattati nel caso di studio al Cap.15. Il mancato rispetto di una deadline non è semplicemente un degrado di prestazioni, è un fallimento del sistema con potenziali conseguenze che comportano la perdita di controllo dell'aereo con conseguente distruzione di esso e con danni mortali al pilota. Nei sistemi *Soft Real-Time*, come ad esempio nei sistemi multimediali il superamento occasionale di una deadline comporta una riduzione della qualità percepita e quindi della effettiva resa di prestazione, ma non comporta nessun tipo di pericolo per quanto riguarda l'utente finale. Il fulcro della progettazione RT è l'algoritmo di **schedulazione** ovvero il meccanismo che il sistema operativo utilizza per decidere quale task eseguire in ogni istante su ogni core della CPU disponibile. In un ambiente multi-core, la complessità aumenta: ai task devono essere assegnate sia le priorità di esecuzione che l'affinità ai core fisici, garantendo che i task più critici abbiano sempre accesso alla risorsa computazionale entro le proprie deadline.

12.1.1 Utilizzo di Rate Monotonic

All' interno dei prossimi capitoli verrà preso in considerazione l'algoritmo hard-real-time **Rate Monotonic** (RM), proposto da Liu e Layland nel 1973. Questo algoritmo è uno dei fondamenti teorici della schedulazione RT, Esso assegna staticamente a ogni task una priorità inversamente proporzionale al suo periodo: il task con il periodo più breve ottiene la priorità più alta. Il suo utilizzo è ottimale per sistemi di task periodici con deadline uguali al periodo, ma non è necessaria questo requisito. La condizione di schedulabilità è la seguente:

$$U = \sum_{i=1}^n \frac{C_i}{T_i} \leq n \left(2^{1/n} - 1 \right) \quad (12.1)$$

dove C_i è il Worst Case Execution Time del task i , T_i è il suo periodo. Il limite a destra converge a $\ln 2 \approx 0.693$ per $n \rightarrow \infty$. In un sistema Linux come quello utilizzato, viene usata la policy `SCHED_FIFO` con RM, la schedulazione quindi viene implementata assegnando priorità numeriche discendenti al crescere del periodo nel momento della creazione del thread con tutti i parametri necessari. Il limite $U_{bound}(n)$ è una funzione decrescente di n che converge a $\ln 2 \approx 0.693$ per $n \rightarrow \infty$.

Importante: il test è *sufficiente ma non necessario*. Un insieme di task che supera il bound potrebbe comunque rispettare tutte le deadline (come dimostra il Processo Set C delle slide ERTS con $U = 1.0$). Il test con Hyperbolic Bound, anch'esso presentato nel corso, è meno pessimistico ma più complesso.

12.2 Obiettivo della Tesi

La finalità di questa tesi è lo sviluppo e la validazione di un sistema completo per l'**analisi dei parametri temporali** di applicazioni Real-Time in esecuzione su un'architettura multi-core e possibilmente con diverse regole di schedulazione, di conseguenza diverse da RM come EDF, PIP e PCP. Il progetto risponde alla seguente domanda: dato un insieme di task periodici implementati in C++ tramite le primitive della libreria POSIX e in esecuzione su Linux, è possibile estrarre automaticamente le metriche temporali fondamentali (WCET, Jitter, Response Time, Deadline Miss) e visualizzarle in modo tale che le statistiche ottenute siano

corrette e graficamente utilizzabili per la validazione dell' applicazione creata dallo sviluppatore.

La risposta proposta si articola in tre componenti principali:

1. Una **libreria per i Marker Temporal**i: in C/C++ che inietta marker temporali nel buffer di tracciamento del kernel Linux tramite il sottosistema **ftrace**, in questo modo attraverso il file generato dalla simulazione chiamato `trace.dat`, si può capire in quale momento attraverso il timestamp una certa funzione periodica viene eseguita. In questo modo la simulazione ha consistenza e validazione. Senza questa funzionalità risulterebbe difficile capire esattamente cosa accade durante la simulazione e quindi anche capire se la creazione dei thread con la loro rispettiva Periodic Task e la gestione della schedulazione sia avvenuta nel modo corretto e che rispetti i vincoli di utilizzo del sistema textit Real-Time.
2. **parser C++**: chiamato nel progetto (`ThreadAnalysisi.cpp`), il seguente codice legge il log degli eventi di scheduling generato da `trace-cmd` e calcola automaticamente le metriche temporali per ogni task, queste verranno poi caricate su un file csv chiamato (`timeline.csv`) che sarà utilizzato dal monitor python (`monitor.py`) per poter visualizzare correttamente tutte le statistiche.
3. **monitor Python**: che legge il file csv (`timeline.csv`) e genera diagrammi di Gantt con tabelle di statistiche leggibili per lo sviluppatore.

Nella trattazione che seguirà, è stato scelto come caso di studio per la validazione del progetto un computer di bordo di un caccia F-16 che simula i controlli eseguiti dal protocollo **Fly-By-Wire**. L'implementazione è basata su **Multi-Thread** concorrenti distribuiti su i 7 core fisici disponibili sulla macchina linux utilizzata, la quale riproduce fedelmente il vero funzionamento di un sistema avionico.

Capitolo 13

Modelli e Strumenti

13.1 Il Modello di Task Periodico

13.1.1 Definizioni Fondamentali

Un **task** è una sequenza di istruzioni che, in assenza di altre attività, verrebbe eseguita dal processore in modo continuo fino al completamento. Nei sistemi Real-Time concorrenti, più task possono essere simultaneamente attivi, ma solo uno alla volta è in esecuzione. I task non in esecuzione che sono pronti a farlo vengono detti *ready* e sono mantenuti in una **coda dei processi pronti** (*ready queue*), gestita dalla politica di scheduling. Il meccanismo di **prelazione** (*preemption*) consente al kernel di sospendere il task in esecuzione in favore di uno più urgente; il task sospeso rientra nella coda dei processi pronti. Ovviamente la parte del sistema operativo che si occupa di assegnare l'esecuzione sulle risorse disponibili è lo schedulatore, infatti è proprio la politica scelta per lo schedulatore in fase di progettazione del sistema che impone in quale modo avviene l'esecuzione dei task nella coda dei processi pronti.

Un **task Real-Time** è un task soggetto a vincoli temporali espliciti: la robustezza del sistema dipende dal risultato dell'esecuzione del task, ma anche dall'istante in cui viene prodotto questo risultato. Per fare chiarezza Il modello formale di un task periodico τ_i è descritto nel modo seguente:

$$\tau_i = (C_i, T_i, D_i) \quad (13.1)$$

- C_i è il **Worst Case Execution Time** (WCET): il tempo massimo di CPU richiesto dal task in un singolo ciclo. Stimare C_i non è scontato: dipende dall'architettura del processore, dalla cache, dai rami condizionali nel codice e dall'input, quindi da come viene definita la **Periodic Task** da eseguire. In questo progetto, C_i viene misurato sperimentalmente tramite il sistema di tracciamento.

- T_i è il **periodo**: l'intervallo temporale fisso tra due attivazioni consecutive. La k -esima istanza viene rilasciata all'istante $r_{i,k} = r_{i,0} + k \cdot T_i$.
- D_i è la **deadline relativa**: il tempo massimo dall'istante di rilascio entro cui il task deve completare, ricordando che questo è il vincolo più importante di un sistema **Real-time**. In questo progetto si adotta il modello **implicito** ($D_i = T_i$), tipico dei sistemi di controllo in cui ogni ciclo deve terminare prima del successivo, questa scelta ovviamente se non rispettata non inifica il funzionamento del progetto è solo una scelta di progetto.

13.1.2 Il Ciclo di Vita di un Task Periodico

Il ciclo di vita di ogni istanza k di τ_i si articola nelle fasi seguenti:

1. **Rilascio**: al tempo $r_{i,k} = r_{i,0} + k \cdot T_i$ il task diventa pronto e viene inserito nella coda di pronto.
2. **Attesa**: il task rimane in coda finché lo scheduler non gli assegna la CPU.
3. **Esecuzione**: il task utilizza la CPU per un tempo $c_{i,k} \leq C_i$.
4. **Completamento**: al tempo $f_{i,k}$, il task termina l'istanza corrente.
5. **Sleep**: il task si sospende fino al prossimo rilascio.

13.1.3 Metriche Temporali Fondamentali

Le metriche definite nel corso ERTS [12] e misurate dal sistema sviluppato in questa tesi sono:

- **Response Time**: $R_{i,k} = f_{i,k} - r_{i,k}$, differenza tra il tempo di completamento e il tempo di rilascio. Include sia il tempo di esecuzione che l'eventuale attesa in coda dovuta alla prelazione.
- **Lateness**: $L_{i,k} = f_{i,k} - d_{i,k}$, ritardo del completamento rispetto alla deadline. Se il task completa in anticipo, la lateness è negativa.

- **Laxity** (Slack Time): $X_{i,k} = d_{i,k} - r_{i,k} - C_i$, il massimo tempo per cui il task può essere ritardato dalla sua attivazione e ancora rispettare la deadline. Equivale al tempo libero residuo prima della scadenza.
- **Deadline Miss**: si verifica quando $f_{i,k} > d_{i,k}$, ovvero quando $L_{i,k} > 0$. In un sistema Hard Real-Time, ogni deadline miss è un fallimento del sistema.
- **Jitter**: variazione dell'intervallo tra attivazioni consecutive reali rispetto al periodo nominale. Il jitter assoluto è $J_{i,k} = |r_{i,k} - (r_{i,0} + k \cdot T_i)|$; il jitter relativo è la deviazione standard degli intervalli misurati. Un jitter elevato indica instabilità nella schedulazione.
- **WCET e ACET**: rispettivamente il massimo e la media dei tempi di esecuzione osservati su tutti i cicli registrati.
- **Utilizzazione CPU**: $U_i = C_i/T_i$ per ogni task. La somma $U = \sum_i U_i$ rappresenta la frazione di CPU complessivamente richiesta dal sistema.

13.2 Schedulazione a Priorità Fissa: Rate Monotonic

13.2.1 Algoritmi di Scheduling Real-Time

Il corso del prof. Brugali (ERTS [12]) classifica gli algoritmi di scheduling RT in due famiglie principali:

- **Algoritmi a priorità fissa**: la priorità di ogni task è assegnata staticamente e non cambia durante l'esecuzione. L'algoritmo usato in questo caso è **Rate Monotonic** (RM).
- **Algoritmi a priorità dinamica**: la priorità cambia a runtime in base non più al periodo ma alla deadline. Il rappresentante è l'**Earliest Deadline First** (EDF), che assegna la priorità più alta al task con deadline assoluta più vicina.

In questo progetto è stato adottato Rate Monotonic, implementato tramite la policy `SCHED_FIFO` di Linux con priorità statiche.

13.2.2 Rate Monotonic: Assegnamento delle Priorità

L'algoritmo RM, proposto da Liu e Layland nel 1973 [12], assegna staticamente a ogni task una priorità inversamente proporzionale al suo periodo: il task con periodo più breve ottiene la priorità più alta. Formalmente, per due task τ_i e τ_j :

$$T_i < T_j \implies P_i > P_j \quad (13.2)$$

RM è **ottimale** nella classe degli algoritmi a priorità fissa: se un insieme di task può essere schedulato con qualsiasi algoritmo a priorità fissa, allora può essere schedulato anche con RM. Questa proprietà di ottimalità è dimostrata nel corso ERTS e costituisce la base teorica dell'assegnamento delle priorità adottato in questo progetto.

13.3 POSIX e DDS per Attività Periodiche e Distribuite

Esempio senza DDS: due task su core dedicati trattato al capitolo 14.2.1.

Il caso più semplice è l'applicazione `app10g` (`src/app/app10g/app10g.c`): due task periodici con periodi differenti, assegnati a core fisici separati per eliminare qualsiasi contesa della CPU. Di seguito viene mostrato il main completo, che illustra l'intera sequenza di creazione dei thread Real-Time POSIX dall'inizializzazione degli attributi fino al `pthread_join`.

Esempio con DDS: comunicazione distribuita trattato al capitolo 14.2.2

Come mostrato nelle slide ERTS [13], quando i task devono comunicare tra processi distinti su macchine diverse, la memoria condivisa non è più disponibile e si utilizza il middleware DDS. L'applicazione `Broadcastner` (`src/app/Broadcastner/Broadcastner.cpp`) mostra l'integrazione completa tra il ciclo periodico POSIX e la pubblicazione DDS: il thread `DDS_Publish` (Core 1) esegue periodicamente la funzione `dds_publish_function`, mentre il thread `Activity_1` (Core 0) simula un carico computazionale concorrente.

Il confronto tra i due esempi evidenzia le differenze architetturali introdotte dal middleware DDS:

- Nel caso `app10g`, la `Periodic Task (activity_load)` è una funzione che serve per far eseguire cicli a vuoto alla CPU per simulare sforzo computazionale e non vi è alcuna comunicazione inter-processo.
- Nel caso `Broadcastner`, la funzione `dds_publish_function` aggiunge alla computazione locale una fase di pubblicazione DDS, incapsulata tra i marker `DDS_MSG_START/END`. Il tempo tra questi due marker include la serializzazione CDR del messaggio, il trasferimento tramite SHM o UDP e la notifica al subscriber. Questa latenza aggiuntiva è visibile nel diagramma di Gantt come un allungamento del blocco `RUN` rispetto a un task puramente computazionale con lo stesso carico `parameter`, questa differenza può essere apprezzata in modo maggiore nel capitolo dedicato al Monitor DDS di eprosima [13.4](#).
- la presenza del thread `Activity_1` (Core 0, periodo 1000 ms) è presente nell'esempio `Broadcastner` per simulare un processo applicativo concorrente alla pubblicazione DDS, riproducendo uno scenario più realistico in cui il nodo publisher non ha la CPU completamente dedicata.

13.3.1 Il Middleware DDS e la Qualità del Servizio

Come descritto nelle slide ERTS [14], DDS (*Data Distribution Service*) è uno standard per la comunicazione publish-subscribe anonima e decentralizzata. I principali concetti sono:

- **DomainParticipant**: entità logica che partecipa a un dominio DDS. Tutti i partecipanti con lo stesso `domain_id` possono comunicare tra loro.
- **Topic**: canale tipizzato identificato da un nome. Solo partecipanti che usano lo stesso topic e lo stesso tipo di messaggio possono scambiarsi dati.
- **DataWriter** e **DataReader**: gli oggetti concreti usati rispettivamente per pubblicare e ricevere dati.
- **QoS** (*Quality of Service*): parametri che controllano il comportamento della comunicazione. I principali adottati in questo progetto sono:
 - **Reliability**: `BEST_EFFORT` (nessuna garanzia di consegna, minima latenza) o `RELIABLE` (consegna garantita con ritrasmissione).
 - **Durability**: `VOLATILE` (nessuna storia salvata) o `TRANSIENT` (storia salvata in memoria locale, disponibile per subscriber che si connettono in ritardo).
 - **Deadline**: specifica il periodo massimo tra due campioni consecutivi; se violato, il middleware notifica l'applicazione.

13.4 Il Monitor DDS: *eProxima Fast DDS Monitor*

eProxima Fast DDS Monitor è un'applicazione grafica open-source scaricabile a questo link: [Monitor DDS](#), sviluppata da eProxima che permette di osservare, in tempo reale e a posteriori, le statistiche di comunicazione di una rete DDS attiva [15]. Si basa sul modulo statistico interno di FastDDS (**FastDDS Statistics Backend**) che, se abilitato, aggiunge automaticamente dei **DataWriter** interni a ogni **DomainParticipant**: questi writer pubblicano metriche di rete su topic statistici riservati (es. **HISTORY_LATENCY_TOPIC**, **DATA_COUNT_TOPIC**), che il monitor ascolta senza interferire con il traffico applicativo.

13.4.1 Prerequisito: Abilitazione del Modulo Statistico

Per far sì che i **DomainParticipant** del progetto pubblicino i propri dati statistici, è necessario abilitare il modulo statistico di FastDDS **prima**, ovvero quando si fa l'installazione sulla propria macchina del modulo **fastdds** e di tutte le librerie. Bisogna assicurarsi che la flag per l'installazione dei moduli statistici sia attivata. Una volta fatto nel codice l'attivazione del tracciamento avviene tramite la variabile d'ambiente **FASTDDS_STATISTICS**:

```
1  export FASTDDS_STATISTICS="HISTORY_LATENCY_TOPIC;      \  
2  NETWORK_LATENCY_TOPIC;                                \  
3  PUBLICATION_THROUGHPUT_TOPIC;                          \  
4  SUBSCRIPTION_THROUGHPUT_TOPIC;                         \  
5  RTPS_SENT_TOPIC;RTPS_LOST_TOPIC;                      \  
6  DATA_COUNT_TOPIC;SAMPLE_DATAS_TOPIC;                 \  
7  DISCOVERY_TOPIC;PHYSICAL_DATA_TOPIC;                  \  
8  MONITOR_SERVICE_TOPIC"
```

LISTING 13.1: Abilitazione di tutti i topic statistici FastDDS

Una volta settata questa variabile, ogni processo FastDDS avviato nello stesso terminale genera automaticamente i writer statistici interni. Il monitor DDS li scoprirà tramite il meccanismo di discovery e inizierà a raccogliere i dati.

Problema di compatibilità di versione: bisogna assicurarsi che la versione del monitor installata sia compatibile con la versione fastDDS presente e nel caso di mancanza di compatibilità il moitor non funziona come deve.

13.4.2 Configurazione del Dominio nel Codice

Il collegamento tra il monitor e le applicazioni del progetto dipende dal **domain ID** DDS: il monitor deve ascoltare esattamente lo stesso dominio numerico usato dai `DomainParticipant` delle applicazioni. Nel codice del progetto, il dominio viene specificato nella chiamata `start()` del wrapper `Broadcastner`:

```

1 // Inizializzazione del DomainParticipant sul dominio 1.
2 // Il monitor DDS deve essere configurato per ascoltare
3 // esattamente questo stesso domain ID (1) per vedere
4 // i DomainParticipant di questa applicazione.
5 eprosima::fastdds::dds::TypeSupport msg_type(new
Point_PubSubType());
6 g_broadcastner.start(&msg_type, "Point", "PointTopic", 1);
7 //
8 // domain_id = 1

```

LISTING 13.2: Configurazione del domain ID nella classe `Broadcastner`

Internamente, il metodo `start()` crea il `DomainParticipant` con il `domain_id` specificato:

```

1 bool Broadcastner::start(TypeSupport *msg_type,
2 std::string msg_name,
3 std::string topic,
4 int domain_id) {
5 // Il DomainParticipant viene creato sul dominio
specificato.
6 // Solo partecipanti con lo stesso domain_id si vedono a
vicenda.
7 participant_ = DomainParticipantFactory::get_instance()
->create_participant(domain_id,
8 PARTICIPANT_QOS_DEFAULT);
9 // ... registrazione TypeSupport, creazione Topic e
10 DataWriter

```

```
11 }
```

LISTING 13.3: Creazione del DomainParticipant sul dominio configurato

Al momento di avviare il monitor, nella finestra di dialogo iniziale si deve inserire il numero **1** (o il domain ID usato dall'applicazione) per sincronizzarsi con i partecipanti corretti. Se il campo viene impostato su un dominio diverso, il monitor non vedrà nessuna entità.

13.4.3 Domain View: La Mappa della Rete DDS

La **Domain View** è la prima schermata di ispezione disponibile nel pannello principale del monitor. Visualizza un grafo che descrive la struttura fisica e logica della rete DDS attiva:

- **Struttura gerarchica fisica:** ogni *Host* (macchina fisica) contiene uno o più *User* (utenti di sistema), che a loro volta contengono i *Process* (processi in esecuzione). Questa gerarchia è resa possibile dall'attivazione del topic `PHYSICAL_DATA_TOPIC` nella variabile d'ambiente.
- **Struttura logica DDS:** ogni processo è associato a uno o più **DomainParticipant**. Ogni partecipante espone i propri **DataWriter** (freccia in entrata nel topic, lato publisher) e **DataReader** (freccia in uscita dal topic, lato subscriber).
- **Visualizzazione del flusso dati:** nella Domain View, il topic è rappresentato come una linea verticale centrale. Il **Broadcastner** è visibile come il nodo con la freccia che punta *verso* il topic (**DataWriter**), mentre il **Listener** è il nodo con la freccia che esce *dal* topic (**DataReader**).

All'interno della Domain View sono disponibili funzionalità aggiuntive:

- **Filtro per topic:** clic destro sul nome del topic → *Filter topic graph*: apre una nuova scheda con il grafo filtrato per quel solo topic, utile quando la rete ha molti topic attivi contemporaneamente.
- **Visualizzazione IDL:** clic destro sul topic → *Data type IDL view*: mostra la definizione IDL del tipo di messaggio, copiabile direttamente.

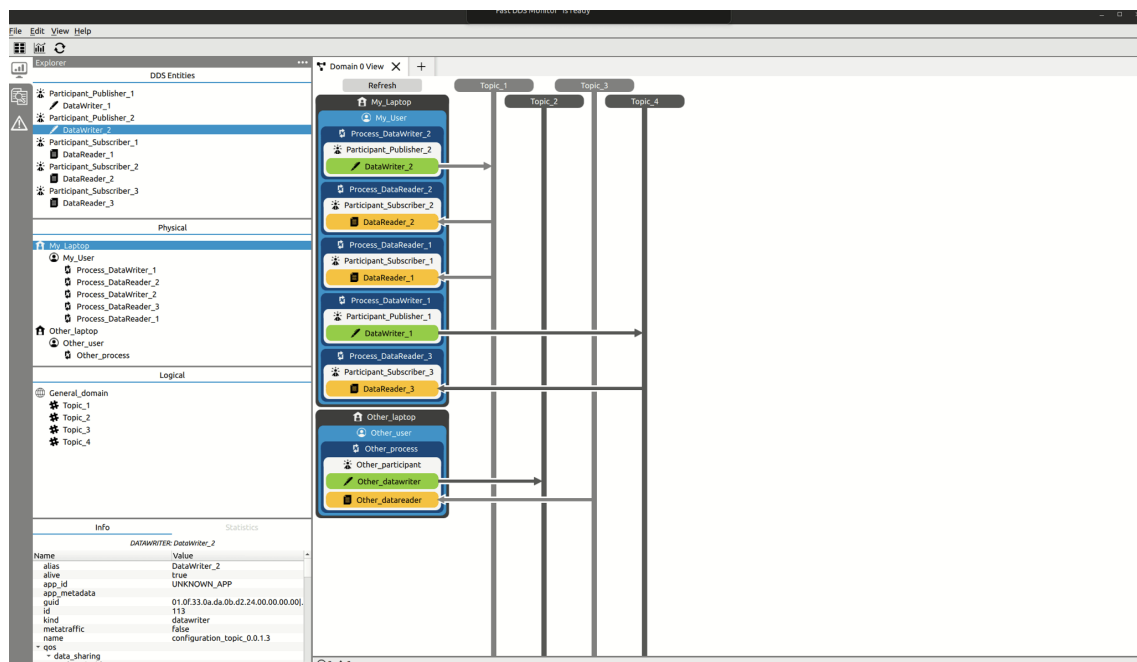


FIGURA 13.1: La figura che mostra il funzionamento del Domain View

- **Cambio alias:** clic destro sull'entità → *Change alias*: rinomina l'entità solo all'interno del monitor, senza modificare la rete DDS reale. Utile per distinguere rapidamente **Broadcastner** e **Listener** che FastDDS identifica entrambi come **RTPSParticipant**.

In questo progetto, la Domain View è stata utilizzata per verificare visivamente che il matching tra il **DataWriter** del **Broadcastner** e il **DataReader** del **Listener** fosse avvenuto correttamente prima di avviare il tracciamento con `trace-cmd`.

13.4.4 Chart View: Visualizzazione Statistica della Comunicazione

La **Chart View** è il cuore analitico del monitor. Permette di creare grafici personalizzati a partire dai topic statistici raccolti. Il monitor distingue due modalità di visualizzazione:

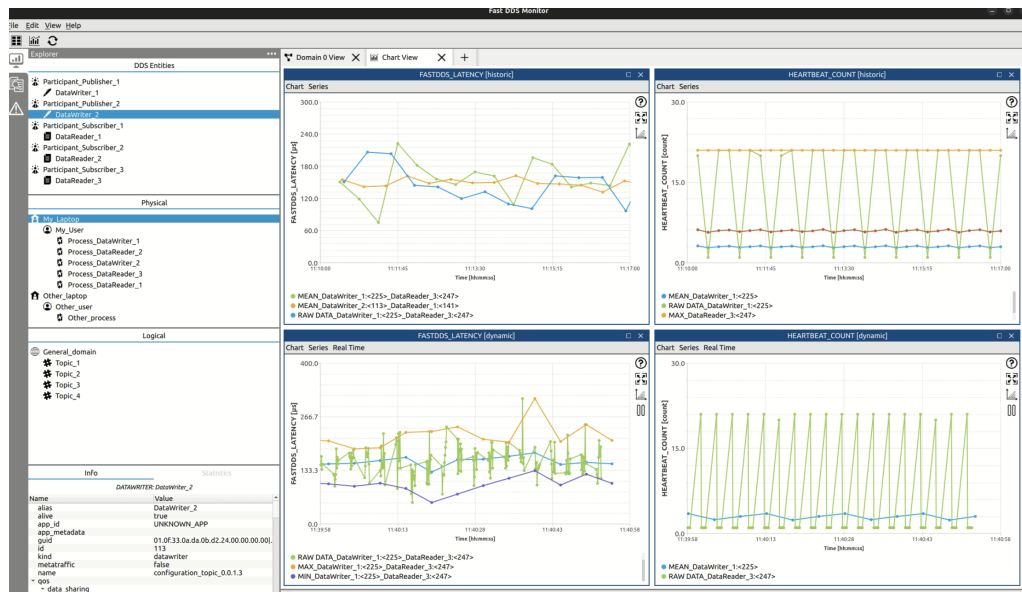


FIGURA 13.2: figura che mostra funzionamento del monitor DDS e del Chart View

Modalità di tracciamento statistico:

La modalità storica interroga tutti i dati raccolti dal monitor dall'avvio fino al momento della creazione del grafico. I parametri configurabili sono:

- **Topic statistico:** il tipo di metrica da visualizzare. I più rilevanti per questo progetto sono:
 - **DATA_COUNT:** numero di campioni pubblicati dal DataWriter in ogni intervallo temporale. Permette di verificare che il publisher stia inviando alla frequenza attesa (un campione ogni 600 ms per il task DDS_Publish).
 - **FASTDDS_LATENCY:** tempo trascorso tra la chiamata `publish()` lato writer e la ricezione del dato nel callback `on_data_available()` lato reader.

Questa è la metrica di latenza end-to-end più significativa per i sistemi RT.

- **PUBLICATION_THROUGHPUT**: byte al secondo pubblicati dal writer, utile per verificare che lo streaming di immagini di **ImageSender** non saturi la banda disponibile.
- **NETWORK_LATENCY**: latenza a livello di rete (RTPS transport), distinta dalla latenza applicativa. In un sistema su loopback o SHM, è tipicamente inferiore al microsecondo.
- **Number of bins**: il numero di intervalli in cui suddividere la finestra temporale. Con **bins = 20**, i dati vengono aggregati in 20 punti. Con **bins = 0**, ogni singolo datapoint ricevuto viene visualizzato individualmente, mostrando la distribuzione grezza.
- **Statistics kind**: il tipo di aggregazione statistica applicata ai campioni in ogni bin:
 - **MEAN**: media aritmetica (latenza media nell'intervallo).
 - **MEDIAN**: mediana (meno sensibile agli outlier rispetto alla media).
 - **MAX**: valore massimo (worst-case della latenza nel bin).
 - **MIN**: valore minimo (best-case).
 - **STANDARD_DEVIATION**: deviazione standard (equivalente del Jitter per la latenza DDS).
 - **SUM**: somma totale (utile con **DATA_COUNT** per contare i messaggi totali inviati).
 - **RAW_DATA**: ogni singolo datapoint senza aggregazione.

Esempio: grafico della latenza Broadcastner→Listener. Per osservare la latenza di comunicazione nel progetto, si seleziona il topic **FASTDDS_LATENCY**, si scelgono come entità sorgente il **DomainParticipant** del **Broadcastner** e come entità destinazione il **DomainParticipant** del **Listener**. Si aggiungono quattro serie con **Statistics kind = MEDIAN, MAX, MIN e STANDARD_DEVIATION** per avere un quadro completo della distribuzione della latenza.

Dati in Tempo Reale

La modalità real-time aggiorna il grafico automaticamente a intervalli configurabili, mostrando una finestra scorrevole dei dati più recenti:

- **Time window:** la durata della finestra temporale visualizzata (es. 1 minuto). I dati più vecchi escono dalla finestra man mano che arrivano nuovi.
- **Update period:** ogni quanti secondi il monitor interroga il backend statistico per nuovi dati (es. ogni 5 secondi). Non rappresenta la frequenza di campionamento DDS, che avviene ad ogni messaggio ricevuto.

Nella modalità real-time, è possibile mettere in pausa l'aggiornamento degli assi per eseguire zoom e ispezione manuale dei dati (il pulsante play/pause), pur continuando a raccogliere nuovi campioni in background. Aggiungendo una serie con `RAW_DATA` si visualizzano i singoli datapoint grezzi, permettendo di comprendere come `MEAN`, `MAX` e `MIN` vengano calcolati a partire da essi.

Sistema di Allerta

Il monitor offre un sistema di allerta configurabile: è possibile definire condizioni che, se soddisfatte, notificano l'utente o eseguono uno script automatico. I tipi di allerta sono:

- **NEW_DATA:** si attiva quando viene ricevuto un `DATA_COUNT` positivo da un'entità corrispondente al filtro. Utile per rilevare che un publisher ha ripreso a inviare dopo un'interruzione.
- **NO_DATA:** si attiva quando il `PUBLICATION_THROUGHPUT` di un'entità scende sotto una soglia configurabile. Permette di rilevare automaticamente un publisher bloccato o lento.

In questo progetto, le allerte non sono state utilizzate attivamente, ma il sistema è stato usato per verificare visivamente che il `Broadcastner` non scendesse mai sotto la soglia di un messaggio per periodo (600 ms).

13.4.5 Utilizzo nel Progetto: Validazione Prima del Tracciamento

Il flusso di utilizzo del monitor DDS in questo progetto è il seguente:

1. **Impostare la variabile d'ambiente** `FASTDDS_STATISTICS` nel terminale prima dell'avvio dei nodi `Broadcastner` e `Listener`.
2. **Avviare il monitor** e selezionare il domain ID corretto (lo stesso usato nel codice, es. 1).
3. **Verificare nella Domain View** che entrambi i processi siano visibili e che il matching `DataWriter` $\leftrightarrow$ `DataReader` sia stato stabilito correttamente.
4. **Creare un grafico** `DATA_COUNT` in modalità real-time per verificare che i messaggi vengano inviati alla frequenza attesa (circa 1.67 messaggi al secondo per il periodo di 600 ms).
5. **Creare un grafico** `FASTDDS_LATENCY` per misurare la latenza end-to-end prima di avviare il tracciamento kernel, in modo da avere un riferimento di confronto.
6. Solo dopo aver validato la comunicazione DDS tramite il monitor, si avvia lo script `marker.sh` per il tracciamento congiunto dei due processi con `trace-cmd`.

Questa procedura garantisce che eventuali problemi di configurazione DDS (es. domain ID sbagliato, QoS incompatibili, topic non trovato) vengano identificati e risolti **prima** di produrre un tracciato kernel che risulterebbe privo di eventi DDS significativi.

13.5 Strumenti di Tracciamento del Kernel: *ftrace*, *trace-cmd* e *KernelShark*

13.5.1 Il Sottosistema *ftrace* del Kernel Linux

ftrace è il framework di tracciamento integrato nel kernel Linux. Rappresenta lo strumento principale per il debug e il profiling del kernel senza necessità di modificare il codice sorgente del kernel o delle applicazioni [16]. Espone un filesystem virtuale denominato *tracefs*, montato tipicamente in `/sys/kernel/debug/tracing/` o `/sys/kernel/tracing/`, attraverso cui si interagisce direttamente con il sottosistema di tracciamento.

Il cuore di *ftrace* è il **buffer circolare per CPU**: il kernel mantiene un ring buffer separato per ogni core fisico. Quando un evento abilitato si verifica, il kernel scrive la relativa entry nel buffer del core su cui è in esecuzione con un timestamp ad alta risoluzione. La struttura per-CPU evita contese su lock globali, rendendo il meccanismo sufficientemente leggero da essere usato in sistemi Real-Time.

I Tracepoint del Sottosistema *sched*

Il sottosistema *sched* del kernel espone diversi tracepoint statici per la schedulazione. In questo progetto vengono utilizzati i seguenti [17]:

- **sched:sched_switch**: registrato ad ogni cambio di contesto della CPU. Ogni evento cattura:
 - **prev_comm**, **prev_pid**, **prev_prio**: nome, PID e priorità del processo che abbandona la CPU.
 - **prev_state**: lo stato con cui il processo uscente si sospende:
 - * **S** (*Sleeping*): sospensione volontaria (es. `clock_nanosleep`).
 - * **R** (*Runnable*): il processo è pronto ma è stato prelazionato da uno a priorità maggiore.
 - * **D** (*Uninterruptible sleep*): bloccato su I/O o mutex kernel.
 - **next_comm**, **next_pid**, **next_prio**: nome, PID e priorità del processo subentrante.

Questo evento è il dato primario da cui il parser C++ ricostruisce gli intervalli RUN, SLEEP e PREEMPT di ogni task

- **sched:sched_wakeup**: registrato quando un thread viene estratto dallo stato di sleep e inserito nella coda di pronto. La differenza temporale tra questo evento e il successivo **sched_switch** che porta il thread in esecuzione è la **latenza di attivazione** (*wakeup latency*), una metrica critica nei sistemi RT.
- **tracing_mark_write**: questo non è un tracepoint kernel ma un meccanismo di iniezione dallo spazio utente. Qualsiasi stringa scritta dall'applicazione nel file virtuale `/sys/kernel/debug/tracing/trace_marker` (tramite la funzione `write_trace_marker()` del progetto) viene inserita nel ring buffer del core corrente con il timestamp al nanosecondo del kernel, sincronizzato con gli eventi **sched_switch**. Questa sincronizzazione è il fondamento dell'intera pipeline di analisi: permette di correlare i marker applicativi (es. `PERIOD_START_Activity_1`) con i cambi di contesto reali del kernel.

13.5.2 trace-cmd: Interfaccia a Riga di Comando per ftrace

`trace-cmd` è il tool in spazio utente che semplifica l'uso di `ftrace` [17]. Internamente, `trace-cmd record` crea un processo di registrazione separato per ogni CPU disponibile: questi processi leggono continuamente il ring buffer del proprio core e scrivono i dati in file temporanei. Al termine della sessione, tutti i file temporanei vengono concatenati nel file binario compresso `trace.dat`.

Utilizzo in `run_trace.sh`: Tracciamento di un Singolo Processo

Negli script `run_trace.sh` e `run_trace_dds.sh`, `trace-cmd record` viene invocato nella sua modalità più diretta: avvia l'eseguibile e registra simultaneamente gli eventi del kernel per tutta la sua durata:

```

1  sudo trace-cmd record \
2  -e sched:sched_switch \    # abilita il tracepoint sched_switch
3  -e sched:sched_wakeup \    # abilita il tracepoint sched_wakeup
4  -o "$OUTPUT_DIR/trace.dat" \ # file di output binario nella
    sandbox

```

```
5 "$EXECUTABLE" # avvia l'eseguibile e lo traccia
```

LISTING 13.4: Invocazione di *trace-cmd record* in *run_trace.sh*

I parametri hanno il seguente significato preciso:

- **-e sched:sched_switch**: abilita il tracepoint `sched_switch` nel sottosistema `sched`. Secondo la documentazione ufficiale [17], il formato dell'argomento è `subsystem:event-name`; è anche possibile abilitare tutti gli eventi di un sottosistema omettendo il nome dell'evento (es. `-e sched`).
- **-e sched:sched_wakeup**: abilita il tracepoint `sched_wakeup`. L'aggiunta di questo evento, pur aumentando leggermente il volume del log, è fondamentale per misurare la latenza tra il risveglio di un thread (fine del `clock_nanosleep`) e il momento in cui ottiene effettivamente la CPU (`sched_switch` con `next_pid` corrispondente).
- **-o \$OUTPUT_DIR/trace.dat**: specifica il percorso del file di output. Ogni sessione viene salvata in una sandbox con timestamp univoco (es. `bin/Test/trace_results_app10g_20260427_212626/`), garantendo che nessuna sessione sovrascriva un'altra.
- **Argomento posizionale \$EXECUTABLE**: quando un comando viene passato a `trace-cmd record` come ultimo argomento, il tool lo avvia e interrompe automaticamente la registrazione al termine del processo. Se nessun comando viene specificato, la registrazione continua finché l'utente non preme `Ctrl-C`: questa è la modalità usata negli script `marker.sh`.

In questo caso **non** viene usata l'opzione `-P` (filtraggio per PID) perché l'eseguibile viene avviato direttamente da `trace-cmd`: il tool conosce già il PID del processo figlio e filtra automaticamente. Tutti gli eventi degli altri processi del sistema vengono scartati dal kernel prima ancora di essere scritti nel ring buffer, riducendo il volume del log.

Utilizzo in `marker.sh`: Tracciamento Passivo Multi-Processo

Negli script `marker.sh` e `marker1.sh`, la modalità di utilizzo è sostanzialmente diversa: i processi sono già in esecuzione quando lo script viene avviato, quindi non

si può usare il meccanismo di lancio diretto. Si usa invece l'opzione `-P` per specificare esplicitamente i PID da tracciare:

```

1  # PIDS_CMD viene costruita iterativamente durante la fase di
   aggancio:
2  # PIDS_CMD="-P 9120 -P 9121 -P 9122 -P 9200 -P 9201"
3  # (PID del processo principale + TID di ogni thread interno)
4
5  sudo trace-cmd record \
6  -e sched:sched_switch \
7  -e sched:sched_wakeup \
8  $PIDS_CMD \           # filtra SOLO i thread elencati
9  -o "$OUTPUT_DIR/trace.dat"
10 # Nessun eseguibile: registra finché l'utente preme Ctrl-C

```

LISTING 13.5: Invocazione di `trace-cmd record` in `marker.sh` con filtraggio per PID

La differenza critica rispetto a `run_trace.sh` è l'assenza dell'eseguibile finale: senza un comando da lanciare, `trace-cmd` registra fino all'interruzione manuale (`Ctrl-C`). Questo permette di controllare esattamente la durata della sessione di tracciamento, indipendentemente dalla vita dei processi.

L'opzione `-P` (per PID, equivalente a `-F` che filtra solo il processo figlio, ma per PID già esistenti) istruisce il kernel a registrare eventi di scheduling solo quando il thread `prev_pid` o `next_pid` dell'evento `sched_switch` corrisponde a uno dei PID specificati. Questo filtraggio avviene **nel kernel**, non nel tool in user space: significa che i thread non elencati non generano mai entries nel ring buffer, eliminando completamente il rumore di sistema.

Il File Prodotto: `trace.dat`

Il file `trace.dat` è un archivio binario proprietario del formato `trace-cmd`. Non è leggibile direttamente da un editor di testo: contiene i metadati del sistema (versione del kernel, numero di CPU, mappa degli eventi abilitati con i loro formati), seguiti dai dati grezzi di ogni CPU compressi [17]. La dimensione tipica per una sessione di 5-10 secondi con i soli eventi `sched_switch` e `sched_wakeup` è dell'ordine di pochi MB.

13.5.3 Il Comando `trace-cmd report`: Decodifica del Log Binario

Al termine della registrazione, il file binario `trace.dat` viene decodificato in un report testuale tramite `trace-cmd report`. Negli script del progetto, la decodifica avviene sempre con la stessa sintassi:

```

1  # Necessario perche' trace-cmd record ha girato con sudo
2  sudo chown -R "$(id -u):$(id -g)" "$OUTPUT_DIR"
3
4  # Decodifica: legge trace.dat e produce il report testuale
5  trace-cmd report "$OUTPUT_DIR/trace.dat" > "$OUTPUT_DIR/
   trace_output.txt"

```

LISTING 13.6: Decodifica del log binario negli script `run_trace.sh` e `marker2.sh`

Il comando `trace-cmd report` legge il file `trace.dat`, interpreta i metadati per conoscere il formato di ogni evento (numero e tipo dei campi), e serializza ogni entry del ring buffer in una riga testuale nel formato standard [17]:

```

1  comm-PID [CPU] timestamp: evento: dettagli
2
3  # Esempi reali:
4  Activity_1-8272 [001] 1234.567810: print:
   tracing_mark_write: PERIOD_START_Activity_1
5  Activity_1-8272 [001] 1234.567890: sched_switch: prev_comm=
   Activity_1 prev_pid=8272
6  prev_prio=98 prev_state=S
7  ==> next_comm=swapper/1 next_pid=0 next_prio=120
8  swapper/1-0 [001] 1234.768000: sched_wakeup: comm=
   Activity_1 pid=8272 prio=98
9  target_cpu=001
10 Activity_1-8272 [001] 1234.768050: sched_switch: prev_comm=
   swapper/1 prev_pid=0
11 ==> next_comm=Activity_1 next_pid=8272
12 Activity_1-8272 [001] 1234.768060: print:
   tracing_mark_write: PERIOD_END_Activity_1

```

LISTING 13.7: Formato delle righe in `trace_output.txt`

L'analisi di questa sequenza illustra la correlazione fondamentale che il parser C++ usa.

1. Il marker `PERIOD_START_Activity_1` a $t = 1234.567810$ segnala l'inizio del ciclo.
2. Il `sched_switch` con `prev_state=S` a $t = 1234.567890$ mostra che il task si è sospeso volontariamente (ha chiamato `clock_nanosleep`): inizia un intervallo `SLEEP`.
3. Il `sched_wakeup` a $t = 1234.768000$ mostra che il timer ha risvegliato il task: finisce il `SLEEP`.
4. Il `sched_switch` a $t = 1234.768050$ con `next_pid=8272` mostra che il task ha ottenuto la CPU: il gap tra questo evento e il precedente risveglio ($50 \mu s$) è la latenza di attivazione.
5. Il marker `PERIOD_END_Activity_1` a $t = 1234.768060$ conclude il ciclo.

File prodotto: `bin/Test/trace_results_<app>_<timestamp>/trace_output.txt`

13.5.4 KernelShark: Visualizzazione Grafica del Tracciato

KernelShark è la GUI ufficiale per la visualizzazione dei file `trace.dat`, sviluppata da Steven Rostedt. [16]. Il suo utilizzo è integrato negli script `marker.sh` del progetto:



FIGURA 13.3: logo dell'applicazione

```

1  if command -v kernelshark >/dev/null 2>&1; then
2  (cd "$OUTPUT_DIR" && kernelshark trace.dat) &
3  fi

```

LISTING 13.8: Apertura automatica di KernelShark negli script `marker`

L'apertura avviene in un sottoprocesso in background (&) per non bloccare il completamento dello script. KernelShark viene lanciato nella directory della sandbox corrente in modo che possa trovare il file `trace.dat` con un percorso relativo.

Le funzionalità usate in questo progetto sono:

- **Timeline per core:** ogni riga della vista principale corrisponde a un core fisico e mostra i processi attivi in ogni istante con barre colorate. È possibile identificare visivamente i periodi di **SLEEP** (barre brevi) e i momenti di prelazione (cambio di colore senza transizione allo **swapper**).
- **Filtro per thread:** cliccando su un thread nel pannello laterale, si visualizzano solo gli eventi di quel thread sull'intera timeline, permettendo di isolare

il comportamento di un singolo task (es. solo `Activity_1`) in un tracciato con 20 thread.

- **Visualizzazione dei marker:** gli eventi `tracing_mark_write` (marker scritti dall'applicazione) appaiono come etichette testuali sulla timeline, sovrapposte ai corrispondenti eventi di scheduling. Questo permette di validare visivamente che i marker `PERIOD_START`/`FUNCTION_START`/`FUNCTION_END`/`PERIOD_END` cadano negli istanti attesi.
- **Zoom e cursori:** due cursori verticali permettono di selezionare un intervallo temporale e misurarne la durata con precisione al microsecondo, utile per verificare manualmente i valori di WCET e Jitter calcolati dal parser C++.



FIGURA 13.4: Interfaccia grafica di KernelShark

13.5.5 La Dashboard: `dashboard.py`

Posizione nel progetto: `src/tools/dashboard.py`

Tutte le operazioni descritte nei capitoli precedenti (avvio delle applicazioni, scelta dello script di tracciamento, controllo della registrazione, analisi e visualizzazione) richiedono tradizionalmente l'apertura di più terminali in parallelo, ricordando l'ordine corretto degli script e gestendo manualmente i percorsi. La `dashboard.py` nasce per eliminare questa complessità operativa: è un'interfaccia grafica desktop

realizzata in Python che unifica in un'unica finestra l'intero flusso di lavoro del progetto.

L'applicazione si avvia senza necessità di argomenti, calcola automaticamente la radice del progetto e gestisce nativamente l'escalation dei privilegi (`sudo`) richiedendo la password una sola volta per l'intera sessione, senza memorizzarla su disco in modo permanente. La finestra è organizzata in tre *tab* (schede) principali, selezionabili dalla barra superiore: *Scripts Manager (Single)*, *DDS Studio (Multi)* e *Trace Manager*.

13.5.6 Prerequisiti di Installazione

Appena scaricato il progetto, prima di avviare gli strumenti di tracciamento o la dashboard, è strettamente necessario soddisfare le dipendenze di sistema. All'interno della cartella `bin/tools/` sono forniti due script eseguibili automatizzati per questo scopo:

- `install_project_deps.sh`: provvede ad installare le librerie Python (come `pandas`, `matplotlib`, `tkinter`, `Pillow`) e i pacchetti di sistema essenziali per la compilazione.
- `install_kernelshark.sh`: automatizza l'installazione di `trace-cmd` e dell'interfaccia grafica `kernelshark`, indispensabili per l'acquisizione e la visualizzazione del tracciato kernel.
- **script per lancio della dashboard**: per avviare la dashboard si può lanciare lo script presente in `bin/tools`, in questo modo `./run_dashboard.sh`

Questi script sollevano l'utente dalla noiosa configurazione manuale dell'ambiente.

Tab 1: Scripts Manager

Questa scheda della dashboard è dedicata all'avvio e al tracciamento di applicazioni a singolo processo (come `app10g`). Il flusso è diviso in due passaggi. A sinistra è presente la lista degli script Bash (`marker1.sh`, `run_trace.sh`, ecc.). A destra l'utente seleziona il *Target Process to Trace* da un menu a tendina che scansiona automaticamente la cartella `bin/app`. I bottoni di avvio permettono di lanciare lo script in background (con l'output reindirizzato alla console integrata) oppure in un emulatore di terminale esterno. Il bottone di arresto invia un segnale `SIGINT` simulando il `Ctrl+C`. Se lo script eseguito è di tipo `marker`, al termine viene proposto automaticamente di completare la pipeline di analisi (parsing e generazione grafici).

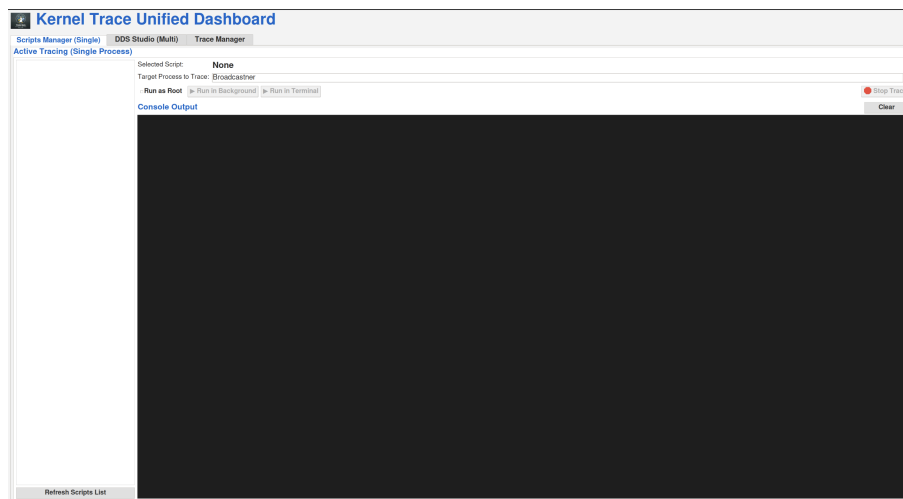


FIGURA 13.5: Tab presente nella dashboard per il tracing di singoli processi

Tab 2: DDS Studio MultiProcesso

Questa scheda estende le funzionalità della precedente per gestire le applicazioni di comunicazione distribuita (come `Broadcastner` e `Listener`). L'utente può popolare una lista multipla di processi target e, con un singolo click su *Launch ALL*, avviare automaticamente tutti i terminali necessari con i rispettivi eseguibili. Allo stesso modo, avviando uno script di tracciamento (es. `marker_dds.sh`), la GUI preleva automaticamente tutti i processi dalla lista e li passa in sequenza allo script Bash, istruendo `trace-cmd` a tracciare l'intero sistema distribuito in modo trasparente.

13.5. Strumenti di Tracciamento del Kernel: *ftrace*, *trace-cmd* e *KernelShark* 47

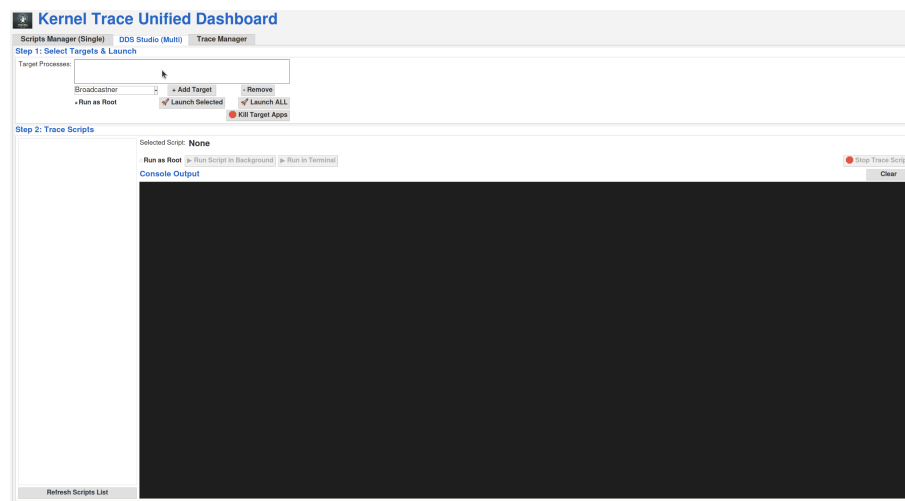


FIGURA 13.6: tab per il tracing di più processi nella dashboard

Tab 3: Trace Manager

Questa scheda funge da archivio e visualizzatore rapido delle sessioni prodotte archiviate in `bin/Test/`. Il *Trace Manager* mostra tutte le sessioni ordinate cronologicamente. Selezionando una voce, il pannello destro mostra l'elenco dei file con le rispettive dimensioni e il resoconto testuale (`risultati_finali.txt`).

I bottoni superiori permettono di interagire con la sessione:

- **Open KernelShark:** avvia l'interfaccia di visualizzazione leggendo il `trace.dat`.
- **View Monitor Image:** apre l'immagine PNG prodotta dal monitor Python.
- **Delete Trace:** elimina la cartella della sessione di test.



FIGURA 13.7

13.5.7 Flow-Chart di Riferimento per ottenere i Documenti di Progetto

Il flusso completo dall'acquisizione alla visualizzazione è riassunto nel flow-chart seguente, che descrive i file prodotti a ogni step.

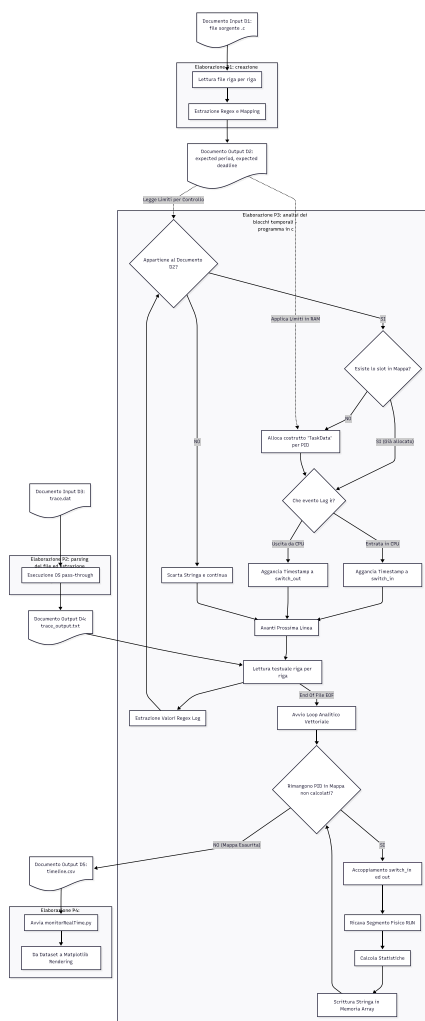


FIGURA 13.8: Flowchart di riferimento e flusso dati

13.6 Librerie Python Utilizzate

I tool sviluppati in questo progetto, come il monitor grafico (`monitor.py`) e l'interfaccia unificata (`dashboard.py`), si affidano a diverse librerie Python:

13.6.1 Librerie per l'Analisi e Visualizzazione dei Dati

- **pandas**: carica il `timeline.csv` in un `DataFrame` strutturato, separa le righe di evento dalle statistiche e raggruppa i dati per task per calcolare l'aggregazione finale.
- **matplotlib**: disegna il diagramma di Gantt, le linee di deadline e la tabella statistica, esportando infine l'immagine in alta risoluzione.
- **numpy**: calcola la deviazione standard degli intervalli tra attivazioni consecutive (Jitter) e normalizza la timeline sottraendo il timestamp iniziale.

13.6.2 Librerie per la GUI e l'Esecuzione di Sistema

- **tkinter e ttk**: librerie standard usata in `dashboard.py` per creare le finestre, i *tab*, le aree di testo, i bottoni e gestire i layout grafici.
- **Pillow (PIL)**: estensione per l'elaborazione delle immagini, utilizzata in `tkinter` per mostrare loghi o icone nell'interfaccia.
- **subprocess e shutil**: essenziali per eseguire gli script Bash in *background*, lanciare emulatori di terminale come `gnome-terminal` e iniettare la password di amministratore per `sudo` senza bloccare la GUI.
- **threading**: utilizzato intensivamente nella dashboard per isolare l'esecuzione dei comandi esterni. Questo evita il blocco della UI durante le registrazioni di `trace-cmd`.
- **os, signal e uuid**: usate per interagire a basso livello con il sistema operativo, gestire percorsi, generare file temporanei sicuri per l'inserimento della password e inviare segnali `SIGINT` per interrompere il tracciamento in corso.

Capitolo 14

Il Lavoro Svolto

All'interno di questo capitolo è presente la descrizione delle cinque fasi di elaborazione del monitor real-time e del caso di studio riguardante il computer di bordo del velivolo F-16. Per ciascuna fase sono indicati i **dati in ingresso**, gli **strumenti utilizzati**, la **logica delle scelte progettuali** e i **risultati prodotti**, corredati da codice sorgente e schermate.

14.1 Fase 1 — Architettura del Progetto e Build System

Dati in ingresso: struttura del workspace di sviluppo.

Strumenti utilizzati: GNU Make, `gcc/g++`, `fastddsgen`.

Risultati prodotti: binari ELF in `bin/app/`, archivi statici `.a` in `bin/lib/`.

Prima di implementare le logiche di schedulazione Real-Time, è necessario costruire un'architettura del workspace in grado di gestire i vari componenti che costituiscono la struttura utilizzata. Questo progetto, che utilizza le librerie in linguaggio C, codice C++, middleware FastDDS, e script Python per l'analisi statistica, necessita che tutti gli elementi precedentemente citati debbano essere gestiti in modo ordinato e univoco.

14.1.1 Struttura delle Directory

La scelta fatta consiste nella **separazione tra sorgenti e artefatti** in due directory distinte: `src/` per il codice e `bin/` per i prodotti della compilazione. Questa scelta garantisce ordine e compattezza alla struttura del codice, permettendo allo sviluppatore di lavorare in un ambiente ottimizzato.

La Directory dei Sorgenti (src/)

- **src/app/**: Contiene i *main()* di ogni processo eseguibile. Ogni applicazione ospita nella propria sottocartella un Makefile dedicato per l'aggiornamento del codice. Questa scelta permette di compilare un singolo applicativo senza scatenare la ricompilazione di tutti gli altri. Le applicazioni presenti sono:
 - **F16_Computer**: simulatore del computer di bordo.
 - **Broadcastner** e **Listener**: nodi publisher e subscriber DDS per messaggi Geometric Point.
 - **ImageSender** e **ImageSubscriber**: scambio di immagini su rete DDS.
 - **app10g**: esempio didattico con due task periodici su core dedicati.
- **src/lib/**: Ospita tutte le librerie:
 - **lib/**: Contiene **activity** (libreria principale dei task periodici), **time** (gestione del tempo), **trace** (iniezione marker nel kernel) e le librerie avioniche F-16. Questi moduli non hanno dipendenze da FastDDS e possono essere compilati ed eseguiti su qualsiasi sistema Linux.
 - **dds_lib/**: contiene un wrapper C++ sopra le API native di FastDDS. Questo perché creare un publisher richiede l'istanziamento di cinque oggetti separati (**DomainParticipant**, **Publisher**, **Topic**, **DataWriter**, **TypeSupport**). Il wrapper riduce tutto questo a una singola chiamata **start()**.
 - **image_lib/**: classi per la serializzazione di frame OpenCV in messaggi DDS **sensor_msgs/Image**.
 - **generated_msg/**: codice generato automaticamente da **fastddsgen** dai file IDL. Il contenuto di questa cartella non viene mai modificato manualmente: è rigenerato ogni volta che cambia un file IDL.
- **src/msg/**: File IDL che definiscono i protocolli di comunicazione DDS (**geometry_msgs.idl**, **sensor_msgs.idl**, **nav_msgs.idl**). Questi file sono la definizione dei tipi di messaggio che le applicazioni DDS possono usare e quando bisogna modificarli e rieseguirli con **fastddsgen** essi vengono rigenerati automaticamente e anche tutti i codici dedicati al loro utilizzo.

- **src/tools/**: Parser C++ (`thread_analysis.cpp`) e monitor Python (`monitor.py`). Si trovano in **src/** poiché sono sorgenti da mantenere e nel caso aggiornare se ci fossero nuove modifiche da applicare.

La Directory degli Artefatti (**bin/**)

- **bin/app/**: Raccoglie tutti i binari in questo modo la ricerca dell'eseguibile all'interno degli script bash che verranno usati (es. `run trace.sh`) è sempre una singola operazione `find $PROJECT_ROOT/bin/app -name $APP_NAME`.
- **bin/lib/**: Raccoglie tutti gli archivi statici `.a`. Gli elementi presenti in questa cartella sono in modo indipendente, quando compilate nelle rispettive directory di appartenenza, messe attraverso i loro makefile fisicamente in questa cartella.
- **bin/tools/**: Script bash che servono a fare il tracing dei vari eseguibili generati dai sorgenti rispettivi, verranno spiegati in modo più completo in questo punto [14.3](#)
- **bin/Test/**: Vengono create delle cartelle dedicate per ogni simulazione con i risultati di ogni sessione di tracciamento. Ogni run genera una sottocartella con timestamp (es. `trace_results_app10g_20260427_212626/`), impedendo che esecuzioni successive sovrascrivano i risultati precedenti e permettendo il confronto storico tra sessioni.

14.1.2 Build System: Architettura Decentralizzata dei Makefile

Perché Non un Makefile Unico?

L'alternativa più semplice sarebbe un unico Makefile alla radice del progetto che gestisce ogni componente e quindi permette di aggiornare ogni componente all'interno del progetto in maniera immediata. Questa soluzione ha però tre problemi gravi per un progetto di questa complessità:

- **Accoppiamento**: ogni modifica al Makefile centrale rischia di impattare la compilazione di componenti non correlati.

- **Scalabilità:** con 20+ componenti, un Makefile monolitico diventa ingestibile e difficile da debuggare.
- **Riusabilità:** è impossibile estrarre un singolo modulo e compilarlo in un altro progetto senza portarsi dietro l'intero Makefile.

Con l'architettura decentralizzata, ogni componente è autonomo: aggiungere un nuovo modulo significa creare una nuova directory con il proprio Makefile, senza toccare nulla di esistente.

Perché il Linking Statico

Il progetto utilizza esclusivamente librerie statiche `.a` invece di librerie condivise `.so`. Nei sistemi Real-Time, il linking dinamico introduce due forme di non-determinismo:

- **Lazy binding:** alla prima chiamata di ogni funzione di libreria dinamica, il linker runtime deve risolvere l'indirizzo della funzione nella tabella PLT (*Procedure Linkage Table*). Questo introduce una latenza variabile — tipicamente decine di microsecondi — alla prima invocazione, che può causare un picco di jitter al primo ciclo di ogni task.
- **Caricamento della libreria:** se una libreria condivisa viene scaricata dalla cache e deve essere ricaricata, il page fault risultante introduce una latenza imprevedibile.

Con il linking statico, tutto il codice è incorporato nel binario a tempo di compilazione. Al momento dell'esecuzione, non c'è nulla da risolvere o caricare dinamicamente: ogni chiamata a funzione è un salto diretto a un indirizzo già noto.

Costruzione Locale e Linking Statico

```
1  # Calcolo del percorso assoluto della root del progetto
2  # partendo dalla posizione del Makefile stesso
3  PROJECT_ROOT = $(abspath ../../..)
4  BIN_DIR      = $(PROJECT_ROOT)/bin/app
5  LIB_DIR      = $(PROJECT_ROOT)/bin/lib
6
7  # Dipendenze esplicite: ogni libreria statica deve essere
```

```

8      # compilata prima che questo target possa essere linkato
9      LIBS_DEPS = $(LIB_DIR)/libActivity_library.a          \
10     $(MSG_LIB_DIR)/libdds_communication.a                \
11     $(wildcard $(MSG_LIB_DIR)/libdds_*.a)

```

LISTING 14.1: Makefile del nodo Listener: risoluzione path e dipendenze

Il costrutto `$(abspath ../../..)` risolve il percorso relativo in percorso assoluto *al momento della valutazione*. Questo è fondamentale: gli script bash invocano `make` da directory arbitrarie tramite la flag `-C`. Senza un percorso assoluto, i percorsi relativi sarebbero risolti rispetto alla directory corrente del processo chiamante, non rispetto alla posizione del Makefile.

La funzione `wildcard` per le librerie DDS cattura tutte le librerie generate da `fastddsgen`: ogni tipo di messaggio IDL produce una libreria separata (es. `libdds_geometry_msgs.a`, `libdds_sensor_msgs.a`), e il numero di queste librerie può variare quando si aggiungono nuovi tipi di messaggio.

Gestione Dinamica delle Dipendenze degli Header

Uno dei problemi classici dei build system è la gestione delle dipendenze dagli header: se si modifica `activity_library.h` senza che Make lo sappia, il binario non viene ricompilato e incorpora una versione obsoleta della struttura.

```

1      # HDR_DEPS cattura tutti gli header che questo modulo include,
2      # diretti o transitivi, attraverso le directory dichiarate
3      HDR_DEPS = $(wildcard *.h) $(wildcard *.hpp)          \
4      $(wildcard $(CORE_HDR)/*.h)                          \
5      $(wildcard $(DDS_HDR)/*.hpp)                         \
6      $(wildcard $(MSG_HDR)/*.h)
7
8      # Il target dipende dal sorgente, dalle librerie E dagli header
9      # Se uno qualsiasi di questi file e' piu' recente del binario,
10     # Make lo ricompila automaticamente
11     $(TARGET): Listener.cpp $(LIBS_DEPS) $(HDR_DEPS)
12     @mkdir -p $(BIN_DIR)
13     @echo ">>> Building $(TARGET)..."

```

```
14 $(CXX) $(CXXFLAGS) $< -o $@ $(LDFLAGS)
```

LISTING 14.2: Tracking automatico degli header per la ricompilazione

Make confronta il timestamp di ogni prerequisito con il timestamp del target. Se almeno un prerequisito è più recente del binario, lo ricompila. Includendo tutti gli header come prerequisiti, si garantisce che qualsiasi modifica a un'interfaccia di libreria propaghi automaticamente la ricompilazione a tutti i moduli che la includono, senza necessità di `make clean`.

Dati in ingresso: file sorgente C/C++ in `src/lib/`.

Risultati prodotti: archivi statici `.a` in `bin/lib/`.

Le librerie mostrate in questo capitolo sono il fondamento su cui poggiano tutte le applicazioni. Ogni libreria ha un'unica funzione ben definita e non dipende da librerie di livello superiore. Questo permette di testare ogni componente in isolamento e di sostituirne uno senza impattare gli altri.

14.1.3 Libreria `time`: Gestione del Tempo Monotónico POSIX

Posizione nel progetto:

- Interfaccia: `src/lib/lib/time/include/time_library.h`
- Implementazione: `src/lib/lib/time/src/time_library.c`

La libreria `time` risolve due problemi sottili che, se ignorati, portano a comportamenti non deterministici all'interno dei sistemi Real-Time.

Perché `CLOCK_MONOTONIC` e non `CLOCK_REALTIME`

Esistono due modalità per gestire il tempo in Linux:

- **`CLOCK_REALTIME`:** Può essere modificato in qualsiasi momento e quindi se sun-corregge un disallineamento di 200 ms mentre un task periodico dorme, il task interessato si sveglierà 200 ms prima del previsto, violando il suo periodo rompendo così uno dei vincoli dei sistemi Real-Time.

- **CLOCK_MONOTONIC**: garantisce che il tempo venga gestito in modo che non possa essere modificato da nessuno in questo modo si garantisce come sorgente sicura per sistemi RT, permettendo di misurare intervalli precisi indipendentemente dal tempo assoluto del sistema.

La scelta di **CLOCK_MONOTONIC** è quindi obbligata per garantire la correttezza della schedulazione periodica in scenari in cui il controllo dei vincoli temporali è critico.

14.1.4 Libreria **trace**: Iniezione dei Marker nel Kernel

Posizione nel progetto:

- Interfacce: `src/lib/lib/trace/include/trace_marker.h` (e `marker.h`)
- Implementazione: `src/lib/lib/trace/src/trace_marker.c`

La libreria **trace** serve per avere un'interfaccia tra lo spazio utente dell'applicazione e il sottosistema **ftrace** del kernel Linux. Il suo scopo è permettere di avere dei marker temporali coincidenti con eventi di scheduling del kernel, i quali hanno come principale funzionalità il facilitare la lettura degli eventi che si verificano durante la simulazione.

Perché scrivere i marker nel Kernel e non in un File Utente

Un'alternativa più semplice sarebbe scrivere i marker in un file di log in spazio utente (es. con `fprintf` su un file). Questa soluzione ha però tre limitazioni critiche:

- **Mancanza di sincronizzazione**: il timestamp del marker sarebbe quello del processo utente che può differire dal timestamp del kernel di decine di microsecondi a causa dei context switch che ci potrebbero essere.
- **Overhead elevato**: `fprintf` su file comporta lock, buffering e potenziali system call bloccanti.

Il file `/sys/kernel/debug/tracing/trace_marker` (o `/sys/kernel/tracing/trace_marker` in sistemi più recenti) risolve tutti questi problemi: la scrittura viene eseguita direttamente nel buffer del kernel con il timestamp del kernel, garantendo un perfetto allineamento temporale con tutti gli eventi critici di sistema, come `sched_switch` e `sched_wakeup`.

Due Versioni: con Stato e Inline

La scelta di offrire due implementazioni risponde a esigenze diverse. La versione con stato (`trace_marker.h/c`) mantiene un file descriptor globale ed è preferibile per applicazioni che scrivono milioni di marker (come i task periodici con periodo di 10 ms): il file descriptor viene aperto una sola volta chiamando la `init_tracing()` e riutilizzato in modo sicuro per le performance e gestendone la chiusura con `close_tracing()`.

La versione inline (nell'header aggiuntivo `marker.h`) utilizza invece una variabile locale statica avendo un'apertura alla prima chiamata. Questo risulta comodo per test semplici dove vengono testati pochi thread.

14.1.5 Libreria `activity`: Struttura del Task Periodico

Posizione nel progetto:

- Interfaccia: `src/lib/lib/activity/include/activity_library.h`
- Implementazione: `src/lib/lib/activity/src/activity_library.c`

La libreria `activity` è il componente più importante. Il suo obiettivo è fornire l'implementazione completa di un task periodico Real-Time, nascondendo allo sviluppatore tutta la complessità nella gestione di scrittura dei marker e di implementazione della periodic task dove l'implementazione viene trattata in questo punto [14.2.1](#).

Il Problema del Drift Cumulativo e la sua Soluzione

Il modo più semplice di implementare un task periodico è usare uno sleep relativo come `usleep(periodo_in_microsecondi)`. Tuttavia, questo approccio è profondamente errato. Se il task impiega t_{exec} ms, il prossimo risveglio sarà a $t_{start} + t_{exec} + T_{sleep}$, non a $t_{start} + T_{periodo}$. Andando avanti la simulazione accumula questi ritardi e i tasks perdono la sincronizzazione con il tempo reale (fenomeno del *drift cumulativo*). La libreria `activity` risolve questo problema con l'utilizzo di `clock_nanosleep` combinata al flag `TIMER_ABSTIME`, è proprio in questo modo che il `PeriodicTask` acquisisce il tempo corrente all'inizio, e in ogni ciclo calcola in

anticipo l'istante assoluto in cui risvegliarsi ignorando totalmente quanto tempo sia stato effettivamente speso in esecuzione.

Utilizzo dei marker da parte del parser. All'interno del ciclo, `PeriodicTask` chiama `write_trace_marker` cinque volte per delimitare il periodo. Questi marker generano una sequenza di eventi nel log del kernel che il parser C++ usa per estrarre le metriche:

- **PERIOD_START → PERIOD_START successivo:** periodo reale misurato. Confrontato con il periodo scelto per calcolare il Jitter.
- **FUNCTION_START → FUNCTION_END:** misurazione del WCET e ACET della funzione applicativa, isolata dalle attese.
- **PERIOD_START → FUNCTION_END:** Response Time, che include eventuali periodi di prelazione o interferenze subite prima che il task ottenesse realmente la CPU.
- **FUNCTION_END → PERIOD_END:** Slack Time, ovvero il tempo rimanente speso a dormire prima della scadenza della deadline o inizio del nuovo periodo.

La Funzione di Carico Computazionale: `activity_load`

Questa libreria espone anche la funzione `activity_load`, il cui scopo è generare un carico computazionale che serve a simulare il reale sforzo di una CPU in un contesto realistico.

Perché `volatile`: il metodo si basa su un ciclo `for` che calcola il risultato di operazioni matematiche assegnando alla variabile `result` il risultato. Senza la keyword `volatile`, il compilatore moderno (`gcc/g++`) analizzerebbe il ciclo, concluderebbe che `result` non viene mai letto o restituito all'esterno, e **eliminerebbe completamente l'intero loop** provocando la mancata visione all'interno del monitor dell'effettiva esecuzione della periodic task. La keyword `volatile` indica al compilatore che il valore potrebbe essere letto da entità esterne impedendo così in modo rigoroso qualsiasi ottimizzazione del ciclo.

Perché operazioni trigonometriche. Il parametro `cost` permette di calibrare il carico computazionale, ogni unità corrisponde a 10^6 iterazioni, dando in questo modo allo sviluppatore la possibilità di modificare agevolmente lo sforzo computazionale della propria periodic task.

14.1.6 Libreria `dds_lib`: Wrapper al Middleware FastDDS

Questa astrazione si compone in due parti: il modulo `Publisher` (`Broadcastner`) e il modulo `Subscriber` (`Listener`).

Modulo `Broadcastner`

Posizione nel progetto:

- Interfaccia: `src/lib/dds_lib/broadcastner/include/Broadcastner.hpp`
- Implementazione: `src/lib/dds_lib/broadcastner/src/Broadcastner.cpp`

Implementazione: Per creare il `Broadcastner` per creare un `DomainParticipant` è stato implementato un `DomainParticipantFactory` per poi registrare il `TypeSupport` e creare un `Publisher` che comunica su un `Topic` e infine la creazione di un `DataWriter`. Ognuno di questi passi richiede la gestione dei parametri QoS (*Quality of Service*) fondamentali per la comunicazione in ambiente DDS. Senza il wrapper, ogni applicazione che vuole pubblicare dati dovrebbe ripetere e mantenere questa ingombrante sequenza di decine di righe di codice. Il wrapper `Broadcastner` incapsula l'intera complessità in due semplici metodi: un `start()` che prende il tipo di messaggio, topic e dominio configurando tutto, e un `publish()` che casta e invia il payload DDS in modo astratto. Internamente, il metodo `start` configura anche il `PubListener`, un listener interno che intercetta in modo trasparente gli eventi asincroni come `on_publication_matched`. Questo permette di stampare a terminale log essenziali quando un subscriber si connette o si disconnette dal topic, un aspetto fondamentale per il debugging e la validazione della fase di discovery della rete distribuita.

Modulo Listener

Posizione nel progetto:

- Interfaccia: `src/lib/dds_lib/listener/include/Listener.hpp`
- Implementazione: `src/lib/dds_lib/listener/src/Listener.cpp`

Il modulo `Listener` implementa il lato subscriber in ricezione. A differenza di molti approcci, la sua architettura è puramente event-driven: invece di fare un polling attivo ed estenuante per verificare se sono arrivati nuovi messaggi (il che consumerebbe CPU inutilmente distruggendo l'efficienza RT del sistema), è il middleware DDS stesso a chiamare automaticamente il callback di callback del metodo `on_data_available()` ogni volta che un messaggio utile viene consegnato.

Ed è all'interno di questo metodo che sono stati iniettati i kernel marker: ciò consente di individuare con assoluta precisione, nel file di trace, l'istante in cui il `Listener` riceve e notifica un messaggio sulla rete DDS. Il `Listener` a sua volta incapsula la creazione e la registrazione sicura del `DomainParticipant`, `Subscriber` e `DataReader`.

14.1.7 Libreria `generated_msg`: Classi di Messaggistica DDS

Posizione nel progetto:

- Destinazione generati: `src/lib/generated_msg/`
- IDL in ingresso: `src/msg/`

La sottocartella `generated_msg` è il punto di intersezione tra le definizioni formali e neutrali dei messaggi (file IDL, *Interface Definition Language*) e il codice C++ direttamente utilizzabile dalle applicazioni.

Il Processo di Generazione

La compilazione esegue il tool `fastddsgen` (fornito dal framework) che legge in automatico ogni file IDL e produce le implementazioni C++, generando:

- **La struttura dati del messaggio:** per ogni campo definito nell'IDL (es. `float64 x; float64 y; float64 z;` per un `Point`), viene tradotto in una classe C++ nativa completa di getter, setter fortemente tipizzati e operatori di assegnamento.
- **La serializzazione CDR:** metodi ottimizzati come `serialize()` e `deserialize()` che si preoccupano di impacchettare in bytes la struttura in formato CDR (*Common Data Representation*), lo standard dell'Object Management Group (OMG). Il CDR è vitale in quanto garantisce l'interoperabilità di rete assoluta tra nodi in architetture hardware o OS profondamente diversi (assicurando l'integrità fra macchine little-endian e big-endian).
- **Il TypeSupport:** come la classe `XxxPubSubType`, che registra formalmente il nuovo tipo custom nel middleware DDS e fornisce tutte le funzioni di supporto hash necessarie per la discovery logica e il matching rapido tra publisher e subscriber sulla rete.

14.1.8 Libreria `math`: Funzioni Algebriche Ausiliarie

Posizione nel progetto:

- Interfaccia: `src/lib/lib/math/include/math_library.h`
- Implementazione: `src/lib/lib/math/src/math_library.c`

La libreria `math` è una libreria di supporto minimale che implementa operazioni aritmetiche banali come il recupero della somma e del valore massimo dagli elementi di un array di interi.

Queste funzioni sono usate principalmente negli applicativi di test ed entry point per calcolare metriche aggregate come le somme di carichi computazionali richiesti o per trovare i picchi massimi tra le iterazioni, evitando ogni volta la reimplementazione di logiche simili e l'inclusione di pesanti macro framework. La loro esistenza come libreria C pura e separata — anziché come funzioni inline ingombranti all'interno dei file `.c` applicativi — è motivata dalla ferrea volontà architetturale di mantenere ogni file sorgente focalizzato solo ed unicamente sulla propria responsabilità principale, in questo modo, i file che definiscono le applicazioni RT descrivono solo la complessa logica di scheduling e la composizione delle activity, ma non come eseguire algoritmi matematici sui dati di configurazione.

14.2 Fase 3 — Le Applicazioni del Progetto

Dati in ingresso: librerie core compilate in `bin/lib/`.

Strumenti utilizzati: primitive POSIX, middleware FastDDS, OpenCV.

Risultati prodotti: binari in `bin/app/` pronti per il tracciamento.

14.2.1 Applicazione `app10g`: Due Task su Core Distinti

L'applicazione `app10g` (`src/app/app10g/app10g.c`) è l'esempio didattico di riferimento: due task periodici su core dedicati, senza alcuna interferenza da prelazione. Il codice viene mostrato e interrotto nei punti rilevanti per evidenziare il funzionamento del codice ed il perché delle scelte di progetto.

Dipendenze e Gestione degli Errori:

Il file apre con la direttiva `_GNU_SOURCE`, indispensabile per sbloccare le estensioni GNU/POSIX non standard (tra cui `pthread_setname_np` e `pthread_attr_setaffinity_np`), e include i tre header del progetto: `time_library.h` (gestione del tempo POSIX), `trace_marker.h` (scrittura su `ftrace`) e `activity_library.h` (struttura dati e task periodico).

```
1  #define _GNU_SOURCE
2  #include <errno.h>
3  #include <math.h>
4  #include <pthread.h>
5  #include <sched.h>
6  #include <stdbool.h>
7  #include <stdint.h>
8  #include <stdio.h>
9  #include <stdlib.h>
10 #include <string.h>
11 #include <time.h>
12
13 #include "time_library.h"
14 #include "trace_marker.h"
15 #include "activity_library.h"
```

LISTING 14.3: `app10g.c` – direttive e include

La macro `handle_error` incapsula il pattern standard di gestione degli errori POSIX: se il codice di ritorno è diverso da zero, imposta `errno`, stampa il messaggio su `stderr` tramite `perror` e termina il processo con `EXIT_FAILURE`.

```
1  #define handle_error(en, msg)
2      \
3  if (en != 0) {
4      \
5      errno = en;
6      \
7      perror(msg);
8      \
9      exit(EXIT_FAILURE);
10     \
```

```
6     }
```

LISTING 14.4: `app10g.c` – macro di gestione errori

Inizializzazione del Tracing e Dichiarazione delle Variabili

All'ingresso del `main`, la prima operazione è `init_tracing()`: apre il file descriptor verso `/sys/kernel/debug/tracing/trace_marker` (o il percorso alternativo `/sys/kernel/tracing/trace_marker` su kernel più recenti). Questo file è il canale privilegiato attraverso cui il codice utente inietta eventi nel ring buffer del kernel, rendendoli visibili a `trace-cmd` e `KernelShark`.

```
1     int main(int argc, char *argv[]) {
2         // Inizializza il file descriptor per i marker di
3         // tracciamento
4         init_tracing();
5
6         // Setup per i thread 1 e 2
7         pthread_t thread1, thread2;
8         pthread_attr_t attr1, attr2;
9         struct sched_param param1, param2;
10        int ret_err;
11
12        // Definiamo due maschere diverse per i core
13        cpu_set_t cpuset_core1;
14        cpu_set_t cpuset_core2;
```

LISTING 14.5: `app10g.c` – apertura del tracing e variabili

Le strutture `pthread_attr_t` conterranno tutta la configurazione del thread prima della creazione; `struct sched_param` trasporta la priorità RT; `cpu_set_t` è la maschera di affinità CPU, un bitmask in cui ogni bit corrisponde a un core logico.

Inizializzazione degli Attributi e Policy RT

I due oggetti attributo vengono inizializzati con `pthread_attr_init` e immediatamente configurati con la policy `SCHED_FIFO`. Il flag `PTHREAD_EXPLICIT_SCHED` è cruciale: senza di esso il figlio erediterebbe la policy `SCHED_OTHER` del processo padre, non permettendo la configurazione real-time.

```

1      pthread_attr_init(&attr1);
2      pthread_attr_init(&attr2);
3
4      // Impostazione della policy SCHED_FIFO (Real-Time) per
    entrambi
5      pthread_attr_setinheritsched(&attr1, PTHREAD_EXPLICIT_SCHED
    );
6      pthread_attr_setschedpolicy(&attr1, SCHED_FIFO);
7
8      pthread_attr_setinheritsched(&attr2, PTHREAD_EXPLICIT_SCHED
    );
9      pthread_attr_setschedpolicy(&attr2, SCHED_FIFO);
10

```

LISTING 14.6: app10g.c – init attributi e policy SCHED_FIFO

Con SCHED_FIFO, lo scheduler Linux (assegna la CPU al thread con la priorità più alta tra quelli pronti; un thread RT non viene mai prelazionato da thread a priorità inferiore se si usa come algoritmo di schedulazione RM(Rate Monotonic).

Affinità di Core

La sezione seguente vincola ogni thread a un core fisico, per permettere di testare varie simulazione in cui si cambiano i core fisici su cui i thread girano effettivamente, distinto tramite la maschera `cpu_set_t`. `CPU_ZERO` azzera la maschera, `CPU_SET(n, ...)` imposta il bit corrispondente al core *n*, e `pthread_attr_setaffinity_np` registra la maschera nell'attributo del thread.

```

1      // =====
2      // CONFIGURAZIONE AFFINITA' CORE
3      // =====
4
5      // Set per il Thread 1 -> CORE 1
6      CPU_ZERO(&cpuset_core1);
7      CPU_SET(1, &cpuset_core1);
8      pthread_attr_setaffinity_np(&attr1, sizeof(cpu_set_t), &
    cpuset_core1);
9
10     // Set per il Thread 2 -> CORE 2
11     CPU_ZERO(&cpuset_core2);

```

```

12     CPU_SET(2, &cpuset_core2);
13     pthread_attr_setaffinity_np(&attr2, sizeof(cpu_set_t), &
14     cpuset_core2);

```

LISTING 14.7: app10g.c – affinità di core

L'isolamento su core distinti garantisce che i due task non si contendano mai la stessa risorsa hardware, nulla vieta di modificare il codice in modo da vedere come va la simulazione nel caso di prelazione reciproca.

Configurazione dei Thread

Ogni thread viene descritto da una struttura `t_activity_par` che aggrega nome, funzione di carico, periodo, parametro e deadline. La priorità Rate Monotonic viene calcolata come $99 - \lfloor T/10 \rfloor$, poi saturata a 1 per evitare valori non validi.

```

1      // =====
2      // -- THREAD 1 -- (Core 1)
3      // =====
4      t_activity_par activity_1;
5      snprintf(activity_1.name, 15, "Activity_1");
6      activity_1.function = activity_load; // Funzione importata
7      dalla libreria
8      activity_1.period = 1000;
9      activity_1.parameter = 10;
10     activity_1.deadline = 1000;
11
12     param1.sched_priority = 99 - (activity_1.period / 10); // =
13     1 (saturato)
14     if (param1.sched_priority < 1) param1.sched_priority = 1;
15     pthread_attr_setschedparam(&attr1, &param1);
16
17     ret_err = pthread_create(&thread1, &attr1, PeriodicTask, (
18     void *)&activity_1);
19     handle_error(ret_err, "Error in creating PeriodicTask 1");
20     pthread_setname_np(thread1, "Activity_1");

```

LISTING 14.8: app10g.c – Thread 1: Core 1, periodo 1000 ms

```

1      // =====
2      // -- THREAD 2 -- (Core 2)
3      // =====
4      t_activity_par activity_2;
5      snprintf(activity_2.name, 15, "Activity_2");
6      activity_2.function = activity_load; // Funzione importata
    dalla libreria
7      activity_2.period    = 1200;
8      activity_2.parameter = 20;
9      activity_2.deadline  = 1200;
10
11     param2.sched_priority = 99 - (activity_2.period / 10);
12     if (param2.sched_priority < 1) param2.sched_priority = 1;
13     pthread_attr_setschedparam(&attr2, &param2);
14
15     ret_err = pthread_create(&thread2, &attr2, PeriodicTask, (
    void *)&activity_2);
16     handle_error(ret_err, "Error in creating PeriodicTask 2");
17     pthread_setname_np(thread2, "Activity_2");
18

```

LISTING 14.9: app10g.c – Thread 2: Core 2, periodo 1200 ms

Il nome assegnato con `pthread_setname_np` (massimo 15 caratteri, limite imposto dal kernel) è quello che appare direttamente in KernelShark e nell'output di `trace-cmd report`, permettendo di identificare immediatamente il task responsabile di ogni evento nel ring buffer.

Cleanup e Attesa

Una volta creati i thread, gli oggetti attributo non servono più: vengono distrutti per liberare le risorse interne allocate da `pthread_attr_init`. Il `main` si blocca poi sulle due `pthread_join`, che in un sistema embedded reale non ritorneranno mai (i task girano all'infinito nel loro loop `while(1)`); tuttavia la chiamata è necessaria per impedire che il processo termini immediatamente. Prima di uscire, `close_tracing()` chiude il file descriptor verso `trace_marker`, assicurando che tutti i byte in sospeso vengano svuotati.

```

1      // Pulizia attributi
2      pthread_attr_destroy(&attr1);

```

```
3     pthread_attr_destroy(&attr2);
4
5     // Attesa completamento (in un sistema reale girano all'
    infinito)
6     pthread_join(thread1, NULL);
7     pthread_join(thread2, NULL);
8
9     // Chiude il file descriptor dei marker
10    close_tracing();
11
12    exit(0);
13 }
```

LISTING 14.10: app10g.c – pulizia e join

Caratteristiche Rilevanti

- **core distinti:** i due task non condividono mai la stessa risorsa hardware; la maschera di affinità garantisce che il kernel non migri mai il thread verso un core diverso.
- **Carico differenziato:** la variabile parameter attraverso la funzione `activity_load`, permette di osservare tempi di esecuzione differenti nelle tracce, infatti il parametro decide all'interno della funzione quanti cicli di CPU vengono impiegati producendo effetti diversi.
- **Tracciamento automatico:** l'strumentazione completa (4 marker per periodo: `PERIOD_START`, `FUNCTION_START`, `FUNCTION_END`, `SLEEP_START/PERIOD_END`) è interamente delegata a `PeriodicTask`, rendendo `app10g.c` libero da codice di tracing.

PeriodicTask eseguita dall' applicazione:

`PeriodicTask` è la funzione eseguita da ogni thread. Segue lo schema classico del task periodico POSIX: campiona il tempo assoluto iniziale con `CLOCK_MONOTONIC`, poi entra in un loop infinito che:

1. Scrive il marker `PERIOD_START_<name>` nel ring buffer del kernel.

2. Calcola il prossimo istante di rilascio aggiungendo il periodo al tempo corrente.
3. Scrive il marker `FUNCTION_START_<name>` e avvia la misurazione del tempo.
4. Esegue la funzione di carico (con la firma corretta).
5. Scrive il marker `FUNCTION_END_<name>`.
6. Scrive il marker `SLEEP_START_<name>` e si sospende con `clock_nanosleep` in modalità `TIMER_ABSTIME`.
7. Alla ripresa scrive `PERIOD_END_<name>` e ricomincia.

Iniziamo analizzando la fase di setup del task, dove vengono preparate le variabili per la temporizzazione e per la generazione dei marker nel kernel:

```

1  /*
2  * activity_library.c
3  */
4  #include "activity_library.h"
5  #include "trace_marker.h"
6  #include <math.h>
7  #include <stdio.h>
8
9  void *PeriodicTask(void *ptr) {
10     t_activity_par activity;
11     activity = *((t_activity_par *)ptr);
12
13     struct timespec exec_release_time;
14     uint64_t exec_start_time;
15     uint64_t exec_end_time;
16     char marker[128];
17
18     // BASELINE: acquisisce il tempo corrente come riferimento
19     // assoluto
20     clock_gettime(CLOCK_MONOTONIC, &exec_release_time);

```

LISTING 14.11: activity_library.c – Inizializzazione di PeriodicTask

A questo punto inizia il ciclo infinito. L'uso di `TIMER_ABSTIME` in `clock_nanosleep` è fondamentale per la correttezza temporale: il thread si sveglia all'istante assoluto $t_0 + k \cdot T$, indipendentemente da quanto tempo ha impiegato la funzione di carico. Un'attesa relativa (`TIMER_RELTIME`) accumulerebbe invece l'errore di ogni iterazione, causando una deriva del periodo nel tempo.

```

1      while (1) {
2          snprintf(marker, sizeof(marker), "PERIOD_START_%s",
activity.name);
3          write_trace_marker(marker);
4
5          // CALCOLO PREVENTIVO del prossimo rilascio
6          time_add_millisecs(&exec_release_time, activity.period)
7
8          ;
9
10         snprintf(marker, sizeof(marker), "FUNCTION_START_%s",
activity.name);
11         write_trace_marker(marker);
12
13         exec_start_time = time_current_millisecs();

```

LISTING 14.12: activity_library.c – Ciclo infinito di PeriodicTask

Il blocco successivo esegue l'effettivo carico computazionale. Per garantire la compatibilità con le diverse funzioni utente, il codice esegue un controllo sul puntatore a funzione:

```

1      // Chiamata con entrambi i parametri per compatibilità
2      if (activity.function == (void (*)(void *, int))
activity_load) {
3          void (*func_single)(int) = (void (*)(int))activity.
function;
4          func_single(activity.parameter);
5      } else {
6          activity.function(activity.instance, activity.
parameter);
7      }
8
9      exec_end_time = time_current_millisecs();

```

```

10
11         snprintf(marker, sizeof(marker), "FUNCTION_END_%s",
activity.name);
12         write_trace_marker(marker);
13
14         snprintf(marker, sizeof(marker), "SLEEP_START_%s",
activity.name);
15         write_trace_marker(marker);
16
17         // BLOCCO ASSOLUTO anti-deriva
18         clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME, &
exec_release_time, NULL);
19
20         snprintf(marker, sizeof(marker), "PERIOD_END_%s",
activity.name);
21         write_trace_marker(marker);
22     }
23
24     pthread_exit((void *)ptr);
25 }

```

LISTING 14.13: activity_library.c – Esecuzione carico e chiusura periodo

activity_load:

activity_load genera un carico computazionale controllato attraverso un ciclo di operazioni floating-point (sin, cos, tan) su cost milioni di iterazioni.

```

1     void activity_load(int cost) {
2         volatile double result = 0;
3         for (long i = 0; i < (long)cost * 1000 * 1000; i++) {
4             result = result + sin(i) + cos(i) + i * tan(i);
5         }
6     }

```

LISTING 14.14: activity_library.c – Funzione activity_load

La variabile `result` è dichiarata `volatile` per impedire al compilatore di ottimizzare ed eliminare il ciclo.

14.2.2 Applicazione Broadcastner: Publisher DDS Periodico

Posizione nel progetto: `src/app/Broadcastner/Broadcastner.cpp`

Il Broadcastner è il nodo publisher della rete DDS: pubblica periodicamente messaggi `geometry_msgs/Point` sul topic `PointTopic`. Il processo ospita due thread a ruoli distinti: `DDS_Publish` (Core 1), responsabile della pubblicazione DDS, e `Activity_1` (Core 0), che esegue un carico computazionale puro senza produrre messaggi. I due thread cooperano nello stesso processo, ma sono isolati su core distinti.

Include e Variabili Globali di Modulo

Il file include i tre layer del progetto: la libreria DDS applicativa (`Broadcastner.hpp`), la libreria di scheduling (`activity_library.h`) e l'infrastruttura di tracing (`trace_marker.h`). Le due variabili statiche di modulo, `g_broadcastner` e `g_counter`, sono condivise tra le funzioni del file ma invisibili agli altri translation unit: l'uso di `static` a livello di file garantisce l'incapsulamento senza ricorrere a classi C++.

```
1  /*
2  * Broadcastner.cpp
3  */
4
5  #include "Broadcastner.hpp"
6  #include "activity_library.h"
7  #include "trace_marker.h"
8
9  #include <cstdint>
10 #include <cstring>
11 #include <fastrtps/rtps/common/Time_t.h>
12 #include <geometry_msgsPubSubTypes.h>
13 #include <pthread.h>
14 #include <sched.h>
15 #include <stdio.h>
16
17 // Usiamo il namespace generato da fastddsgen v2+
18 using namespace geometry_msgs::msg::dds_;
19
```

```
20 static Broadcastner g_broadcastner;  
21 static double g_counter = 0.0;
```

LISTING 14.15: Broadcastner.cpp – Include e variabili globali di modulo

La Funzione di Pubblicazione con Marker

dds_publish_function è la funzione eseguita dal PeriodicTask ad ogni periodo. La struttura segue uno schema preciso: prima genera il carico computazionale sintetico con activity_load, poi prepara il messaggio, e infine lo pubblica racchiuso tra i due marker DDS_MSG_START/END. Questi due marker svolgono un ruolo chiave nell'analisi: il parser li riconosce e li usa per isolare nel log del kernel l'intervallo di latenza DDS — il tempo reale che il middleware impiega per serializzare il payload CDR e trasmetterlo sulla rete — separandolo dal tempo di calcolo puro. Il risultato viene esportato nelle statistiche di comunicazione in timeline.csv.

```
1 void dds_publish_function(void *instance, int parameter) {  
2     (void)instance;  
3  
4     // Carico computazionale calibrato (WCET sintetico)  
5     activity_load(parameter);  
6  
7     // Costruzione del messaggio Point con coordinate  
8     progressive  
9     g_counter += 0.1;  
10    Point_ point_msg;  
11    point_msg.x(1.0 * g_counter);  
12    point_msg.y(2.0 * g_counter);  
13    point_msg.z(3.0 * g_counter);  
14  
15    // Marker [DDS_START]: inizio serializzazione e invio sul  
16    DataWriter  
17    write_trace_marker("DDS_MSG_START_DDS_Publish");  
18    g_broadcastner.publish(&point_msg);  
19    // Marker [DDS_END]: il middleware ha consegnato il  
20    messaggio al layer RTPS  
21    write_trace_marker("DDS_MSG_END_DDS_Publish");  
22 }
```

```

20     printf("DDS_Publish: sent Point(%.1f, %.1f, %.1f)\n",
21           point_msg.x(), point_msg.y(), point_msg.z());
22 }

```

LISTING 14.16: Broadcastner.cpp – Funzione di pubblicazione con marker DDS

Accanto a `dds_publish_function` c'è anche la funzione `computation_function`, un semplice wrapper che chiama solo `activity_load` senza produrre alcun messaggio. Questa separazione permette di avere nel processo un secondo thread puramente computazionale, utile per osservare nelle tracce come un carico esterno su Core 0 non interferisce con la temporizzazione di pubblicazione su Core 1.

```

1     void computation_function(void *instance, int parameter) {
2         (void)instance;
3         activity_load(parameter);
4     }

```

LISTING 14.17: Broadcastner.cpp – Funzione di carico puro (senza DDS)

Inizializzazione DDS e Creazione dei Thread

Il main apre il file descriptor verso `trace_marker`, inizializza il middleware DDS tramite il wrapper `Broadcastner` e procede con la creazione dei due thread RT.

```

1     int main(int argc, char **argv) {
2         init_tracing();
3
4         // Crea DomainParticipant, Publisher, Topic e DataWriter in
         una sola chiamata
5         eprosima::fastdds::dds::TypeSupport msg_type(new
Point_PubSubType());
6         g_broadcastner.start(&msg_type, "Point", "PointTopic", 1);
7         printf("\nDDS inizializzato. Press Ctrl-C to stop.\n\n");
8     }

```

LISTING 14.18: Broadcastner.cpp – Avvio DDS nel main

La chiamata a `g_broadcastner.start()` è l'unico punto in cui il codice tocca direttamente il middleware: tutta la complessità di configurazione QoS, creazione del `DomainParticipant` e registrazione del `TypeSupport` è incapsulata nel wrapper

presente in `dds_lib`. A seguire, i thread vengono configurati e creati secondo la sequenza POSIX standard già descritta nella Sezione 14.1.5. Il thread `DDS_Publish` è vincolato al Core 1 con priorità Rate Monotonic $99 - (600/10) = 39$.

```

1      cpu_set_t cpuset;
2      CPU_ZERO(&cpuset);
3      CPU_SET(1, &cpuset);    // Isolamento su Core 1
4
5      pthread_t thread1;
6      pthread_attr_t attr1;
7      struct sched_param param1;
8
9      pthread_attr_init(&attr1);
10     pthread_attr_setinheritsched(&attr1, PTHREAD_EXPLICIT_SCHED
11 );
12     pthread_attr_setschedpolicy(&attr1, SCHED_FIFO);
13     pthread_attr_setaffinity_np(&attr1, sizeof(cpu_set_t), &
14 cpuset);
15
16     t_activity_par activity_1;
17     snprintf(activity_1.name, sizeof(activity_1.name), "
18 DDS_Publish");
19     activity_1.function = dds_publish_function;
20     activity_1.period = 600;
21     activity_1.parameter = 6;
22     activity_1.deadline = 600;
23     activity_1.instance = NULL;
24     activity_1.print = true;
25
26     param1.sched_priority = 99 - (activity_1.period / 10); // =
27     39
28     if (param1.sched_priority < 1) param1.sched_priority = 1;
29     pthread_attr_setschedparam(&attr1, &param1);
30
31     int ret_err = pthread_create(&thread1, &attr1, PeriodicTask
32 , (void *)&activity_1);
33     if (ret_err != 0) { perror("Error creating thread 1"); exit
34 (1); }
35     pthread_setname_np(thread1, "DDS_Publish");
36     pthread_attr_destroy(&attr1);

```

31

LISTING 14.19: `Broadcastner.cpp` – Thread 1: `DDS_Publish` su
Core 1, periodo 600 ms

Il secondo thread, `Activity_1`, è destinato al Core 0 con periodo più lungo (1000 ms) e carico inferiore (`parameter = 4`). La priorità RM risultante è $99 - (1000/10) = 89$, che è numericamente più bassa, ma poiché i due thread sono su core distinti la priorità non determina prelazione reciproca — è utilizzata solo dallo scheduler del kernel per eventuali decisioni di migrazione.

```

1      cpu_set_t cpuset2;
2      CPU_ZERO(&cpuset2);
3      CPU_SET(0, &cpuset2); // Isolamento su Core 0
4
5      pthread_t thread2;
6      pthread_attr_t attr2;
7      struct sched_param param2;
8
9      pthread_attr_init(&attr2);
10     pthread_attr_setinheritsched(&attr2, PTHREAD_EXPLICIT_SCHED
11 );
12     pthread_attr_setschedpolicy(&attr2, SCHED_FIFO);
13     pthread_attr_setaffinity_np(&attr2, sizeof(cpu_set_t), &
14 cpuset2);
15
16     t_activity_par activity_2;
17     snprintf(activity_2.name, sizeof(activity_2.name), "
18 Activity_1");
19     activity_2.function = computation_function;
20     activity_2.period = 1000;
21     activity_2.parameter = 4;
22     activity_2.deadline = 1000;
23     activity_2.instance = NULL;
24     activity_2.print = true;
25
26     param2.sched_priority = 99 - (activity_2.period / 10); // =
89
27     if (param2.sched_priority < 1) param2.sched_priority = 1;
28     pthread_attr_setschedparam(&attr2, &param2);

```



```
27     ret_err = pthread_create(&thread2, &attr2, PeriodicTask, (  
void *)&activity_2);  
28     if (ret_err != 0) { perror("Error creating thread 2"); exit  
(1); }  
29     pthread_setname_np(thread2, "Activity_1");  
30     pthread_attr_destroy(&attr2);  
31
```

LISTING 14.20: Broadcastner.cpp – Thread 2: Activity_1 su Core
0, periodo 1000 ms

Infine, il main attende la terminazione dei thread (che in esecuzione reale non avviene mai) e chiude il file descriptor verso `trace_marker`.

```
1     pthread_join(thread1, NULL);  
2     pthread_join(thread2, NULL);  
3  
4     close_tracing();  
5     return 0;  
6 }
```

LISTING 14.21: Broadcastner.cpp – Join e chiusura tracing

14.2.3 Applicazione Listener: Subscriber DDS Event-Driven

Posizione nel progetto: `src/app/Listener/Listener.cpp`

Il `Listener` implementa il nodo subscriber della rete DDS. A differenza del `Broadcastner`, non crea thread POSIX esplicitamente: il middleware DDS gestisce internamente un thread di ricezione che invoca automaticamente i callback dell'applicazione all'arrivo di ogni messaggio. L'architettura è quindi puramente event-driven.

Include e Namespace

Il file include direttamente le API di FastDDS (senza passare dal wrapper `dds_lib`) perché la logica del subscriber è intrinsecamente più semplice e non giustifica un livello di astrazione aggiuntivo. Il namespace DDS viene importato esplicitamente per rendere leggibile il codice.

```
1      /*
2      * Listener.cpp
3      */
4
5      #include "Listener.hpp"
6      #include "activity_library.h"
7      #include "trace_marker.h"
8
9      #include <cstdint>
10     #include <iostream>
11     #include <limits>
12     #include <sstream>
13
14     #include <fastdds/dds/domain/DomainParticipantFactory.hpp>
15     #include <fastdds/dds/subscriber/DataReader.hpp>
16     #include <fastdds/dds/subscriber/SampleInfo.hpp>
17     #include <fastdds/dds/subscriber/Subscriber.hpp>
18     #include <fastdds/dds/subscriber/qos/DataReaderQos.hpp>
19     #include <fastrtps/attributes/ParticipantAttributes.h>
20     #include <fastrtps/attributes/SubscriberAttributes.h>
21     #include <fastrtps/rtps/common/Time_t.h>
22     #include <geometry_msgsPubSubTypes.h>
```

```
23
24     using namespace eprosima::fastdds::dds;
25     using namespace geometry_msgs::msg::dds_;
```

LISTING 14.22: Listener.cpp – Include e namespace

La Classe GeometryListener: Architettura a Callback

Il cuore del subscriber è la classe `GeometryListener`, che estende `DataReaderListener`. Il pattern scelto è quello dell'overriding virtuale: si sovrascrivono solo i due metodi di interesse, lasciando il middleware libero di gestire tutti gli altri eventi del ciclo di vita DDS.

```
1     class GeometryListener : public DataReaderListener {
2     public:
3         GeometryListener() {}
4         ~GeometryListener() override {}
5     }
```

LISTING 14.23: Listener.cpp – Dichiarazione della classe `GeometryListener`

Il metodo `on_data_available` è il callback del Listener: questa funzione viene invocata dal thread interno del middleware e ogni volta che un nuovo campione è disponibile nel `DataReader`. I due marker `DDS_READ_START/END` racchiudono l'intera operazione di lettura, permettendo al parser di calcolare la latenza end-to-end dalla pubblicazione alla ricezione confrontandoli con i marker `DDS_MSG_START/END` del `Broadcastner`.

```
1     void on_data_available(DataReader *reader) override {
2         // Marker [DDS_READ_START]: inizio lettura dal
3         DataReader
4         write_trace_marker("DDS_READ_START");
5
6         SampleInfo info;
7         Point_ point_msg;
8
9         // take_next_sample: estrae il campione e lo copia in
10        point_msg
11        // RETCODE_OK garantisce che il dato sia valido e non
12        corrotto
```

```

10         if (reader->take_next_sample(&point_msg, &info) ==
11             ReturnCode_t::RETCODE_OK) {
12             // ALIVE\INSTANCE\STATE: il publisher e ancora
attivo
13             if (info.instance_state == ALIVE_INSTANCE_STATE) {
14                 std::cout << "Received Point(" << point_msg.x()
15                 << ", "
16                 << point_msg.y() << ", " << point_msg.z()
17                 << ")" << std::endl;
18             }
19         }
20         // Marker [DDS\READ\END]: il dato e stato estratto e
processato
21         write_trace_marker("DDS_READ_END");
22     }
23

```

LISTING 14.24: Listener.cpp – Callback on_data_available con marker DDS

Il secondo metodo, `on_subscription_matched`, è un callback diagnostico: il middleware lo invoca ogni volta che un publisher si connette o si disconnette dal topic. Non partecipa alla misurazione delle performance, ma è essenziale durante il debugging per verificare che la fase di discovery DDS sia avvenuta correttamente tra i nodi della rete.

```

1     void on_subscription_matched(DataReader *reader,
2     const SubscriptionMatchedStatus &info) override {
3         if (info.current_count_change == 1) {
4             std::cout << "Publisher matched." << std::endl;
5         } else if (info.current_count_change == -1) {
6             std::cout << "Publisher unmatched." << std::endl;
7         }
8     }
9 };

```

LISTING 14.25: Listener.cpp – Callback on_subscription_matched

Il main: Avvio della Sottoscrizione

Il main del Listener è volutamente minimale: delega tutta la complessità al wrapper `dds_lib` e alla classe `GeometryListener`. Con una sola chiamata a `listener.start()`, il middleware crea internamente `DomainParticipant`, `Subscriber` e `DataReader`, registra la classe `GeometryListener` come handler degli eventi in arrivo e avvia il thread di ricezione. Il main non ha bisogno di un loop attivo: si blocca implicitamente in attesa degli eventi gestiti dal middleware.

```
1  int main(int argc, char **argv) {
2      init_tracing();
3
4      // TypeSupport generato da fastddsgen: registra il tipo
   Point nel middleware
5      eprosima::fastdds::dds::TypeSupport msg_type(new
   Point_PubSubType());
6
7      // Il costruttore di Listener riceve il callback, il tipo e
   il nome del tipo
8      Listener listener(new GeometryListener(), &msg_type, "Point
   ");
9
10     // start() crea DomainParticipant, Subscriber e DataReader
   sul topic indicato
11     // Il domain_id=1 deve coincidere con quello del
   Broadcastner per la discovery
12     listener.start("PointTopic", 1);
13
14     close_tracing();
15     return 0;
16 }
```

LISTING 14.26: Listener.cpp – main: avvio della sottoscrizione
DDS

La simmetria tra i due applicativi è evidente: il `Broadcastner` usa `g_broadcastner.start()` per pubblicare su `PointTopic` nel dominio 1, e il `Listener` usa `listener.start("PointTopic", 1)` per sottoscrivere allo stesso topic nello stesso dominio. Il protocollo di discovery RTPS si occupa di far incontrare i due nodi automaticamente, evento notificato dai

callback `on_publication_matched` (lato publisher) e `on_subscription_matched` (lato subscriber).

14.3 Fase 4 — Infrastruttura di Tracciamento e Automazione

Dati in ingresso: binari in `bin/app/`.

Strumenti utilizzati: `trace-cmd`, `marker.sh`, `marker_dds.sh`, `run_trace.sh`, `run_trace_dds.sh` `bash`, `g++`.

Risultati prodotti: `trace.dat`, `trace_output.txt`, `timeline.csv`, `risultati_finali.txt`, diagramma Gantt PNG.

La motivazione che ha portato a due famiglie di script distinte nasce da una limitazione tecnica: nella comunicazione DDS, il sistema si compone di almeno due processi distinti in parallelo; verranno quindi distinti due modi per tracciare le simulazioni, il metodo attivo che consiste nel tracciare i processi in modo immediato attraverso gli script (`run_trace.sh`), mentre quello passivo agganciandosi ai pid dei processi da tracciare già in esecuzione riesce a fare il tracing sia dei processi che dei thread generati da quel processo lo script `bash` usato è (`marker.sh`).

14.3.1 Tracciamento Attivo: `run_trace.sh`

Posizione nel progetto: `bin/tools/run_trace.sh`

Lo script `run_trace.sh` è il punto di ingresso dell'intera pipeline di analisi. Viene invocato con il nome dell'eseguibile da osservare come unico argomento:

```
1 ./run_trace.sh [nome eseguibile da simulare]
```

Se non viene fornito nessun argomento, lo script entra in modalità interattiva e chiede il nome all'utente. Da quel singolo parametro, in sequenza, risolve il percorso del binario, individua il sorgente C/C++ corrispondente, crea la cartella di output, lancia la registrazione kernel, decodifica il trace, compila ed esegue il parser C++ e avvia il monitor Python. I cinque passi sono descritti di seguito seguendo il codice riga per riga.

Risoluzione dei Percorsi e Alias dei Nomi

Le prime righe dello script calcolano i percorsi assoluti a partire dalla posizione dello script stesso, usando la variabile speciale `${BASH_SOURCE[0]}` che contiene il path del file corrente anche quando lo script viene invocato da una directory diversa. Risalire di due livelli rispetto a `bin/tools/` porta alla radice del progetto `Project_Kernel_Trace/`, che viene salvata in `PROJECT_ROOT`: tutte le directory costruite successivamente (`bin/app/`, `src/tools/`, `bin/Test/`) sono espresse come sottopercorsi di questa variabile, rendendo lo script portabile su qualsiasi macchina senza modifiche.

```
1  #!/bin/bash
2  # run_trace.sh - Tracciamento per Project_Kernel_Trace (launch
   mode)
3
4  # Salviamo la cartella principale del progetto
5  SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
6  PROJECT_ROOT="$(cd "$SCRIPT_DIR/../../" && pwd)"
7  BASE_DIR="$PROJECT_ROOT"
```

LISTING 14.27: `run_trace.sh` – Calcolo percorsi assoluti e alias nomi

Prima ancora di cercare il binario, lo script normalizza il nome ricevuto tramite la funzione `resolve_app_name`. Questo è necessario perché le versioni storiche del progetto usavano nomi come `GeometryBroadcastner` e `GeometryListener`, poi rinominati in `Broadcastner` e `Listener` durante il refactoring. La funzione agisce come un dizionario di alias: se il nome è riconosciuto restituisce quello moderno, altrimenti lo lascia invariato. Questo garantisce la retrocompatibilità con i log di sessioni precedenti.

```
1  # Funzione per risolvere alias dei nomi
2  resolve_app_name() {
3      case $1 in
4          GeometryBroadcastner) echo "Broadcastner" ;;
5          GeometryListener)    echo "Listener" ;;
6          *)                    echo "$1" ;;
7      esac
8  }
9
10 # Chiediamo l'eseguibile (interattivo o da argomento)
11 if [ "$#" -eq 0 ]; then
12     read -p "Inserisci l'eseguibile da tracciare (es. app10c o
GeometryListener): " RAW_NAME
13 else
14     RAW_NAME=$1
15 fi
16
17 EXECUTABLE_NAME=$(resolve_app_name "$RAW_NAME")
```

LISTING 14.28: `run_trace.sh` – Normalizzazione nomi storici e lettura input

Discovery Dinamica di Binario e Sorgente

Risolto il nome, lo script cerca il binario compilato dentro `bin/app/` usando `find` con il filtro `-executable`: in questo modo non viene mai confuso con file di testo o script omonimi che potrebbero trovarsi nella stessa cartella. Se il binario non viene trovato, lo script termina con un errore chiaro prima di fare qualsiasi altra operazione.

```
1  # Cerca l'eseguibile in bin/app/
```



```

2 EXECUTABLE=$(find "$PROJECT_ROOT/bin/app" -type f -executable \
3 -name "$EXECUTABLE_NAME" | head -n 1)
4
5 if [ -z "$EXECUTABLE" ]; then
6 echo "ERRORE: Eseguibile '$EXECUTABLE_NAME' non trovato in bin/
7 app/!"
8 exit 1
9 fi
10 APP_NAME=$(basename "$EXECUTABLE")

```

LISTING 14.29: run_trace.sh – Ricerca del binario compilato

Oltre al binario, serve anche il file sorgente: il parser C++ lo legge per estrarre i periodi nominali dichiarati nel codice (i valori di `activity.period`) e confrontarli con i periodi misurati nel trace. La funzione `find_src` implementa una strategia di ricerca con priorità: cerca prima in `src/app/` (dove stanno le applicazioni), e solo se non trova nulla allarga la ricerca a tutto `src/`. Se nemmeno questa seconda ricerca produce risultati, lo script si ferma con un errore critico prima di avviare la registrazione kernel, evitando di lasciare tracce incomplete su disco.

```

1 # Troviamo il file sorgente per l'analisi
2 find_src() {
3     local name=$1
4     # Priorita a src/app/
5     local candidate=$(find "$PROJECT_ROOT/src/app" \
6 -name "${name}.cpp" -o -name "${name}.c" | head -n 1)
7     if [ -n "$candidate" ]; then
8         echo "$candidate"
9     else
10        # Cerca ovunque in src/
11        find "$PROJECT_ROOT/src" -name "${name}.cpp" -o -name "${
12 name}.c" | head -n 1
13    fi
14 }
15 SRC_FILE=$(find_src "$APP_NAME")
16
17 if [ ! -f "$SRC_FILE" ]; then
18 echo "ERRORE CRITICO: Non trovo il file sorgente per $APP_NAME!"
19 "

```

```

19     exit 1
20     fi

```

LISTING 14.30: `run_trace.sh` – Ricerca del sorgente C/C++ con fallback

Creazione della Directory di Output Univoca

Prima di avviare la registrazione, lo script risolve i percorsi assoluti degli strumenti (`monitor.py` e `thread_analysis.cpp`) e crea la directory di output. Il nome della directory incorpora il timestamp della sessione nel formato `YYYYMMDD_HHMMSS`: ogni esecuzione produce una cartella separata, impedendo che i dati di sessioni diverse si sovrascrivano. Il banner stampato a terminale riassume i tre percorsi chiave prima di avviare qualsiasi operazione privilegiata.

```

1     SRC_FILE_ABS="$SRC_FILE"
2     MONITOR_SCRIPT_ABS="$PROJECT_ROOT/src/tools/monitor.py"
3     PARSER_SRC="$PROJECT_ROOT/src/tools/thread_analysis.cpp"
4
5     TIMESTAMP=$(date +%Y%m%d_%H%M%S)
6     OUTPUT_DIR="$PROJECT_ROOT/bin/Test/trace_results_${APP_NAME}_${
7     TIMESTAMP}"
8
9     echo "=====
10
11     echo " Inizio tracciamento per: $APP_NAME"
12     echo " Sorgente trovato:          $SRC_FILE_ABS"
13     echo " Cartella di output:         $OUTPUT_DIR"
14     echo "=====

```

LISTING 14.31: `run_trace.sh` – Percorsi degli strumenti e directory di output

Step 1 — Registrazione con `trace-cmd record`

Il primo passo è il cuore dell'intera pipeline: `trace-cmd record` avvia l'applicazione come processo figlio e contemporaneamente abilita nel kernel la registrazione degli

eventi `sched_switch` (ogni context switch tra thread) e `sched_wakeup` (ogni risveglio da sleep). L'applicazione gira normalmente con la propria schedulazione RT; quando termina (o viene interrotta con Ctrl-C), `trace-cmd` scrive il ring buffer del kernel nel file binario compresso `trace.dat`. Il flag `-o` specifica il percorso di output dentro la directory di sessione appena creata.

```

1  echo "[1/5] Avvio simulatore e registrazione trace-cmd..."
2  sudo trace-cmd record \
3  -e sched:sched_switch \
4  -e sched:sched_wakeup \
5  -o "$OUTPUT_DIR/trace.dat" \
6  "$EXECUTABLE"
7
8  if [ ! -f "$OUTPUT_DIR/trace.dat" ]; then
9  echo "ERRORE CRITICO: trace.dat non creato."
10 exit 1
11 fi
12
13 sudo chown -R "$(id -u):$(id -g)" "$OUTPUT_DIR"

```

LISTING 14.32: `run_trace.sh` – Step 1: acquisizione eventi kernel con `trace-cmd`

File prodotto: `trace.dat`.

Il `chown` dopo la registrazione è necessario perché `trace-cmd` è stato eseguito con `sudo`: i file prodotti appartengono a `root`, e senza questa operazione i passi successivi (parser e monitor, eseguiti senza privilegi) non potrebbero leggerli.

Step 2 — Decodifica con `trace-cmd report`

Il file `trace.dat` è un formato binario non leggibile direttamente. `trace-cmd report` è un comando che lo decodifica e produce un file di testo in cui ogni riga rappresenta un evento con: timestamp in secondi dal boot, CPU, nome e PID del processo interrotto, e dettagli dell'evento (stato del thread uscente, nome e PID del thread entrante per `sched_switch`; nome e PID del thread svegliato per `sched_wakeup`). Questo file è l'ingresso del parser C++.

```

1  echo "[2/5] Generazione del report testuale..."

```

2

```
trace-cmd report "$OUTPUT_DIR/trace.dat" > "$OUTPUT_DIR/
trace_output.txt"
```

LISTING 14.33: `run_trace.sh` – Step 2: decodifica in testo leggibile

File prodotto: `trace_output.txt`

```
cpus=8
<idle>-0 [004] 183.059336: sched_switch: swapper/4:0 [120] R ==> app10g:4559 [120]
app10g-4558 [003] 183.059343: sched_wakeup: migration/4:41 [0] CPU:004
Activity_1-4559 [004] 183.059346: sched_switch: app10g:4559 [120] S ==> migration/4:41 [0]
app10g-4558 [003] 183.059347: sched_switch: app10g:4558 [120] D ==> swapper/3:0 [120]
migration/4-41 [004] 183.059350: sched_switch: migration/4:41 [0] S ==> swapper/4:0 [120]
<idle>-0 [003] 183.059350: sched_wakeup: app10g:4558 [120] CPU:003
<idle>-0 [003] 183.059351: sched_switch: swapper/3:0 [120] R ==> app10g:4558 [120]
<idle>-0 [001] 183.059357: sched_wakeup: app10g:4559 [98] CPU:001
<idle>-0 [001] 183.059359: sched_switch: swapper/1:0 [120] R ==> app10g:4559 [98]
Activity_1-4559 [001] 183.059374: print: tracing_mark_write: PERIOD_START_Activity_1
Activity_1-4559 [001] 183.059375: print: tracing_mark_write: FUNCTION_START_Activity_1
<idle>-0 [004] 183.059390: sched_switch: swapper/4:0 [120] R ==> app10g:4560 [120]
app10g-4558 [003] 183.059392: sched_wakeup: migration/4:41 [0] CPU:004
Activity_2-4560 [004] 183.059394: sched_switch: app10g:4560 [120] S ==> migration/4:41 [0]
migration/4-41 [004] 183.059395: sched_switch: migration/4:41 [0] S ==> swapper/4:0 [120]
app10g-4558 [003] 183.059396: sched_switch: app10g:4558 [120] D ==> swapper/3:0 [120]
<idle>-0 [003] 183.059397: sched_wakeup: app10g:4558 [120] CPU:003
<idle>-0 [003] 183.059397: sched_switch: swapper/3:0 [120] R ==> app10g:4558 [120]
app10g-4558 [003] 183.059401: sched_wakeup: app10g:4560 [98] CPU:002
app10g-4558 [003] 183.059411: sched_switch: app10g:4558 [120] S ==> swapper/3:0 [120]
<idle>-0 [004] 183.060485: sched_wakeup: rcu_tasks_trace:67 [120] CPU:004
<idle>-0 [004] 183.060499: sched_switch: swapper/4:0 [120] R ==> rcu_tasks_trace:67 [120]
rcu_tasks_trace-67 [004] 183.060502: sched_switch: rcu_tasks_trace:67 [120] D ==> swapper/4:0 [120]
<idle>-0 [004] 183.060999: sched_wakeup: gnome-terminal-:4290 [120] CPU:004
<idle>-0 [004] 183.061004: sched_switch: swapper/4:0 [120] R ==> gnome-terminal-:4290 [120]
gnome-terminal-:4290 [004] 183.061027: sched_switch: gnome-terminal-:4290 [120] S ==> swapper/4:0 [120]
<idle>-0 [005] 183.061262: sched_wakeup: MathWorksServic:3604 [120] CPU:005
<idle>-0 [005] 183.061264: sched_switch: swapper/5:0 [120] R ==> MathWorksServic:3604 [120]
<idle>-0 [004] 183.061297: sched_wakeup: MathWorksServic:3600 [120] CPU:004
<idle>-0 [004] 183.061297: sched_switch: swapper/4:0 [120] R ==> MathWorksServic:3600 [120]
MathWorksServic-3604 [005] 183.061302: sched_switch: MathWorksServic:3604 [120] S ==> swapper/5:0 [120]
<idle>-0 [005] 183.061331: sched_wakeup: MathWorksServic:3608 [120] CPU:005
MathWorksServic-3600 [004] 183.061331: sched_switch: MathWorksServic:3600 [120] S ==> swapper/4:0 [120]
<idle>-0 [005] 183.061332: sched_switch: swapper/5:0 [120] R ==> MathWorksServic:3608 [120]
<idle>-0 [004] 183.061340: sched_wakeup: MathWorksServic:3600 [120] CPU:004
```

FIGURA 14.1: Estratto di `trace_output.txt`: ogni riga documenta un evento di scheduling con timestamp, nome processo, PID e stato di transizione.

Step 3 — Compilazione ed Esecuzione del Parser C++

Il parser `thread_analysis.cpp` viene compilato a runtime direttamente nella directory di output della sessione corrente. La scelta di compilare *just-in-time* anziché distribuire un binario precompilato ha una ragione precisa: il parser viene sviluppato reciprocamente con le applicazioni durante lo sviluppo, e compilarlo ogni volta garantisce che venga sempre usata la versione corrispondente al sorgente presente in `src/tools/`, senza rischio di disallineamento tra parser e formato dei marker.

I flag `-O2 -std=c++17` bilanciano velocità di compilazione e ottimizzazione del codice prodotto. L'eseguibile viene scritto dentro `OUTPUT_DIR`: anche il binario

del parser fa parte degli artefatti della sessione, facilitando il debugging in caso di risultati anomali.

```
1  echo "[3/5] Compilazione analizzatore C++..."
2  PARSER_EXE="$OUTPUT_DIR/thread_analysis"
3  if [ -f "$PARSER_SRC" ]; then
4  g++ -O2 -std=c++17 "$PARSER_SRC" -o "$PARSER_EXE"
5  else
6  echo "ATTENZIONE: File $PARSER_SRC non trovato!"
7  PARSER_EXE=""
8  fi
9
10 echo "[4/5] Avvio Analisi (Lettura periodi dal codice)..."
11 if [ -n "$PARSER_EXE" ]; then
12 cd "$OUTPUT_DIR" || exit
13 ./thread_analysis "trace_output.txt" "$SRC_FILE_ABS" "ALL" > "
risultati_finali.txt"
14 echo ""
15 cat "risultati_finali.txt"
```

LISTING 14.34: `run_trace.sh` – Step 3: compilazione e lancio del parser C++

Il parser spiegato interamente nel suo funzionamento al Capitolo 14.4, riceve tre argomenti: il file di testo del trace, il sorgente dell'applicazione (da cui estrae i periodi) e la stringa "ALL" che indica di analizzare tutti i thread trovati nel log. L'output viene rediretto in `risultati_finali.txt` e contemporaneamente stampato a terminale con `cat`.

File prodotti: `timeline.csv`, `risultati_finali.txt`

Analisi del file `timeline.csv` Il file `timeline.csv` contiene tutte le misurazioni temporali dei thread e le statistiche finali estratte dal trace kernel. Nella Tabella 14.1 è riportato il significato di ciascuna metrica statistica (identificata dal prefisso `STAT_`) prodotta dall'analizzatore C++ e utile per valutare le prestazioni *real-time* di ogni applicazione tracciata. Questo processo di analisi viene eseguito per qualsiasi script bash utilizzato durante la simulazione, come mostrato ad esempio nel listato 14.3.2. In Figura 14.3 viene riportato un esempio del risultato grafico ottenuto in seguito alla generazione del file di timeline.

```

-----
TASK: Activity_1 (PID: 4514)
-----
[Esecuzione] WCET : 234.213 ms | ACET: 225.174 ms
[T.Risposta] WCRT : 234.205 ms
[Periodo] Mis. : 999.904 ms | Jitter: 0.063 ms
[Parametri] Periodo: 1000.000 ms | Deadline: 1000.000 ms
[Scadenze] Miss : 0 | Slack Peggior: 765.795 ms
[Stati] Sleep : 15 | Preempt: 0 | Run n.: 16
[Globale] Util% : 24.015 % | Gaps : 15 (781.594 ms max)
-----

TASK: Activity_2 (PID: 4515)
-----
[Esecuzione] WCET : 452.551 ms | ACET: 438.739 ms
[T.Risposta] WCRT : 452.544 ms
[Periodo] Mis. : 1199.906 ms | Jitter: 0.122 ms
[Parametri] Periodo: 1200.000 ms | Deadline: 1200.000 ms
[Scadenze] Miss : 0 | Slack Peggior: 747.456 ms
[Stati] Sleep : 12 | Preempt: 2 | Run n.: 15
[Globale] Util% : 39.636 % | Gaps : 14 (771.995 ms max)
-----

```

FIGURA 14.2: Output a terminale di `thread_analysis`: metriche RT per ogni task.

	A	B	C	D	E	F
52	Activity_1	MARKER_FUNCTION_START_Activity_1	5391.505329	5391.505329	0	
53	Activity_1	MARKER_FUNCTION_END_Activity_1	5391.660698	5391.660698	0	
54	Activity_1	MARKER_SLEEP_START_Activity_1	5391.660699	5391.660699	0	
55	Activity_1	MARKER_PERIOD_END_Activity_1	5392.428482	5392.428482	0	
56	Activity_1	MARKER_PERIOD_START_Activity_1	5392.428483	5392.428483	0	
57	Activity_1	MARKER_FUNCTION_START_Activity_1	5392.428483	5392.428483	0	
58	Activity_1	MARKER_FUNCTION_END_Activity_1	5392.566115	5392.566115	0	
59	Activity_1	MARKER_SLEEP_START_Activity_1	5392.566115	5392.566115	0	
60	Activity_1	MARKER_PERIOD_END_Activity_1	5393.351587	5393.351587	0	
61	Activity_1	MARKER_PERIOD_START_Activity_1	5393.351588	5393.351588	0	
62	Activity_1	MARKER_FUNCTION_START_Activity_1	5393.351588	5393.351588	0	
63	Activity_1	MARKER_FUNCTION_END_Activity_1	5393.492043	5393.492043	0	
64	Activity_1	MARKER_SLEEP_START_Activity_1	5393.492044	5393.492044	0	
65	Activity_1	MARKER_PERIOD_END_Activity_1	5394.275848	5394.275848	0	
66	Activity_1	MARKER_PERIOD_START_Activity_1	5394.275849	5394.275849	0	
67	Activity_1	MARKER_FUNCTION_START_Activity_1	5394.275849	5394.275849	0	
68	Activity_1	MARKER_FUNCTION_END_Activity_1	5394.412865	5394.412865	0	
69	Activity_1	MARKER_SLEEP_START_Activity_1	5394.412865	5394.412865	0	
70	Activity_1	MARKER_PERIOD_END_Activity_1	5395.199509	5395.199509	0	
71	Activity_1	MARKER_PERIOD_START_Activity_1	5395.199509	5395.199509	0	
72	Activity_1	MARKER_FUNCTION_START_Activity_1	5395.19951	5395.19951	0	
73	Activity_1	MARKER_FUNCTION_END_Activity_1	5395.337022	5395.337022	0	
74	Activity_1	MARKER_SLEEP_START_Activity_1	5395.337023	5395.337023	0	
75	Activity_1	MARKER_PERIOD_END_Activity_1	5396.120827	5396.120827	0	
76	Activity_1	MARKER_PERIOD_START_Activity_1	5396.120828	5396.120828	0	
77	Activity_1	MARKER_FUNCTION_START_Activity_1	5396.120829	5396.120829	0	
78	Activity_1	MARKER_FUNCTION_END_Activity_1	5396.268288	5396.268288	0	
79	Activity_1	MARKER_SLEEP_START_Activity_1	5396.268289	5396.268289	0	
80	Activity_1	MARKER_PERIOD_END_Activity_1	5397.043823	5397.043823	0	
81	Activity_1	MARKER_PERIOD_START_Activity_1	5397.043824	5397.043824	0	
82	Activity_1	MARKER_FUNCTION_START_Activity_1	5397.043824	5397.043824	0	
83	Activity_1	MARKER_FUNCTION_END_Activity_1	5397.187516	5397.187516	0	
84	Activity_1	MARKER_SLEEP_START_Activity_1	5397.187516	5397.187516	0	
85	Activity_1	MARKER_PERIOD_END_Activity_1	5397.966948	5397.966948	0	
86	Activity_1	MARKER_PERIOD_START_Activity_1	5397.966949	5397.966949	0	
87	Activity_1	MARKER_FUNCTION_START_Activity_1	5397.966949	5397.966949	0	
88	Activity_1	MARKER_FUNCTION_END_Activity_1	5398.1081	5398.1081	0	
89	Activity_1	MARKER_SLEEP_START_Activity_1	5398.108101	5398.108101	0	
90	Activity_1	MARKER_PERIOD_END_Activity_1	5398.890028	5398.890028	0	
91	Activity_1	MARKER_PERIOD_START_Activity_1	5398.890029	5398.890029	0	
92	Activity_1	MARKER_FUNCTION_START_Activity_1	5398.89003	5398.89003	0	
93	Activity_1	MARKER_FUNCTION_END_Activity_1	5399.02791	5399.02791	0	
94	Activity_1	MARKER_SLEEP_START_Activity_1	5399.027911	5399.027911	0	
95	Activity_1	MARKER_PERIOD_END_Activity_1	5399.813141	5399.813141	0	
96	Activity_1	MARKER_PERIOD_START_Activity_1	5399.813142	5399.813142	0	
97	Activity_1	MARKER_FUNCTION_START_Activity_1	5399.813142	5399.813142	0	
98	Activity_1	MARKER_FUNCTION_END_Activity_1	5399.950718	5399.950718	0	
99	Activity_1	MARKER_SLEEP_START_Activity_1	5399.950719	5399.950719	0	
100	Activity_1	MARKER_PERIOD_END_Activity_1	5400.73624	5400.73624	0	
101	Activity_1	MARKER_PERIOD_START_Activity_1	5400.736241	5400.736241	0	
102	Activity_1	MARKER_FUNCTION_START_Activity_1	5400.736241	5400.736241	0	

FIGURA 14.3: Esempio di risultato ottenuto nella creazione del file `timeline.csv`.

Statistica	Significato
STAT_PERIOD	Periodo nominale del task dichiarato a codice.
STAT_MEASURED	Periodo medio effettivo misurato tra due esecuzioni consecutive.
STAT_DEADLINE	Scadenza temporale (deadline) entro cui il task deve completare l'esecuzione.
STAT_WCET	<i>Worst-Case Execution Time</i> : il tempo massimo di esecuzione registrato.
STAT_ACET	<i>Average-Case Execution Time</i> : il tempo medio di esecuzione.
STAT_JITTER	Jitter misurato sull'attivazione o sull'esecuzione del task.
STAT_UTIL	Fattore di utilizzo della CPU (percentuale di tempo trascorso in esecuzione).
STAT_NRUN	Numero totale di attivazioni (run) del task.
STAT_NPRE	Numero totale di <i>preemption</i> (interruzioni da parte di altri task) subite.
STAT_NSLP	Numero di volte in cui il task è entrato in stato di <i>sleep</i> .
STAT_NMISS	Numero di <i>deadline</i> mancate durante la sessione di tracciamento.
STAT_WSLACK	<i>Worst-case slack time</i> : il margine temporale minimo registrato tra il termine dell'esecuzione e la deadline.
STAT_HGAPS	Numero di <i>huge gaps</i> , ovvero anomalie in cui il task non è stato attivato per un periodo superiore alla norma.
STAT_NGAPS	Numero totale di gap temporali (ritardi di risveglio) rilevati.
STAT_MAXGAP	Durata massima registrata per un gap anomalo.
STAT_RUNTOT	Tempo totale in cui il task è stato attivamente in esecuzione sulla CPU.
STAT_PRETOT	Tempo totale trascorso in stato di <i>preemption</i> (pronto per eseguire ma bloccato da task a priorità maggiore).
STAT_SLPTOT	Tempo totale trascorso in stato di <i>sleep</i> .

TABELLA 14.1: Descrizione delle metriche statistiche presenti nel file `timeline.csv`

Step 4 — Monitor Visivo Python e Chiusura

L'ultimo passo della pipeline lancia `monitor.py` passandogli il file `timeline.csv` generato dal parser. Lo script Python legge le colonne del CSV (timestamp di inizio e fine di ogni finestra temporale per ogni thread) e produce un diagramma di Gantt che rende immediatamente visibili i periodi, le durate di esecuzione, i tempi di sleep e le eventuali anomalie temporali.

```
1 echo "[5/5] Avvio monitor visivo (Python)..."
2 if [ -f "$MONITOR_SCRIPT_ABS" ]; then
3 python3 "$MONITOR_SCRIPT_ABS" "timeline.csv"
4 else
5 echo "ATTENZIONE: Non trovo lo script $MONITOR_SCRIPT_ABS."
6 fi
7 cd "$PROJECT_ROOT" || exit
8 fi
9
10 echo "=====
11
12 echo "Tracciamento completato! Dati in: $OUTPUT_DIR"
```

LISTING 14.35: `run_trace.sh` – Step 4: monitor visivo e banner finale

File prodotto: timeline_monitor.png

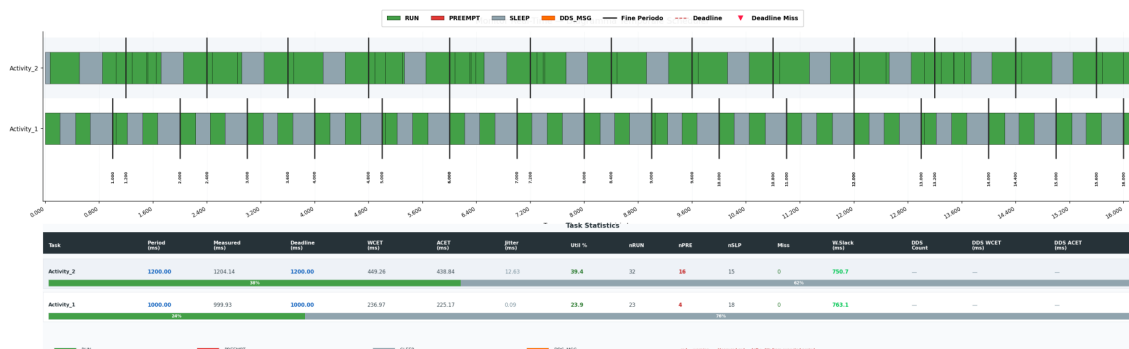


FIGURA 14.4: Diagramma di Gantt generato da `monitor.py` per `app10g`.

14.3.2 Tracciamento Passivo Multi-Processo: `marker.sh`

Posizione nel progetto: `bin/tools/marker.sh`

A differenza di `run_trace.sh`, che lancia l'eseguibile e lo registra in un unico processo, `marker.sh` opera in modalità **passiva**: i processi da osservare sono già in esecuzione in terminali separati prima che lo script venga invocato. Il suo compito è agganciare il tracer kernel a quei processi esistenti senza interromperne l'esecuzione. Viene invocato con i nomi degli eseguibili già avviati:

```
1 ./marker.sh Broadcastner Listener
```

Questo approccio è necessario per le applicazioni distribuite: `Broadcastner` e `Listener` devono partire contemporaneamente (su terminali distinti o nodi distinti) e completare la fase di discovery DDS tra di loro prima che l'osservazione abbia senso. Solo una volta che i due nodi si sono scoperti e il flusso di messaggi è attivo, si aggancia il tracer.

Risoluzione dei Percorsi e Lettura degli Argomenti

Le prime righe sono identiche a `run_trace.sh`: calcolo di `PROJECT_ROOT` tramite `BASH_SOURCE[0]` e normalizzazione dei nomi storici tramite `resolve_app_name`. La differenza principale è nella lettura degli argomenti: lo script accetta fino a due nomi di eseguibili, permettendo il tracciamento congiunto di due processi distinti nella stessa sessione.

```
1 #!/bin/bash
2 # marker.sh - Estrazione TID e tracciamento passivo (standard app
   version)
3
4 # Salviamo la cartella principale del progetto
5 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
6 PROJECT_ROOT="$(cd "$SCRIPT_DIR/../../.." && pwd)"
7
8 # Se siamo già root (es. dashboard), non serve sudo
9 if [ "$EUID" -eq 0 ]; then
10 SUDO_CMD=""
11 else
12 SUDO_CMD="sudo "
13 fi
```

```

14 BASE_DIR="$PROJECT_ROOT"
15
16 # Funzione per risolvere alias dei nomi
17 resolve_app_name() {
18     case $1 in
19         GeometryBroadcastner) echo "Broadcastner" ;;
20         GeometryListener)    echo "Listener" ;;
21         *)                    echo "$1" ;;
22     esac
23 }
24
25 # Chiediamo gli eseguibili (devono essere GIÀ AVVIATI negli altri
    terminali)
26 if [ "$#" -eq 0 ]; then
27     read -p "Inserisci il PRIMO eseguibile (già avviato, es.
        Broadcastner o app10f): " RAW_NAME
28     read -p "[Opzionale] Inserisci il SECONDO eseguibile: "
        RAW_SEC_NAME
29 else
30     RAW_NAME=$1
31     RAW_SEC_NAME=$2
32 fi
33
34 EXECUTABLE_NAME=$(resolve_app_name "$RAW_NAME")
35 APP_NAME=$(basename "$EXECUTABLE_NAME")

```

LISTING 14.36: marker.sh – Percorsi, alias e lettura argomenti

Aggancio Passivo: Estrazione dei TID da /proc

Il cuore dello script è la funzione `aggiungi_pid_e_thread`, che implementa la logica di aggancio passivo. Il problema da risolvere è il seguente: `trace-cmd record` accetta il flag `-P` per filtrare gli eventi di un processo specifico, ma se si specifica solo il PID del processo padre si perdono gli eventi dei thread interni, che hanno TID distinti dal PID padre. La soluzione è navigare il filesystem virtuale `/proc/[PID]/task/`: il kernel Linux espone lì una directory per ogni thread del processo, il cui nome è il TID. Per ognuno, il file `comm` contiene il nome leggibile del thread (quello assegnato con `pthread_setname_np`). La funzione costruisce incrementalmente la stringa `PIDS_CMD` aggiungendo un flag `-P` per ogni TID trovato.

```

1
2 # ESTRAZIONE DEI THREAD (TID)
3
4 PIDS_1=$(pidof "$APP_NAME")
5 if [ -z "$PIDS_1" ]; then
6 echo "ERRORE: Il processo '$APP_NAME' non è in esecuzione!"
7 echo "Avvia prima l'app in un altro terminale, poi lancia questo
   script."
8 echo ""
9 echo "  Per le app DDS (Broadcastner, Listener, ecc.) serve sudo:"
10 echo "      sudo ./ $APP_NAME"
11 exit 1
12 fi
13
14 PIDS_CMD=""
15
16 # Funzione per estrarre tutti i thread
17 aggiungi_pid_e_thread() {
18     local process_name=$1
19     local pids=$2
20     for p in $pids; do
21         PIDS_CMD="$PIDS_CMD -P $p"
22         echo ">> Trovato processo principale: $process_name (PID: $p)"
23
24         # Esploriamo /proc/$p/task/ per trovare tutti i thread interni
25         # (TID)
26         if [ -d "/proc/$p/task" ]; then
27             for tid_path in /proc/$p/task/*; do
28                 tid=$(basename "$tid_path")
29                 if [ "$tid" != "$p" ]; then
30                     PIDS_CMD="$PIDS_CMD -P $tid"
31                     # Leggiamo il nome del thread per conferma a video
32                     thread_name=$(cat "$tid_path/comm" 2>/dev/null)
33                     echo "    -> Agganciato thread: $thread_name (TID: $tid)"
34                 fi
35             done
36         fi
37     }

```

LISTING 14.37: marker.sh – Estrazione ricorsiva dei TID da /proc

La funzione viene chiamata prima per il processo primario, poi, se è stato fornito un secondo argomento, anche per il processo secondario. In entrambi i casi, se il processo non è trovato in esecuzione, lo script lo segnala con un avviso ma non termina: il tracciamento del primo processo continua comunque.

```

1  # 1. Processiamo la prima app e i suoi thread
2  aggiungi_pid_e_thread "$APP_NAME" "$PIDS_1"
3
4  # 2. Se e' presente un secondo eseguibile, lo si cerca insieme
   ai suoi thread
5  if [ -n "$RAW_SEC_NAME" ]; then
6  SEC_EXECUTABLE_NAME=$(resolve_app_name "$RAW_SEC_NAME")
7  SEC_APP_NAME=$(basename "$SEC_EXECUTABLE_NAME")
8  PIDS_2=$(pidof "$SEC_APP_NAME")
9  if [ -n "$PIDS_2" ]; then
10 echo "-----"
11
12 aggiungi_pid_e_thread "$SEC_APP_NAME" "$PIDS_2"
13 echo ">> Tracciamento congiunto pronto."
14 else
15 echo ">> ATTENZIONE: '$SEC_APP_NAME' non trovato. Traccio solo
   $APP_NAME."
16 fi
17 else
18 echo ">> Tracciamento singolo pronto."
19 fi

```

LISTING 14.38: `marker.sh` – Chiamata per primo e secondo processo

Discovery del Sorgente, Directory di Output e Registrazione

Completata la costruzione di `PIDS_CMD`, lo script cerca il sorgente C/C++ dell'applicazione con la stessa funzione `find_src` vista in `run_trace.sh`, crea la directory di sessione con timestamp e poi avvia `trace-cmd record`. La differenza cruciale rispetto a `run_trace.sh` è che in tale sede non viene passato alcun eseguibile alla fine del comando: la registrazione non lancia nulla, ma filtra gli eventi del kernel esclusivamente sui PID e TID elencati in `PIDS_CMD`. La registrazione rimane attiva finché l'utente preme `Ctrl+C`.

```

1 # Troviamo il file sorgente per l'analisi

```

```

>> Trovato processo principale: Broadcastner (PID: 16694)
-> Agganciato thread: dds.shm.wdog (TID: 16695)
-> Agganciato thread: dds.ev.0 (TID: 16696)
-> Agganciato thread: dds.udp.7660 (TID: 16697)
-> Agganciato thread: dds.udp.7650 (TID: 16698)
-> Agganciato thread: dds.shm.7661 (TID: 16699)
-> Agganciato thread: dds.udp.7661 (TID: 16700)
-> Agganciato thread: dds.asyn.0.0 (TID: 16701)
-> Agganciato thread: dds.asyn.0.2 (TID: 16702)
-> Agganciato thread: DDS_Publish (TID: 16703)
-> Agganciato thread: Activity_1 (TID: 16704)
-----
>> Trovato processo principale: Listener (PID: 16746)
-> Agganciato thread: dds.shm.wdog (TID: 16747)
-> Agganciato thread: dds.ev.0 (TID: 16748)
-> Agganciato thread: dds.udp.7662 (TID: 16749)
-> Agganciato thread: dds.udp.7650 (TID: 16750)
-> Agganciato thread: dds.shm.7663 (TID: 16751)
-> Agganciato thread: dds.udp.7663 (TID: 16752)
-> Agganciato thread: dds.asyn.0.0 (TID: 16753)
-> Agganciato thread: dds.asyn.0.2 (TID: 16754)
-> Agganciato thread: dds.dsha.260 (TID: 16755)
>> Tracciamento congiunto pronto.

```

FIGURA 14.5: Output di `marker.sh`: aggancio congiunto con estrazione di tutti i TID prima di avviare il trace.

```

2 find_src() {
3     local name=$1
4     # Priorità a src/app/
5     local candidate=$(find "$PROJECT_ROOT/src/app" -name "${name}.
  cpp" -o -name "${name}.c" | head -n 1)
6     if [ -n "$candidate" ]; then
7         echo "$candidate"
8     else
9         # Cerca ovunque in src/
10        find "$PROJECT_ROOT/src" -name "${name}.cpp" -o -name "${name}.
  c" | head -n 1
11        fi
12    }
13
14    SRC_FILE=$(find_src "$APP_NAME")
15
16    if [ ! -f "$SRC_FILE" ]; then
17        echo "ERRORE CRITICO: Non trovo il file sorgente per $APP_NAME!"
18    exit 1

```

```

19 fi
20
21 SRC_FILE_ABS="$SRC_FILE"
22 MONITOR_SCRIPT_ABS="$PROJECT_ROOT/src/tools/monitor.py"
23 PARSER_SRC="$PROJECT_ROOT/src/tools/thread_analysis.cpp"
24
25 TIMESTAMP=$(date +%Y%m%d_%H%M%S)
26 OUTPUT_DIR="$PROJECT_ROOT/bin/Test/trace_marker_${APP_NAME}_${
    TIMESTAMP}"
27
28 echo "===== "
29 echo " Inizio tracciamento per: $APP_NAME"
30 echo " Sorgente trovato: $SRC_FILE_ABS"
31 echo " Cartella di output: $OUTPUT_DIR"
32 echo "===== "
33
34 mkdir -p "$OUTPUT_DIR"
35
36 echo "[1/5] Registrazione trace-cmd sui PID e THREAD scelti..."
37 echo "--- PREMI Ctrl+C PER FERMARE LA REGISTRAZIONE QUANDO HAI
    FINITO! ---"
38 # Lanciamo trace-cmd in background e catturiamo il suo PID per
    gestire il segnale
39 TRACE_CMD_PID=""
40 cleanup() {
41     echo ""
42     echo "--- Segnale ricevuto, arresto trace-cmd... ---"
43     if [ -n "$TRACE_CMD_PID" ] && kill -0 "$TRACE_CMD_PID" 2>/dev/
        null; then
44         kill -INT "$TRACE_CMD_PID" 2>/dev/null
45         wait "$TRACE_CMD_PID" 2>/dev/null
46     fi
47 }
48 trap cleanup INT TERM
49
50 $SUDO_CMD trace-cmd record -e sched:sched_switch -e sched:
    sched_wakeup $PIDS_CMD -o "$OUTPUT_DIR/trace.dat" &
51 TRACE_CMD_PID=$!
52 wait $TRACE_CMD_PID 2>/dev/null
53 trap - INT TERM

```

```

54
55 if [ ! -f "$OUTPUT_DIR/trace.dat" ]; then
56 echo "ERRORE CRITICO: trace.dat non creato."
57 exit 1
58 fi
59
60 $SUDO_CMD chown -R "${SUDO_UID:-$(id -u)}:${SUDO_GID:-$(id -g)}" "
    $OUTPUT_DIR"

```

LISTING 14.39: `marker.sh` – Discovery sorgente, output e registrazione passiva

Decodifica, Analisi, Monitor e KernelShark

I passi 2, 3, 4 e 5 sono identici a `run_trace.sh`: decodifica del binario in testo, compilazione just-in-time del parser C++ e suo lancio, visualizzazione con il monitor Python. `marker.sh` aggiunge però un sesto passo che `run_trace.sh` non ha: l'apertura automatica di KernelShark.

```

1  echo "[2/5] Generazione del report testuale..."
2  trace-cmd report "$OUTPUT_DIR/trace.dat" > "$OUTPUT_DIR/
    trace_output.txt"
3
4  echo "[3/5] Compilazione analizzatore C++..."
5  PARSEX_EXE="$OUTPUT_DIR/thread_analysis"
6  if [ -f "$PARSER_SRC" ]; then
7  g++ -O2 -std=c++17 "$PARSER_SRC" -o "$PARSER_EXE"
8  else
9  echo "ATTENZIONE: File $PARSER_SRC non trovato!"
10 PARSEX_EXE=""
11 fi
12
13 echo "[4/5] Avvio Analisi (Lettura periodi dal codice)... "
14 if [ -n "$PARSER_EXE" ]; then
15 cd "$OUTPUT_DIR" || exit
16 ./thread_analysis "trace_output.txt" "$SRC_FILE_ABS" "ALL" > "
    risultati_finali.txt"
17 echo ""
18 cat "risultati_finali.txt"
19

```

```
20 echo "[5/5] Avvio monitor visivo (Python)..."
21 if [ -f "$MONITOR_SCRIPT_ABS" ]; then
22     PYTHON_BIN="python3"
23     "$PYTHON_BIN" "$MONITOR_SCRIPT_ABS" "timeline.csv"
24 else
25     echo "ATTENZIONE: Non trovo lo script $MONITOR_SCRIPT_ABS."
26 fi
27 cd "$PROJECT_ROOT" || exit
28 fi
29
30 echo "=====
31 "
32 echo "Tracciamento completato! Dati in: $OUTPUT_DIR"
33
34 echo "[6/6] Avvio KernelShark..."
35 if command -v kernelshark >/dev/null 2>&1; then
36     (cd "$OUTPUT_DIR" && kernelshark trace.dat) &
37 else
38     echo "ATTENZIONE: KernelShark non trovato."
39 fi
```

LISTING 14.40: `marker.sh` – Decodifica, analisi, monitor e apertura
KernelShark

KernelShark viene lanciato in background (&) all'interno di una subshell che prima cambia directory nella cartella di output. In questo modo KernelShark trova `trace.dat` nella propria directory di lavoro e può aprirlo direttamente senza percorso assoluto. L'operatore & lascia libero il terminale: lo script termina e stampa il banner finale, mentre KernelShark rimane aperto come processo separato.



FIGURA 14.6: Monitor Python con risultato della simulazione Broadcastner–Listener. Si può notare nel monitor la barra arancio riguardante il tempo impiegato per la comunicazione DDS.

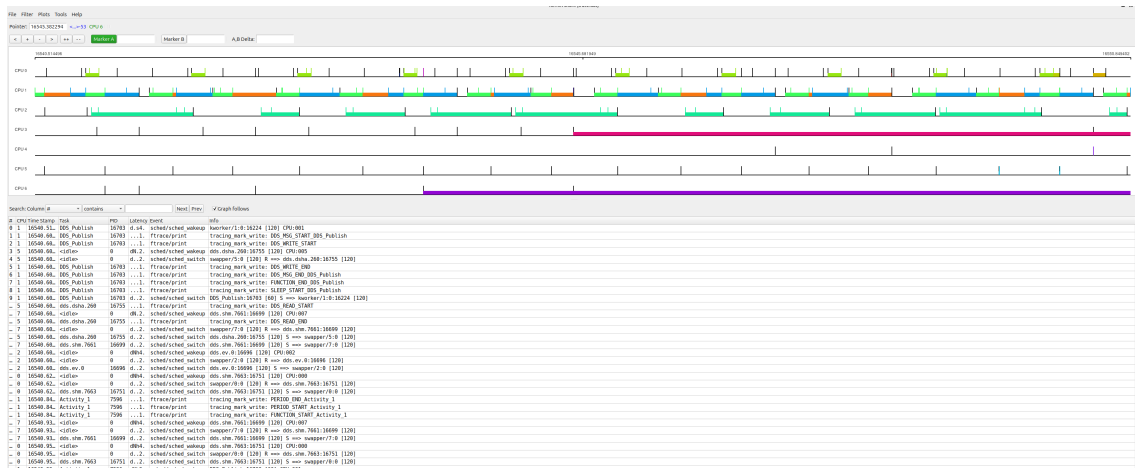


FIGURA 14.7: Interfaccia KernelShark con marker applicativi visibili sulla timeline.

14.3.3 Le Versioni Didattiche: `marker1.sh` – `marker4.sh`

Posizione nel progetto: `bin/tools/marker1.sh`, `marker2.sh`, `marker3.sh`, `marker4.sh`

La logica di `marker.sh` è suddivisa in quattro script indipendenti che eseguono esattamente gli stessi passi nell'identico ordine, ma separati in processi distinti. Il valore aggiunto non è funzionale ma didattico e diagnostico: ogni step può essere eseguito individualmente, ispezionato, ripetuto o saltato. Se ad esempio il parser produce risultati anomali, si può rieseguire solo `marker3.sh` su una cartella di sessione già esistente senza dover ripetere la registrazione. Il meccanismo che tiene insieme i quattro script è il file temporaneo `/tmp/last_marker_dir`.

`marker1.sh`: Registrazione

`marker1.sh` contiene la logica completa di aggancio TID, creazione della directory di output e avvio di `trace-cmd record`. Al termine, invece di procedere con i passi successivi, scrive il percorso della cartella di output in `/tmp/last_marker_dir` e il nome dell'applicazione in `app_name.txt` dentro la stessa cartella. Questi due file fungono da canale di comunicazione verso gli script successivi: ogni script successivo legge da lì il contesto della sessione corrente.

```

1  #!/bin/bash
2  # marker1.sh - Estrazione TID e tracciamento passivo (
  Registrazione)
3
4  SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
5  PROJECT_ROOT="$(cd "$SCRIPT_DIR/../../.." && pwd)"
6
7  # Funzione per risolvere alias dei nomi
8  resolve_app_name() {
9      case $1 in
10         GeometryBroadcastner) echo "Broadcastner" ;;
11         GeometryListener)     echo "Listener" ;;
12         *)                     echo "$1" ;;
13     esac
14 }
15
16 if [ "$#" -eq 0 ]; then
```

```

17  read -p "Inserisci il PRIMO eseguibile (già avviato, es.
Broadcastner o app10f): " RAW_NAME
18  read -p "[Opzionale] Inserisci il SECONDO eseguibile: "
RAW_SEC_NAME
19  else
20  RAW_NAME=$1
21  RAW_SEC_NAME=$2
22  fi
23
24  EXECUTABLE_NAME=$(resolve_app_name "$RAW_NAME")
25  APP_NAME=$(basename "$EXECUTABLE_NAME")
26
27  PIDS_1=$(pidof "$APP_NAME")
28  if [ -z "$PIDS_1" ]; then
29  echo "ERRORE: Il processo '$APP_NAME' non è in esecuzione!"
30  exit 1
31  fi
32
33  PIDS_CMD=""
34
35  aggiungi_pid_e_thread() {
36  local process_name=$1
37  local pids=$2
38  for p in $pids; do
39  PIDS_CMD="$PIDS_CMD -P $p"
40  echo ">> Trovato processo principale: $process_name (PID:
$p)"
41  if [ -d "/proc/$p/task" ]; then
42  for tid_path in /proc/$p/task/*; do
43  tid=$(basename "$tid_path")
44  if [ "$tid" != "$p" ]; then
45  PIDS_CMD="$PIDS_CMD -P $tid"
46  thread_name=$(cat "$tid_path/comm" 2>/dev/null)
47  echo "    -> Agganciato thread: $thread_name (TID: $tid)"
48  fi
49  done
50  fi
51  done
52  }
53

```

```

54     aggiungi_pid_e_thread "$APP_NAME" "$PIDS_1"
55
56     if [ -n "$RAW_SEC_NAME" ]; then
57         SEC_EXECUTABLE_NAME=$(resolve_app_name "$RAW_SEC_NAME")
58         SEC_APP_NAME=$(basename "$SEC_EXECUTABLE_NAME")
59         PIDS_2=$(pidof "$SEC_APP_NAME")
60         if [ -n "$PIDS_2" ]; then
61             aggiungi_pid_e_thread "$SEC_APP_NAME" "$PIDS_2"
62         fi
63     fi
64
65     TIMESTAMP=$(date +"%Y%m%d_%H%M%S")
66     OUTPUT_DIR="$PROJECT_ROOT/bin/Test/trace_marker_${APP_NAME}_${
TIMESTAMP}"
67     mkdir -p "$OUTPUT_DIR"
68
69     # Salviamo la cartella di output per i prossimi script
70     echo "$OUTPUT_DIR" > /tmp/last_marker_dir
71     # Salviamo anche l'app name per trovare il sorgente in marker3
72     echo "$APP_NAME" > "$OUTPUT_DIR/app_name.txt"
73
74     echo "=====
"
75     echo " Inizio tracciamento per: $APP_NAME"
76     echo " Cartella di output: $OUTPUT_DIR"
77     echo "=====
"
78
79     echo "[1/4] Registrazione trace-cmd..."
80     echo "--- PREMI Ctrl+C PER FERMARE LA REGISTRAZIONE ---"
81     sudo trace-cmd record -e sched:sched_switch -e sched:
sched_wakeup $PIDS_CMD -o "$OUTPUT_DIR/trace.dat"
82
83     if [ ! -f "$OUTPUT_DIR/trace.dat" ]; then
84         echo "ERRORE: trace.dat non creato."
85         exit 1
86     fi
87
88     sudo chown -R "${SUDO_UID:-$(id -u)}:${SUDO_GID:-$(id -g)}" "
$OUTPUT_DIR"

```

```

89  echo "Registrazione completata. Usa marker2.sh per generare il
    report."

```

LISTING 14.41: `marker1.sh` – Aggancio TID, registrazione e salvataggio contesto

File prodotti: `trace.dat`, `/tmp/last_marker_dir`, `app_name.txt`

marker2.sh: Decodifica

`marker2.sh` legge il percorso della cartella di sessione da `/tmp/last_marker_dir`, ma accetta anche un argomento esplicito: `${1:-$DEFAULT_DIR}` usa il primo argomento se presente, altrimenti cade sul valore letto dal file temporaneo. Questo permette di puntare manualmente a una sessione precedente senza dover rieseguire la registrazione. Decodifica il binario `trace.dat` in testo e guida l'utente al passo successivo.

```

1  #!/bin/bash
2  # marker2.sh - Generazione report testuale
3
4  if [ -f /tmp/last_marker_dir ]; then
5  DEFAULT_DIR=$(cat /tmp/last_marker_dir)
6  fi
7
8  OUTPUT_DIR=${1:-$DEFAULT_DIR}
9
10 if [ -z "$OUTPUT_DIR" ] || [ ! -d "$OUTPUT_DIR" ]; then
11 echo "ERRORE: Cartella di output non trovata. Esegui prima marker1.
    sh o specifica la cartella come argomento."
12 exit 1
13 fi
14
15 echo "[2/4] Generazione del report testuale in $OUTPUT_DIR..."
16 if [ ! -f "$OUTPUT_DIR/trace.dat" ]; then
17 echo "ERRORE: $OUTPUT_DIR/trace.dat non trovato."
18 exit 1
19 fi
20
21 trace-cmd report "$OUTPUT_DIR/trace.dat" > "$OUTPUT_DIR/
    trace_output.txt"
22 echo "Report generato: $OUTPUT_DIR/trace_output.txt"

```

```
23 echo "Usa marker3.sh per l'analisi dei thread."
```

LISTING 14.42: `marker2.sh` – Decodifica con fallback esplicito

File prodotto: `trace_output.txt`

marker3.sh: Analisi con Parser C++

`marker3.sh` recupera il nome dell'applicazione da `app_name.txt` (scritto da `marker1.sh`) per ricostruire il percorso del sorgente C/C++ tramite `find_src`. Compila il parser just-in-time nella cartella di sessione e lo esegue. La dipendenza da `trace_output.txt` è verificata esplicitamente: se il file non esiste, lo script segnala all'utente di eseguire prima `marker2.sh`.

```
1  #!/bin/bash
2  # marker3.sh - Analisi dei thread con thread_analysis
3
4  SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
5  PROJECT_ROOT="$(cd "$SCRIPT_DIR/../../.." && pwd)"
6
7  if [ -f /tmp/last_marker_dir ]; then
8  DEFAULT_DIR=$(cat /tmp/last_marker_dir)
9  fi
10
11 OUTPUT_DIR=${1:-$DEFAULT_DIR}
12
13 if [ -z "$OUTPUT_DIR" ] || [ ! -d "$OUTPUT_DIR" ]; then
14 echo "ERRORE: Cartella di output non trovata."
15 exit 1
16 fi
17
18 APP_NAME=$(cat "$OUTPUT_DIR/app_name.txt" 2>/dev/null)
19
20 find_src() {
21     local name=$1
22     local candidate=$(find "$PROJECT_ROOT/src/app" -name "${name}.cpp" -o -name "${name}.c" | head -n 1)
23     if [ -n "$candidate" ]; then
24     echo "$candidate"
25     else
```

```

26     find "$PROJECT_ROOT/src" -name "${name}.cpp" -o -name "${
name}.c" | head -n 1
27     fi
28 }
29
30 SRC_FILE_ABS=$(find_src "$APP_NAME")
31 PARSER_SRC="$PROJECT_ROOT/src/tools/thread_analysis.cpp"
32 PARSER_EXE="$OUTPUT_DIR/thread_analysis"
33
34 echo "[3/4] Compilazione analizzatore C++..."
35 if [ -f "$PARSER_SRC" ]; then
36     g++ -O2 -std=c++17 "$PARSER_SRC" -o "$PARSER_EXE"
37 else
38     echo "ERRORE: $PARSER_SRC non trovato!"
39     exit 1
40 fi
41
42 echo "[4/4] Avvio Analisi..."
43 if [ ! -f "$OUTPUT_DIR/trace_output.txt" ]; then
44     echo "ERRORE: $OUTPUT_DIR/trace_output.txt non trovato. Esegui
marker2.sh prima."
45     exit 1
46 fi
47
48 cd "$OUTPUT_DIR" || exit
49 ./thread_analysis "trace_output.txt" "$SRC_FILE_ABS" "ALL" > "
risultati_finali.txt"
50 echo ""
51 cat "risultati_finali.txt"
52 cd "$PROJECT_ROOT" || exit
53
54 echo "Analisi completata. Usa marker4.sh per visualizzare i
risultati."

```

LISTING 14.43: `marker3.sh` – Compilazione parser e analisi con
verifica dipendenze

File prodotti: `timeline.csv`, `risultati_finali.txt`

marker4.sh: Visualizzazione

`marker4.sh` conclude la pipeline lanciando il monitor Python e aprendo KernelShark. Verifica esplicitamente la presenza di `timeline.csv` prima di lanciare il monitor, guidando l'utente verso `marker3.sh` se il file manca. Anche in questo caso KernelShark viene aperto in background con la stessa tecnica della subshell, lasciando il terminale libero.

```

1  #!/bin/bash
2  # marker4.sh - Visualizzazione risultati (Monitor Python e
   KernelShark)
3
4  SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
5  PROJECT_ROOT="$(cd "$SCRIPT_DIR/../../.." && pwd)"
6
7  if [ -f /tmp/last_marker_dir ]; then
8  DEFAULT_DIR=$(cat /tmp/last_marker_dir)
9  fi
10
11  OUTPUT_DIR=${1:-$DEFAULT_DIR}
12
13  if [ -z "$OUTPUT_DIR" ] || [ ! -d "$OUTPUT_DIR" ]; then
14  echo "ERRORE: Cartella di output non trovata."
15  exit 1
16  fi
17
18  MONITOR_SCRIPT_ABS="$PROJECT_ROOT/src/tools/monitor.py"
19
20  echo "[5/6] Avvio monitor visivo (Python)..."
21  if [ -f "$MONITOR_SCRIPT_ABS" ]; then
22  if [ -f "$OUTPUT_DIR/timeline.csv" ]; then
23  cd "$OUTPUT_DIR" || exit
24  PYTHON_BIN="python3"
25  "$PYTHON_BIN" "$MONITOR_SCRIPT_ABS" "timeline.csv"
26  cd "$PROJECT_ROOT" || exit
27  else
28  echo "ERRORE: timeline.csv non trovato in $OUTPUT_DIR. Esegui
   marker3.sh prima."
29  fi
30  else

```



```
31     echo "ATTENZIONE: Non trovo lo script $MONITOR_SCRIPT_ABS."
32     fi
33
34     echo "[6/6] Avvio KernelShark..."
35     if command -v kernelshark >/dev/null 2>&1; then
36     if [ -f "$OUTPUT_DIR/trace.dat" ]; then
37     (cd "$OUTPUT_DIR" && kernelshark trace.dat) &
38     else
39     echo "ERRORE: trace.dat non trovato in $OUTPUT_DIR."
40     fi
41     else
42     echo "ATTENZIONE: KernelShark non trovato."
43     fi
44
45     echo "Visualizzazione avviata."
```

LISTING 14.44: `marker4.sh` – Monitor Python e apertura
KernelShark

File prodotto: `timeline_monitor.png`

La sequenza `marker1` → `marker2` → `marker3` → `marker4` riproduce esattamente `marker.sh`, con il valore aggiunto che ogni passaggio è verificabile, ripetibile e ispezionabile separatamente. Il file `/tmp/last_marker_dir` è il filo conduttore che collega i quattro script senza che l'utente debba ricordarsi o riscrivere il percorso della cartella di sessione.

14.4 Fase 5 — Analisi del Tracciato e Strumenti di Monitoraggio

Dati in ingresso: `trace_output.txt`, file sorgente C/C++ dell'applicazione.

Strumenti utilizzati: `thread_analysis.cpp` (C++17), `monitor.py` (Python 3 + pandas + matplotlib).

Risultati prodotti: `timeline.csv`, `risultati_finali.txt`, `timeline_monitor.png`.

14.4.1 L'Analizzatore C++: `thread_analysis.cpp`

Posizione nel progetto: `src/tools/thread_analysis.cpp`

Il parser è un programma che riceve tre argomenti dalla shell: il file testuale del trace, il file sorgente C/C++ dell'applicazione e il filtro CPU (ALL per tutti i core, oppure un numero intero per un core specifico). Dalla loro combinazione produce `timeline.csv` e stampa a terminale le statistiche RT per ogni thread. L'elaborazione avviene in quattro fasi sequenziali, ciascuna descritta di seguito seguendo il codice sorgente.

Include, Strutture Dati e Helper Functions

Il file dipende esclusivamente dalla standard library C++17: nessuna dipendenza esterna è necessaria. Le strutture dati principali sono tre:

- **Interval:** rappresenta un intervallo temporale con inizio e fine in secondi assoluti.
- **OutEvent:** associa al timestamp di uscita dalla CPU lo stato del thread uscente, codificato con il carattere POSIX (S = sleeping, R = runnable/preempted, D = uninterruptible, ecc.).
- **TaskData:** è il contenitore principale. Per ogni thread (identificato dal PID/TID) raccoglie i vettori di switch-in, switch-out, intervalli RUN, SLEEP, PREEMPT e i marker applicativi letti dal *trace*.

```

1  #include <algorithm>
2  #include <cmath>
3  #include <fstream>
4  #include <iomanip>
5  #include <iostream>
6  #include <limits>
7  #include <map>
8  #include <regex>
9  #include <string>
10 #include <vector>
11
12 struct Interval { double start; double end; };
13 struct OutEvent { double time; char state; };
14
15 struct TaskData {
16     std::string name;
17     double expected_period_sec = 0.0;
18     double expected_deadline_sec = 0.0;
19     std::vector<double> switch_in;
20     std::vector<OutEvent> switch_out;
21     std::vector<Interval> run;
22     std::vector<Interval> sleep_int;
23     std::vector<Interval> preempt_int;
24     std::vector<std::pair<double, std::string>> markers;
25 };
26
27 struct MissLogEntry {
28     std::string task;
29     int period_idx;
30     double ps_abs;
31     double ps_rel;
32     double deadline_ms;
33     double response_ms;
34     double overshoot_ms;
35 };
36
37 std::string trim(const std::string &str) {
38     size_t first = str.find_first_not_of(" \t");
39     if (std::string::npos == first) return "";
40     size_t last = str.find_last_not_of(" \t\r\n");

```

```

41         return str.substr(first, (last - first + 1));
42     }
43
44     // Somma delle durate degli intervalli che si sovrappongono a [
45     lo, hi]
46     double state_time_in(const std::vector<Interval> &intervals,
47     double lo, double hi) {
48         double total = 0;
49         for (const auto &iv : intervals) {
50             double overlap_start = std::max(iv.start, lo);
51             double overlap_end    = std::min(iv.end,   hi);
52             if (overlap_end > overlap_start)
53                 total += (overlap_end - overlap_start);
54         }
55         return total;
56     }

```

LISTING 14.45: thread_analysis.cpp – Include e strutture dati

Il main apre con la validazione degli argomenti e la dichiarazione delle tre mappe principali. Il terzo argomento (`target_cpu`) permette di filtrare gli eventi per core: se viene passato `ALL` il filtro è disabilitato (`target_cpu = -1`) e tutti i core vengono analizzati insieme.

```

1     int main(int argc, char *argv[]) {
2         if (argc < 3) {
3             std::cerr << "Uso: " << argv[0]
4             << " <trace.txt> <sorgente.c> [CPU_ID]\n";
5             return 1;
6         }
7
8         std::string trace_file_path = argv[1];
9         std::string source_file_path = argv[2];
10
11         int target_cpu = -1;
12         if (argc >= 4) {
13             std::string cpu_arg = argv[3];
14             if (cpu_arg != "ALL" && cpu_arg != "all")
15                 target_cpu = std::stoi(cpu_arg);
16         }
17

```

```

18     std::map<int, TaskData>          tasks;
19     std::map<std::string, double> expected_periods;
20     std::map<std::string, double> expected_deadlines;
21

```

LISTING 14.46: `thread_analysis.cpp` – Ingresso del main e mappe di stato

Fase 1: Estrazione dei Parametri Teorici dal Sorgente C/C++

La prima fase legge il file sorgente dell'applicazione riga per riga e applica tre espressioni regolari per estrarre, dalle istruzioni di configurazione dell'attività, il nome, il periodo e la deadline di ogni task. Questo approccio consente al parser di essere completamente indipendente dal codice di tracing: i parametri teorici non vengono scritti nei marker, ma sono letti direttamente dal sorgente che ha generato il trace.

Le tre regex coprono le varianti di `snprintf/sprintf` usate nel progetto per il nome, e le assegnazioni dirette per periodo e deadline. I valori di periodo e deadline sono espressi in millisecondi nel sorgente e convertiti in secondi dividendo per 1000, per uniformarli ai timestamp di `trace_output.txt` che sono in secondi assoluti.

```

1      //
      =====

2      // FASE 1: LETTURA DEL SORGENTE C PER ESTRARRE PERIODI E
      DEADLINE
3      //
      =====

4      std::ifstream src_file(source_file_path);
5      if (src_file.is_open()) {
6          std::string src_line;
7
8          // Riconosce: snprintf(activity_1.name, 15, "Activity_1
      ")
9          std::regex name_re(
10             R"DELIM(s(?:n)?printf\s*\(\s*([a-zA-Z0-9_\[\]]+)\)\.name\s*,)DELIM"
11             R"DELIM(\s*(?:\d+|sizeof\s*\([^)]+\)|[a-zA-Z0-9_]+\)\s*,\s*)?"([^\"]+)\s*\))DELIM");

```

```

12
13         // Riconosce: activity_1.period = 1000
14         std::regex period_re(
15         R"DELIM(([a-zA-Z0-9_\[\\]]+)\.period\s*=\s*(\d+))DELIM"
16
17         ;
18
19         // Riconosce: activity_1.deadline = 1000
20         std::regex deadline_re(
21         R"DELIM(([a-zA-Z0-9_\[\\]]+)\.deadline\s*=\s*(\d+))DELIM
22
23         ");
24
25         std::map<std::string, std::string> var_to_name;
26         std::map<std::string, double>      var_to_period;
27         std::map<std::string, double>      var_to_deadline;
28
29         while (std::getline(src_file, src_line)) {
30             std::smatch m;
31             if (std::regex_search(src_line, m, name_re))
32                 var_to_name[m[1]] = m[2];
33             if (std::regex_search(src_line, m, period_re))
34                 var_to_period[m[1]] = std::stod(m[2]) / 1000.0;
35             if (std::regex_search(src_line, m, deadline_re))
36                 var_to_deadline[m[1]] = std::stod(m[2]) / 1000.0;
37         }
38
39         // Associa nome -> periodo/deadline tramite la
40         // variabile comune
41         for (const auto &[var_name, act_name] : var_to_name) {
42             if (var_to_period.count(var_name))
43                 expected_periods[act_name] = var_to_period[var_name];
44             if (var_to_deadline.count(var_name))
45                 expected_deadlines[act_name] = var_to_deadline[
46                 var_name];
47         }
48         src_file.close();
49     } else {
50         std::cerr << "Attenzione: Impossibile aprire "
51         << source_file_path << "\n";
52     }
53 }

```

47

LISTING 14.47: `thread_analysis.cpp` – Fase 1: parsing regex del sorgente

La variabile intermedia `var_to_name` è necessaria perché nel sorgente la stessa struct può comparire in righe distinte: `activity_1.name = "Activity_1"` e `activity_1.period = 1000` condividono solo il prefisso `activity_1`. La mappa `var_to_name` funge da chiave di join: prima si raccolgono tutte e tre le associazioni per nome variabile, poi si costruisce la mappa finale indicizzata per nome attività.

Fase 2: Discovery Dinamica dei Thread dal Trace

La seconda fase scorre `trace_output.txt` per la prima volta. La regex principale `line_re` decompone ogni riga del trace nei suoi campi: nome del processo/thread, PID, CPU, timestamp e tipo di evento. Per ogni riga, il parser cerca una corrispondenza tra il nome del thread (`comm`) e i nomi estratti nella Fase 1. Questa corrispondenza viene usata per popolare la mappa `tasks`, assegnando ad ogni PID/TID il nome completo del task, il periodo e la deadline attesi. Un secondo ciclo gestisce i thread che hanno scritto marker nel trace ma non sono stati associati in modo diretto a un nome del sorgente: questo copre i casi in cui il nome del thread nel trace è un prefisso del nome dichiarato nel sorgente.

```

1      //
      =====
2      // FASE 2: PARSING DEL TRACE.TXT (DISCOVERY DINAMICA)
3      //
      =====

4      std::ifstream file(trace_file_path);
5      if (!file.is_open()) return 1;
6
7      // Regex che decompone ogni riga del trace in: comm, pid,
      cpu, time, event, details
8      std::regex line_re(
9      R"DELIM(~\s*(.*?)-(\d+)\s+[(\d+)\]\s+(\d+\.\d+):\s+(.*?):\s+(.*)$)DELIM");
10

```

```

11     std::string line;
12     std::map<int, std::string> marker_pids;
13
14     while (std::getline(file, line)) {
15         std::smatch m;
16         if (std::regex_search(line, m, line_re)) {
17             std::string comm = trim(m[1]);
18             int pid = std::stoi(m[2]);
19             std::string event = trim(m[5]);
20
21             // Raccoglie i PID che hanno scritto marker
applicativi
22             if (event.find("print") != std::string::npos ||
23                 event.find("tracing_mark_write") != std::string::
npos) {
24                 if (marker_pids.count(pid) == 0)
25                     marker_pids[pid] = comm;
26             }
27
28             // Associa PID -> TaskData se il comm corrisponde a
un nome noto
29             if (tasks.count(pid) == 0) {
30                 for (const auto &kv : expected_periods) {
31                     if (kv.first.find(comm) == 0) {
32                         tasks[pid].name = kv.
first;
33                         tasks[pid].expected_period_sec = kv.
second;
34                         tasks[pid].expected_deadline_sec =
35                             expected_deadlines.count(kv.first)
36                             ? expected_deadlines[kv.first]
37                             : kv.second;
38                         break;
39                     }
40                 }
41             }
42         }
43     }
44

```



```

45     // Secondo ciclo: aggiunge i thread con marker non ancora
in tasks
46     for (const auto &[pid, comm] : marker_pids) {
47         if (tasks.count(pid)) continue;
48
49         std::string full_name = "";
50         for (const auto &kv : expected_periods)
51             if (kv.first.find(comm) == 0) { full_name = kv.first;
break; }
52
53         // Esclusione thread interni DDS
54         if (full_name.empty()) {
55             if (comm.find("dds.") == 0 || comm.find("
tpool") == 0 ||
56                 comm.find("fastrtps") == 0 || comm.find("fastdds"
) == 0 ||
57                 comm.find("reception") == 0 || comm.find("
non_blocking") == 0)
58                 continue;
59             full_name = comm;
60         }
61
62         tasks[pid].name = full_name;
63         if (expected_periods.count(full_name))
64             tasks[pid].expected_period_sec = expected_periods[
full_name];
65         if (expected_deadlines.count(full_name))
66             tasks[pid].expected_deadline_sec = expected_deadlines[
full_name];
67         else if (expected_periods.count(full_name))
68             tasks[pid].expected_deadline_sec = expected_periods[
full_name];
69         }
70
71         // Riavvolge il file per la Fase 3
72         file.clear();
73         file.seekg(0);
74

```

LISTING 14.48: thread_analysis.cpp – Fase 2: prima passata,
discovery thread

Al termine della Fase 2 la mappa `tasks` contiene un record per ogni thread applicativo trovato nel trace, con nome e parametri RT già associati. Il file viene riavvolto a inizio con `seekg(0)` per la seconda passata.

Fase 3: Estrazione degli Sched_Switch e Logica VPID

La terza fase è la più critica: scorre il trace una seconda volta raccogliendo tutti gli eventi `sched_switch` e i marker applicativi. Per ogni `sched_switch` aggiorna i vettori `switch_in` e `switch_out` del thread uscente e di quello entrante, registrando anche il carattere di stato del thread uscente. Un meccanismo aggiuntivo per gestire nel caso librerie che gestiscono cambi a real-time di periodo o priorità che genererebbero problematiche nella visualizzazione all'interno del monitor, i **Virtual PID (VPID)**, gestisce il caso speciale dell'inversione di priorità. Quando un task RT muta la propria priorità a runtime (marcando l'evento con il marker `INVERSION_DONE_...`), il kernel assegna al thread lo stesso PID ma con caratteristiche temporali diverse. Trattare i due "momenti" del thread come un unico task produrrebbe statistiche errate. La mappa `real_to_vpid` risolve il problema: finché non viene trovato un marker di inversione, il VPID coincide con il PID reale e gli eventi vengono registrati normalmente. All'intercettazione del marker, viene creato un VPID sintetico (`old_pid + 1000000`), il vecchio task viene chiuso fittiziamente e il nuovo task virtuale inizia ad accumulare i propri eventi. Tutte le righe successive con il PID originale faranno riferimento al nuovo VPID tramite la mappa.

```
1      //
2      // FASE 3: ESTRAZIONE SCHED_SWITCH E LOGICA PID VIRTUALI (
  VPID)
3      //
4      double global_min_time = std::numeric_limits<double>::
infinity();
5      double global_max_time = 0;
6
7      // Mappa di routing: real\_pid -> vpid (identita se non c'e
inversione)
8      std::map<int, int> real_to_vpid;
9
10     std::regex switch_re(
```

```

11      R"DELIM(. *?:(\d+)\s+\[(\d+)\]\s+([A-Z]) .*?==>\s+. *?:(\d+)\s
12      +\[ \] DELIM)";
13
14      while (std::getline(file, line)) {
15          std::smatch m;
16          if (std::regex_search(line, m, line_re)) {
17              int pid = std::stoi(m[2]);
18              int cpu = std::stoi(m[3]);
19              double t = std::stod(m[4]);
20              std::string event = trim(m[5]);
21              std::string details = m[6];
22
23              if (t < global_min_time) global_min_time = t;
24              if (t > global_max_time) global_max_time = t;
25
26              // Filtro per core specifico se richiesto
27              if (target_cpu != -1 && cpu != target_cpu) continue
28          ;
29
30              // Risolve il VPID corrente per questo PID reale
31              int vpid = real_to_vpid.count(pid) ? real_to_vpid[
pid] : pid;

```

LISTING 14.49: `thread_analysis.cpp` – Fase 3: seconda passata, `sched_switch` e VPID

La gestione dei `sched_switch` costruisce incrementalmente i vettori `switch_in/switch_out`: ogni context switch produce un `switch_out` per il thread che cede la CPU (con relativo stato) e un `switch_in` per quello che la prende. Entrambi vengono registrati con il VPID corrente, non il PID reale.

```

1          if (event == "sched_switch") {
2              std::smatch sm;
3              if (std::regex_search(details, sm, switch_re))
4          {
5              int prev_pid = std::stoi(sm[1]); // thread
           uscente
           char state = sm[2].str()[0]; // stato:
           S, R, D...

```

```

6         int next_pid = std::stoi(sm[3]); // thread
    entrante
7
8         // Routing attraverso la mappa VPID
9         int v_prev = real_to_vpid.count(prev_pid)
10        ? real_to_vpid[prev_pid] : prev_pid;
11        int v_next = real_to_vpid.count(next_pid)
12        ? real_to_vpid[next_pid] : next_pid;
13
14        if (tasks.count(v_next)) tasks[v_next].
switch_in.push_back(t);
15        if (tasks.count(v_prev)) tasks[v_prev].
switch_out.push_back({t, state});
16    }
17

```

LISTING 14.50: thread_analysis.cpp – Registrazione switch_in/switch_out

Il blocco seguente gestisce i marker applicativi. Dopo aver estratto il testo del marker (eliminando il prefisso `:` che `trace-cmd report` aggiunge), il parser verifica se si tratta del marker speciale di inversione. In caso affermativo, crea il nuovo VPID e aggiorna la mappa; in caso contrario, registra il marker nel vettore del task corrente.

```

1         } else if (event.find("print") != std::string::npos
    ||
2         event.find("tracing_mark_write") != std::string::
npos) {
3
4             std::string marker_text = trim(details);
5             size_t colon_pos = marker_text.find(":");
6             if (colon_pos != std::string::npos)
7                 marker_text = trim(marker_text.substr(colon_pos
+ 1));
8
9             //
=====
10             // Logica inversione di priorit 
11             //
=====

```

```

12         if (marker_text.find("INVERSION_DONE") != std::
string::npos) {
13
14             std::regex inv_re(
15                 R"(INVERSION_DONE_([a-zA-Z0-9_+)_PER_(\d+))");
16             std::smatch sm_inv;
17
18             if (std::regex_search(marker_text, sm_inv,
inv_re)) {
19                 std::string base_name = trim(sm_inv
[1]);
20                 int new_period_ms = std::stoi(
sm_inv[3]);
21
22                 int old_vpid = vpid;
23                 int new_vpid = old_vpid + 1000000;
24
25                 // 1. Chiude fittiziamente il blocco
RUN del task vecchio
26                 if (tasks.count(old_vpid))
27                     tasks[old_vpid].switch_out.push_back({t
, 'R'});
28
29                 // 2. Aggiorna la mappa di routing
30                 real_to_vpid[pid] = new_vpid;
31
32                 // 3. Inizializza il nuovo task
virtuale
33                 tasks[new_vpid].name = base_name + "_P2
";
34                 tasks[new_vpid].expected_period_sec =
new_period_ms / 1000.0;
35                 tasks[new_vpid].expected_deadline_sec =
new_period_ms / 1000.0;
36
37                 // 4. Apre fittiziamente il blocco RUN
del nuovo task
38                 tasks[new_vpid].switch_in.push_back(t);

```

```
39         tasks[new_vpid].markers.push_back({t,  
marker_text});  
40     }  
41     continue; // Evita di registrare il marker  
nel task vecchio  
42 }  
43  
44     // Routing standard per tutti gli altri marker  
45     if (tasks.count(vpid))  
46         tasks[vpid].markers.push_back({t, marker_text})  
;  
47 }  
48 }  
49 }  
50
```

LISTING 14.51: `thread_analysis.cpp` – Gestione marker e logica
inversione VPID

Fase 4: Costruzione Intervalli, Metriche RT e Lettura file CSV

La quarta fase elabora la mappa `tasks` risultante e, per ogni thread, esegue quattro operazioni: costruisce gli intervalli RUN abbinando switch-in e switch-out; classifica i gap tra blocchi RUN come SLEEP o PREEMPT in base allo stato del thread uscente; calcola le metriche RT sfruttando i marker (`PERIOD_START/FUNCTION_END`); serializza tutto su `timeline.csv` e stampa il riepilogo a terminale. Il file CSV viene aperto con `std::ofstream` prima del ciclo principale. Ogni riga ha il formato `task,tipo,start,end,duration_ms`.

```

1      //
=====

2      // FASE 4: COSTRUZIONE INTERVALLI + CSV + STATISTICHE
    AVANZATE
3      //
=====

4      std::ofstream csv("timeline.csv");
5      csv << "task,type,start,end,duration_ms\n";
6
7      std::vector<MissLogEntry> all_misses;
8      double csv_span_s = global_max_time - global_min_time;
9
10     for (auto &pair : tasks) {
11         TaskData &tdata = pair.second;
12         // --- Costruzione intervalli RUN ---
13         // Abbina ogni switch_in con il primo switch_out
    successivo
14         size_t i = 0, j = 0;
15         while (i < tdata.switch_in.size() && j < tdata.
switch_out.size()) {
16             if (tdata.switch_out[j].time > tdata.switch_in[i])
17             {
18                 tdata.run.push_back({tdata.switch_in[i], tdata.
switch_out[j].time});
19                 i++; j++;
20             } else { j++; }
    }

```

LISTING 14.52: `thread_analysis.cpp` – Apertura CSV e costruzione intervalli RUN

Per i thread privi di marker (es. thread di supporto senza strumentazione), il parser genera intervalli `WAIT` sintetici basandosi sul periodo atteso estratto nella Fase 1. Questo permette al monitor Python di visualizzare anche thread non strumentati, mostrando i periodi vuoti come assenze di attività.

```

1         if (tdata.run.empty()) continue;
2
3         // --- Scrivi RUN al CSV ---
4         for (auto &r : tdata.run) {
5             double d = r.end - r.start;
6             csv << tdata.name << ",RUN," << std::fixed << std:::
7             setprecision(6)
8             << r.start << "," << r.end << "," << d * 1e3 << "\n
9             ";
10        }
11
12        // --- Classifica i gap come SLEEP (S/D/Z/X) o PREEMPT
13        (R) ---
14        for (size_t k = 0; k < tdata.run.size() - 1 && k <
15        tdata.switch_out.size(); k++) {
16            double gap_start = tdata.run[k].end;
17            double gap_end    = tdata.run[k + 1].start;
18            if (gap_end <= gap_start) continue;
19
20            char state = tdata.switch_out[k].state;
21            if (state == 'S' || state == 'D' || state == 'Z' ||
22            state == 'X') {
23                tdata.sleep_int.push_back({gap_start, gap_end});
24            };
25
26            csv << tdata.name << ",SLEEP," << std::fixed <<
27            std:::setprecision(6)
28            << gap_start << "," << gap_end << ","
29            << (gap_end - gap_start) * 1e3 << "\n";
30        } else if (state == 'R') {
31            tdata.preempt_int.push_back({gap_start, gap_end
32            });

```



```

24         csv << tdata.name << ",PREEMPT," << std::fixed
    << std::setprecision(6)
25         << gap_start << "," << gap_end << ","
26         << (gap_end - gap_start) * 1e3 << "\n";
27     }
28 }
29
30 // --- Scrivi MARKER al CSV ---
31 for (const auto &m : tdata.markers) {
32     csv << tdata.name << ",MARKER_" << m.second << ","
33     << std::fixed << std::setprecision(6)
34     << m.first << "," << m.first << ",0.0\n";
35 }
36

```

LISTING 14.53: thread_analysis.cpp – Scrivi RUN, SLEEP, PREEMPT e MARKER al CSV

I marker DDS_MSG_START/END vengono abbinati in coppia per calcolare la durata di ogni operazione di pubblicazione DDS. Anche questi intervalli vengono scritti nel CSV come righe di tipo DDS_MSG, con WCET e ACET DDS calcolati e scritti come righe di statistica.

```

1 // --- Abbinamento marker DDS_MSG_START/END ---
2 std::vector<double> dds_starts, dds_ends;
3 for (const auto &m : tdata.markers) {
4     if (m.second.find("DDS_MSG_START") != std::string::
npos ||
5     m.second.find("DDS_WRITE_START") != std::string::
npos)
6         dds_starts.push_back(m.first);
7     else if (m.second.find("DDS_MSG_END") != std::
string::npos ||
8     m.second.find("DDS_WRITE_END") != std::string::npos
9     )
10        dds_ends.push_back(m.first);
11 }
12 std::sort(dds_starts.begin(), dds_starts.end());
13 std::sort(dds_ends.begin(), dds_ends.end());
14 // [abbinamento e scrittura CSV omissi per brevità]

```

```

15         // --- Estrazione marker di periodo ---
16         std::vector<double> pstart, pend, func_ends;
17         for (const auto &m : tdata.markers) {
18             if (m.second.find("PERIOD_START") != std::
string::npos) pstart.push_back(m.first);
19             else if (m.second.find("PERIOD_END") != std::
string::npos) pend.push_back(m.first);
20             else if (m.second.find("FUNCTION_END") != std::
string::npos) func_ends.push_back(m.first);
21         }
22         std::sort(pstart.begin(), pstart.end());
23         std::sort(pend.begin(), pend.end());
24         std::sort(func_ends.begin(), func_ends.end());
25
26         // Periodo misurato: mediana degli intervalli tra
PERIOD_START consecutivi
27         double measured_ms = -1, jitter_ms = -1;
28         if (pstart.size() >= 2) {
29             std::vector<double> gaps;
30             for (size_t g = 1; g < pstart.size(); g++)
31                 gaps.push_back(pstart[g] - pstart[g - 1]);
32             std::sort(gaps.begin(), gaps.end());
33             measured_ms = gaps[gaps.size() / 2] * 1000.0;
34
35             // Jitter: deviazione standard dei gap
36             double mean_gap = 0;
37             for (double g : gaps) mean_gap += g;
38             mean_gap /= gaps.size();
39             double var = 0;
40             for (double g : gaps) var += (g - mean_gap) * (g -
mean_gap);
41             jitter_ms = std::sqrt(var / gaps.size()) * 1000.0;
42         }
43

```

LISTING 14.54: `thread_analysis.cpp` – Intervalli DDS e calcolo metriche RT

Il calcolo dello Slack Time e il rilevamento delle Deadline Miss avvengono iterando sui periodi. Per ogni istante `PERIOD_START`, il parser cerca il corrispondente `FUNCTION_END` all'interno della stessa finestra temporale. La differenza tra i due

è il Response Time; sottraendolo dalla deadline si ottiene lo Slack. Se lo Slack è negativo, l'evento viene registrato in `MissLogEntry` e contato in `n_misses`.

```

1      double wcet_ms = 0, sum_run_ms = 0, worst_slack_ms = 1
      e9;
2      int job_count = 0, n_misses = 0;
3
4      for (size_t k = 0; k < pstart.size(); k++) {
5          double ps = pstart[k];
6          // Cerca PERIOD_END corrispondente
7          double pe = -1;
8          for (double cand : pend)
9              if (cand > ps) { pe = cand; break; }
10         if (pe == -1 && k + 1 < pstart.size()) pe = pstart[
            k + 1];
11
12         double pe_upper = (pe != -1) ? pe
13             : std::numeric_limits<double>::infinity();
14
15         // Cerca FUNCTION_END all'interno del periodo
16         double func_end_t = -1;
17         for (double fe : func_ends)
18             if (fe > ps && fe < pe_upper) func_end_t = fe;
19
20         if (func_end_t != -1 && calc_deadline_sec > 0) {
21             double response_ms = (func_end_t - ps) *
            1000.0;
22             double slack_ms      = (calc_deadline_sec *
            1000.0) - response_ms;
23             worst_slack_ms = std::min(worst_slack_ms,
            slack_ms);
24
25             if (slack_ms < 0) {
26                 n_misses++;
27                 MissLogEntry mle;
28                 mle.task      = tdata.name;
29                 mle.period_idx = k + 1;
30                 mle.ps_abs     = ps;
31                 mle.ps_rel     = ps - global_min_time;

```

```

32         mle.deadline_ms = calc_deadline_sec *
1000.0;
33         mle.response_ms = response_ms;
34         mle.overshoot_ms = -slack_ms;
35         all_misses.push_back(mle);
36     }
37 }
38
39 // WCET e ACET per periodo
40 double inst_run_ms = (pe != -1)
41 ? state_time_in(tdata.run, ps, pe) * 1000.0 : 0;
42 if (inst_run_ms > 0) {
43     wcet_ms = std::max(wcet_ms, inst_run_ms);
44     sum_run_ms += inst_run_ms;
45     job_count++;
46 }
47 }
48 double acet_ms = job_count > 0 ? (sum_run_ms /
job_count) : 0;
49

```

LISTING 14.55: `thread_analysis.cpp` – Slack Time e rilevamento Deadline Miss

Al termine del ciclo per ogni task, tutte le metriche vengono scritte nel CSV come righe `STAT_*` e stampate a terminale in formato tabulare. Le righe `MISS_LOG` vengono scritte alla fine del CSV, dopo tutti i task.

```

1 // --- Scrivi righe STAT_* al CSV ---
2 csv << std::fixed << std::setprecision(6);
3 csv << tdata.name << ",STAT_PERIOD,0,0," << (
calc_period_sec * 1000.0) << "\n";
4 csv << tdata.name << ",STAT_MEASURED,0,0," << (
measured_ms > 0 ? measured_ms : -1) << "\n";
5 csv << tdata.name << ",STAT_DEADLINE,0,0," << (
calc_deadline_sec * 1000.0) << "\n";
6 csv << tdata.name << ",STAT_WCET,0,0," << wcet_ms
<< "\n";
7 csv << tdata.name << ",STAT_ACET,0,0," << acet_ms
<< "\n";

```

```

8         csv << tdata.name << ",STAT_JITTER,0,0," << (
jitter_ms > 0 ? jitter_ms : -1) << "\n";
9         csv << tdata.name << ",STAT_UTIL,0,0," << util
<< "\n";
10        csv << tdata.name << ",STAT_NMISS,0,0," << n_misses
<< "\n";
11        csv << tdata.name << ",STAT_WSLACK,0,0," << w_slack
<< "\n";
12        // [ulteriori STAT_* omesse per brevit ]
13
14        // --- Output a terminale ---
15        std::cout << "\n
-----\n";
16        std::cout << " TASK: " << tdata.name << " (PID: " <<
pair.first << ")\n";
17        std::cout << "
-----\n";
18        std::cout << std::fixed << std::setprecision(3);
19        std::cout << " [Esecuzione] WCET : " << std::setw(10)
<< wcet_ms
20        << " ms | ACET: " << std::setw(10) << acet_ms << " ms
\n";
21        std::cout << " [Periodo] Mis. : " << std::setw(10)
<< (measured_ms > 0 ? measured_ms : 0)
22        << " ms | Jitter: " << std::setw(8)
<< (jitter_ms > 0 ? jitter_ms : 0) << " ms\n";
23        std::cout << " [Parametri] Periodo: " << std::setw(8)
<< calc_period_sec * 1000
24        << " ms | Deadline: " << std::setw(8) <<
calc_deadline_sec * 1000 << " ms\n";
25        std::cout << " [Scadenze] Miss : " << std::setw(8)
<< n_misses
26        << " | Slack Peggior : " << std::setw(8)
<< (worst_slack_ms >= 1e9 ? 0.0 : worst_slack_ms) << "
ms\n";
27        std::cout << " [Stati] Sleep : " << std::setw(8)
<< tdata.sleep_int.size()
28        << " | Preempt: " << std::setw(8) << tdata.
preempt_int.size()
29        << " | Run n.: " << tdata.run.size() << "\n";
30
31
32

```

```
33         std::cout << " [Globale]      Util% : " << std::setw(8)
    << util * 100.0 << " %\n";
34     }
35
36     // --- Scrivi MISS_LOG al CSV ---
37     for (const auto &mle : all_misses) {
38         csv << mle.task << ",MISS_LOG," << std::fixed << std::
    setprecision(6)
39         << mle.ps_abs << "," << mle.ps_rel << "," << mle.
    deadline_ms << "|"
40         << mle.response_ms << "|" << mle.overshoot_ms << "|"
41         << mle.period_idx << "\n";
42     }
43
44     csv.close();
45     std::cout << "\n[+] Dati esportati su timeline.csv\n";
46     return 0;
47 }
48
```

LISTING 14.56: `thread_analysis.cpp` – Serializzazione statistiche e output terminale

Il campo `STAT_UTIL` merita una nota: l'utilizzo viene calcolato come rapporto tra il tempo totale di esecuzione e la durata complessiva della finestra di osservazione (normalizzata sul numero di periodi se i marker sono presenti, o sull'intera durata del trace in caso contrario). Un utilizzo vicino a 1.0 indica un task saturo; un utilizzo basso con deadline miss segnala prelezioni o interferenze esterne.

Capitolo 15

Caso di Studio: Il Computer di Bordo dell’F-16

Il caso di studio analizzato dal progetto tratta la simulazione del computer di bordo di un caccia F-16, l’implementazione in `src/app/F16_Computer/F16_Computer.cpp`. Questo programma istanzia simultaneamente su 7 core fisici, ciascuno dei quali è un sottosistema avionico distinto del velivolo, infatti ogni sotto-sistema esegue la propria Periodic Task attraverso la politica di schedulazione *Rate Monotonic*. L’obiettivo è duplice: dimostrare che l’architettura delle librerie sviluppate (si veda il Capitolo [14.1.2](#)) riesce a gestire correttamente scenari ad alto parallelismo e a validare che il sistema di tracciamento e analisi sia in grado di gestire e rappresentare graficamente thread concorrenti, producendo metriche temporali coerenti per ogni sottosistema creato.

15.1 Motivazione Aeronautica e Architettura del Sistema

Un computer di bordo avionico reale gestisce in parallelo subsistemi con requisiti temporali diversi la creazione che seguirà avrà come bibliografia di base la seguente [18], ad esempio i loop di controllo del volo devono reagire in pochi millisecondi, mentre i radar e i display possono tollerare periodi più lunghi. Questo permette di testare se la schedulazione *Rate Monotonic*, o nel caso si utilizzi un qualsiasi altro algoritmo di schedulazione, ad ogni task viene permesso di eseguire la propria Periodic Task senza superare la deadline, garantendo che i task più critici ottengano sempre la CPU per eseguire in modo che non si presentino deadline miss che facciano degradare la risposta del sistema. Il simulatore F-16 è stato strutturato in sei **sub-systems** rispecchiando l'architettura avionica reale del velivolo preso in esame:

TABELLA 15.1: Mappa completa dei 20 thread del simulatore F-16

Sottosistema	Thread	Core	Periodo (ms)	Priorità
FCS Longitudinal	PITCH_A	0	10	98
	PITCH_B	1	10	98
	PITCH_C	2	10	98
FCS Lateral	ROLL_A	3	10	98
	ROLL_B	4	10	98
	ROLL_C	5	10	98
FCS Altitude	ALT_A	6	20	97
	ALT_B	0	20	97
Navigation GPS	GPS_1	1	50	94
	GPS_2	2	50	94
Navigation INS	INS_1	3	10	98
	INS_2	4	10	98
	INS_3	5	10	98
Radar Aereo	RADAR_A	6	50	94
Radar Terrestre	RADAR_G	0	100	89
Engine FADEC	FADEC_A	1	10	98
	FADEC_B	2	10	98
Weapons	WEAPONS	3	100	89
Displays	HUD	4	30	96
	MFD	5	30	96

15.2 Librerie Avioniche Utilizzate

Il simulatore non utilizza la libreria `activity` generica (descritta nella Sezione 14.1.5), ma impiega le librerie specifiche create appositamente per il modello F-16 illustrate nella Sezione 15.2.1. Ogni sottosistema corrisponde quindi a una libreria statica dedicata:

```

1  #include "f16_fcs_long.hpp"    // FCS Longitudinale (Beccheggio
    )
2  #include "f16_fcs_lat.hpp"    // FCS Laterale (Rollio)
3  #include "f16_fcs_alt.hpp"    // FCS Quota (Altitudine)
4  #include "f16_nav_gps.hpp"    // Navigazione GPS
5  #include "f16_nav_ins.hpp"    // Navigazione Inerziale
6  #include "f16_radar_air.hpp"  // Radar aria-aria
7  #include "f16_radar_gnd.hpp"  // Radar aria-terra
8  #include "f16_engine_fadec.hpp" // Controllo Motore FADEC
9  #include "f16_weapons.hpp"    // Sistema d'Arma
10 #include "f16_displays.hpp"   // Display Avionici (HUD/MFD)
11 #include "trace_marker.h"     // Marker di tracciamento kernel

```

LISTING 15.1: Include delle librerie avioniche in F16_Computer.cpp

15.2.1 Modulo F-16: Librerie di Simulazione Aerodinamica

I moduli F-16 estendono il pattern della libreria `activity` con due funzionalità aggiuntive:

Rilevamento Runtime delle Deadline Miss e Flag Skip

```

1  exec_end_time = time_current_millisecs();
2
3  if (exec_end_time > exec_next_release_time) {
4      // La funzione ha usato piu' tempo del periodo disponibile:
5      // segnala la violazione e attiva il flag di skip
6      printf("%s:  -MISS-    cost = %ld ms\n",
7          activity.name, computational_cost);
8      skip = true;
9  } else {
10     printf("%s:  DO JOB    cost = %ld ms\n",

```

```
11     activity.name, computational_cost);
12 }
13
14 // Al ciclo successivo, se skip e' true:
15 if (skip) {
16     printf("%s:  *SKIP*\n", activity.name);
17     skip = false;
18     // Salta l'esecuzione e va direttamente al clock_nanosleep
19     goto wait_next_release;
20 }
```

LISTING 15.2: Logica di rilevamento deadline miss e skip nei moduli
F-16

Perché skippare il ciclo successivo. Quando un task supera il proprio periodo, al ciclo successivo il tempo di rilascio calcolato (`exec_next_release_time`) è già nel passato. Se il task eseguisse normalmente, uscirebbe immediatamente dal `clock_nanosleep` senza effettivamente dormire poiché l’istante target è già trascorso quindi di conseguenza inizierebbe subito un nuovo ciclo di esecuzione. Il risultato sarebbe che il task esegue **due cicli consecutivi senza pausa**, consumando quasi il doppio della propria quantità di CPU e causando in questo modo certamente una seconda deadline miss che va a degradare le capacità del sistema. Il flag `skip` interrompe questo effetto a catena: al ciclo immediatamente successivo alla miss, il task non esegue la propria funzione applicativa ma va direttamente al `clock_nanosleep`, lasciando il tempo al sistema di riallinearsi. Il costo di questo meccanismo è un ciclo “vuoto” in cui nessun dato viene elaborato, ma è il prezzo necessario per evitare un’instabilità a cascata.

Elenco Completo dei Moduli F-16

Ciascuno dei seguenti moduli basato sulla creazione di un **FCS** (flight control system), compila un archivio statico dedicato in `bin/lib/`. Il parametro `parameter` presente ogni modulo è stato creato per produrre un carico computazionale, che durante l’esecuzione della simulazione, che assicuri la fine del periodic task entro la deadline di riferimento:

- **engine_fadec** → `libf16_engine_fadec.a`: simula il controllo digitale del motore tramite calcoli del flusso di carburante ($\exp(\sin(i)) * \cos(i)$). Ha il meccanismo di skip attivo perché il controllo motore è safety-critical: una miss non può essere ignorata senza conseguenze.
- **fcs_long** → `libf16_fcs_long.a`: sistema di controllo longitudinale (beccheggio). Periodo nominale 10 ms.
- **fcs_lat** → `libf16_fcs_lat.a`: sistema di controllo laterale (rollio). Periodo nominale 10 ms.
- **fcs_alt** → `libf16_fcs_alt.a`: controllo automatico della quota. Periodo nominale 20 ms.
- **nav_gps** → `libf16_nav_gps.a`: simulazione ricezione e filtraggio segnali GPS. Periodo nominale 50 ms.
- **nav_ins** → `libf16_nav_ins.a`: navigazione inerziale con integrazione degli accelerometri. Periodo nominale 10 ms (stessa frequenza del FCS).
- **radar_air** → `libf16_radar_air.a`: elaborazione segnali radar aria-aria. Periodo nominale 50 ms, carico più elevato (`parameter = 10`).
- **radar_gnd** → `libf16_radar_gnd.a`: radar di mappatura terrestre. Periodo nominale 100 ms, carico massimo (`parameter = 15`).
- **weapons** → `libf16_weapons.a`: logica del sistema d'arma. Periodo nominale 100 ms.
- **displays** → `libf16_displays.a`: aggiornamento display avionici HUD e MFD. Periodo nominale 30 ms.

Ogni libreria espone la propria struttura di parametrizzazione (es. `t_f16_fcs_long_par`) e la propria funzione `PeriodicTask_<NOME>` che internamente utilizza le primitive della libreria `time` (Sezione 14.1.3) per il sincronismo assoluto e la libreria `trace` (Sezione 14.1.4) per la marcatura del kernel.

15.3 Il Sistema di Controllo del Volo (FCS)

Il Flight Control System [19], è il sottosistema più importante ed infatti è proprio questo sistema che garantisce la sicurezza durante il volo in condizione di volo anche da combattimento. I controlli più importanti sono i loop di controllo per beccheggio (*pitch*), rollio (*roll*) e quota (*altitude*) quest'ultimi devono rispettare deadline strettissime affinché il velivolo rimanga stabile.

15.3.1 Controllo del Beccheggio (PITCH_A, PITCH_B, PITCH_C)

I tre thread PITCH_A, PITCH_B e PITCH_C implementano un controllo longitudinale: operano in parallelo su tre core distinti (Core 0, 1 e 2) con lo stesso periodo di **10 ms**. La presenza di tre thread è una caratteristica di sicurezza.

```

1  // Affinita': Core 0 dedicato esclusivamente a PITCH_A
2  CPU_ZERO(&cpuset); CPU_SET(0, &cpuset);
3  pthread_attr_t attr_pitch_a; pthread_attr_t attr;
   pthread_attr_t attr; pthread_attr_t attr;
   pthread_attr_t attr; pthread_attr_t attr;
4
5  t_f16_fcs_long_par activity_pitch_a;
6  sprintf(activity_pitch_a.name, "PITCH_A");
7  activity_pitch_a.period = 10; // ms deadline critica
8  activity_pitch_a.parameter = 2; // carico computazionale
   ridotto
9  activity_pitch_a.deadline = 10; // ms coincidente con il
   periodo
10
11 // Priorita Rate Monotonic: 99 - (10/10) = 98 (altissima)
12 param_pitch_a.sched_priority = 99 - (activity_pitch_a.period /
   10);
13 pthread_attr_t attr; pthread_attr_t attr; pthread_attr_t attr;
   pthread_attr_t attr; pthread_attr_t attr; pthread_attr_t attr;
14
15 ret_err = pthread_create(&thread_pitch_a, &attr_pitch_a,
16 PeriodicTask_FCS_LONG, (void *)&activity_pitch_a);
17 pthread_setname_np(thread_pitch_a, "PITCH_A");

```

LISTING 15.3: Creazione del thread PITCH_A su Core 0 con periodo

10 ms

La scelta del parametro 2 (carico computazionale) è calibrata per garantire che l'esecuzione della funzione aviconica sia completata entro i 10 ms di deadline. Analogamente vengono creati PITCH_B (Core 1) e PITCH_C (Core 2).

15.3.2 Controllo del Rollio (ROLL_A, ROLL_B, ROLL_C)

Il controllo laterale segue la stessa logica. I tre thread ROLL_A, ROLL_B, ROLL_C vengono assegnati ai Core 3, 4 e 5 con periodo **10 ms**, garantendo che il controllo degli alettoni sia completamente isolato dal controllo del beccheggio a livello hardware.

15.3.3 Controllo della Quota (ALT_A, ALT_B)

I thread ALT_A e ALT_B gestiscono il mantenimento automatico della quota. Con periodo **20 ms** (il doppio dei loop di controllo primari), ottengono una priorità leggermente inferiore (97 vs 98), rispettando la gerarchia *Rate Monotonic*: il controllo degli assetti primari deve sempre precedere il controllo della quota in caso di contesa.

```
1   t_f16_fcs_alt_par activity_alt_a;  
2   sprintf(activity_alt_a.name, "ALT_A");  
3   activity_alt_a.period      = 20;  
4   activity_alt_a.parameter   = 3;  
5   activity_alt_a.deadline    = 20;  
6  
7   // Priorita RM: 99 - (20/10) = 97  
8   param_alt_a.sched_priority = 99 - (activity_alt_a.period / 10);
```

LISTING 15.4: Configurazione di ALT_A: periodo 20 ms, priorità 97

15.4 Il Sistema di Navigazione

Il sistema di navigazione implementato in real-time si basa sull'architettura descritta in [20]. In tale contesto viene simulata l'azione di due tecnologie complementari: la ricezione GPS per la posizione assoluta e la navigazione inerziale per la stima continua dello stato cinematico del velivolo.

15.4.1 Navigazione GPS (GPS_1, GPS_2)

I due thread GPS operano a **50 ms** (20 Hz), frequenza compatibile con la banda tipica dei ricevitori GPS ad alta precisione. Vengono assegnati ai Core 1 e 2.

15.4.2 Navigazione Inerziale (INS_1, INS_2, INS_3)

Il sistema inerziale è il più critico nell'architettura di navigazione: con periodo **10 ms** gestito su i tre (Core 3, 4 e 5), i tre thread INS ottengono priorità 98, identica ai loop FCS. La navigazione inerziale deve essere aggiornata alla stessa frequenza dei controlli di volo perché fornisce i dati di assetto in tempo reale utilizzati dal FCS:

```
1  // INS_1 su Core 3
2  t_f16_nav_ins_par activity_ins_1;
3  sprintf(activity_ins_1.name, "INS_1");
4  activity_ins_1.period    = 10;    // ms stessa frequenza del
   FCS
5  activity_ins_1.parameter = 4;    // carico maggiore del GPS (
   elaborazione IMU)
6  activity_ins_1.deadline  = 10;
7
8  // INS_2 su Core 4, INS_3 su Core 5 stessi parametri
```

LISTING 15.5: Configurazione del sistema INS

15.5 Il Sistema Radar

Il computer di bordo creato implementa due modalità radar con requisiti temporali differenziati, riflettendo le caratteristiche operative reali del radar AN/APG-68 [21], montato sull'F-16.

15.5.1 Radar Aria-Aria (RADAR_A)

Il thread `RADAR_A` opera con periodo **50 ms** sul Core 6. Con parametro 10, genera il carico computazionale più elevato dell'intera architettura, simulando l'elaborazione dei segnali radio-frequenza per il tracciamento di bersagli aerei.

15.5.2 Radar Aria-Terra (RADAR_G)

Il radar di mappatura terrestre `RADAR_G` opera con periodo **100 ms** sul Core 0, ottenendo la priorità più bassa tra i radar (89). L'aggiornamento meno frequente riflette la minore urgenza della mappatura del suolo rispetto al tracciamento di minacce aeree:

```
1  t_f16_radar_gnd_par activity_radar_g;  
2  sprintf(activity_radar_g.name, "RADAR_G");  
3  activity_radar_g.period    = 100;  // ms aggiornamento meno  
critico  
4  activity_radar_g.parameter = 15;    // massimo carico  
computazionale  
5  activity_radar_g.deadline  = 100;  
6  
7  // Priorita RM: 99 - (100/10) = 89  
8  param_radar_g.sched_priority = 99 - (activity_radar_g.period /  
10);
```

LISTING 15.6: Configurazione di `RADAR_G`: periodo 100 ms, massimo carico

15.6 Il Sistema FADEC

Il FADEC (*Full Authority Digital Engine Control*)[22], è il sistema che gestisce in modo digitale il controllo completo del motore turbofan General Electric F110 dell'F-16. I due thread FADEC_A e FADEC_B operano con periodo **10 ms** sui Core 1 e 2, con priorità 98, affiancando i loop di controllo del volo nella fascia temporale più critica.

Il parametro computazionale 3 genera all'interno della libreria `f16_engine_fadec` (Sezione 15.2.1) una simulazione del calcolo del flusso di carburante tramite la funzione `exp(sin(i)) * cos(i)`, ed è dotato del meccanismo di rilevamento delle deadline miss con flag `skip` descritto nel capitolo delle librerie:

```
1   t_f16_engine_fadec_par activity_fadec_a;
2   sprintf(activity_fadec_a.name, "FADEC_A");
3   activity_fadec_a.period    = 10;    // ms controllo motore
   critico
4   activity_fadec_a.parameter = 3;
5   activity_fadec_a.deadline  = 10;
6
7   // Priorita 98: identica ai loop FCS e INS
8   param_fadec_a.sched_priority = 99 - (activity_fadec_a.period /
   10);
9   ret_err = pthread_create(&thread_fadec_a, &attr_fadec_a,
10  PeriodicTask_ENGINE_FADEC, (void *)&activity_fadec_a);
11  pthread_setname_np(thread_fadec_a, "FADEC_A");
```

LISTING 15.7: Configurazione del thread FADEC_A

15.7 Il Sistema d'Arma e i Display Avionici

15.7.1 Sistema d'ingaggio

Il thread `WEAPONS` gestisce la logica del sistema missilistico e del cannone M61A1. Con periodo **100 ms** sul Core 3, ottiene la priorità più bassa (89), riflettendo la natura non continua delle operazioni d'arma rispetto ai loop di controllo del volo

15.7.2 Display HUD e MFD

I due thread di visualizzazione (HUD e MFD) operano a **30 ms** (circa 33 Hz) rispettivamente su Core 4 e Core 5. Il periodo di 30 ms garantisce una frequenza di aggiornamento fluida per il pilota mantenendo un margine adeguato rispetto ai loop critici da 10 ms.

15.8 Distribuzione dei Core e Analisi dell'Architettura

La distribuzione dei 20 thread sui 7 core fisici disponibili è il risultato di un bilanciamento architetturale che tiene conto sia della criticità temporale che del carico computazionale di ogni sottosistema. La Figura ?? illustra la mappa di assegnazione.

Va notato che alcuni core ospitano thread di diversi sottosistemi con periodi differenti:

- **Core 0:** ospita PITCH_A (10 ms, priorità 98), ALT_B (20 ms, priorità 97) e RADAR_G (100 ms, priorità 89). Lo scheduler SCHED_FIFO garantisce che PITCH_A possa sempre prelazionare i task a priorità inferiore.
- **Core 1:** ospita PITCH_B (10 ms), GPS_1 (50 ms) e FADEC_A (10 ms). La coesistenza di due task con periodo 10 ms sullo stesso core è possibile solo se la loro esecuzione combinata non supera i 10 ms disponibili nel periodo.

15.9 Sincronizzazione Finale e Terminazione

Al termine della sezione di creazione dei thread, il main esegue una `pthread_join` per ognuno dei 20 thread prima di chiudere il file descriptor dei marker:

```

1  pthread_join(thread_pitch_a, NULL);
2  pthread_join(thread_pitch_b, NULL);
3  pthread_join(thread_pitch_c, NULL);
4  pthread_join(thread_roll_a, NULL);
5  pthread_join(thread_roll_b, NULL);
6  pthread_join(thread_roll_c, NULL);
7  pthread_join(thread_alt_a, NULL);

```

```
8   pthread_join(thread_alt_b, NULL);
9   pthread_join(thread_gps_1, NULL);
10  pthread_join(thread_gps_2, NULL);
11  pthread_join(thread_ins_1, NULL);
12  pthread_join(thread_ins_2, NULL);
13  pthread_join(thread_ins_3, NULL);
14  pthread_join(thread_radar_a, NULL);
15  pthread_join(thread_radar_g, NULL);
16  pthread_join(thread_fadec_a, NULL);
17  pthread_join(thread_fadec_b, NULL);
18  pthread_join(thread_weapons, NULL);
19  pthread_join(thread_hud, NULL);
20  pthread_join(thread_mfd, NULL);
21
22  close_tracing();
23  exit(0);
```

LISTING 15.8: Attesa di terminazione di tutti i 20 thread

Poiché ogni thread esegue un loop infinito (`while(1)`), nella pratica il programma non termina spontaneamente: il `main` rimane bloccato sulle `pthread_join` fino all'interruzione esterna da parte del sistema operativo (tipicamente tramite `Ctrl+C` o il termine dello script di tracciamento).

15.10 Risultati dell'Analisi di Tracciamento

Il tracciamento di **F16_Computer** viene avviato tramite lo script `run_trace.sh`:

```
1 cd bin/tools/  
2 ./run_trace.sh F16_Computer
```

Al termine dell'acquisizione, il parser C++ `thread_analysis` analizza il file `trace_output.txt` riconoscendo tutti i 20 thread tramite i loro marker `PERIOD_START_<NOME>` e producendo il file `timeline.csv` con le statistiche complete.

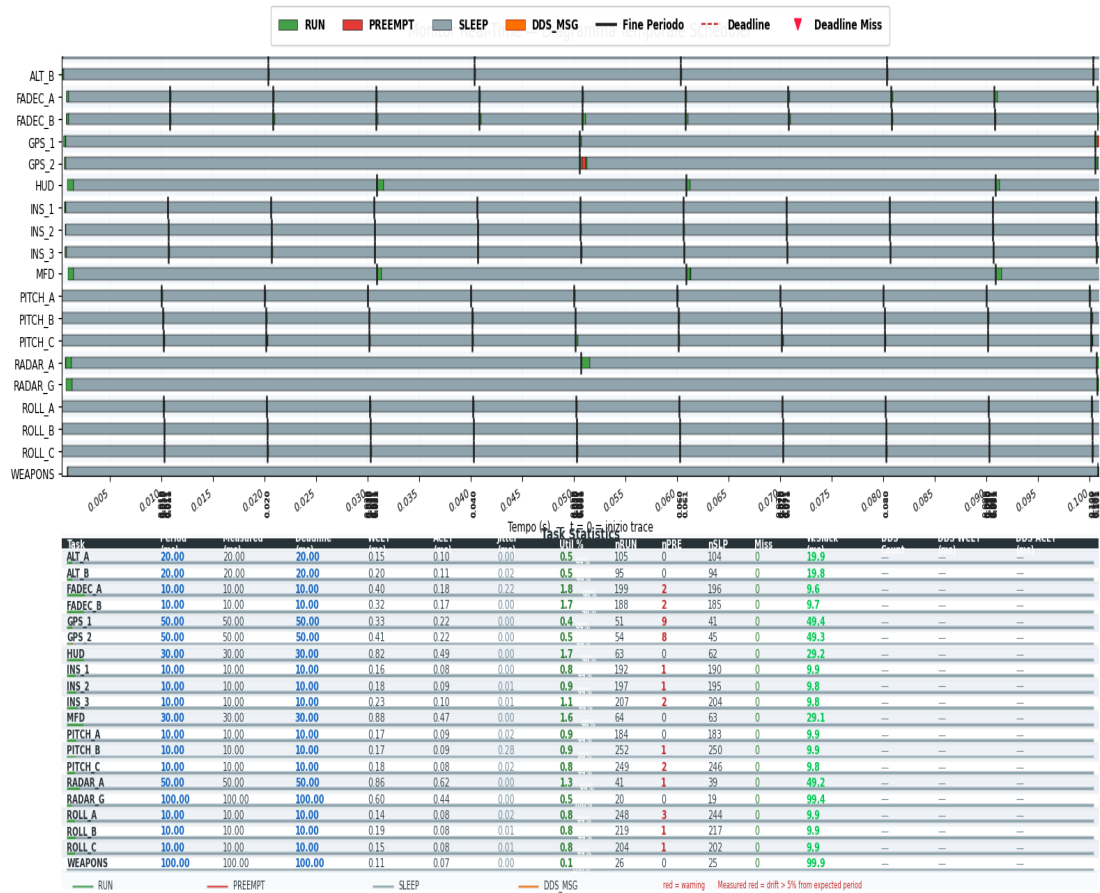


FIGURA 15.1: Diagramma di Gantt del simulatore F-16 prodotto da `monitor.py`: le 20 corsie temporali mostrano l'alternanza di stati RUN (verde), SLEEP (grigio) e PREEMPT (rosso) per ogni sottosistema avionico.

```

1  -----
2  TASK: PITCH_A (PID: 22440)
3  -----
4  [Esecuzione] WCET :      0.175 ms | ACET:      0.089 ms
5  [T.Risposta] WCRT :      0.130 ms
6  [Periodo]    Mis. :      9.999 ms | Jitter:      0.021 ms
7  [Parametri]  Periodo:    10.000 ms | Deadline:    10.000 ms
8  [Scadenze]   Miss   :           0 | Slack Peggior:  9.870
   ms
9  [Stati]      Sleep   :      183   | Preempt:      0   |
   Run n.: 184
10 [Globale]    Util%   :      0.909 % | Gaps    :      183
   (10.008 ms max)
11
12 -----
13 TASK: PITCH_B (PID: 22441)
14 -----
15 [Esecuzione] WCET :      0.170 ms | ACET:      0.088 ms
16 [T.Risposta] WCRT :      0.126 ms
17 [Periodo]    Mis. :     10.001 ms | Jitter:      0.278 ms
18 [Parametri]  Periodo:    10.000 ms | Deadline:    10.000 ms
19 [Scadenze]   Miss   :           0 | Slack Peggior:  9.874
   ms
20 [Stati]      Sleep   :      250   | Preempt:      1   |
   Run n.: 252
21 [Globale]    Util%   :      0.891 % | Gaps    :      243
   (12.530 ms max)
22
23 -----
24 TASK: PITCH_C (PID: 22442)
25 -----
26 [Esecuzione] WCET :      0.181 ms | ACET:      0.078 ms
27 [T.Risposta] WCRT :      0.154 ms
28 [Periodo]    Mis. :     10.000 ms | Jitter:      0.023 ms
29 [Parametri]  Periodo:    10.000 ms | Deadline:    10.000 ms
30 [Scadenze]   Miss   :           0 | Slack Peggior:  9.846
   ms
31 [Stati]      Sleep   :      246   | Preempt:      2   |
   Run n.: 249

```

```

32  [Globale]    Util%   :    0.799 %    |   Gaps    :    236 (9.991
    ms max)
33
34  -----
35  TASK: ROLL_A (PID: 22443)
36  -----
37  [Esecuzione] WCET :    0.139 ms   |   ACET:    0.076 ms
38  [T.Risposta] WCRT :    0.120 ms
39  [Periodo]    Mis.  :   10.001 ms   |   Jitter:    0.022 ms
40  [Parametri]  Periodo: 10.000 ms   |   Deadline: 10.000 ms
41  [Scadenze]   Miss   :         0     |   Slack Peggior: 9.880
    ms
42  [Stati]      Sleep  :    244       |   Preempt:    3       |
    Run n.: 248
43  [Globale]    Util%   :    0.778 %    |   Gaps    :    238
    (10.028 ms max)
44
45  -----
46  TASK: ROLL_B (PID: 22444)
47  -----
48  [Esecuzione] WCET :    0.188 ms   |   ACET:    0.081 ms
49  [T.Risposta] WCRT :    0.148 ms
50  [Periodo]    Mis.  :   10.000 ms   |   Jitter:    0.005 ms
51  [Parametri]  Periodo: 10.000 ms   |   Deadline: 10.000 ms
52  [Scadenze]   Miss   :         0     |   Slack Peggior: 9.852
    ms
53  [Stati]      Sleep  :    217       |   Preempt:    1       |
    Run n.: 219
54  [Globale]    Util%   :    0.827 %    |   Gaps    :    214 (9.972
    ms max)
55
56  -----
57  TASK: ROLL_C (PID: 22445)
58  -----
59  [Esecuzione] WCET :    0.149 ms   |   ACET:    0.078 ms
60  [T.Risposta] WCRT :    0.126 ms
61  [Periodo]    Mis.  :   10.000 ms   |   Jitter:    0.006 ms
62  [Parametri]  Periodo: 10.000 ms   |   Deadline: 10.000 ms
63  [Scadenze]   Miss   :         0     |   Slack Peggior: 9.874
    ms

```

```

64  [Stati]      Sleep :      202      | Preempt:      1      |
    Run n.: 204
65  [Globale]   Util% :      0.790 %   | Gaps :      202 (9.975
    ms max)
66
67  -----
68  TASK: ALT_A (PID: 22446)
69  -----
70  [Esecuzione] WCET :      0.149 ms   | ACET:      0.100 ms
71  [T.Risposta] WCRT :      0.132 ms
72  [Periodo]   Mis. :      20.001 ms   | Jitter:      0.002 ms
73  [Parametri] Periodo: 20.000 ms   | Deadline: 20.000 ms
74  [Scadenze]  Miss :      0          | Slack Peggior: 19.868
    ms
75  [Stati]      Sleep :      104      | Preempt:      0      |
    Run n.: 105
76  [Globale]   Util% :      0.509 %   | Gaps :      104
    (19.921 ms max)
77
78  -----
79  TASK: ALT_B (PID: 22447)
80  -----
81  [Esecuzione] WCET :      0.201 ms   | ACET:      0.105 ms
82  [T.Risposta] WCRT :      0.182 ms
83  [Periodo]   Mis. :      20.001 ms   | Jitter:      0.021 ms
84  [Parametri] Periodo: 20.000 ms   | Deadline: 20.000 ms
85  [Scadenze]  Miss :      0          | Slack Peggior: 19.818
    ms
86  [Stati]      Sleep :      94       | Preempt:      0      |
    Run n.: 95
87  [Globale]   Util% :      0.537 %   | Gaps :      94
    (20.034 ms max)
88
89  -----
90  TASK: GPS_1 (PID: 22448)
91  -----
92  [Esecuzione] WCET :      0.326 ms   | ACET:      0.218 ms
93  [T.Risposta] WCRT :      0.575 ms
94  [Periodo]   Mis. :      50.002 ms   | Jitter:      0.001 ms
95  [Parametri] Periodo: 50.000 ms   | Deadline: 50.000 ms

```

```

96  [Scadenze]  Miss   :      0      |  Slack Peggior:  49.425
    ms
97  [Stati]    Sleep  :      41      |  Preempt:      9      |
    Run n.: 51
98  [Globale]  Util%  :    0.449 %   |  Gaps   :      49
    (49.812 ms max)
99
100 -----
101 TASK: GPS_2 (PID: 22449)
102 -----
103 [Esecuzione] WCET :    0.407 ms   |  ACET:    0.224 ms
104 [T.Risposta] WCRT :    0.696 ms
105 [Periodo]   Mis.  :    50.002 ms   |  Jitter:    0.002 ms
106 [Parametri] Periodo: 50.000 ms   |  Deadline:  50.000 ms
107 [Scadenze]  Miss   :      0      |  Slack Peggior:  49.304
    ms
108 [Stati]    Sleep  :      45      |  Preempt:      8      |
    Run n.: 54
109 [Globale]  Util%  :    0.462 %   |  Gaps   :      50
    (49.824 ms max)
110
111 -----
112 TASK: INS_1 (PID: 22450)
113 -----
114 [Esecuzione] WCET :    0.156 ms   |  ACET:    0.082 ms
115 [T.Risposta] WCRT :    0.141 ms
116 [Periodo]   Mis.  :    10.000 ms   |  Jitter:    0.004 ms
117 [Parametri] Periodo: 10.000 ms   |  Deadline:  10.000 ms
118 [Scadenze]  Miss   :      0      |  Slack Peggior:   9.859
    ms
119 [Stati]    Sleep  :     190      |  Preempt:      1      |
    Run n.: 192
120 [Globale]  Util%  :    0.830 %   |  Gaps   :     188 (9.939
    ms max)
121
122 -----
123 TASK: INS_2 (PID: 22451)
124 -----
125 [Esecuzione] WCET :    0.182 ms   |  ACET:    0.090 ms
126 [T.Risposta] WCRT :    0.164 ms

```



```

127 [Periodo] Mis. : 10.000 ms | Jitter: 0.008 ms
128 [Parametri] Periodo: 10.000 ms | Deadline: 10.000 ms
129 [Scadenze] Miss : 0 | Slack Peggior: 9.836
ms
130 [Stati] Sleep : 195 | Preempt: 1 |
Run n.: 197
131 [Globale] Util% : 0.913 % | Gaps : 195 (9.963
ms max)
132
133 -----
134 TASK: INS_3 (PID: 22452)
135 -----
136 [Esecuzione] WCET : 0.231 ms | ACET: 0.105 ms
137 [T.Risposta] WCRT : 0.198 ms
138 [Periodo] Mis. : 10.000 ms | Jitter: 0.010 ms
139 [Parametri] Periodo: 10.000 ms | Deadline: 10.000 ms
140 [Scadenze] Miss : 0 | Slack Peggior: 9.802
ms
141 [Stati] Sleep : 204 | Preempt: 2 |
Run n.: 207
142 [Globale] Util% : 1.060 % | Gaps : 204 (9.950
ms max)
143
144 -----
145 TASK: RADAR_A (PID: 22453)
146 -----
147 [Esecuzione] WCET : 0.859 ms | ACET: 0.621 ms
148 [T.Risposta] WCRT : 0.849 ms
149 [Periodo] Mis. : 50.002 ms | Jitter: 0.003 ms
150 [Parametri] Periodo: 50.000 ms | Deadline: 50.000 ms
151 [Scadenze] Miss : 0 | Slack Peggior: 49.151
ms
152 [Stati] Sleep : 39 | Preempt: 1 |
Run n.: 41
153 [Globale] Util% : 1.278 % | Gaps : 39
(49.448 ms max)
154
155 -----
156 TASK: RADAR_G (PID: 22454)
157 -----

```

```

158 [Esecuzione] WCET :      0.599 ms | ACET:      0.436 ms
159 [T.Risposta] WCRT :      0.608 ms
160 [Periodo]    Mis. :      100.004 ms | Jitter:      0.001 ms
161 [Parametri]  Periodo: 100.000 ms | Deadline: 100.000 ms
162 [Scadenze]   Miss :          0 | Slack Peggiore: 99.392
ms
163 [Stati]      Sleep :          19 | Preempt:          0 |
Run n.: 20
164 [Globale]    Util% :      0.472 % | Gaps :          19
(99.667 ms max)
165
166 -----
167 TASK: FADEC_A (PID: 22455)
168 -----
169 [Esecuzione] WCET :      0.404 ms | ACET:      0.175 ms
170 [T.Risposta] WCRT :      0.383 ms
171 [Periodo]    Mis. :      10.000 ms | Jitter:      0.218 ms
172 [Parametri]  Periodo: 10.000 ms | Deadline: 10.000 ms
173 [Scadenze]   Miss :          0 | Slack Peggiore: 9.617
ms
174 [Stati]      Sleep :          196 | Preempt:          2 |
Run n.: 199
175 [Globale]    Util% :      1.765 % | Gaps :          198
(11.945 ms max)
176
177 -----
178 TASK: FADEC_B (PID: 22456)
179 -----
180 [Esecuzione] WCET :      0.319 ms | ACET:      0.167 ms
181 [T.Risposta] WCRT :      0.300 ms
182 [Periodo]    Mis. :      10.000 ms | Jitter:      0.005 ms
183 [Parametri]  Periodo: 10.000 ms | Deadline: 10.000 ms
184 [Scadenze]   Miss :          0 | Slack Peggiore: 9.700
ms
185 [Stati]      Sleep :          185 | Preempt:          2 |
Run n.: 188
186 [Globale]    Util% :      1.680 % | Gaps :          187 (9.884
ms max)
187
188 -----

```

```

189 TASK: WEAPONS (PID: 22457)
190 -----
191 [Esecuzione] WCET :      0.114 ms | ACET:      0.074 ms
192 [T.Risposta] WCRT :      0.092 ms
193 [Periodo]   Mis. :    100.004 ms | Jitter:     0.002 ms
194 [Parametri] Periodo: 100.000 ms | Deadline: 100.000 ms
195 [Scadenze]  Miss  :          0 | Slack Peggior: 99.908
ms
196 [Stati]     Sleep :        25 | Preempt:      0 |
Run n.: 26
197 [Globale]   Util% :    0.079 % | Gaps :        24
(99.948 ms max)
198
199 -----
200 TASK: HUD (PID: 22458)
201 -----
202 [Esecuzione] WCET :      0.824 ms | ACET:      0.494 ms
203 [T.Risposta] WCRT :      0.806 ms
204 [Periodo]   Mis. :    30.001 ms | Jitter:     0.002 ms
205 [Parametri] Periodo: 30.000 ms | Deadline: 30.000 ms
206 [Scadenze]  Miss  :          0 | Slack Peggior: 29.194
ms
207 [Stati]     Sleep :        62 | Preempt:      0 |
Run n.: 63
208 [Globale]   Util% :    1.685 % | Gaps :        62
(29.642 ms max)
209
210 -----
211 TASK: MFD (PID: 22459)
212 -----
213 [Esecuzione] WCET :      0.880 ms | ACET:      0.469 ms
214 [T.Risposta] WCRT :      0.865 ms
215 [Periodo]   Mis. :    30.001 ms | Jitter:     0.002 ms
216 [Parametri] Periodo: 30.000 ms | Deadline: 30.000 ms
217 [Scadenze]  Miss  :          0 | Slack Peggior: 29.135
ms
218 [Stati]     Sleep :        63 | Preempt:      0 |
Run n.: 64
219 [Globale]   Util% :    1.590 % | Gaps :        62
(29.630 ms max)

```

LISTING 15.9: Attesa di terminazione di tutti i 20 thread

Capitolo 16

Conclusioni

16.1 Risultati Ottenuti

Questo lavoro ha dimostrato i risultati ottenuti attraverso l'automazione del controllo longitudinale attraverso script matlba di analisi e l'utilizzo di un simulatore per validare il lavoro svolto con:

- **Script MATLAB:** script per l'analisi MIMO e di creazione di modelli di controllo LQR e MPC, che si trovano all'interno della repository GitHub dedicata. Disponibile al seguente indirizzo: <https://github.com/leoleg2004/AutomationF16>
- **Simulatore Flight Gear:** simulatore open-source usato per validare le strategie ocntrollo ottenuta con irsultato visivo che si poggia sul modello simulink di un velivolo F-16 non lineare del prof.Bhattachrya[1].

inoltre si è sviluppato un sistema integrato basato un computer di bordo di un F-16 attraverso l'analisi di parametri temporali di applicazioni Real-Time su architettura multi-core Linux. Il sistema sviluppato composto dalla libreria di istruzione C, dagli script bash, dal parser C++ e dal monitor Python è in grado di:

- **Repository dedicata:** il repository con tutti gli elementi presenti nella tesi sono disponibili al seguente indirizzo: <https://github.com/leoleg2004/Fly-By-Wire-project>
- **Acquisire automaticamente** il log degli eventi di scheduling tramite `trace-cmd` con overhead minimo sui task in esecuzione.
- **Estrarre le metriche fondamentali** (WCET, ACET, Jitter, Response Time, Deadline Miss, Utilizzazione CPU) per ogni task periodico, senza modificare il codice applicativo al di là dell'inclusione della libreria `trace_marker`.

- **Tracciare scenari multi-processo** tipici della comunicazione DDS, grazie agli script `marker.sh` per l'aggancio passivo a processi già in esecuzione tramite `/proc`.
- **Visualizzare i risultati** in diagrammi di Gantt e tabelle statistiche con evidenziazione condizionale delle anomalie.

Il caso di studio del computer di bordo dell'F-16 ha validato il sistema in condizioni di carico elevato (20 thread POSIX su 7 core fisici), confermando che la schedulazione *Rate Monotonic* implementata tramite `SCHED_FIFO` garantisce il rispetto delle deadline per tutti i sottosistemi critici (FCS, INS, FADEC) con periodo di 10 ms.

16.2 Limiti del Sistema

- **Overhead dei marker:** La scrittura nel buffer `ftrace` introduce un overhead per ogni chiamata. Nei task con WCET dell'ordine dei microsecondi questo overhead può alterare le misure.
- **Buffer circolare:** In sessioni di tracciamento molto lunghe o ad alta frequenza di eventi, il buffer di `ftrace` può andare in overflow, perdendo eventi e rendendo le misure incomplete.
- **Parsing statico limitato:** Il parsing regex del sorgente funziona solo per i pattern di dichiarazione riconoscibili (`.period = ...`, `.deadline = ...`). Parametri calcolati a runtime non vengono estratti.
- **Risoluzione dei nomi:** La discovery del parser si basa sul matching tra nomi dei thread nel log e marker testuali. Thread che non iniettano marker non vengono riconosciuti dall'analizzatore.

16.3 Sviluppi Futuri

- **Analisi a runtime:** Sostituire l'analisi *post-mortem* del log con un agente in-process che calcoli le metriche in tempo reale e notifichi immediatamente eventuali violazioni di deadline durante l'esecuzione.

- **Supporto a task aperiodici e sporadici:** Estendere il parser per riconoscere task con periodo variabile o attivazione basata su eventi, tipici dei sistemi a interrupt.
- **Analisi multi-nodo DDS:** Estendere il monitor Python per correlare temporalmente i marker di publisher e subscriber su macchine fisiche diverse, calcolando la latenza end-to-end della comunicazione DDS su rete reale.

Ringraziamenti

A conclusione di questo lavoro di tesi e di questo percorso accademico, desidero ringraziare il Prof. Antonio Russo e il Prof. Davide Brugali, che sono stati un costante punto di riferimento e di crescita personale. Ringrazio il Prof. Russo per i continui confronti su ogni mio dubbio e per i preziosi consigli che mi hanno guidato verso le scelte per il mio futuro. Esprimo la mia gratitudine anche al Prof. Brugali per avermi dato l'opportunità di misurarmi con nuove sfide che hanno arricchito enormemente il mio bagaglio di conoscenze. Considererò sempre entrambi come due mentori che hanno saputo mettermi nelle condizioni di migliorare continuamente. Un ringraziamento speciale va alla mia famiglia. A mia mamma, Federica Galli, per essere sempre stata in prima linea a spingermi verso i miei obiettivi. A mio padre, Emanuele Candido Leggeri, per gli sforzi non passati inosservati: senza di essi non avrei potuto concludere questo percorso e aprirne un altro, altrettanto complesso e sfidante. Infine, a mia sorella Angelica, che non ha mai smesso di farmi sentire il supporto necessario per superare ogni problema. Vi ringrazio per essere stati una famiglia così presente e per avermi permesso di completare questi studi con la massima serenità. Un grazie anche ai miei compagni di università: Gabriele Bregolin, Samuele Vecchi, Raul Beltramelli, Riccardo Motta e Davide Scorsetti. Siete stati tutti fondamentali per superare insieme gli ostacoli e i momenti più complessi di questi anni impegnativi; a tutti voi auguro la migliore fortuna nei rispettivi percorsi personali. Ringrazio anche i miei amici d'infanzia: Nicholas Cresci, Stefan Ifteme, Matteo Lupi, Jhair Alejandro Guerra, Giorgia Milanese e Gaia Capelli, che negli anni sono stati continua fonte di ispirazione e di confronto. A loro riconosco un aiuto fondamentale durante questi anni di studio e auguro il meglio per i rispettivi progetti di vita. Infine, grazie alla mia ragazza, Michela. Sei stata una fonte inesauribile di sostegno, sempre pronta ad ascoltare ogni mio problema o pensiero. Anche nei momenti in cui provavo ad abbassare la testa, la tua voglia di vedermi trionfare è bastata per entrambi; devo dunque anche a te il raggiungimento di questo traguardo così importante per la mia vita.

*«Gli scienziati studiano il mondo così com'è;
gli ingegneri creano il mondo che non è mai stato.»*

– Theodore von Kármán

Appendice A

Codici MATLAB: Controllo Predittivo MPC

In questa appendice vengono riportati i codici MATLAB utilizzati. Il documento è strutturato per essere facilmente navigabile: partendo dallo script principale, è possibile esplorare tutte le funzioni richiamate.

Script Principale MPC

File: MPC.m

Script MATLAB principale per la simulazione MPC dell'F-16 (Configurazione, Inizializzazione, Ciclo Closed-Loop). In questo script vengono richiamate le seguenti funzioni, definite nelle sezioni successive:

- `startup.m`
- `funzioni_mpc/discretizza_modello.m`
- `funzioni_mpc/imposta_vincoli.m`
- `funzioni_mpc/setup_mpc.m`
- `funzioni_mpc/simula_mpc.m`
- `funzioni_mpc/cis.m`
- `funzioni_mpc/plot_cis.m`
- `funzioni_mpc/plot_mpc_cost_3d.m`
- `funzioni_mpc/plot_risultati.m`

```

1 %
  =====

2 % Tesi Triennale - MPC Longitudinale F-16
3 % Simulazione con Conversione Continuo-Discreto (c2d) e Radianti
4 % Ing. Leggeri Leonardo
5 %
  =====

6
7 % Inizializzazione Path
8 % Aggiungiamo prima la cartella locale Startup in cima al path per
  evitare
9 % conflitti con startup_project di altri progetti (es. Tre Camere)
10 addpath(fullfile(pwd, 'Startup'), '-begin');
11 startup_project();
12
13 %
  =====

14 % TABS AUTOMATICI: Inserisce tutti i grafici in un unico pannello a
  schede
15 %
  =====

16 set(0, 'DefaultFigureWindowStyle', 'docked');
17
18 %% 1. Definizione del Sistema (TEMPO CONTINUO)
19 nx = 4; % Stati: [theta,q,U,W]'
20 nu = 3; % Ingressi: [T, dele, dlef]'
21 Ts = 0.1; % Tempo di campionamento in secondi
22
23 disp('Conversione del modello da Continuo a Discreto...');
24 [A_long_ds, B_ctrl_ds] = discretizza_modello(A_long, B_ctrl, nx, nu
  , Ts);
25
26 %% 2. Progetto LQR tramite funzione dedicata
27 %
  =====

```

```

28 % NOTA SUI PESI Q e R:
29 % Se vuoi modificare l'aggressività del controllore (pesi Q e R),
30 % apri il file "funzioni_lqr/progetta_LQR_discreto.m" e modificali
    lì dentro
31 %
    =====

32 disp('Progetto LQR discreto tramite funzione dedicata...');
33 [K, P, Q, R, A_cl] = progetta_LQR_discreto(A_long_ds, B_ctrl_ds);
34
35 % I vincoli FISICI assoluti dell'F-16:
36 % Thrust: [1000, 19000] lbf -> Newton
37 % Elevator: [-25, 25] gradi
38 % Flaps (LEF): [0, 25] gradi
39 lbf2N = 4.44822;
40 ft2m = 0.3048;
41
42 U_min_abs = [1000 * lbf2N; -25 * pi/180; 0];
43 U_max_abs = [19000 * lbf2N; 25 * pi/180; 25 * pi/180];
44
45 % Poiché l'MPC ragiona in "perturbazioni" (u_err), i vincoli su
    u_err
46 % devono essere sottratti del punto di trim (best_u)
47 U_min = U_min_abs - [best_u(1); best_u(2); best_u(3)];
48 U_max = U_max_abs - [best_u(1); best_u(2); best_u(3)];
49
50 Rate_max = [10000 * lbf2N; 60 * pi/180; 25 * pi/180];
51
52 % Stati: theta(rad), q(rad/s), u(m/s), w(m/s)
53 X_max = [ 45 * pi/180; 60 * pi/180; 100 * ft2m; 85 * ft2m];
54 X_min = [-45 * pi/180; -60 * pi/180; -100 * ft2m; -85 * ft2m];
55
56 [Fx, fx, Fu, fu, dU_min, dU_max, Gx, gx] = imposta_vincoli(X_min,
    X_max, U_min, U_max, Rate_max, Ts);
57
58 % Riferimento di Default (Origine del sistema linearizzato)
59 x_ref = zeros(nx, 1);
60 u_ref = zeros(nu, 1);
61

```

```

62 %
    =====

63 % ESEMPIO con target non nullo
64 % Affinché l'MPC funzioni correttamente sul modello linearizzato,
    il nuovo
65 % target deve essere un punto di equilibrio valido:  $x_{ref} = A_{ds} * x_{ref} + B_{ds} * u_{ref}$ 
66 %
67 %  $u_{ref} = [100; 2 * \pi/180; 0];$  % Es: +1000 lbs di spinta e 2 gradi
    di equilibratore
68 %  $x_{ref} = (eye(nx) - A_{long\_ds}) \setminus (B_{ctrl\_ds} * u_{ref});$  % Stato di
    equilibrio esatto associato
69 %
    =====

70
71 %% 4. Calcolo Control Invariant Set e Plot
72 disp('--- CALCOLO del Control Invariant Set ---');
73 [G_inf, g_inf] = cis(A_long_ds, B_ctrl_ds, x_ref, u_ref, Fx, fx, Fu
    , fu, Q, R);
74
75 %% 5. Setup Problema MPC
76 N = 40; % Orizzonte predittivo
77 mpc_prob = setup_mpc(N, nx, nu, A_long_ds, B_ctrl_ds, Q, P, R,
    U_min, U_max, Gx, gx, G_inf, g_inf, x_ref, u_ref);
78
79 %% 4b. Plot 3D dei Set Invarianti e Feasible Set N-Step
80 % Inizializza Dashboard
81 tabs = crea_dashboard_mpc();
82 plot_cis(G_inf, g_inf, x_ref, mpc_prob, A_long_ds, dU_max, tabs.
    cis_3d);
83
84 %% 6. Simulazione MPC Completa
85 disp('--- Avvio Ottimizzazione e Simulazione MPC ---');
86 % Ordine: [theta; q; u; w]
87 x_iniziale = [25 * (pi/180); % theta: +25 gradi (cabrata)
88              3 * (pi/180); % q: +20 gradi/s (velocità di
    beccheggio)

```

```

89         2;                % u: +20 ft/s (deviazione velocità
        forward)
90         5];                % w: +20 ft/s (deviazione velocità
        verticale)
91 t_sim = 100; % tempo di simulazione
92
93 [storia_x, storia_u, storia_costo] = simula_mpc(mpc_prob,
        x_iniziale, t_sim, A_long_ds, B_ctrl_ds, dU_max);
94 disp('Ottimizzazione Riuscita. Il modello è matematicamente solido.
        ');
95
96 disp('-----');
97 disp('VERIFICA RAGGIUNGIMENTO TARGET (Ultimo Step)');
98 disp('Stato Finale Raggiunto (storia_x(:,end))');
99 disp(storia_x(:,end));
100 disp('Target Desiderato (x_ref)');
101 disp(x_ref);
102 disp('Errore Assoluto [theta; q; U; W]');
103 disp(abs(storia_x(:,end) - x_ref));
104 disp('-----');
105
106 %% 7. Grafici
107 plot_risultati(t_sim, storia_x, storia_u, U_min, U_max, x_ref,
        u_ref, tabs.stati, tabs.attuatori);
108
109 %% 8. Plot Funzionale di Costo MPC 3D
110 plot_mpc_cost_3d(mpc_prob, A_long_ds, dU_max, storia_x,
        storia_costo, Ts, tabs.cost_3d);

```

LISTING A.1: Script Principale MPC

Inizializzazione Workspace

File: startup.m

Script di inizializzazione dell'ambiente di lavoro e configurazione automatica dei percorsi di progetto.


```

1 %
  =====

2 % Copyright (c) 2026 Ing. Leggeri Leonardo
3 % Tutti i diritti riservati.
4 %
5 % ATTENZIONE: Questo software e il relativo codice sorgente sono di
  proprietà
6 % esclusiva dell'Ing. Leggeri Leonardo. É severamente vietata la
  copia,
7 % la distribuzione, la modifica o la vendita a terzi senza l'
  esplicito
8 % consenso scritto dell'autore. La distribuzione o la vendita non
  autorizzata
9 % costituisce reato ed è perseguibile penalmente secondo le leggi
  vigenti.
10 %
  =====

11
12 % STARTUP.m
13 % Questo script viene eseguito automaticamente da MATLAB all'avvio
14 % (se la cartella corrente è impostata su questa directory)
15 % oppure all'apertura del Progetto MATLAB.
16
17 disp('--- Esecuzione Automatica di Startup ---');
18 addpath(fullfile(pwd, 'Startup'));
19 startup_project();

```

LISTING A.2: Inizializzazione Workspace

Discretizzazione del Modello

File: funzioni_mpc/discretizza_modello.m

Funzione per la discretizzazione del modello continuo dello spazio di stato (c2d).

```

1 %
  =====

```

```

2 % Copyright (c) 2026 Ing. Leggeri Leonardo
3 % Tutti i diritti riservati.
4 %
5 % ATTENZIONE: Questo software e il relativo codice sorgente sono di
    proprietà
6 % esclusiva dell'Ing. Leggeri Leonardo. É severamente vietata la
    copia,
7 % la distribuzione, la modifica o la vendita a terzi senza l'
    esplicito
8 % consenso scritto dell'autore. La distribuzione o la vendita non
    autorizzata
9 % costituisce reato ed è perseguibile penalmente secondo le leggi
    vigenti.
10 %
    =====
11
12 function [A_long_ds, B_ctrl_ds] = discretizza_modello(A_long,
    B_ctrl, nx, nu, Ts)
13     % Converte il sistema da tempo continuo a discreto
14     sys_c = ss(A_long, B_ctrl, eye(nx), zeros(nx, nu));
15     sys_d = c2d(sys_c, Ts, 'zoh'); % Zero-Order Hold
16     A_long_ds = sys_d.A;
17     B_ctrl_ds = sys_d.B;
18 end

```

LISTING A.3: Discretizzazione del Modello

Definizione dei Vincoli (Constraints)

File: funzioni_mpc/imposta_vincoli.m

Funzione per la definizione rigorosa dei vincoli fisici e operativi (su stati e ingressi).

```

1 %
    =====
2 % Copyright (c) 2026 Ing. Leggeri Leonardo
3 % Tutti i diritti riservati.
4 %

```

```

5 % ATTENZIONE: Questo software e il relativo codice sorgente sono di
  proprietà
6 % esclusiva dell'Ing. Leggeri Leonardo. É severamente vietata la
  copia,
7 % la distribuzione, la modifica o la vendita a terzi senza l'
  esplicito
8 % consenso scritto dell'autore. La distribuzione o la vendita non
  autorizzata
9 % costituisce reato ed è perseguibile penalmente secondo le leggi
  vigenti.
10 %
    =====

11
12 function [Fx, fx, Fu, fu, dU_min, dU_max, Gx, gx] = imposta_vincoli
  (X_min, X_max, U_min, U_max, Rate_max, Ts)
13   % IMPOSTA_VINCOLI Costruisce le matrici poliedriche dei vincoli
  per l'MPC.
14   % É una funzione generalizzata per qualsiasi sistema.
15   %
16   % Input:
17   %   X_min, X_max: Limiti inferiori e superiori degli stati (nx
  x 1)
18   %   U_min, U_max: Limiti inferiori e superiori degli ingressi (
  nu x 1)
19   %   Rate_max: Massimo rateo di variazione degli ingressi (nu x
  1)
20   %   Ts: Tempo di campionamento
21   %
22   % Output:
23   %   Fx, fx: Matrici vincoli di stato  $Fx * x \leq fx$ 
24   %   Fu, fu: Matrici vincoli di ingresso  $Fu * u \leq fu$ 
25   %   dU_min, dU_max: Vincoli di rateo
26   %   Gx, gx: Copia di Fx, fx (per retrocompatibilità)
27
28   nx = length(X_min);
29   nu = length(U_min);
30
31   % Vincoli di Rateo
32   dU_max = Rate_max * Ts;

```

```

33     dU_min = -dU_max;
34
35     % Vincoli Poliedrici di Stato
36     Fx = [eye(nx); -eye(nx)];
37     fx = [X_max; -X_min];
38
39     % Vincoli Poliedrici di Ingresso
40     Fu = [eye(nu); -eye(nu)];
41     fu = [U_max; -U_min];
42
43     % Per retrocompatibilità con altri codici MPC pre-esistenti
44     Gx = Fx;
45     gx = fx;
46 end

```

LISTING A.4: Definizione dei Vincoli (Constraints)

Setup e Configurazione MPC

File: funzioni_mpc/setup_mpc.m

Funzione per la sintesi e l'impostazione del controllore predittivo (pesi Q e R, orizzonte predittivo, vincoli terminali).

```

1 %
  =====
2 % Copyright (c) 2026 Ing. Leggeri Leonardo
3 % Tutti i diritti riservati.
4 %
5 % ATTENZIONE: Questo software e il relativo codice sorgente sono di
  proprietà
6 % esclusiva dell'Ing. Leggeri Leonardo. É severamente vietata la
  copia,
7 % la distribuzione, la modifica o la vendita a terzi senza l'
  esplicito
8 % consenso scritto dell'autore. La distribuzione o la vendita non
  autorizzata
9 % costituisce reato ed è perseguibile penalmente secondo le leggi
  vigenti.

```

```

10 %
=====
11
12 function mpc_prob = setup_mpc(N, nx, nu, A_long_ds, B_ctrl_ds, Q, P
    , R, U_min, U_max, Gx, gx, G_inf, g_inf, x_ref, u_ref)
13 % SETUP_MPC Prepara tutte le matrici per il quadprog
14 n_vars = N*nu + N*nx;
15
16 R_blk = kron(eye(N), R);
17 Q_blk = blkdiag(kron(eye(N-1), Q), P);
18 H = 2 * blkdiag(R_blk, Q_blk);
19
20 % Termine lineare della funzione di costo per l'inseguimento
del riferimento
21 f = zeros(n_vars, 1);
22 for k = 1:N
23     f((k-1)*nu + 1 : k*nu) = -2 * R * u_ref;
24 end
25 for k = 1:N-1
26     f(N*nu + (k-1)*nx + 1 : N*nu + k*nx) = -2 * Q * x_ref;
27 end
28 f(N*nu + (N-1)*nx + 1 : N*nu + N*nx) = -2 * P * x_ref;
29
30 Aeq_base = zeros(N*nx, n_vars);
31 for k = 1:N
32     Aeq_base((k-1)*nx + 1 : k*nx, (k-1)*nu + 1 : k*nu) = -
B_ctrl_ds;
33     Aeq_base((k-1)*nx + 1 : k*nx, N*nu + (k-1)*nx + 1 : N*nu +
k*nx) = eye(nx);
34     if k > 1
35         Aeq_base((k-1)*nx + 1 : k*nx, N*nu + (k-2)*nx + 1 : N*
nu + (k-1)*nx) = -A_long_ds;
36     end
37 end
38
39 lb = -inf(n_vars, 1); ub = inf(n_vars, 1);
40 for k = 1:N
41     lb((k-1)*nu + 1 : k*nu) = U_min;
42     ub((k-1)*nu + 1 : k*nu) = U_max;

```

```

43     end
44
45     A_ineq_stat = []; b_ineq_stat = [];
46     for j = 1:N-1
47         A_t = zeros(size(Gx, 1), n_vars);
48         A_t(:, N*nu + (j-1)*nx + 1 : N*nu + j*nx) = Gx;
49         A_ineq_stat = [A_ineq_stat; A_t];
50         b_ineq_stat = [b_ineq_stat; gx];
51     end
52
53     A_t = zeros(size(G_inf, 1), n_vars);
54     A_t(:, N*nu + (N-1)*nx + 1 : N*nu + N*nx) = G_inf;
55     A_ineq_stat = [A_ineq_stat; A_t];
56     b_ineq_stat = [b_ineq_stat; g_inf];
57
58     % Salvataggio in struttura
59     mpc_prob.H = H;
60     mpc_prob.f = f;
61     mpc_prob.Aeq_base = Aeq_base;
62     mpc_prob.lb = lb;
63     mpc_prob.ub = ub;
64     mpc_prob.A_ineq_stat = A_ineq_stat;
65     mpc_prob.b_ineq_stat = b_ineq_stat;
66     mpc_prob.n_vars = n_vars;
67     mpc_prob.N = N;
68     mpc_prob.nx = nx;
69     mpc_prob.nu = nu;
70 end

```

LISTING A.5: Setup e Configurazione MPC

Simulatore MPC Closed-Loop

File: `funzioni_mpc/simula_mpc.m`

Funzione core per la simulazione temporale e l'evoluzione del ciclo chiuso con controllo MPC.

```

1 %
  =====

2 % Copyright (c) 2026 Ing. Leggeri Leonardo
3 % Tutti i diritti riservati.
4 %
5 % ATTENZIONE: Questo software e il relativo codice sorgente sono di
  proprietà
6 % esclusiva dell'Ing. Leggeri Leonardo. É severamente vietata la
  copia,
7 % la distribuzione, la modifica o la vendita a terzi senza l'
  esplicito
8 % consenso scritto dell'autore. La distribuzione o la vendita non
  autorizzata
9 % costituisce reato ed è perseguibile penalmente secondo le leggi
  vigenti.
10 %
  =====

11
12 function [storia_x, storia_u, storia_costo] = simula_mpc(mpc_prob,
  x_iniziale, t_sim, A_long_ds, B_ctrl_ds, dU_max)
13   % SIMULA_MPC Risolve il problema quadratico e simula l'
  evoluzione del sistema
14   nx = mpc_prob.nx;
15   nu = mpc_prob.nu;
16   N = mpc_prob.N;
17   n_vars = mpc_prob.n_vars;
18
19   storia_x = zeros(nx, t_sim+1); storia_x(:,1) = x_iniziale;
20   storia_u = zeros(nu, t_sim);
21   storia_costo = zeros(1, t_sim);
22   u_previous = [0;0;0];
23   options = optimoptions('quadprog', 'Display', 'off');
24   % --- Precomputazione Matrici Invarianti ---
25   A_rate = zeros(nu*2*N, n_vars);
26   b_rate_base = zeros(nu*2*N, 1);
27   for k = 1:N
28       idx_u = (k-1)*nu+1:k*nu;
29       A_rate((k-1)*2*nu+1:k*2*nu, idx_u) = [eye(nu); -eye(nu)];

```

```

30         if k > 1
31             idx_u_prev = (k-2)*nu+1:(k-1)*nu;
32             A_rate((k-1)*2*nu+1:k*2*nu, idx_u_prev) = [-eye(nu);
eye(nu)];
33             b_rate_base((k-1)*2*nu+1:k*2*nu) = [dU_max; dU_max];
34         end
35     end
36     A_ineq_tot = [mpc_prob.A_ineq_stat; A_rate];
37     % -----
38
39     for t = 1:t_sim
40         beq = zeros(N*nx, 1);
41         beq(1:nx) = A_long_ds * storia_x(:,t);
42
43         b_rate = b_rate_base;
44         b_rate(1:2*nu) = [dU_max + u_previous; dU_max - u_previous
];
45         b_ineq_tot = [mpc_prob.b_ineq_stat; b_rate];
46         [z_opt, fval, exitflag] = quadprog(mpc_prob.H, mpc_prob.f,
A_ineq_tot, b_ineq_tot, mpc_prob.Aeq_base, beq, mpc_prob.lb,
mpc_prob.ub, [], options);
47         if exitflag < 0
48             error('Infeasible! Il punto allo step %d è fuori da X_N
. Riduci leggermente la severità di x_iniziale.', t);
49         end
50
51         storia_costo(t) = fval;
52
53         % L'animazione live è stata rimossa per massimizzare le
prestazioni.
54
55         u_applicata = z_opt(1:nu);
56         storia_u(:, t) = u_applicata;
57
58         storia_x(:, t+1) = A_long_ds * storia_x(:,t) + B_ctrl_ds *
u_applicata;
59         u_previous = u_applicata;
60     end
61
62     % Plot della traiettoria completa tutto in una volta alla fine

```



```

63     plot3(storia_x(3,:), storia_x(4,:), storia_x(2,:), '-b', '
        LineWidth', 2.5, 'DisplayName', 'Traiettoria Effettiva');
64
65     % Plot punto di partenza
66     plot3(x_iniziale(3), x_iniziale(4), x_iniziale(2), 'k*', '
        MarkerSize', 10, 'LineWidth', 2, 'DisplayName', 'Partenza');
67     text(x_iniziale(3), x_iniziale(4), x_iniziale(2)+0.05, '
        Partenza', 'FontWeight', 'bold');
68
69     % Aggiungi un pallino rosso per ogni passo effettivamente
        compiuto alla fine
70     plot3(storia_x(3,:), storia_x(4,:), storia_x(2,:), 'ro', '
        MarkerSize', 6, 'MarkerFaceColor', 'r', 'DisplayName', 'Passi
        MPC');
71
72     % Plot punto di arrivo finale
73     plot3(storia_x(3,end), storia_x(4,end), storia_x(2,end), 'kp',
        'MarkerSize', 15, 'MarkerFaceColor', 'k', 'DisplayName', 'Arrivo
        Effettivo');
74     text(storia_x(3,end), storia_x(4,end), storia_x(2,end)-0.05, '
        Arrivo', 'FontWeight', 'bold', 'Color', 'k');
75
76     % Aggiorna la legenda includendo i nuovi plot
77     legend('show', 'Location', 'best', 'Interpreter', 'latex');
78 end

```

LISTING A.6: Simulatore MPC Closed-Loop

Calcolo Insieme Invariante (CIS)

File: funzioni_mpc/cis.m

Funzione per il calcolo del Control Invariant Set e del Maximum Control Invariant Set (Insieme Invariante di Controllo).

```

1 %
    =====

2 % Copyright (c) 2026 Ing. Leggeri Leonardo
3 % Tutti i diritti riservati.

```

```

4 %
5 % ATTENZIONE: Questo software e il relativo codice sorgente sono di
    proprietà
6 % esclusiva dell'Ing. Leggeri Leonardo. É severamente vietata la
    copia,
7 % la distribuzione, la modifica o la vendita a terzi senza l'
    esplicito
8 % consenso scritto dell'autore. La distribuzione o la vendita non
    autorizzata
9 % costituisce reato ed è perseguibile penalmente secondo le leggi
    vigenti.
10 %
    =====
11
12 function [G_inf, g_inf] = cis(A, B, x_ref, u_ref, Fx, fx, Fu, fu, Q
    , R)
13     % Control Invariant Set Iterativo tramite Metodo Matriciale (No
    Polyhedron)
14     [K, ~, ~] = dlqr(A, B, Q, R);
15     A_cl = A - B*K;
16
17     % Traslazione dei vincoli rispetto al riferimento z = x - x_ref
18     Gx = Fx;
19     gx = fx - Fx*x_ref;
20
21     Gu_x = [-K; K];
22     gu_x = [fu(1:3) - Fu(1:3,:)*u_ref; fu(4:6) - Fu(4:6,:)*u_ref];
23
24     G = [Gx; Gu_x];
25     g = [gx; gu_x];
26
27     G_inf = G; g_inf = g;
28     max_iter = 100; tol = 1e-6;
29     for i = 1:max_iter
30         G_next = G * (A_cl^i);
31         if norm(G_next, inf) < tol
32             break;
33         end
34         G_inf = [G_inf; G_next];

```

```

35     g_inf = [g_inf; g];
36     end
37
38     % Ritraslazione di g_inf in modo che  $G\_inf * x \leq g\_inf$ 
39     % (dato che attualmente  $G\_inf * z \leq g\_inf$ )
40     g_inf = g_inf + G_inf * x_ref;
41 end

```

LISTING A.7: Calcolo Insieme Invariante (CIS)

Visualizzazione Insiemi Invarianti

File: funzioni_mpc/plot_cis.m

Funzione per la visualizzazione 3D e 2D (tramite proiezioni) degli insiemi invarianti calcolati.

```

1 %
  =====

2 % Copyright (c) 2026 Ing. Leggeri Leonardo
3 % Tutti i diritti riservati.
4 %
5 % ATTENZIONE: Questo software e il relativo codice sorgente sono di
  proprietà
6 % esclusiva dell'Ing. Leggeri Leonardo. É severamente vietata la
  copia,
7 % la distribuzione, la modifica o la vendita a terzi senza l'
  esplicito
8 % consenso scritto dell'autore. La distribuzione o la vendita non
  autorizzata
9 % costituisce reato ed è perseguibile penalmente secondo le leggi
  vigenti.
10 %
  =====

11
12 function plot_cis(G_inf, g_inf, x_ref, mpc_prob, A_long_ds, dU_max,
  parent_tab)
13     if nargin < 3

```

```

14     x_ref = zeros(4,1);
15 end
16 if nargin < 7
17     fig1 = figure('Name', 'Control Invariant Set 3D', 'Color',
18 'w', 'Position', [50, 50, 900, 700]);
19     parent_tab = fig1;
20 end
21 % PLOT_CIS Disegna in 3D il Control Invariant Set
22 ax = axes('Parent', parent_tab);
23 hold(ax, 'on'); grid(ax, 'on'); view(ax, 3);
24 title(ax, ['Poliedro  $X_f$  in 3D (Fetta  $\theta =$ ',
num2str(x_ref(1)), ' $\theta$ '), 'Interpreter', 'latex', 'FontSize',
14);
25
26 % Assi aggiornati al tuo vettore di stato: [theta, q, U, W]
27 xlabel('u [m/s]'); ylabel('w [m/s]'); zlabel('q [rad/s]');
28
29 % Griglia ad alta risoluzione centrata vicino all'origine per
trovare il CIS
30 [X_U, X_W, X_q] = meshgrid(linspace(-30, 30, 40), ... % Range
per u (Stato 3)
31                               linspace(-30, 30, 40), ... % Range
per w (Stato 4)
32                               linspace(-40 * pi/180, 40 * pi/180,
40)); % Range per q (Stato 2)
33
34 X_U_f = X_U(:); X_W_f = X_W(:); X_q_f = X_q(:);
35
36 % Imposta la fetta in modo che combaci con la theta del target
37 theta_slice = x_ref(1);
38 X_theta_f = theta_slice * ones(size(X_U_f)); % Fetta
corrispondente al target
39
40 Validi_inf = true(size(X_U_f));
41 for j = 1:size(G_inf, 1)
42     Valore = G_inf(j,1)*X_theta_f + G_inf(j,2)*X_q_f + G_inf(j
,3)*X_U_f + G_inf(j,4)*X_W_f;
43     Validi_inf = Validi_inf & (Valore <= g_inf(j));
44 end

```

```

45
46     PX_inf = X_U_f(Validi_inf); PY_inf = X_W_f(Validi_inf); PZ_inf
    = X_q_f(Validi_inf);
47
48     if length(PX_inf) > 4
49         K_hull_inf = convhull(PX_inf, PY_inf, PZ_inf);
50         h_inf = trisurf(K_hull_inf, PX_inf, PY_inf, PZ_inf, 'Parent
    ', ax, 'FaceColor', 'c', 'FaceAlpha', 0.8, 'EdgeColor', 'b', '
    EdgeAlpha', 0.3);
51         leg_handles = h_inf;
52         leg_names = {'Control Invariant Set  $\mathcal{X}_0^\infty$ '};
53     else
54         disp('ATTENZIONE: Nessun punto valido trovato per il plot
    di  $X_0$  sulla fetta scelta.');
```

55 leg_handles = [];

56 leg_names = {};

57 end

58

59 % --- NOVITÀ: Calcolo e Plot N-Step Feasible Set ---

60 if nargin >= 6

61 disp('Calcolo del Set Feasible N-Step X_N in corso (

esplorazione vincoli)...');

62 nx = mpc_prob.nx;

63 nu = mpc_prob.nu;

64 N = mpc_prob.N;

65 n_vars = mpc_prob.n_vars;

66

67 options = optimoptions('linprog', 'Display', 'off');

68 A_rate = zeros(nu*2*N, n_vars);

69 b_rate_base = zeros(nu*2*N, 1);

70 for k = 1:N

71 idx_u = (k-1)*nu+1:k*nu;

72 A_rate((k-1)*2*nu+1:k*2*nu, idx_u) = [eye(nu); -eye(nu)

];

73 if k > 1

74 idx_u_prev = (k-2)*nu+1:(k-1)*nu;

75 A_rate((k-1)*2*nu+1:k*2*nu, idx_u_prev) = [-eye(nu)

; eye(nu)];

76 b_rate_base((k-1)*2*nu+1:k*2*nu) = [dU_max; dU_max

];

```

77         end
78     end
79     A_ineq_tot = [mpc_prob.A_ineq_stat; A_rate];
80
81     u_previous = [0;0;0];
82     b_rate = b_rate_base;
83     b_rate(1:2*nu) = [dU_max + u_previous; dU_max - u_previous
84 ];
85     b_ineq_tot = [mpc_prob.b_ineq_stat; b_rate];
86
87     % Griglia a bassa risoluzione dedicata SOLO all'N-Step Set
88     per velocizzare linprog
89     [X_U_N, X_W_N, X_q_N] = meshgrid(linspace(-40, 40, 15), ...
90                                     linspace(-40, 40, 15), ...
91                                     linspace(-40 * pi/180, 40
92 * pi/180, 15));
93
94     X_U_N_f = X_U_N(:); X_W_N_f = X_W_N(:); X_q_N_f = X_q_N(:);
95     f_lin = zeros(n_vars, 1);
96     Validi_N = false(size(X_U_N_f));
97
98     for i = 1:length(X_U_N_f)
99         % Check rapido se è già in O_inf per saltare linprog
100         x0_test = [theta_slice; X_q_N_f(i); X_U_N_f(i); X_W_N_f
101 (i)];
102
103         is_in_O_inf = true;
104         for j = 1:size(G_inf, 1)
105             if G_inf(j,:) * x0_test > g_inf(j)
106                 is_in_O_inf = false;
107                 break;
108             end
109         end
110
111         if is_in_O_inf
112             Validi_N(i) = true;
113             continue;
114         end
115
116         % Se non è in O_inf, testa con linprog

```

```

113         beq = zeros(N*nx, 1);
114         beq(1:nx) = A_long_ds * x0_test;
115
116         [~, ~, exitflag] = linprog(f_lin, A_ineq_tot,
b_ineq_tot, mpc_prob.Aeq_base, beq, mpc_prob.lb, mpc_prob.ub,
options);
117         if exitflag >= 0 || exitflag == -3
118             Validi_N(i) = true;
119         end
120     end
121
122     PX_N = X_U_N_f(Validi_N); PY_N = X_W_N_f(Validi_N); PZ_N =
X_q_N_f(Validi_N);
123     if length(PX_N) > 4
124         K_hull_N = convhull(PX_N, PY_N, PZ_N);
125         h_N = trisurf(K_hull_N, PX_N, PY_N, PZ_N, 'Parent', ax,
'FaceColor', 'y', 'FaceAlpha', 0.2, 'EdgeColor', 'y', '
EdgeAlpha', 0.1);
126         leg_handles(end+1) = h_N;
127         leg_names{end+1} = sprintf('Feasible N-Step Set  $\mathcal{X}_{\{N\}}$ ', N);
128         title(ax, ['Poliedri  $\mathcal{X}_{\{0\}}^{\infty}$  e  $\mathcal{X}_{\{N\}}$  in 3D (Fetta  $\theta =$ ', num2str(
theta_slice), '$)'], 'Interpreter', 'latex', 'FontSize', 14);
129     end
130 end
131
132 if ~isempty(leg_handles)
133     legend(leg_handles, leg_names, 'Location', 'best', '
Interpreter', 'latex');
134 end
135 end

```

LISTING A.8: Visualizzazione Insiemi Invarianti

Analisi Costo MPC 3D

File: funzioni_mpc/plot_mpc_cost_3d.m

Funzione per il plotting tridimensionale della funzione di costo associata al regolatore MPC.

```

1 %
  =====

2 % Copyright (c) 2026 Ing. Leggeri Leonardo
3 % Tutti i diritti riservati.
4 %
5 % ATTENZIONE: Questo software e il relativo codice sorgente sono di
  proprietà
6 % esclusiva dell'Ing. Leggeri Leonardo. É severamente vietata la
  copia,
7 % la distribuzione, la modifica o la vendita a terzi senza l'
  esplicito
8 % consenso scritto dell'autore. La distribuzione o la vendita non
  autorizzata
9 % costituisce reato ed è perseguibile penalmente secondo le leggi
  vigenti.
10 %
  =====

11
12 function plot_mpc_cost_3d(mpc_prob, A_long_ds, dU_max, storia_x,
  storia_costo, Ts, parent_tab)
13 %
  =====

14 % PLOT_MPC_COST_3D - Visualizza il Funzionale di Costo Ottimo J
  *(x) dell'MPC
15 %
  =====

16
17 if nargin < 6
18     Ts = 0.1;

```



```

19     end
20     if nargin < 7
21         fig1 = figure('Name', sprintf('Costo MPC 3D (Ts = %g s)',
Ts), 'Color', 'w', 'Position', [150 150 850 650]);
22         parent_tab = fig1;
23     end
24     w_lim = 40; % m/s
25     q_lim = 1.0; % rad/s
26
27     % Usiamo una griglia 20x20 per un calcolo rapido
28     [W_grid, Q_rad] = meshgrid(linspace(-w_lim, w_lim, 20),
linspace(-q_lim, q_lim, 20));
29
30     nx = mpc_prob.nx;
31     nu = mpc_prob.nu;
32     N = mpc_prob.N;
33     n_vars = mpc_prob.n_vars;
34
35     % Recuperiamo P_ds approssimato (o potresti passarlo da MPC.m)
36     [~, P_ds, ~] = dlqr(A_long_ds, mpc_prob.Aeq_base(1:nx, 1:nu)
*(-1), blkdiag(10,10,1,1), eye(nu));
37
38     V_surf = zeros(size(W_grid));
39     for i = 1:size(W_grid, 1)
40         for j = 1:size(W_grid, 2)
41             stato_surf = [0; Q_rad(i,j); 0; W_grid(i,j)];
42             V_surf(i,j) = stato_surf' * P_ds * stato_surf;
43         end
44     end
45
46     %% --- FIGURA 3D ---
47     ax = axes('Parent', parent_tab);
48     hold(ax, 'on'); grid(ax, 'on');
49
50     h_surf = surf(ax, W_grid, Q_rad, V_surf, 'EdgeColor', 'none', '
FaceAlpha', 0.65);
51     colormap(ax, jet);
52     cb = colorbar(ax);
53     ylabel(cb, 'Costo Ottimo MPC  $J^*(x_k)$ ', 'Interpreter', 'latex
', 'FontSize', 12);

```

```

54
55     max_v_traj = max(storia_costo);
56     if isempty(max_v_traj) || isnan(max_v_traj)
57         max_v_traj = 1;
58     end
59
60     % Traiettorie
61     N_steps = length(storia_costo);
62     t_discrete = 1:N_steps;
63     t_fine = linspace(1, N_steps, N_steps * 10);
64
65     W_smooth = pchip(t_discrete, storia_x(4,1:N_steps), t_fine);
66     Q_smooth = pchip(t_discrete, storia_x(2,1:N_steps), t_fine);
67     V_smooth = pchip(t_discrete, storia_costo, t_fine);
68
69     h_line = plot3(ax, W_smooth, Q_smooth, V_smooth, '-r', '
LineWidth', 2);
70
71     % Punti discreti veri
72     plot3(ax, storia_x(4,1:N_steps), storia_x(2,1:N_steps),
73         storia_costo, 'o', ...
74         'MarkerEdgeColor', 'r', 'MarkerFaceColor', 'w', '
MarkerSize', 4);
75
76     % Start
77     h_start = plot3(ax, storia_x(4,1), storia_x(2,1), storia_costo
78         (1), 'o', ...
79         'MarkerFaceColor', 'r', 'MarkerEdgeColor', 'k',
80         'MarkerSize', 8);
81
82     % End
83     h_end = plot3(ax, storia_x(4,N_steps), storia_x(2,N_steps),
84         storia_costo(N_steps), 's', ...
85         'MarkerFaceColor', 'g', 'MarkerEdgeColor', 'k', '
MarkerSize', 8);
86
87     title(ax, sprintf('\textbf{Funzionale di Costo MPC }$J^*(x_k)$
a Tempo Discreto ($T_s = \%g$ s)}', Ts), 'Interpreter', 'latex',
'FontSize', 16);

```

```

84     xlabel(ax, 'Velocit\'a Verticale $w$ [m/s]', 'Interpreter', '
latex', 'FontSize', 12);
85     ylabel(ax, 'Velocit\'a di Beccheggio $q$ [rad/s]', 'Interpreter
', 'latex', 'FontSize', 12);
86     zlabel(ax, 'Costo $J^*(x_k)$', 'Interpreter', 'latex', '
FontSize', 12);
87
88     legend(ax, [h_surf, h_line, h_start, h_end], ...
89         {'Superficie $J^*(x_k)$', 'Evoluzione di Stato', '
Partenza ($k=0$)', 'Arrivo'}, ...
90         'Interpreter', 'latex', 'FontSize', 12, 'Location', '
northeast');
91
92     zlim(ax, [0, max(max(V_surf(:)), max_v_traj) * 1.2]);
93     xlim(ax, [-w_lim, w_lim]);
94     ylim(ax, [-q_lim, q_lim]);
95     view(ax, -35, 30);
96 end

```

LISTING A.9: Analisi Costo MPC 3D

Plot e Grafici dei Risultati

File: funzioni_mpc/plot_risultati.m

Funzione per la rappresentazione grafica delle risposte del sistema, evoluzione degli stati e comandi degli attuatori.

```

1 %
    =====

2 % Copyright (c) 2026 Ing. Leggeri Leonardo
3 % Tutti i diritti riservati.
4 %
5 % ATTENZIONE: Questo software e il relativo codice sorgente sono di
    proprietà
6 % esclusiva dell'Ing. Leggeri Leonardo. É severamente vietata la
    copia,
7 % la distribuzione, la modifica o la vendita a terzi senza l'
    esplicito

```

```

8 % consenso scritto dell'autore. La distribuzione o la vendita non
   autorizzata
9 % costituisce reato ed è perseguibile penalmente secondo le leggi
   vigenti.
10 %
   =====
11
12 %funzione per la creazione dei grafici dopo esecuzione script MPC.m
13 function plot_risultati(t_sim, storia_x, storia_u, U_min, U_max,
   x_ref, u_ref, tab_stati, tab_attuatori)
14     if nargin < 6
15         x_ref = zeros(4,1);
16         u_ref = zeros(3,1);
17     end
18     if nargin < 8
19         fig1 = figure('Name', 'Risultati MPC: Stati del Velivolo',
   'Color', 'w', 'Position', [100 100 900 600]);
20         tab_stati = fig1;
21         fig2 = figure('Name', 'Risultati MPC: Sforzo degli
   Attuatori', 'Color', 'w', 'Position', [150 150 700 800]);
22         tab_attuatori = fig2;
23     end
24
25     % PLOT_RISULTATI Disegna i grafici dell'evoluzione di stati e
   attuatori
26     t_layout_x = tiledlayout(tab_stati, 2, 2, 'Padding', 'compact',
   'TileSpacing', 'compact');
27
28     nexttile(t_layout_x); plot(0:t_sim, storia_x(1,:), '-b', '
   LineWidth', 1.5); hold on; yline(x_ref(1), 'r--', 'LineWidth',
   1.2); title('Pitch Angle (\theta) [rad]'); grid on;
29     nexttile(t_layout_x); plot(0:t_sim, storia_x(2,:), '-r', '
   LineWidth', 1.5); hold on; yline(x_ref(2), 'r--', 'LineWidth',
   1.2); title('Pitch Rate (q) [rad/s]'); grid on;
30     nexttile(t_layout_x); plot(0:t_sim, storia_x(3,:), '-m', '
   LineWidth', 1.5); hold on; yline(x_ref(3), 'r--', 'LineWidth',
   1.2); title('Velocità Forward (u) [m/s]'); grid on;

```

```

31     nexttile(t_layout_x); plot(0:t_sim, storia_x(4,:), '-c', '
LineWidth', 1.5); hold on; yline(x_ref(4), 'r--', 'LineWidth',
1.2); title('Velocità Verticale (w) [m/s]'); grid on;
32     lgd = legend({'Traiettoria', 'Target (x_{ref})'}, 'Orientation'
, 'horizontal');
33     lgd.Layout.Tile = 'north';
34
35     t_layout_u = tiledlayout(tab_attuatori, 3, 1, 'Padding', '
compact', 'TileSpacing', 'compact');
36
37     nexttile(t_layout_u); stairs(0:t_sim-1, storia_u(1,:), '-g', '
LineWidth', 1.5); hold on;
38     yline(U_max(1), 'k--'); yline(U_min(1), 'k--'); yline(u_ref(1),
'r--', 'LineWidth', 1.2); title(' Spinta [N]'); grid on;
39
40     nexttile(t_layout_u); stairs(0:t_sim-1, storia_u(2,:) * 180 /
pi, '-g', 'LineWidth', 1.5); hold on;
41     yline(U_max(2) * 180 / pi, 'k--'); yline(U_min(2) * 180 / pi, '
k--'); yline(u_ref(2) * 180 / pi, 'r--', 'LineWidth', 1.2);
42     title(' Elevatore [deg]'); grid on;
43
44     nexttile(t_layout_u); stairs(0:t_sim-1, storia_u(3,:) * 180 /
pi, '-g', 'LineWidth', 1.5); hold on;
45     yline(U_max(3) * 180 / pi, 'k--'); yline(U_min(3) * 180 / pi, '
k--'); yline(u_ref(3) * 180 / pi, 'r--', 'LineWidth', 1.2);
46     title(' Flap [deg]'); grid on;
47 end

```

LISTING A.10: Plot e Grafici dei Risultati

Appendice B

Codici MATLAB: Controllo Ottimo LQR

In questa appendice vengono riportati i codici MATLAB utilizzati. Il documento è strutturato per essere facilmente navigabile: partendo dallo script principale, è possibile esplorare tutte le funzioni richiamate.

Script Principale LQR

File: LQRAnalysis.m

Script MATLAB principale per l'analisi di stabilità, tuning dei pesi tramite regola di Bryson e progettazione del controllo LQR. In questo script vengono richiamate le seguenti funzioni, definite nelle sezioni successive:

- `plot_lyapunov_lqr.m`
- `funzioni_lqr/progetta_LQR_continuo.m`
- `funzioni_lqr/progetta_LQR_discreto.m`
- `funzioni_lqr/plot_lyapunov.m`
- `funzioni_lqr/plot_lyapunov_discrete.m`

```

1 %
   =====
2 % Tesi Triennale - LQR Longitudinale F-16
3 % Ing. Leggeri Leonardo
4 %
   =====

```

```

5 close all
6 % Inizializzazione Path
7 startup_project();
8
9 %% 1. IMPOSTAZIONE DEL SISTEMA (Solo veri attuatori)
10 % Estraiamo le dimensioni dalle matrici
11 nx = size(A_long, 1);
12 nu_long = size(B_long, 2);
13
14 B_ctrl = B_long(:, 1:3);
15 D_ctrl = D_long(:, 1:3);
16 nu = size(B_ctrl, 2);
17
18 %% 2. SINTESI DEL CONTROLLORE LQR
19 %
20 % =====
21 %
22 % NOTA SUI PESI Q e R:
23 % Se vuoi modificare l'aggressività del controllore (pesi Q e R),
24 % apri il file "funzioni_lqr/progetta_LQR_continuo.m" e modificali
25 %
26 % =====
27
28 disp('--- Sintesi LQR Continuo ---');
29 [K, P, Q, R, A_cl] = progetta_LQR_continuo(A_long, B_ctrl);
30
31 disp('Guadagno K:');
32 disp(K);
33 disp('Autovalori a ciclo chiuso (devono avere parte reale negativa)');
34 disp(eig(A_cl));
35
36 % Creiamo il sistema State-Space a ciclo chiuso (Continuo)
37 sys_cl = ss(A_cl, zeros(nx, nu), eye(nx), zeros(nx, nu));
38
39 % Vettore tempo per la simulazione
40 t = 0:0.05:10;
41
42 % Condizione iniziale estrema (Looping/Candela)
43 x0 = [0; 0.3; 0; 20]; % theta, q, U, W

```

```

40 % Simulazione della risposta libera (initial) del sistema
41 [y_sim, t_sim, x_sim] = initial(sys_cl, x0, t);
42
43 % Traspongo x_sim per avere gli stati sulle righe (come nel tuo
    script precedente)
44 xf = x_sim';
45
46 % Calcolo dei comandi u generati dall'LQR in ogni istante (u = -K*x
    )
47 uf = -K * xf;
48
49 %% 4. GRAFICI (Senza vincoli, con nuovi colori)
50 rad2deg = 180 / pi; % Fattore di conversione utile per la
    leggibilità
51
52 % Definizione del colore Giallo leggibile per sfondo bianco
53 colore_giallo = [0.9290, 0.6940, 0.1250];
54
55 %
    -----
56 % FIGURA 1: DINAMICA DEGLI STATI (Variabili Separate)
57 %
    -----
58 figure('Name', 'Controllo LQR Puro: Stati a Ciclo Chiuso', 'Color',
    'w', 'Position', [100, 100, 800, 800])
59
60 % 1. Angolo di Beccheggio (theta)
61 subplot(4, 1, 1)
62 plot(t, xf(1,:) * rad2deg, 'b', 'LineWidth', 2); hold on;
63 yline(0, 'k--', 'LineWidth', 1);
64 ylabel('Ampiezza [deg]', 'Interpreter', 'latex')
65 title('$\theta$ (Pitch Angle)', 'Interpreter', 'latex')
66 grid on;
67
68 % 2. Rateo di Beccheggio (q)
69 subplot(4, 1, 2)
70 plot(t, xf(2,:) * rad2deg, 'r', 'LineWidth', 2); hold on;
71 yline(0, 'k--', 'LineWidth', 1);

```



```

72 ylabel('Ampiezza [deg/s]', 'Interpreter', 'latex')
73 title('$q$ (Pitch Rate)', 'Interpreter', 'latex')
74 grid on;
75
76 % 3. Velocità asse X (U)
77 subplot(4, 1, 3)
78 plot(t, xf(3,:), 'Color', colore_giallo, 'LineWidth', 2); hold on;
79 yline(0, 'k--', 'LineWidth', 1);
80 ylabel('Ampiezza [ft/s]', 'Interpreter', 'latex')
81 title('$U$ (Velocità Orizzontale)', 'Interpreter', 'latex')
82 grid on;
83
84 % 4. Velocità asse Z (W)
85 subplot(4, 1, 4)
86 plot(t, xf(4,:), 'g', 'LineWidth', 2); hold on;
87 yline(0, 'k--', 'LineWidth', 1);
88 xlabel('Tempo [s]', 'Interpreter', 'latex')
89 ylabel('Ampiezza [ft/s]', 'Interpreter', 'latex')
90 title('$W$ (Velocità Verticale $\approx \alpha$)', 'Interpreter',
    'latex')
91 grid on;
92
93 %
    -----
94 % FIGURA 2: SFORZO DEGLI ATTUATORI (Senza linee di saturazione)
95 %
    -----

96 figure('Name', 'Controllo LQR Puro: Sforzo Attuatori', 'Color', 'w'
    , 'Position', [150, 150, 800, 800])
97
98 % 1. Thrust (Spinta del Motore)
99 subplot(3, 1, 1)
100 plot(t, uf(1,:), 'b', 'LineWidth', 2); hold on; % Blu
101 yline(0, 'k--', 'LineWidth', 1);
102 ylabel('Spinta [lbs]')
103 title('Azione di Controllo Libera: Manetta (Thrust)')
104 grid on;
105

```

```

106 % 2. Elevator (Equilibratore)
107 subplot(3, 1, 2)
108 plot(t, uf(2,:) * rad2deg, 'r', 'LineWidth', 2); hold on; % Rosso
109 yline(0, 'k--', 'LineWidth', 1);
110 ylabel('Deflessione [deg]')
111 title('Azione di Controllo Libera: Equilibratore')
112 grid on;
113
114 % 3. Leading Edge Flap (LEF)
115 subplot(3, 1, 3)
116 plot(t, uf(3,:) * rad2deg, 'g', 'LineWidth', 2); hold on; % Verde
117 yline(0, 'k--', 'LineWidth', 1);
118 xlabel('Tempo [s]')
119 ylabel('Deflessione [deg]')
120 title('Azione di Controllo Libera: Leading Edge Flap')
121 grid on;
122
123 %
=====
124 % VISUALIZZAZIONE GRAFICA DEI NUOVI POLI (Ciclo Chiuso)
125 %
=====

126 sys_cl_map = ss(A_cl, B_ctrl, C_long, D_ctrl);
127
128 figure('Name', 'Mappa Poli-Zeri a Ciclo Chiuso (LQR Continuo)', '
    Color', 'w', 'Position', [200, 200, 600, 500]);
129 pzmap(sys_cl_map);
130 grid on;
131
132 title('Verifica Stabilità LQR: Posizione dei Nuovi Poli');
133 xlabel('Asse Reale (Velocità di convergenza/divergenza)');
134 ylabel('Asse Immaginario (Frequenza di oscillazione)');
135
136 %% 5. RITRATTO DI FASE E FUNZIONE DI LYAPUNOV (LQR)
137 % Per visualizzare la stabilità, isoliamo la dinamica a corto
    periodo
138 % variando W (velocità verticale) e q (pitch rate) e fissando U=0,
    theta=0.

```

```

139
140 % 5.1 Definizione della griglia spaziale
141 w_range = linspace(-25, 25, 50); % ft/s
142 q_range_rad = linspace(-1.5, 1.5, 50); % rad/s
143 [W_grid, Q_rad] = meshgrid(w_range, q_range_rad);
144 Q_deg = Q_rad * rad2deg; % Conversione per asse grafico
145
146 % 5.2 Calcolo della funzione di Lyapunov  $V(x) = x^T * P * x$  sulla
    griglia
147 V_surf = zeros(size(W_grid));
148 for i = 1:size(W_grid, 1)
149     for j = 1:size(W_grid, 2)
150         % Fissiamo theta=0 e U=0
151         stato_surf = [0; Q_rad(i,j); 0; W_grid(i,j)];
152         V_surf(i,j) = stato_surf' * P * stato_surf;
153     end
154 end
155
156 % 5.3 Dinamica a ciclo chiuso per ode45
157 dxdt = @(t, x) A_cl * x;
158
159 %% 5. RITRATTO DI FASE E FUNZIONE DI LYAPUNOV 3D (LQR)
160 % Usa la nuova funzione dedicata per generare il ritratto di fase 2
    D e
161 % la funzione di Lyapunov in 3D
162 plot_lyapunov_lqr(P, A_cl);

```

LISTING B.1: Script Principale LQR

Analisi Funzionale di Lyapunov

File: plot_lyapunov_lqr.m

Script principale per il plotting, l'analisi e lo studio dell'ellissoide associato al funzionale di Lyapunov del sistema LQR.

```

1 %
    =====
2 % Copyright (c) 2026 Ing. Leggeri Leonardo

```

```

3 % Tutti i diritti riservati.
4 %
5 % ATTENZIONE: Questo software e il relativo codice sorgente sono di
    proprietà
6 % esclusiva dell'Ing. Leggeri Leonardo. É severamente vietata la
    copia,
7 % la distribuzione, la modifica o la vendita a terzi senza l'
    esplicito
8 % consenso scritto dell'autore. La distribuzione o la vendita non
    autorizzata
9 % costituisce reato ed è perseguibile penalmente secondo le leggi
    vigenti.
10 %
    =====
11
12 function plot_lyapunov_lqr(P, A_cl)
13     %
    =====
14
15     % PLOT_LYAPUNOV - Ritratto di Fase e Funzione di Lyapunov 3D
16     % Riceve in ingresso la matrice P e la matrice a ciclo chiuso
    A_cl
17     %
    =====
18
19     set(0,'DefaultLineLineWidth',1.5);
20     set(0,'DefaultAxesFontSize',14);
21     set(0,'DefaultTextInterpreter','latex');
22
23     %% 1. Traiettorie (Solo dinamiche veloci)
24     dxdt = @(t,x) A_cl * x;
25
26     % Matrice delle condizioni iniziali multiple (8 scenari di
    partenza)
27     % Ordine degli stati: [theta; q; u; w]
28     x0_mult = [
29         0,      0,      0,      0,      0,      0,      0,      0;
        1.2, -1.2,  0.5, -0.5,  0.8, -0.8,  0.3, -0.3;
    
```

```

30         0,    0,    0,    0,    0,    0,    0,    0;
31         20,   -20,   15,   -15, -10,   10,   22,   -22
32     ];
33
34     figure('Name', 'Ritratto di Fase LQR', 'Color', 'w', 'Position'
35     , [50 100 800 600]);
36     hold on; grid on;
37
38     % Generiamo automaticamente una palette di colori per le N
39     % traiettorie
40
41     num_simulazioni = size(x0_mult, 2);
42     colori = lines(num_simulazioni);
43
44     for i = 1:num_simulazioni
45         % Simuliamo solo per 3 secondi per il corto periodo
46         [~, x] = ode45(dxdt, [0 3], x0_mult(:, i));
47
48         % Salviamo gli handle solo della prima iterazione per una
49         % legenda pulita
50         if i == 1
51             h_traj = plot(x(:,4), rad2deg(x(:,2)), 'Color', colori(
52             i,:));
53             h_start = plot(x(1,4), rad2deg(x(1,2)), '*', 'Color',
54             colori(i,:), 'MarkerSize', 8);
55         else
56             plot(x(:,4), rad2deg(x(:,2)), 'Color', colori(i,:));
57             plot(x(1,4), rad2deg(x(1,2)), '*', 'Color', colori(i,:)
58             , 'MarkerSize', 8);
59         end
60     end
61
62     %% 2. Campo Vettoriale e Curve di Livello
63
64     % Riduciamo la griglia per "abbracciare" esattamente le tue
65     % condizioni iniziali
66     w_lim = 45;
67     q_lim = 75;
68     [W_grid, Q_deg] = meshgrid(linspace(-w_lim, w_lim, 80),
69     linspace(-q_lim, q_lim, 80));

```

```

62     Q_rad = deg2rad(Q_deg);
63
64     W_dot = zeros(size(W_grid));
65     Q_dot = zeros(size(Q_rad));
66     V_contour = zeros(size(W_grid));
67
68     for i = 1:numel(W_grid)
69         % Slicing: imponiamo theta=0 e u=0
70         stato = [0; Q_rad(i); 0; W_grid(i)];
71         derivata = A_cl * stato;
72
73         Q_dot(i) = derivata(2);
74         W_dot(i) = derivata(4);
75         V_contour(i) = stato' * P * stato;
76     end
77
78     % Campo vettoriale
79     L = sqrt(W_dot.^2 + Q_dot.^2) + 1e-6;
80     h_quiv = quiver(W_grid, Q_deg, W_dot./L, rad2deg(Q_dot)./L,
81                     0.45, 'Color', [0.7 0.7 0.7]);
82
83     % Curve di Livello (Spaziatura Logaritmica)
84     val_min = min(V_contour(:));
85     val_max = max(V_contour(:));
86     livelli_log = logspace(log10(val_min + 1e-3), log10(val_max),
87                             18);
88
89     [~, h_cont] = contour(W_grid, Q_deg, V_contour, livelli_log, '
90                           LineWidth', 1.5);
91
92     xlabel('Vertical Velocity $w$ [ft/s]')
93     ylabel('Pitch Rate $q$ [deg/s]')
94     title('Ritratto di fase')
95
96     xlim([-w_lim, w_lim]);
97     ylim([-q_lim, q_lim]);
98
99     % Legenda dinamica e pulita (solo 4 voci invece di 18)
100    legend([h_traj, h_start, h_quiv, h_cont], ...

```

```

98         {'Traiettorie di Stato', 'Punti di Partenza', 'Direzione
Flusso', 'Contour $V(x) = x^T P x$'}, ...
99         'Interpreter','latex', 'Location', 'bestoutside');
100
101     %% 3. Visualizzazione 3D della Funzione di Lyapunov LQR
102     figure('Name', 'Funzione di Lyapunov 3D - LQR', 'Color', 'w');
103     hold on; grid on;
104
105     % Superficie 3D
106     V_surf = zeros(size(W_grid));
107     for i = 1:size(W_grid, 1)
108         for j = 1:size(W_grid, 2)
109             stato_surf = [0; Q_rad(i,j); 0; W_grid(i,j)];
110             V_surf(i,j) = stato_surf' * P * stato_surf;
111         end
112     end
113
114     h_surf = surf(W_grid, Q_deg, V_surf, 'EdgeColor', 'none', '
FaceAlpha', 0.7);
115     colormap jet;
116     colorbar;
117
118     % --- AGGIUNTA FONDAMENTALE: Inizializziamo max_v_traj ---
119     max_v_traj = 0;
120
121     % Proiezione traiettorie 3D
122     for i = 1:num_simulazioni
123         % Integrazione più fitta per fluidità visiva
124         [~, x] = ode45(dxdt, linspace(0, 3, 300), x0_mult(:, i));
125
126         V_trajettoria = sum((x * P) .* x, 2);
127
128         % --- Calcoliamo il picco di energia ---
129         if max(V_trajettoria) > max_v_traj
130             max_v_traj = max(V_trajettoria);
131         end
132
133         % Usiamo i colori estratti dalla palette per mantenere
coerenza col 2D
134         if i == 1

```

```

135         h_traj3 = plot3(x(:,4), rad2deg(x(:,2)), V_traiettoria,
136             '-', 'Color', colori(i,:), 'LineWidth', 2.5);
137         h_start3 = plot3(x(1,4), rad2deg(x(1,2)), V_traiettoria
138             (1), 'o', 'MarkerFaceColor', colori(i,:), 'MarkerEdgeColor', 'k'
139             , 'MarkerSize', 6);
140     else
141         plot3(x(:,4), rad2deg(x(:,2)), V_traiettoria, '-', '
142             Color', colori(i,:), 'LineWidth', 2.5);
143         plot3(x(1,4), rad2deg(x(1,2)), V_traiettoria(1), 'o', '
144             MarkerFaceColor', colori(i,:), 'MarkerEdgeColor', 'k', '
145             MarkerSize', 6);
146     end
147 end
148
149 title('Funzione di Lyapunov  $V(x) = x^T P x$  a Ciclo Chiuso', '
150     Interpreter', 'latex')
151 xlabel('Velocit\'a Verticale  $w$  [ft/s]', 'Interpreter', 'latex
152 ')
153 ylabel('Pitch Rate  $q$  [deg/s]', 'Interpreter', 'latex')
154 zlabel('Energia  $V(x)$ ', 'Interpreter', 'latex')
155
156 % Taglio dinamico dell'asse Z per non tagliare il paraboloide
157 zmax_plot = max(max(V_surf(:)), max_v_traj);
158 zlim([0, zmax_plot * 1.1]);
159 xlim([-w_lim, w_lim]);
160 ylim([-q_lim, q_lim]);
161 view(-45, 30);
162
163 % Legenda 3D
164 legend([h_surf, h_traj3, h_start3], ...
165     {'Funzione di Lyapunov  $V(x)$ ', 'Traiettoria di Stato',
166     'Condizione Iniziale'}, ...
167     'Interpreter', 'latex', 'Location', 'bestoutside');
168 end

```

LISTING B.2: Analisi Funzionale di Lyapunov

Progetto LQR (Tempo Continuo)

File: funzioni_lqr/progetta_LQR_continuo.m

Funzione per la risoluzione dell'equazione algebrica di Riccati (ARE) e il calcolo del guadagno ottimo LQR nel dominio del tempo continuo.

```

1 %
  =====

2 % Copyright (c) 2026 Ing. Leggeri Leonardo
3 % Tutti i diritti riservati.
4 %
5 % ATTENZIONE: Questo software e il relativo codice sorgente sono di
  proprietà
6 % esclusiva dell'Ing. Leggeri Leonardo. È severamente vietata la
  copia,
7 % la distribuzione, la modifica o la vendita a terzi senza l'
  esplicito
8 % consenso scritto dell'autore. La distribuzione o la vendita non
  autorizzata
9 % costituisce reato ed è perseguibile penalmente secondo le leggi
  vigenti.
10 %
  =====

11
12 function [K, P, Q, R, A_cl] = progetta_LQR_continuo(A, B)
13 %
  =====

14 % Calcolo pesi LQR (Regola di Bryson) coerenti con il modello F
  -16
15 % Ordine stati: [theta (rad); q (rad/s); U (ft/s); W (ft/s)]
16 % Ordine input: [Thrust (lbs); Elevator (rad); LEF (rad)]
17 %
  =====

18
19 % Massimi scostamenti tollerabili per gli stati (Bryson's rule)

```

```

20     max_theta = deg2rad(15); % 15 gradi max tollerati per il pitch
21     max_q     = deg2rad(30); % 30 deg/s max tollerati per il pitch
    rate
22     max_U     = 30;          % 30 ft/s max per la velocità X
23     max_W     = 20;          % 20 ft/s max per la velocità Z (
    incide su alpha)
24
25     Q = diag([1/max_theta^2, 1/max_q^2, 1/max_U^2, 1/max_W^2]);
26
27     % Massimi scostamenti tollerabili per gli attuatori
28     max_thrust = 5000;       % 5000 lbs variazione manetta
29     max_elev   = deg2rad(25); % 25 gradi max deflessione
    equilibratore
30     max_lef    = deg2rad(25); % 25 gradi max deflessione LEF
31
32     R = diag([1/max_thrust^2, 1/max_elev^2, 1/max_lef^2]);
33
34     % Fattori di tuning per dare più aggressività globale (se
    necessario)
35     rho_q = 1;
36     rho_r = 100;
37
38     Q = rho_q * Q;
39     R = rho_r * R;
40
41     % Sintesi LQR
42     [K, P, ~] = lqr(A, B, Q, R);
43     A_cl = A - B*K;
44 end

```

LISTING B.3: Progetto LQR (Tempo Continuo)

Progetto LQR

File: `funzioni_lqr/progetta_LQR_discreto.m`

Funzione per il calcolo del guadagno ottimo LQR nel dominio del tempo discreto.

```

1 %
  =====

2 % Copyright (c) 2026 Ing. Leggeri Leonardo
3 % Tutti i diritti riservati.
4 %
5 % ATTENZIONE: Questo software e il relativo codice sorgente sono di
  proprietà
6 % esclusiva dell'Ing. Leggeri Leonardo. É severamente vietata la
  copia,
7 % la distribuzione, la modifica o la vendita a terzi senza l'
  esplicito
8 % consenso scritto dell'autore. La distribuzione o la vendita non
  autorizzata
9 % costituisce reato ed è perseguibile penalmente secondo le leggi
  vigenti.
10 %
  =====

11
12 function [K, P, Q, R, A_cl] = progetta_LQR_discreto(A, B)
13 %
  =====

14 % Calcolo pesi LQR (Regola di Bryson) coerenti con il modello F
  -16
15 % Ordine stati: [theta (rad); q (rad/s); U (m/s); W (m/s)]
16 % Ordine input: [Thrust (N); Elevator (rad); LEF (rad)]
17 %
  =====

18
19 ft2m = 0.3048;
20 lbf2N = 4.44822;
21
22 % Massimi scostamenti tollerabili per gli stati (Bryson's rule)
23 max_theta = 15 * pi/180; % 15 gradi max tollerati per il pitch
24 max_q      = 30 * pi/180; % 30 deg/s max tollerati per il pitch
  rate

```

```

25     max_U      = 30 * ft2m;    % 30 ft/s convertiti in m/s max per la
    velocità X
26     max_W      = 20 * ft2m;    % 20 ft/s convertiti in m/s max per la
    velocità Z
27
28     Q = diag([1/max_theta^2, 1/max_q^2, 1/max_U^2, 1/max_W^2]);
29
30     % Massimi scostamenti tollerabili per gli attuatori
31     max_thrust = 5000 * lbf2N;    % 5000 lbs convertite in Newton
32     max_elev    = 25 * pi/180;    % 25 gradi max deflessione
    equilibratore
33     max_lef     = 25 * pi/180;    % 25 gradi max deflessione LEF
34
35     R = diag([1/max_thrust^2, 1/max_elev^2, 1/max_lef^2]);
36
37     % Fattori di tuning per dare più aggressività globale (se
    necessario)
38     rho_q = 20;    % Bilanciamento: molto reattivo ma senza rendere
    il problema Infeasible
39     rho_r = 1;    % Mantiene uno sforzo di controllo ragionevole per
    non violare i vincoli con x_iniziale elevato
40
41     Q = rho_q * Q;
42     R = rho_r * R;
43
44     % Sintesi LQR
45     fprintf('DEBUG progetta_LQR_discreto:\n');
46     fprintf('Size of A: %dx%d\n', size(A,1), size(A,2));
47     fprintf('Size of B: %dx%d\n', size(B,1), size(B,2));
48     fprintf('Size of Q before dlqr: %dx%d\n', size(Q,1), size(Q,2))
    ;
49     fprintf('Size of R before dlqr: %dx%d\n', size(R,1), size(R,2))
    ;
50
51     % Assicuriamoci che Q e R siano delle dimensioni esatte
52     Nx = size(A, 1);
53     Nu = size(B, 2);
54
55     if size(Q, 1) ~= Nx || size(Q, 2) ~= Nx

```

```

56         disp('ATTENZIONE: Q non è della dimensione corretta!
Ricostruzione sicura in corso...');
57         Q_new = eye(Nx);
58         n_min = min(Nx, size(Q,1));
59         Q_new(1:n_min, 1:n_min) = Q(1:n_min, 1:n_min);
60         Q = Q_new;
61     end
62
63     if size(R, 1) ~= Nu || size(R, 2) ~= Nu
64         disp('ATTENZIONE: R non è della dimensione corretta!
Ricostruzione sicura in corso...');
65         R = eye(Nu);
66     end
67
68     [K, P, ~] = dlqr(A, B, Q, R);
69     A_cl = A - B*K;
70 end

```

LISTING B.4: Progetto LQR (Tempo Discreto)

Plot Lyapunov (Tempo Continuo)

File: funzioni_lqr/plot_lyapunov.m

Funzione per la visualizzazione grafica dell'ellissoide del funzionale di Lyapunov continuo.

```

1 %
    =====

2 % Copyright (c) 2026 Ing. Leggeri Leonardo
3 % Tutti i diritti riservati.
4 %
5 % ATTENZIONE: Questo software e il relativo codice sorgente sono di
    proprietà
6 % esclusiva dell'Ing. Leggeri Leonardo. È severamente vietata la
    copia,
7 % la distribuzione, la modifica o la vendita a terzi senza l'
    esplicito

```

```

8 % consenso scritto dell'autore. La distribuzione o la vendita non
  autorizzata
9 % costituisce reato ed è perseguibile penalmente secondo le leggi
  vigenti.
10 %
  =====
11
12 function plot_lyapunov(P, A_cl)
13     %
  =====
14     % PLOT_LYAPUNOV - Ritratto di Fase e Funzione di Lyapunov 3D
15     % Riceve in ingresso la matrice P e la matrice a ciclo chiuso
  A_cl
16     %
  =====
17
18     set(0,'DefaultLineLineWidth',1.5);
19     set(0,'DefaultAxesFontSize',14);
20     set(0,'DefaultTextInterpreter','latex');
21
22     %% 1. Traiettorie (Solo dinamiche veloci)
23     dxdt = @(t,x) A_cl * x;
24
25     % Condizioni iniziali
26     % Matrice delle condizioni iniziali multiple (8 scenari di
  partenza)
27 % Ordine degli stati: [theta; q; u; w]
28 x0_mult = [
29     0,    0,    0,    0,    0,    0,    0,    0;
30     1.2, -1.2,  0.5, -0.5,  0.8, -0.8,  0.3, -0.3;
31     0,    0,    0,    0,    0,    0,    0,    0;
32     20,  -20,  15,  -15, -10,  10,  22,  -22
33 ];
34
35     figure('Name', 'Corto Periodo: w vs q', 'Color', 'w', 'Position
  ', [50 100 800 600]);
36     hold on; grid on;

```

```

37     colori = {'r', 'g', 'b'};
38
39     for i = 1:size(x0, 2)
40         % Simuliamo solo per 3 secondi per il corto periodo
41         [t, x] = ode45(dxdt, [0 3], x0(:, i));
42         % Plot: Asse X = w (ft/s), Asse Y = q (deg/s)
43         plot(x(:,4), rad2deg(x(:,2)), 'Color', colori{i});
44         plot(x(1,4), rad2deg(x(1,2)), '*', 'Color', colori{i}, '
MarkerSize', 8);
45     end
46
47     %% 2. Campo Vettoriale e Curve di Livello
48     w_lim = 150;
49     q_lim = 150;
50
51     [W_grid, Q_deg] = meshgrid(linspace(-w_lim, w_lim, 35),
linspace(-q_lim, q_lim, 35));
52     Q_rad = deg2rad(Q_deg);
53
54     W_dot = zeros(size(W_grid));
55     Q_dot = zeros(size(Q_rad));
56     V_contour = zeros(size(W_grid));
57
58     for i = 1:numel(W_grid)
59         % Slicing: imponiamo theta=0 e u=0
60         stato = [0; Q_rad(i); 0; W_grid(i)];
61         derivata = A_cl * stato;
62
63         Q_dot(i) = derivata(2);
64         W_dot(i) = derivata(4);
65         V_contour(i) = stato' * P * stato;
66     end
67
68     % Campo vettoriale
69     L = sqrt(W_dot.^2 + Q_dot.^2) + 1e-6;
70     quiver(W_grid, Q_deg, W_dot./L, rad2deg(Q_dot)./L, 0.45, 'Color
', [0.7 0.7 0.7]);
71
72     % Curve di Livello (Spaziatura Logaritmica)
73     val_min = min(V_contour(:));

```

```

74     val_max = max(V_contour(:));
75     livelli_log = logspace(log10(val_min + 1e-3), log10(val_max),
18);
76
77     contour(W_grid, Q_deg, V_contour, livelli_log, 'LineWidth',
1.5);
78     xlabel('Vertical Velocity $w$ [ft/s]')
79     ylabel('Pitch Rate $q$ [deg/s]')
80     title('Moto di Corto Periodo (Dinamica Veloce)')
81
82     xlim([-w_lim, w_lim]);
83     ylim([-q_lim, q_lim]);
84     legend({'Traiettoria 1', 'Start 1', 'Traiettoria 2', 'Start 2',
'Traiettoria 3', 'Start 3', ...
85         'Direzione Flusso', 'Contour $V(x) = x^T P x$'}, '
Interpreter','latex', 'Location', 'bestoutside');
86
87     %% 3. Visualizzazione 3D della Funzione di Lyapunov LQR
88     figure('Name', 'Funzione di Lyapunov 3D - LQR', 'Color', 'w');
89     hold on; grid on;
90
91     % Superficie 3D
92     V_surf = zeros(size(W_grid));
93     for i = 1:size(W_grid, 1)
94         for j = 1:size(W_grid, 2)
95             stato_surf = [0; Q_rad(i,j); 0; W_grid(i,j)];
96             V_surf(i,j) = stato_surf' * P * stato_surf;
97         end
98     end
99
100     surf(W_grid, Q_deg, V_surf, 'EdgeColor', 'none', 'FaceAlpha',
0.7);
101     colormap jet;
102     colorbar;
103
104     % Proiezione traiettorie 3D
105     for i = 1:size(x0, 2)
106         % Integrazione più fitta per fluidità visiva
107         [t, x] = ode45(dxdt, linspace(0, 3, 300), x0(:, i));
108

```



```

109     V_traiettoria = sum((x * P) .* x, 2);
110
111     plot3(x(:,4), rad2deg(x(:,2)), V_traiettoria, 'k-', '
LineWidth', 2);
112     plot3(x(1,4), rad2deg(x(1,2)), V_traiettoria(1), 'ro', '
MarkerFaceColor', 'r', 'MarkerSize', 6);
113     end
114
115     title('Funzione di Lyapunov  $V(x) = x^T P x$  (LQR)', '
Interpreter', 'latex')
116     xlabel('w$ [ft/s]', 'Interpreter', 'latex')
117     ylabel('q$ [deg/s]', 'Interpreter', 'latex')
118     zlabel('V(x)$', 'Interpreter', 'latex')
119     view(-45, 30);
120 end

```

LISTING B.5: Plot Lyapunov (Tempo Continuo)

Plot Lyapunov (Tempo Discreto)

File: funzioni_lqr/plot_lyapunov_discrete.m

Funzione per la visualizzazione grafica dell'ellissoide del funzionale di Lyapunov discreto.

```

1 %
  =====

2 % Copyright (c) 2026 Ing. Leggeri Leonardo
3 % Tutti i diritti riservati.
4 %
5 % ATTENZIONE: Questo software e il relativo codice sorgente sono di
  proprietà
6 % esclusiva dell'Ing. Leggeri Leonardo. É severamente vietata la
  copia,
7 % la distribuzione, la modifica o la vendita a terzi senza l'
  esplicito
8 % consenso scritto dell'autore. La distribuzione o la vendita non
  autorizzata

```

```

9 % costituisce reato ed è perseguibile penalmente secondo le leggi
    vigenti.
10 %
    =====
11
12 function plot_lyapunov_discrete(P_ds, A_cl_ds, Ts)
13     %
    =====
14
15     % PLOT_LYAPUNOV_DISCRETE - Funzione di Lyapunov 3D per sistemi
    a tempo discreto
16     % Grafico migliorato: Traiettoria interpolata fluida +
    Campionamenti discreti
17     %
    =====
18
19     if nargin < 3
20         Ts = 0.05;
21     end
22
23     rad2deg = 180 / pi;
24
25     % --- 1. DEFINIZIONE LIMITI VISIVI ---
26     w_lim = 50; % ft/s
27     q_lim = 50; % deg/s
28
29     [W_grid, Q_deg] = meshgrid(linspace(-w_lim, w_lim, 80),
    linspace(-q_lim, q_lim, 80));
30     Q_rad = Q_deg / rad2deg;
31
32     % --- 2. CALCOLO SUPERFICIE DI LYAPUNOV ---
33     V_surf = zeros(size(W_grid));
34     for i = 1:size(W_grid, 1)
35         for j = 1:size(W_grid, 2)
36             stato_surf = [0; Q_rad(i,j); 0; W_grid(i,j)];
37             V_surf(i,j) = stato_surf' * P_ds * stato_surf;
38         end
39     end

```

```

39
40 % --- 3. DINAMICA DISCRETA E CONDIZIONI INIZIALI ---
41 N_steps = round(10 / Ts);
42
43 x0 = [0, 0.5, 0, 30;
44       0, -0.6, 0, -40;
45       0, 0.8, 0, -20]';
46
47 %% --- FIGURA 3D ---
48 figure('Name', sprintf('Lyapunov 3D Discreto (Ts = %g s)', Ts),
49       'Color', 'w', 'Position', [150 150 850 650]);
50 hold on; grid on;
51
52 % 1. Disegno Superficie
53 h_surf = surf(W_grid, Q_deg, V_surf, 'EdgeColor', 'none', '
54 FaceAlpha', 0.65);
55 colormap jet;
56 cb = colorbar;
57 ylabel(cb, 'Energia  $V(x_k)$ ', 'Interpreter', 'latex', '
58 FontSize', 12);
59
60 colori = {'r', 'g', 'b'};
61 max_v_traj = 0;
62
63 % 2. Simulazione e Disegno Traiettorie
64 for i = 1:size(x0, 2)
65     x_traj = zeros(4, N_steps);
66     x_traj(:, 1) = x0(:, i);
67
68     for k = 1:N_steps-1
69         x_traj(:, k+1) = A_cl_ds * x_traj(:, k);
70     end
71
72     V_traj = zeros(N_steps, 1);
73     for k = 1:N_steps
74         V_traj(k) = x_traj(:, k)' * P_ds * x_traj(:, k);
75     end
76
77     if max(V_traj) > max_v_traj
78         max_v_traj = max(V_traj);
79     end
80 end

```

```

76         end
77
78         % --- MAGIA VISIVA: INTERPOLAZIONE FLUIDA DELLA TRAIETTORIA
79         ---
80         % Usiamo 'pchip' (interpolazione cubica che non crea finti
81         rimbalzi)
82         % per generare 10 volte più punti e rendere la curva
83         morbida
84         t_discrete = 1:N_steps;
85         t_fine = linspace(1, N_steps, N_steps * 10);
86
87         W_smooth = pchip(t_discrete, x_traj(4,:), t_fine);
88         Q_smooth = pchip(t_discrete, x_traj(2,:) * rad2deg, t_fine)
89         ;
90         V_smooth = pchip(t_discrete, V_traj, t_fine);
91
92         % 1. Disegniamo prima la SCIA FLUIDA (linea continua)
93         if i == 1
94             h_line = plot3(W_smooth, Q_smooth, V_smooth, '-', '
95             Color', colori{i}, 'LineWidth', 2);
96         else
97             plot3(W_smooth, Q_smooth, V_smooth, '-', 'Color',
98             colori{i}, 'LineWidth', 2);
99         end
100
101         % 2. Disegniamo sopra i PUNTI DISCRETI (solo pallini, senza
102         linee)
103         plot3(x_traj(4,:), x_traj(2,:) * rad2deg, V_traj, 'o', ...
104             'MarkerEdgeColor', colori{i}, 'MarkerFaceColor', 'w',
105             'MarkerSize', 4);
106
107         % Partenza (Start)
108         if i == 1
109             h_start = plot3(x_traj(4,1), x_traj(2,1) * rad2deg,
110             V_traj(1), 'o', ...
111                 'MarkerFaceColor', colori{i}, '
112             MarkerEdgeColor', 'k', 'MarkerSize', 8);
113         else
114             plot3(x_traj(4,1), x_traj(2,1) * rad2deg, V_traj(1), 'o
115             ', ...

```

```

105         'MarkerFaceColor', colori{i}, 'MarkerEdgeColor',
    'k', 'MarkerSize', 8);
106     end
107
108     % Arrivo (End)
109     plot3(x_traj(4,end), x_traj(2,end) * rad2deg, V_traj(end),
    's', ...
110         'MarkerFaceColor', colori{i}, 'MarkerEdgeColor', 'k',
    'MarkerSize', 8);
111     end
112
113     % --- ORIGINE ---
114     h_end = plot3(0, 0, 0, 'p', 'MarkerFaceColor', 'y', '
    MarkerEdgeColor', 'k', 'MarkerSize', 16);
115
116     % --- FORMATTAZIONE E LEGENDA ---
117     title(sprintf('\textbf{Funzione di Lyapunov $V(x_k)$ a Tempo
    Discreto ($T_s = %g$ s)}', Ts), 'Interpreter', 'latex', '
    FontSize', 16);
118     xlabel('Velocit\'a Verticale $w$ [ft/s]', 'Interpreter', 'latex
    ', 'FontSize', 12);
119     ylabel('Pitch Rate $q$ [deg/s]', 'Interpreter', 'latex', '
    FontSize', 12);
120     zlabel('Energia $V(x_k) = x_k^T P_d x_k$', 'Interpreter', '
    latex', 'FontSize', 12);
121
122     legend([h_surf, h_line, h_start, h_end], ...
123         {'Superficie $V(x_k)$', 'Evoluzione di Stato (
    Interpolata)', 'Partenza ($k=0$)', 'Origine ($k \to \infty$)'},
    ...
124         'Interpreter', 'latex', 'FontSize', 12, 'Location', '
    northeast');
125
126     zlim([0, max_v_traj * 1.2]);
127     xlim([-w_lim, w_lim]);
128     ylim([-q_lim, q_lim]);
129     view(-35, 30);
130 end

```

LISTING B.6: Plot Lyapunov (Tempo Discreto)

Bibliografia

- [1] Raktim Bhattacharya. *F16-Model-Matlab: TrimF16.m*. Implementazione MATLAB del modello non lineare dell’F-16. URL: <https://github.com/isrlab/F16-Model-Matlab/blob/main/TrimF16.m> (visitato il giorno 26/03/2026).
- [2] Karl J. Åström e Richard M. Murray. *Feedback Systems: An Introduction for Scientists and Engineers*. 1st. Princeton, NJ: Princeton University Press, 2008. ISBN: 978-0-691-13576-2. URL: https://fbswiki.org/wiki/index.php/Feedback_Systems:_An_Introduction_for_Scientists_and_Engineers.
- [3] Brian L Stevens, Frank L Lewis e Eric N Johnson. *Aircraft control and simulation: dynamics, controls design, and autonomous systems*. John Wiley & Sons, 2015.
- [4] Brian L. Stevens, Frank L. Lewis e Eric N. Johnson. “Modeling the Aircraft”. In: *Aircraft Control and Simulation: Dynamics, Controls Design, and Autonomous Systems*. Analisi aerodinamica, linearizzazione e separazione dei moti. John Wiley & Sons, Ltd, 2015. Cap. 2, pp. 63–141. ISBN: 9781119174882. DOI: [10.1002/9781119174882.ch2](https://onlinelibrary.wiley.com/doi/abs/10.1002/9781119174882.ch2). URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/9781119174882.ch2>.
- [5] Frank L. Lewis. “Modeling, Design, and Simulation Tools”. In: *Aircraft Control and Simulation: Dynamics, Controls Design, and Autonomous Systems*. Estratto per l’analisi dei modelli di stato continui e simulazione del volo. John Wiley & Sons, Ltd, 2015. Cap. 3, pp. 142–249. ISBN: 9781119174882. DOI: [10.1002/9781119174882.ch3](https://onlinelibrary.wiley.com/doi/abs/10.1002/9781119174882.ch3). URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/9781119174882.ch3>.
- [6] Massachusetts Institute of Technology (MIT). *Quaternions lecture*. MIT. URL: <https://ocw.mit.edu/courses/6-801-machine-vision-fall-2020/resources/lecture-18-rotation-and-how-to-represent-it-unit-quaternions-the-space-of-rotations/> (visitato il giorno 26/03/2026).
- [7] Massachusetts Institute of Technology (MIT). *Differential Equations Notes: The Runge-Kutta Method*. Course 10.001 Web Notes. MIT. URL: <https://>

- web.mit.edu/10.001/Web/Course_Notes/Differential_Equations_Notes/node5.html (visitato il giorno 26/03/2026).
- [8] Bernard Etkin e Lloyd Duff Reid. *Dynamics of flight: stability and control*. Vol. 3. New York: John Wiley & Sons, 1996.
 - [9] *Particle_Swarm_Optimization_toolbox*. URL: <https://it.mathworks.com/matlabcentral/fileexchange/7506-particle-swarm-optimization-toolbox> (visitato il giorno 26/03/2026).
 - [10] Antonio Ferramosca. *Lezione 06 - Schemi di controllo multivariabili*. Materiale didattico per il corso di Controllo Avanzato. Control and Automation Laboratory (CAL). URL: https://cal.unibg.it/wp-content/uploads/Controllo_avanzato/Lezione-06-Schemi-di-controllo-multivariabili.pdf (visitato il giorno 26/03/2026).
 - [11] Katsuhiko Ogata. *Modern Control Engineering*. 5th. Boston, MA: Prentice Hall, 2010.
 - [12] Davide Brugali. *Embedded Real-Time Systems — Lecture Slides*. Materiale didattico del corso di Embedded Real-Time Systems, Università degli Studi di Bergamo. Dispense del corso a.a. 2025–2026, Prof. Davide Brugali. 2025.
 - [13] Davide Brugali. *POSIX Threads — Lecture Notes*. Dispense 19.2–19.3, corso Embedded Real-Time Systems, Università degli Studi di Bergamo, a.a. 2025–2026. 2025.
 - [14] Davide Brugali. *DDS — Lecture Notes*. Dispensa 19.3, corso Embedded Real-Time Systems, Università degli Studi di Bergamo, a.a. 2025–2026. 2025.
 - [15] eProxima. *Fast DDS Monitor – Documentation v4.0.0*. <https://fast-dds-monitor.readthedocs.io/en/latest/>. Accessed: 2026-05-05. 2024.
 - [16] Steven Rostedt. *Debugging the Kernel Using ftrace*. <https://lwn.net/Articles/322666/>. Linux Weekly News (LWN.net). 2009.
 - [17] Steven Rostedt. *trace-cmd-record(1) — Manual Page*. <https://www.trace-cmd.org/Documentation/trace-cmd/trace-cmd-record.1.html>. Copyright Red Hat Inc., GNU Public License. 2010.

- [18] Wikipedia. *Radar*. WikiPedia. URL: https://www.faa.gov/sites/faa.gov/files/aircraft/air_cert/design_approvals/air_software/05-27_RTSA.pdf (visitato il giorno 06/05/2026).
- [19] *AFTI/F-16 Digital Flight Control System Experience*. Rapp. tecn. NASA, 1984. URL: <https://ntrs.nasa.gov/api/citations/19840012524/downloads/19840012524.pdf>.
- [20] *Testing the Latest Embedded GPS/INS Hybrid Navigation System for the F-16 Fighting Falco*. Rapp. tecn. U.S airforce. URL: https://ndia.dtic.mil/wp-content/uploads/2007/test/JKillian_SessionD3.pdf.
- [21] Wikipedia. *Radar*. WikiPedia. URL: <https://en.wikipedia.org/wiki/AN/APG-68> (visitato il giorno 06/05/2026).
- [22] Cary Spitzer, Uma Ferrell e Thomas Ferrell. *Digital avionics handbook*. CRC press, 2017. URL: https://books.google.it/books?hl=it&lr=&id=kvjRBQAAQBAJ&oi=fnd&pg=PP1&dq=spitzer+2017+digital,&ots=TDwms27puj&sig=oq3DEpEi0WTz_eHcTUWwSFQ2xTk&redir_esc=y#v=onepage&q=spitzer%202017%20digital%2C&f=false.
- [23] Antonio Ferramosca. *Lezione 08 - Regolatore Lineare Quadratico LQR*. Materiale didattico per il corso di Controllo Avanzato. Control and Automation Laboratory (CAL). URL: https://cal.unibg.it/wp-content/uploads/Controllo_avanzato/Lezione-08-LQR.pdf (visitato il giorno 26/03/2026).
- [24] Karl J. Åström e Tore Hägglund. *Advanced PID Control*. Research Triangle Park, NC: ISA - The Instrumentation, Systems e Automation Society, 2006. ISBN: 978-1556179426.
- [25] Brian L. Stevens, Frank L. Lewis e Eric N. Johnson. “The Kinematics and Dynamics of Aircraft Motion”. In: *Aircraft Control and Simulation: Dynamics, Controls Design, and Autonomous Systems*. Estrazione delle equazioni cinematiche e dinamiche del moto. John Wiley & Sons, Ltd, 2015. Cap. 1, pp. 1–62. ISBN: 9781119174882. DOI: [10.1002/9781119174882.ch1](https://doi.org/10.1002/9781119174882.ch1). URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/9781119174882.ch1>.
- [26] Davide Brugali. *Modeling Real-Time Activities — Lecture Notes*. Dispensa 09, corso Embedded Real-Time Systems, Università degli Studi di Bergamo, a.a. 2025–2026. 2025.

- [27] Davide Brugali. *Priority-Driven RT Scheduling — Lecture Notes*. Dispensa 11.0, corso Embedded Real-Time Systems, Università degli Studi di Bergamo, a.a. 2025–2026. 2025.
- [28] Davide Brugali. *Handling Overrun — Lecture Notes*. Dispensa 14, corso Embedded Real-Time Systems, Università degli Studi di Bergamo, a.a. 2025–2026. 2025.
- [29] eProxima. *Fast DDS Monitor*. Sito ufficiale e download dell'applicazione. eProxima. URL: <https://www.eprosima.com/middleware/tools/fast-dds-monitor> (visitato il giorno 06/05/2026).